Chương 0. Mục tiêu học tập và bản đồ khối kiến trúc

▶ MỤC TIÊU HỌC TẬP

Sau khi hoàn thành hệ thống 15 chương trong tập tài liệu này (Chương 0 đến Chương 14), người học sẽ làm chủ trọn vẹn quy trình chuyển đổi giải thuật đo cao độ giọng nói F_0 chuẩn mực phòng thí nghiệm của Paul Boersma (1993, phần mềm Praat) sang một hệ thống phần cứng chuyên dụng chạy theo thời gian thực trên kit FPGA DE10-Standard (chip Intel Cyclone V 5CSXFC6D6F31C6N). Cụ thể, bạn sẽ:

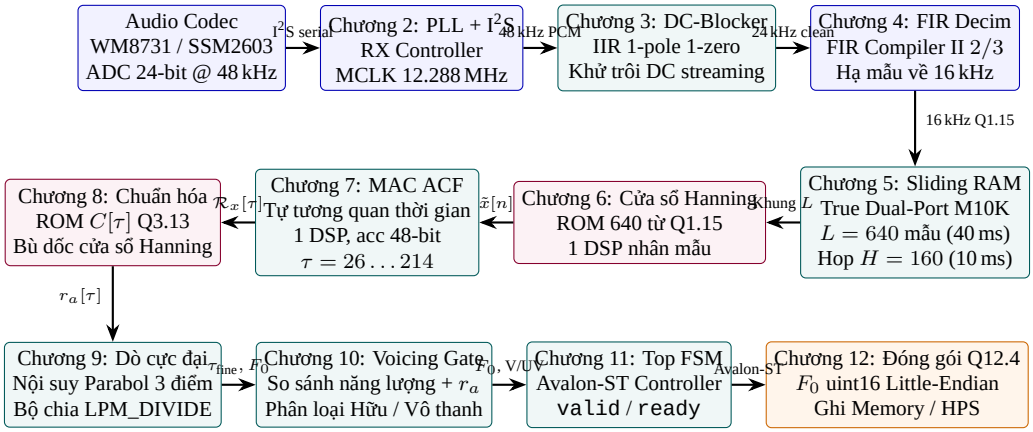
1. Thấu hiểu bản chất vật lý của dao động dây thanh quản, giải thích được từng công thức giải tích trong mã nguồn gốc C++ của Praat (`Sound_to_Pitch.cpp` và `Pitch.cpp`), từ bước khử trôi mức một chiều (DC offset), cửa sổ hóa Hanning, tự tương quan ngắn hạn, chuẩn hóa cửa sổ, đến nội suy đỉnh dưới mẫu và phân loại hữu thanh/vô thanh.
2. Cấu hình chính xác từng khối lõi sở hữu trí tuệ (Intel IP cores) trên phần mềm Quartus Prime Standard / Lite Edition: Altera PLL, On-Chip Memory RAM/ROM 2-PORT, FIR Compiler II, LPM_MULT, LPM_DIVIDE và hiểu rõ cấu tạo vi kiến trúc bên trong các khối DSP blocks và khối nhớ nhúng M10K.
3. Tự tay thiết kế toàn bộ đường truyền dữ liệu (datapath) và máy trạng thái hữu hạn (FSM) bằng ngôn ngữ Verilog HDL chuẩn công nghiệp, tuân thủ chặt chẽ giao thức luồng dữ liệu chuẩn Avalon-ST (`valid`/`ready`).
4. Xây dựng mô hình phần mềm song sinh khớp từng bit (bit-exact Python twin), bảo đảm từng phép cộng, trừ, nhân, dịch bit và bão hòa hai's complement trong phần cứng đều cho kết quả hoàn toàn trùng khớp với phần mềm trên máy tính.
5. Nắm vững phương pháp định lượng tài nguyên silicon (ALM, LUT, Register, DSP, M10K, Fmax, Latency, RTF), bảo đảm thiết kế đạt thông lượng thời gian thực với chi phí dưới 2% diện tích chip Cyclone V.

0.1. Bản đồ đường tín hiệu toàn diện (Complete Signal Path Roadmap)

Trong các hệ thống trí tuệ nhân tạo giọng nói ở biên (Edge Voice AI), tín hiệu âm thanh thu nhận từ micro cần được phân tách thành hai luồng xử lý song song độc lập:

- **Luồng nhận dạng âm học (Log-Mel Frontend):** Gom năng lượng phổ qua 80 bộ lọc tam giác Mel để nhận biết khẩu hình và cộng hưởng ống thanh âm, phục vụ các mô hình nơ-ron nhận dạng giọng nói tự động (ASR như Whisper). Quá trình tích phân dải rộng này cố tình san phẳng và xóa bỏ thông tin cao độ.
- **Luồng đo cao độ (F_0 Stream):** Thực hiện đo chính xác tần số rung cơ học $F_0 = 1/T_0$ của dây thanh quản theo thời gian thực với độ phân giải dưới 1 Hertz, phục vụ nhận diện thanh điệu tiếng Việt (ngang, huyền, sắc, hỏi, ngã, nặng), ngữ điệu câu và chấm điểm phát âm đối sánh tương quan Pearson (r) với chuẩn Praat.

Hình 7 trình bày bản đồ tổng thể đường tín hiệu F_0 trên FPGA từ khi sóng âm chạm vào màng rung micro đến khi xuất dữ liệu cao độ chuẩn định dạng Q12.4 ra thanh ghi giao tiếp.



Hình 7: Bản đồ khối kiến trúc phần cứng ước lượng cao độ F_0 thời gian thực trên kit DE10-Standard (Chương 2 đến 12). Chương 13 thiết lập quy trình kiểm chứng 3 tầng; Chương 14 cung cấp bảng kiểm tra (checklist) nghiệm thu.

0.2. Khung chuẩn mực tám yếu tố của mỗi bài học

Để loại bỏ hoàn toàn tình trạng các chương đọc như bản tóm tắt công thức rời rạc, mỗi bài học trong tài liệu này (từ giải thuật Praat ở Chương 1 đến các khối phần cứng ở Chương 2–12) đều được biên soạn theo một cấu trúc sư phạm chặt chẽ gồm đúng **tám yếu tố bắt buộc**:

1. **Hiện tượng vật lý thật và trực giác cơ bản (Physical Insight):** Phân tích bản chất sinh lý học hoặc điện tử học ngoài đời thực bằng hình ảnh trực quan và liên hệ thực tế; phân định rạch ròi đâu là hiện tượng tự nhiên của âm thanh, đâu chỉ là quy ước toán học hoặc kỹ thuật số.

2. **Toán học từ nguyên lý đầu tiên (First Principles Math):** Dẫn dắt công thức tường minh từ định nghĩa gốc, biến đổi chi tiết từng bước, tuyệt đối không nhảy cóc; định nghĩa rõ ràng đơn vị và ý nghĩa vật lý của mọi ký hiệu trước khi đưa vào biểu thức.

3. **Ví dụ tính tay với số liệu nhỏ (Hand-Worked Example):** Thực hiện tính toán thủ công từng bước với các chuỗi số nhỏ mẫu mực, hiển thị toàn bộ giá trị trung gian. Toàn bộ các con số trong ví dụ đều được kiểm chứng độc lập bằng tập lệnh Python lưu trữ trong thư mục `verify/`.

4. **Vi kiến trúc phần cứng và Intel IP chuẩn (Hardware Architecture):** Phân tích nguyên lý làm việc bên trong các khối IP chuẩn của Intel Quartus (Altera PLL, On-Chip Memory RAM/ROM, FIR Compiler II, LPM_MULT, LPM_DIVIDE) và các khối DSP cứng trên chip Cyclone V. Nêu rõ các ràng buộc silicon thật, ví dụ bộ nhớ M10K không hỗ trợ chế độ đọc-khi-ghi "New Data" trên cùng địa chỉ mà chỉ hỗ trợ "Old Data" hoặc "Don't Care".

5. **Thiết kế RTL Verilog tự viết (Self-Written RTL Alternative):** Cung cấp đoạn mã Verilog HDL tổng hợp được cho khối chức năng tương đương, giải thích chi tiết ý nghĩa kỹ thuật của từng dòng lệnh và cơ chế bắt tay Avalon-ST.

6. **Mô hình song sinh phần mềm khớp bit (Bit-Exact Twin):** Viết hàm Python mô phỏng chính xác hành vi số học dấu phẩy tĩnh (cùng định dạng Q$m.n$, cùng quy tắc làm tròn và bão hòa), đóng vai trò làm thước đo kiểm định khuôn vàng thước ngọc cho phần cứng.

7. **Hộp chi phí silicon chi tiết (Cost Box):** Mở rộng và giải thích đầy đủ mọi thuật ngữ viết tắt ngay lần đầu xuất hiện (ALM, LUT, Register, DSP, M10K, MLAB, Fmax, Latency, RTF) và giải thích cặn kẽ tại sao lựa chọn kiến trúc này là tối ưu cho ngân sách kit DE10-Standard.

8. **Hệ thống bài tập củng cố kèm lời giải chi tiết (Exercises with Explained Solutions):** Các câu hỏi kiểm tra tư duy sâu sắc, có lời giải giải thích nguyên nhân gốc rễ, không chấp nhận câu trả lời một từ.

0.3. Phần cứng mục tiêu và Ngân sách Silicon Kit DE10-Standard

Hệ thống được thiết kế và kiểm chứng thực tế trên kit phát triển **Terasic DE10-Standard**, trang bị vi mạch SoC **Intel Cyclone V SX** (mã linh kiện: `5CSXFC6D6F31C6N`, Speed Grade 6), kết nối với chip giải mã âm thanh analog **Wolfson WM8731** (hoặc SSM2603) thông qua bus dữ liệu nối tiếp I^2S và bus điều khiển I^2C. Nguồn dao động thạch anh chính trên bo mạch có tần số $50\,\text{MHz}$.

Bảng 7 tổng hợp bảng thông số ngân sách tài nguyên vật lý thật trên chip Cyclone V SoC và định mức tối đa dành riêng cho toàn bộ đường xử lý cao độ F_0.

Bảng 7: Bảng phân bổ ngân sách tài nguyên trên chip Intel Cyclone V 5CSXFC6D6F31C6N

Tài nguyên Silicon	**Tổng có sẵn trên Chip**	**Ngân sách luồng F_0**	**Tỷ lệ chiếm dụng (%)**
Logic Modules (ALMs)	41,910 ALMs ($\sim 110\,\text{K}$ LEs)	≈ 850 ALMs	2.03%
Bản ghi logic (Registers)	166,036 Flip-Flops	$\approx 1,200$ Flip-Flops	0.72%
Khối DSP cứng (18×19)	112 khối (hoặc 56 khối 27×27)	3 khối DSP	2.68%
Khối nhớ nhúng M10K	553 khối ($5,530\,\text{Kbit}$)	4 khối M10K ($40\,\text{Kbit}$)	0.72%
Bộ nhớ phân tán MLAB	2,883 khối ($1,845\,\text{Kbit}$)	0 khối (tùy chọn ROM nhỏ)	0.00%
Khối vòng khóa pha (PLL)	6 bộ Fractional PLLs	1 bộ PLL (cho audio)	16.67%

► NHÌN THẤY VẬT LÝ

Toàn bộ thuật toán đo cao độ F_0 chỉ chiếm vỏn vẹn khoảng 2% tài nguyên logic và dưới 3% khối DSP của một chip FPGA tầm trung. Hơn 97% silicon còn lại được giải phóng hoàn toàn để triển khai các mạng nơ-ron sâu nhận dạng giọng nói, xử lý thị giác máy tính hoặc trao đổi dữ liệu tốc độ cao với vi xử lý ARM Cortex-A9 MPCore kép (HPS).

0.4. Từ điển trực giác cốt lõi — Bắt cầu Feynman

Dưới đây là hệ thống định nghĩa sâu sắc về các khái niệm kỹ thuật cốt lõi sẽ xuất hiện xuyên suốt từ Chương 1 đến Chương 14. Mỗi mục được giải thích bằng văn xuôi trôi chảy, trả lời rõ ba câu hỏi: *nó là gì, nó là hiện tượng vật lý thật hay chỉ là công cụ tính toán,* và *tại sao nó ảnh hưởng trực tiếp đến độ chính xác của* F_0.

★ BẮC CẦU FEYNMAN: 1. Tín hiệu liên tục (Continuous-Time), ADC và Quá trình Lấy Mẫu

Sóng âm lan truyền trong không khí là dao động áp suất **liên tục theo thời gian**: tại bất kỳ khoảnh khắc vô cùng nhỏ nào, áp suất khí quyển luôn có một giá trị xác định và biến thiên một cách mượt mà. Màng rung micro chuyển áp suất này thành điện áp tương tự liên tục. Khối **ADC** (*Analog-to-Digital Converter* — Bộ chuyển đổi tương tự sang số) thực hiện phép **lấy mẫu**: cứ sau mỗi chu kỳ thời gian $T_s = 1/f_s$ giây (với f_s là tần số lấy mẫu, ví dụ $48\,\text{kHz}$ hoặc $16\,\text{kHz}$), bộ so sánh điện áp sẽ đo độ lớn tín hiệu và chuyển thành một số nhị phân hữu hạn bit (ví dụ 16 bit hoặc 24 bit), hoàn toàn bỏ qua những biến động xảy ra giữa hai lần đo.

Phép lấy mẫu là một thao tác kỹ thuật nhân tạo, nhưng nó dẫn đến một **hệ quả vật lý thực sự**: phổ tần số của chuỗi số rời rạc bị **lặp lại tuần hoàn với chu kỳ** f_s trên toàn bộ trục tần số, tạo ra vô số các bản sao của phổ gốc tại các vị trí $\pm f_s, \pm 2f_s, \ldots$. Nếu tín hiệu âm thanh ban đầu chứa bất kỳ thành phần tần số nào cao hơn $f_s/2$, các bản sao này sẽ lấn sang và chồng đè lên phổ gốc — gây ra hiện tượng méo chồng phổ (*aliasing*). Đối với việc ước lượng F_0: ta làm việc ở $f_s = 16\,\text{kHz}$, nghĩa là mọi rung động âm học trên $8\,\text{kHz}$ bắt buộc phải được triệt tiêu hoàn toàn trước khi vào bộ tính tương quan; nếu không, năng lượng cao tần sẽ biến thành các chu kỳ giả mạo làm sai lệch cao độ.

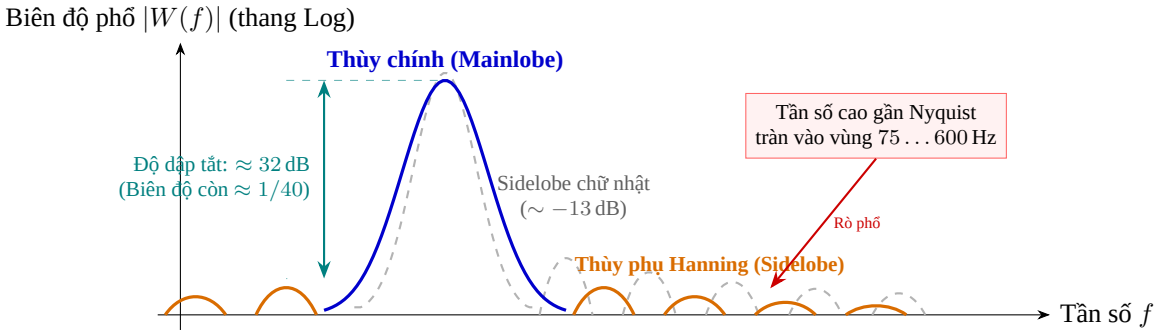
★ BẮC CẦU FEYNMAN: 2. Tần số Nyquist, Hiện tượng Chồng Phổ (Aliasing) và Hình Ảnh Phản Xạ Gương

Tần số Nyquist được định nghĩa là $f_N = f_s/2$, đại diện cho ngưỡng tần số vật lý cao nhất mà một hệ thống rời rạc có thể ghi nhận chính xác theo Định lý lấy mẫu Shannon–Nyquist. **Aliasing** là hiện tượng một tần số cao $f > f_s/2$ bị gập ngược qua ranh giới Nyquist và giả dạng hoàn hảo thành một tần số thấp hơn $f_{\text{alias}} = |f_s - f|$ (hoặc $|k f_s \pm f|$). *Ví dụ số trực quan:* Tại tần số lấy mẫu $f_s = 16\,\text{kHz}$, tần số Nyquist là $f_N = 8\,\text{kHz}$. Một sóng âm có tần số thật là $9\,\text{kHz}$ sau khi qua ADC sẽ tạo ra các điểm mẫu giống hệt như một sóng âm $7\,\text{kHz}$ ($16 - 9 = 7\,\text{kHz}$). Ranh giới $f_s/2$ hoạt động như một **tấm gương phản xạ**: mọi tần số vượt qua mép gương đều bị phản xạ quay ngược vào trong dải tần làm việc. Nếu màng rung micro thu phải tiếng ồn cơ học, tiếng rít quạt gió hay sóng hài bậc cao ở $9\,\text{kHz}$, nó sẽ biến thành một thành phần tần số $7\,\text{kHz}$ sạch sẽ trong dữ liệu số. Thành phần này kết hợp với hiện tượng rò phổ của cửa sổ phân khung sẽ tạo ra các đỉnh tự tương quan giả nằm đúng vào vùng tìm kiếm cao độ con người ($75 \ldots 600\,\text{Hz}$), phá hỏng kết quả đo.

★ BẮC CẦU FEYNMAN: 3. Cắt Khung (Framing), Cửa Sổ (Window), Rò Phổ (Spectral Leakage), Mainlobe, Sidelobe và Đơn Vị dB

Giọng nói con người liên tục thay đổi khẩu hình theo từng âm vị, nên ta không thể xử lý toàn bộ câu nói một lúc mà phải **cắt khung**: trích ra một đoạn ngắn khoảng $40\,\text{ms}$ ($L = 640$ mẫu ở $16\,\text{kHz}$) để phân tích. Phép cắt đột ngột (cửa sổ chữ nhật — Rectangular window) tương đương với việc nhân tín hiệu liên tục với một xung vuông; trong miền tần số, điều này tương đương với phép **tích chập (convolution)** phổ tín hiệu với hàm $\text{sinc}(f) = \sin(\pi f)/(\pi f)$. Hàm sinc sở hữu một búp sóng trung tâm gọi là **thùy chính (mainlobe)** và vô số các gợn sóng lan rộng hai bên gọi là **thùy phụ (sidelobe)**.

Hiện tượng năng lượng của một tần số đơn lẻ bị phát tán lan man sang toàn bộ các tần số khác qua các thùy phụ được gọi là **rò phổ (spectral leakage)**. Thùy phụ đầu tiên của cửa sổ chữ nhật chỉ suy giảm khoảng $-13\,\text{dB}$ so với đỉnh thùy chính. Đơn vị **Decibel (dB)** là thang đo logarit của tỷ số biên độ: $-13\,\text{dB} = 20\log_{10}(A/A_0) \Rightarrow A/A_0 = 10^{-13/20} \approx 0.224 \approx 1/4.5$. Nghĩa là năng lượng rò rỉ vẫn còn tới hơn 22% biên độ gốc! Cửa sổ **Hanning** vuốt hai đầu khung mượt mà về gần 0 bằng một nửa chu kỳ hàm cosin, kéo thùy phụ đầu tiên tụt xuống mức $-32\,\text{dB}$ ($10^{-32/20} \approx 0.025 \approx 1/40$ biên độ, giảm hơn 97% năng lượng rò so với cửa sổ chữ nhật), giúp bảo vệ dải tần F_0 khỏi các bóng ma tần số cao.



Hình 8: So sánh phổ biên độ của cửa sổ Chữ nhật (nét đứt) và cửa sổ Hanning (nét liền). Cửa sổ Hanning chấp nhận thùy chính rộng hơn đôi chút để dập tắt thùy phụ từ mức $-13\,\text{dB}$ xuống $-32\,\text{dB}$, ngăn chặn hiện tượng rò phổ làm bẩn dải đo cao độ F_0.

★ BẮC CẦU FEYNMAN: 4. Phát Ngôn (Utterance), Thuật Toán FFT/IFFT, Mặt Nạ Phổ và Zero-Padding

Trong xử lý ngôn ngữ, một **phát ngôn (utterance)** là một chuỗi âm thanh liên tục tương ứng với một câu nói hoàn chỉnh, ngăn cách bởi các khoảng lặng hít thở. **FFT** (*Fast Fourier Transform*) là thuật toán kinh điển tính nhanh biến đổi Fourier rời rạc (DFT), chuyển chuỗi thời gian $x[n]$ sang các thành phần phổ phức $X[k]$. **IFFT** là phép biến đổi ngược đưa phổ phức trở về miền thời gian.

Mặt nạ phổ (Spectral mask) là một hàm trọng số $G[k]$ được nhân trực tiếp vào từng bin phổ $X[k]$ để triệt tiêu có chủ đích một dải tần số xác định. **Zero-padding** là kỹ thuật chèn thêm các số 0 vào chuỗi dữ liệu. Khi chèn các số 0 vào giữa nửa phổ dương và nửa phổ âm trước khi thực hiện IFFT dài gấp đôi ($2N$), chuỗi thời gian đầu ra sẽ có độ dài tăng gấp hai lần. Đây thực chất là phép nội suy sin(c) lý tưởng: các mẫu mới được tạo ra một cách trơn tru giữa các mẫu cũ mà không tạo ra bất kỳ tần số hài giả nào ngoài dải. Bước 1 của Praat sử dụng tổ hợp FFT cả câu nói $\rightarrow$ nhân mặt nạ hạ dốc từ $0.95 f_N$ về $0 \rightarrow$ zero-padding để upsample $\times 2$ mềm, dọn sạch mép Nyquist.

★ BẮC CẦU FEYNMAN: 5. Hàm Tự Tương Quan (Autocorrelation), Độ Trễ (Lag τ) và Định Lý Wiener–Khinchin

Hàm tự tương quan không vòng của một khung âm thanh $x[n]$ độ dài N mẫu được định nghĩa là:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{N-1-\tau} x[n] \cdot x[n+\tau], \qquad \tau \geq 0.$$

Đại lượng τ được gọi là **độ trễ (lag)**, tính bằng số mẫu. Hàm tự tương quan đo lường độ giống nhau giữa tín hiệu và bản sao dịch chuyển thời gian của chính nó. Khi dây thanh quản rung tuần hoàn với chu kỳ cơ bản T_0 giây ($\tau_0 = f_s \cdot T_0$ mẫu), sóng âm tại thời điểm n sẽ lặp lại tại $n + \tau_0$. Khi trượt đúng khoảng trễ $\tau = \tau_0$, tích các cặp mẫu cùng pha sẽ đạt giá trị cực đại, tạo nên một đỉnh nổi bật trên đồ thị $\mathcal{R}_x[\tau]$. Đo được vị trí đỉnh τ_0, ta suy ra ngay cao độ: $F_0 = f_s/\tau_0$.

Định lý Wiener–Khinchin là một định lý toán học nền tảng chứng minh rằng: hàm tự tương quan của một tín hiệu chính là biến đổi Fourier ngược của mật độ phổ công suất của nó: $\mathcal{R}_x[\tau] = \mathcal{F}^{-1}\{|X[k]|^2\}$. Định lý này mở ra hai con đường thực thi phần cứng hoàn toàn tương đương về mặt toán học: tính tự tương quan bằng cặp biến đổi FFT $\rightarrow |X|^2 \rightarrow$ IFFT, hoặc tính trực tiếp bằng khối nhân dồn thời gian (MAC).

★ BẮC CẦU FEYNMAN: 6. Cánh Bướm (Butterfly), Hệ Số Xoay (Twiddle Factor) và Định Dạng Khối Chấm Động (BFP)

Trong thuật toán FFT cơ số 2 (Radix-2 Cooley-Tukey), phép tính cơ sở nhỏ nhất lấy hai số phức đầu vào a và b, nhân b với một hệ số pha $W_N^k = e^{-j2\pi k/N}$ (được gọi là **hệ số xoay — twiddle factor**), rồi tạo ra hai kết quả đầu ra: $a + W_N^k b$ và $a - W_N^k b$. Đồ hình đường tín hiệu bắt chéo này trông giống hình đôi cánh bướm nên được gọi là phép tính **cánh bướm (butterfly)**.

Khi tính toán FFT trên phần cứng số cố định bit (ví dụ 16 bit), các phép cộng cánh bướm có thể làm biên độ tăng gấp đôi ở mỗi tầng, gây tràn số. Để khắc phục mà không cần chuyển sang bộ số chấm động (Floating-point) đắt đỏ, Intel FFT IP áp dụng cơ chế **Block Floating Point (BFP — Số chấm động khối)**: toàn bộ các phần tử trong một vector chia sẻ chung một số mũ nguyên e. Mỗi khi một tầng cánh bướm có nguy cơ tràn, toàn bộ vector được dịch phải 1 bit và số mũ e tăng thêm 1. Khi bình phương phổ $|X[k]|^2 = \text{Re}^2 + \text{Im}^2$, độ lớn phổ bị nâng lũy thừa bậc hai, khiến số mũ thực tế của năng lượng bị nhân đôi thành $2e$. Nếu kỹ sư quên không dịch bit bù trừ hệ số 2^{2e} trước khi đưa vào khối IFFT, tỷ lệ biên độ của toàn bộ hàm tự tương quan sẽ bị sai lệch hoàn toàn, làm sụp đổ các ngưỡng so sánh voicing.

★ BẮC CẦU FEYNMAN: 7. Số Dấu Phẩy Tĩnh (Fixed-Point Q$m.n$), Bão Hòa (Saturation) và Khớp Từng Bit (Bit-Exact Twin)

Số **Q**$m.n$ là quy ước biểu diễn số thực có dấu bằng số nguyên hai's complement: dùng m bit cho phần nguyên (bao gồm cả bit dấu) và n bit cho phần thập phân, với trọng số nhỏ nhất (LSB) bằng 2^{-n}. Ví dụ: định dạng **Q1.15** dùng 16 bit, có dải biểu diễn từ -1.0 đến $+1.0 - 2^{-15}$ (tức từ -32768 đến $+32767$), trong đó giá trị số thực x được ánh xạ thành số nguyên $X_{\text{int}} = \text{round}(x \cdot 2^{15})$.

Hiện tượng bão hòa (Saturation): Khi kết quả của một phép tính toán vượt ra ngoài dải biểu diễn của thanh ghi, phép bão hòa sẽ ghim giá trị về ngưỡng cực đại ($+32767$) hoặc cực tiểu (-32768). Nếu để xảy ra hiện tượng tràn vòng tự nhiên (overflow/wrap-around), một số dương cực lớn sẽ đột ngột biến thành một số âm cực lớn, tạo ra tiếng nổ lách cách trong âm thanh và bẻ gãy hoàn toàn hàm tự tương quan. **Mô hình song sinh khớp bit (Bit-exact twin)** là chương trình viết bằng Python/C++ mô phỏng lại chính xác từng phép cộng, trừ, nhân, dịch bit và bão hòa của mạch RTL. Đầu ra của mô hình twin và kết quả mô phỏng ModelSim trên cùng một tệp dữ liệu âm thanh kiểm tra bắt buộc phải giống nhau đến từng bit đơn lẻ (100% bit-match).

★ BẮC CẦU FEYNMAN: 8. Vòng Khóa Pha (PLL), Xung Chủ (MCLK), Giao Thức I²S và Hàng Đợi Bất Đồng Bộ (DCFIFO Mã Gray)

PLL (*Phase-Locked Loop* — Vòng khóa pha) là mạch điện tử analog-số tích hợp bên trong chip FPGA, sử dụng bộ dao động điều khiển bằng điện áp (VCO) để khóa đồng pha với dao động thạch anh ngoại vi $50\,\text{MHz}$, từ đó nhân và chia tần số để tạo ra các xung nhịp đồng hồ cực kỳ ổn định. Chip giải mã âm thanh codec WM8731 yêu cầu một xung nhịp chủ riêng biệt **MCLK** $= 12.288\,\text{MHz}$ (đúng bằng $256 \times 48\,\text{kHz}$) để vận hành bộ lọc oversampling và bộ chuyển đổi delta-sigma bên trong nó.
I²S (*Inter-IC Sound*) là chuẩn giao tiếp nối tiếp 3 dây chuyên dụng truyền âm thanh số: dây chọn kênh `WS` (LRCK, tần số $48\,\text{kHz}$, chọn tai trái/phải), dây xung đồng hồ bit `SCLK` (BCLK, tần số $3.072\,\text{MHz} = 64 \times 48\,\text{kHz}$) và dây dữ liệu nối tiếp `SDAT`. Do miền đồng hồ âm thanh ($12.288\,\text{MHz}$) và miền đồng hồ xử lý tín hiệu lõi ($50\,\text{MHz}$) dao động độc lập không đồng bộ, việc truyền mẫu âm thanh qua lại bắt buộc phải đi qua một bộ đệm **DCFIFO** (*Dual-Clock FIFO*). Các con trỏ ghi/đọc của DCFIFO được mã hóa theo chuẩn **Mã Gray** (mỗi bước tăng chỉ có duy nhất 1 bit thay đổi trạng thái) để triệt tiêu hoàn toàn rủi ro tranh chấp trạng thái giả (*metastability*). Sai lệch tần số MCLK dù chỉ vài chục phần triệu (ppm) cũng sẽ làm trôi chu kỳ lấy mẫu theo thời gian, dẫn đến việc đo lệch cao độ F_0.

★ BẮC CẦU FEYNMAN: 9. Bộ Lọc FIR, Taps, Cấu Trúc Đa Pha (Polyphase Decimation) và Dải Chuyển Tiếp

Bộ lọc FIR (*Finite Impulse Response*) tính toán mẫu đầu ra bằng tổng chập có trọng số của các mẫu đầu vào quá khứ: $y[n] = \sum_{k=0}^{M-1} h[k]x[n-k]$. Số lượng hệ số M được gọi là số **vòi lọc (taps)**. **Hạ mẫu (Decimation)** là thao tác giảm tần số lấy mẫu bằng cách loại bỏ bớt các mẫu thừa (ví dụ từ $24\,\text{kHz}$ xuống $16\,\text{kHz}$, tỷ số $2/3$: cứ nhận 3 mẫu thì tính toán xuất ra 2 mẫu). Trước khi hạ mẫu, bắt buộc phải có bộ lọc thông thấp chặn đứng mọi tần số trên $8\,\text{kHz}$ để ngăn ngừa méo aliasing.
Cấu trúc đa pha (Polyphase) phân rã một bộ lọc FIR dài thành các nhánh lọc con ngắn hơn hoạt động song song ở tần số thấp hơn, chỉ kích hoạt nhân hệ số khi cần xuất mẫu, giúp tiết kiệm đến 66% số phép nhân DSP. **Dải chuyển tiếp (Transition band)** là khoảng tần số nằm giữa dải thông (passband, cho qua nguyên vẹn) và dải chắn (stopband, dập tắt hoàn toàn). Dải chuyển tiếp càng dốc và hẹp thì đòi hỏi số vòi lọc M càng lớn, làm tăng số khối nhân DSP và tài nguyên logic trên FPGA.

★ BẮC CẦU FEYNMAN: 10. Khối Nhớ Nhúng M10K, True Dual-Port, Chế Độ Đọc Khi Ghi (Old Data) và Bộ Nhân DSP

M10K là khối bộ nhớ RAM tĩnh chuyên dụng tích hợp sẵn trên silicon Cyclone V, mỗi khối chứa chính xác $10,240$ bit dữ liệu (khoảng 10 Kbit). Chế độ **True Dual-Port** (hai cổng thực) cho phép hai mạch logic độc lập thực hiện đọc hoặc ghi đồng thời vào hai địa chỉ bất kỳ trong cùng một chu kỳ xung nhịp. Khi một cổng thực hiện ghi dữ liệu mới vào địa chỉ A trong khi cổng kia đọc đúng địa chỉ A tại cùng một cạnh xung nhịp (*read-during-write collision*), phần cứng silicon của Cyclone V **không hỗ trợ chế độ xuất dữ liệu mới (New Data)**, mà chỉ bảo đảm chế độ xuất dữ liệu cũ (**Old Data**) hoặc không xác định (Don't Care). Mọi thuật toán hàng đợi trượt cửa sổ F_0 đều phải được thiết kế để dựa trên dữ liệu Old Data hoặc lệch pha ghi/đọc.
Khối **DSP block** trên Cyclone V là mạch nhân số học silicon cứng tốc độ cao, có khả năng thực hiện 2 phép nhân độc lập 18×19 bit có dấu (hoặc 1 phép nhân 27×27 bit) trong đúng 1 chu kỳ xung nhịp. **ALM** (*Adaptive Logic Module*) là đơn vị logic cơ bản của Cyclone V, bao gồm bảng tra cứu LUT 8 ngõ vào, hai bộ cộng đầy đủ và hai flip-flop, được phần mềm tổng hợp dùng để đếm tài nguyên logic. **Fmax** là tần số xung nhịp làm việc tối đa mà mạch có thể chạy ổn định sau khi đã bố trí và định tuyến dây (Place and Route) mà không bị vi phạm thời gian thiết lập tín hiệu (Setup time violation).

★ BẮC CẦU FEYNMAN: 11. Đỉnh Cực Đại Cục Bộ, Nội Suy Parabol, Nội Suy Sinc, Sai Quãng Tám và Bộ Lọc Median-3

Giá trị cực đại của hàm tự tương quan hiếm khi nằm chính xác tại một số nguyên mẫu trễ τ, mà thường rơi vào khoảng giữa hai mẫu rời rạc. **Nội suy Parabol (Parabolic interpolation)** sử dụng 3 điểm quanh đỉnh để khớp một đường cong bậc hai, tìm ra tọa độ đỉnh thực với độ chính xác dưới mẫu (sub-sample) chỉ bằng 2 phép trừ và 1 phép chia. **Nội suy Sinc** sử dụng khai triển giải tích hàm sinc trên cửa sổ 70 hoặc 700 mẫu (SINC70/SINC700) để tiệm cận lời giải chính xác tuyệt đối, nhưng tốn hàng trăm phép nhân trên mỗi đỉnh. FPGA chọn nội suy Parabol vì nó cân bằng hoàn hảo giữa độ chính xác (sai số < 2 cent) và tài nguyên silicon.
Sai quãng tám (Octave error): Sóng âm tuần hoàn chu kỳ T_0 cũng tuần hoàn chu kỳ $2T_0$, khiến hàm tự tương quan luôn xuất hiện đỉnh phụ tại $\tau = 2T_0$. Nếu biên độ đỉnh phụ bị nhiễu đẩy cao hơn đỉnh chính, thuật toán sẽ nhận nhầm tần số bị tụt xuống một nửa ($F_0/2$). Praat áp dụng **Octave cost** để phạt nhẹ các trễ lớn nhằm triệt tiêu thiên vị này. **Voicing** là phán quyết khung âm thanh là Hữu thanh (dây thanh rung, có F_0) hay Vô thanh / Im lặng ($F_0 = 0$). Để làm sạch đường cong cao độ khỏi các xung nhiễu đơn lẻ mà không phải chạy thuật toán quy hoạch động Viterbi phức tạp, phần cứng sử dụng **Bộ lọc trung vị 3 khung (Median-3 filter)**: lấy giá trị nằm giữa của 3 khung liên tiếp, tạo độ trễ xử lý chỉ đúng $10\,\text{ms}$.

★ BẮC CẦU FEYNMAN: 12. Giao Thức Luồng Avalon-ST (`valid`/`ready`), Điểm Cực và Điểm Không

Avalon-ST (*Avalon Streaming Interface*) là chuẩn giao tiếp truyền dữ liệu điểm-nối-điểm một chiều của Intel dành cho các hệ thống DSP băng thông cao. Giao thức vận hành dựa trên cặp tín hiệu bắt tay cơ bản: bên phát nâng tín hiệu `valid` $= 1$ khi dữ liệu trên bus hợp lệ; bên thu nâng tín hiệu `ready` $= 1$ khi bộ đệm bên trong còn chỗ trống tiếp nhận. Giao dịch chuyển dữ liệu chỉ xảy ra tại cạnh xung nhịp mà cả hai tín hiệu `valid` và `ready` đồng thời bằng 1. Nếu bên thu hạ `ready` $= 0$ (áp lực ngược — *backpressure*), bên phát bắt buộc phải giữ nguyên dữ liệu trên bus; việc lơ là tín hiệu `ready` sẽ làm rơi rớt mẫu âm thanh và phá vỡ cấu trúc khung F_0.
Trong lý thuyết xử lý tín hiệu số, hàm truyền của một bộ lọc rời rạc được biểu diễn dưới dạng phân thức $H(z) = B(z)/A(z)$. Nghiệm của tử số $B(z) = 0$ được gọi là các **điểm không (zeros)**, có tác dụng triệt tiêu hoàn toàn năng lượng tại các tần số tương ứng. Nghiệm của mẫu số $A(z) = 0$ được gọi là các **điểm cực (poles)**, quyết định mức khuếch đại cộng hưởng và tính ổn định của hệ thống. Bộ chặn DC ở Chương 3 đặt một điểm không đúng tại $z = 1$ để triệt tiêu hoàn toàn thành phần một chiều ($0\,\text{Hz}$) và đặt một điểm cực rất gần đó ($z = 0.995$) để duy trì đáp ứng phẳng cho toàn bộ dải âm tần nghe thấy.

★ BẮC CẦU FEYNMAN: 13. Hệ Số Thời Gian Thực (Real-Time Factor – RTF)

Hệ số thời gian thực (RTF) được định nghĩa là tỷ số giữa thời gian phần cứng tiêu tốn để tính toán xong một khung dữ liệu chia cho thời lượng thực tế của khung dữ liệu đó:

$$\text{RTF} = \frac{T_{\text{compute}}}{T_{\text{audio}}}.$$

Một hệ thống được xem là đáp ứng thời gian thực khi và chỉ khi $\text{RTF} \leq 1.0$. Nếu $\text{RTF} > 1.0$, hệ thống tính toán chậm hơn tốc độ âm thanh đi vào, khiến bộ đệm bị tràn và mất dữ liệu. Trong thiết kế này, với bước nhảy khung $H = 160$ mẫu ($10\,\text{ms}$), tại tần số xung nhịp hệ thống $50\,\text{MHz}$, phần cứng mất $120,960$ chu kỳ ($\approx 2.42\,\text{ms}$) để tính xong toàn bộ 189 giá trị tương quan và trích xuất đỉnh cao độ. Hệ số thời gian thực đạt được là:

$$\text{RTF} = \frac{2.42\,\text{ms}}{10\,\text{ms}} = 0.242 \ll 1.0.$$

Hệ số này chứng minh phần cứng chạy nhanh gấp hơn 4.1 lần so với tốc độ thời gian thực, hoàn toàn bảo đảm độ trễ cực thấp cho các ứng dụng tương tác giọng nói trực tiếp.

0.5. Ngân sách học tập và Bản đồ thời gian biểu

CHI PHÍ (LUT / DSP / chu kỳ)

Kế hoạch phân bổ thời gian học tập và thực hành (Tổng thời lượng: ≈ 24 giờ làm việc):

- **Chương 0 & Chương 1 (Nền tảng Praat và Toán giải tích):** Dành từ 4 đến 6 giờ đọc kỹ từng chứng minh toán học, tự tay chạy lại tập lệnh Python `verify/verify_chuong_00_01.py` để kiểm tra từng con số tính tay. *Nguyên lý cốt lõi:* Sai một công thức toán ở Chương 1 thì không một khối DSP hay bộ nhớ M10K nào ở các chương sau có thể cứu vãn được hệ thống.
- **Chương 2 đến Chương 6 (Tiền xử lý âm thanh và Chuẩn bị khung):** Dành từ 5 đến 6 giờ. Nắm vững cấu hình PLL, giao thức I^2S, bộ lọc chặn DC, bộ lọc hạ mẫu FIR $2/3$ và cơ chế đệm khung trượt trên M10K.
- **Chương 7 đến Chương 10 (Trái tim tính toán cao độ):** Dành từ 6 đến 8 giờ. Đây là phần trọng tâm phần cứng gồm bộ nhân dồn MAC tự tương quan, bảng ROM bù dốc cửa sổ, bộ chia nội suy parabol và mạch so sánh voicing.
- **Chương 11 đến Chương 14 (Tích hợp, Đóng gói và Kiểm chứng 3 tầng):** Dành từ 4 đến 5 giờ. Kết nối toàn hệ thống qua FSM Avalon-ST, đóng gói dữ liệu đầu ra Q12.4, chạy mô phỏng ModelSim và đối chiếu sai số với ngưỡng 20 cent của Praat.

0.6. Bài tập tự đánh giá và Lời giải chi tiết

Nhằm giúp người học tự củng cố tư duy kiến trúc và các nguyên lý nền tảng trước khi bước vào chi tiết giải thuật Praat và phần cứng, dưới đây là hệ thống 4 bài tập phân tích sâu kèm lời giải tường minh:

Bài tập 0.1 (Hiện tượng Aliasing và Méo Cao Độ): Một micro thu âm ghi nhận tín hiệu tiếng nói bị lẫn một xung nhiễu điện cơ học tần số cao $f_{\text{noise}} = 9\,\text{kHz}$. Hệ thống xử lý hoạt động tại tần số lấy mẫu $f_s = 16\,\text{kHz}$.

a) Xác định tần số Nyquist của hệ thống và tính tần số phản xạ giả mạo (f_{alias}) của tín hiệu nhiễu sau quá trình lấy mẫu.

b) Tần số giả mạo này có rơi vào vùng tìm kiếm cao độ con người ($75\,\text{Hz} \ldots 600\,\text{Hz}$) hay không? Nếu tín hiệu này tiếp tục đi qua khối hạ mẫu Decimation $2/3$ hoặc bị rò phổ bởi cửa sổ phân khung, hậu quả gì sẽ xảy ra đối với đỉnh tự tương quan?

Lời giải giải thích:

a) Tần số Nyquist của hệ thống là $f_N = f_s/2 = 16\,\text{kHz}/2 = 8\,\text{kHz}$. Do tần số nhiễu $f_{\text{noise}} = 9\,\text{kHz} > f_N$, hiện tượng aliasing chắc chắn xảy ra. Tần số giả mạo xuất hiện trong miền gốc được tính theo công thức phản xạ gương qua ranh giới Nyquist:

$$f_{\text{alias}} = |f_s - f_{\text{noise}}| = |16\,\text{kHz} - 9\,\text{kHz}| = 7\,\text{kHz}.$$

b) Tại tần số $7\,\text{kHz}$, sóng phản xạ chưa rơi trực tiếp vào dải cao độ cơ bản $75 \ldots 600\,\text{Hz}$. Tuy nhiên, trong miền thời gian, một thành phần $7\,\text{kHz}$ có biên độ lớn khi bị nhân với cửa sổ phân khung Hanning dài $40\,\text{ms}$ sẽ sinh ra các thùy phụ lan rộng. Nguy hiểm hơn, nếu hệ thống tiếp tục áp dụng các bước hạ mẫu số mà bộ lọc chống chồng phổ (anti-aliasing) không đạt độ suy giảm tối thiểu $60\,\text{dB}$, thành phần $7\,\text{kHz}$ này sẽ tiếp tục gập qua các ranh giới Nyquist kế tiếp và đổ thẳng vào dải vài trăm Hertz, tạo ra các đỉnh tương quan giả mạo cạnh tranh trực tiếp với tần số dây thanh quản thật. Do đó, việc dọn sạch phổ trước khi phân khung là yêu cầu sinh tử.

Bài tập 0.2 (Định mức Thời gian thực và Hệ số RTF): Hệ thống đo cao độ F_0 hoạt động với xung nhịp đồng hồ hệ thống $f_{\text{clk}} = 50\,\text{MHz}$. Bước nhảy khung thời gian là $H = 160$ mẫu tại $f_s = 16\,\text{kHz}$ ($10\,\text{ms}$). Để quét dải cao độ từ $75\,\text{Hz}$ đến $600\,\text{Hz}$, khối MAC tự tương quan phải tính toán 189 giá trị trễ ($\tau = 26 \ldots 214$), mỗi giá trị trễ tích lũy $L = 640$ tích mẫu.

a) Tính số chu kỳ xung nhịp tối đa mà hệ thống có sẵn trong một bước nhảy khung $10\,\text{ms}$.

b) Giả sử kiến trúc sử dụng 1 khối DSP nhân dồn tuần tự (1 phép tính nhân-cộng trên mỗi chu kỳ đồng hồ). Tính tổng thời gian cần thiết để xử lý xong một khung và suy ra hệ số thời gian thực RTF. Đưa ra nhận xét về biên độ an toàn (margin) của thiết kế.

Lời giải giải thích:

a) Số chu kỳ xung nhịp hệ thống có sẵn trong khoảng thời gian một bước nhảy khung $T_{\text{audio}} = 10\,\text{ms} = 0.01\,\text{s}$ là:

$$N_{\text{cycles}} = f_{\text{clk}} \times T_{\text{audio}} = 50 \times 10^6\,\text{Hz} \times 0.01\,\text{s} = 500,000\,\text{chu kỳ}.$$

b) Tổng số phép nhân dồn MAC cần thực hiện cho 189 giá trị trễ là:

$$N_{\text{MAC}} = 189 \times 640 = 120,960\,\text{phép tính}.$$

Với 1 khối DSP thực thi 1 phép MAC trên 1 chu kỳ, thời gian tính toán tiêu tốn là $T_{\text{compute}} = 120,960/(50\times 10^6\,\text{Hz}) = 2.4192\,\text{ms}$. Hệ số thời gian thực là:

$$\text{RTF} = \frac{T_{\text{compute}}}{T_{\text{audio}}} = \frac{2.4192\,\text{ms}}{10.0\,\text{ms}} \approx 0.2419.$$

Vì $\text{RTF} = 0.2419 \ll 1.0$, phần cứng chỉ dùng chưa đến 25% thời gian có sẵn của mỗi khung. Hệ thống có biên an toàn dồi dào ($> 75\%$ thời gian rảnh rỗi), cho phép xung nhịp có thể hạ xuống $25\,\text{MHz}$ để tiết kiệm năng lượng hoặc dùng chung khối DSP cho các thuật toán khác mà không bao giờ lo tràn bộ đệm.

Bài tập 0.3 (Đặc tính Bộ nhớ M10K và Xung đột Đọc-khi-Ghi): Một kỹ sư đề xuất thiết kế bộ đệm trượt $L = 640$ mẫu âm thanh bằng khối RAM On-Chip Memory 2-PORT trên Cyclone V, trong đó Cổng A liên tục ghi mẫu mới từ ADC vào địa chỉ k trong khi Cổng B đọc mẫu cũ tại chính địa chỉ k đó để chuyển cho khối tự tương quan tính toán. Kỹ sư này cấu hình chế độ ngõ ra là "New Data" để hy vọng đọc được mẫu vừa ghi tức thì.

a) Kiến trúc silicon thật của khối M10K trên Intel Cyclone V có hỗ trợ chế độ "New Data" cho hai cổng độc lập (mixed-port read-during-write) hay không?

b) Điều gì sẽ xảy ra nếu phần mềm tổng hợp Quartus gặp cấu hình này? Giải pháp thiết kế đúng đắn để tránh lỗi dữ liệu rác là gì?

Lời giải giải thích:

a) Không. Cấu tạo vật lý của các khối nhớ nhúng M10K trên dòng vi mạch Intel Cyclone V **hoàn toàn không hỗ trợ** chế độ "New Data" khi đọc và ghi đồng thời vào cùng một địa chỉ từ hai cổng khác nhau. Trong tài liệu chính thức *Cyclone V Device Handbook*, Intel quy định rõ: đối với chế độ True Dual-Port hoặc Simple Dual-Port trên các xung nhịp khác nhau hoặc cùng một xung nhịp nhưng khác cổng, ngõ ra dữ liệu khi xảy ra xung đột địa chỉ chỉ có thể là **"Old Data"** (xuất dữ liệu trước khi ghi) hoặc **"Don't Care"** (dữ liệu không xác định).

b) Nếu cố tình ép chế độ "New Data", Quartus Prime bắt buộc phải chèn thêm hàng chục khối logic ALM ngoại vi để tạo mạch ghép kênh đường vòng (bypass multiplexer logic), làm giảm đáng kể tần số hoạt động cực đại F_{max} và tăng diện tích chip. Giải pháp thiết kế chuẩn mực là: phân định rõ thời gian ghi mẫu mới và đọc khung cũ theo cơ chế lệch pha (ping-pong buffer hoặc quản lý con trỏ modulo vòng tròn với khoảng cách đọc/ghi cách xa nhau ít nhất $H = 160$ địa chỉ), bảo đảm Cổng A và Cổng B không bao giờ truy cập trùng một địa chỉ RAM trong cùng một chu kỳ xung nhịp.

Bài tập 0.4 (Rò Phổ và Đơn Vị Decibel): Biên độ thùy phụ đầu tiên của cửa sổ chữ nhật là $-13\,\text{dB}$ so với thùy chính, trong khi cửa sổ Hanning đạt mức suy giảm $-32\,\text{dB}$.

a) Đổi các mức suy giảm $-13\,\text{dB}$ và $-32\,\text{dB}$ sang tỷ số biên độ tuyến tính tương đối (A/A_0).

b) Giải thích tại sao việc suy giảm năng lượng rò rỉ từ 22.4% xuống 2.5% lại có ý nghĩa sống còn trong việc ngăn chặn hiện tượng nhảy quãng tám (octave error) của hàm tự tương quan.

Lời giải giải thích:

a) Theo định nghĩa đơn vị decibel cho biên độ tín hiệu áp suất âm thanh hoặc điện áp: $L_{\text{dB}} = 20\log_{10}(A/A_0) \Rightarrow A/A_0 = 10^{L_{\text{dB}}/20}$.

- Với cửa sổ chữ nhật: $A/A_0 = 10^{-13/20} = 10^{-0.65} \approx 0.22387$ (tương đương 22.39%, tức biên độ chỉ giảm khoảng 4.47 lần).
- Với cửa sổ Hanning: $A/A_0 = 10^{-32/20} = 10^{-1.60} \approx 0.02512$ (tương đương 2.51%, tức biên độ giảm tới 39.81 lần).

b) Năng lượng sóng âm tỷ lệ với bình phương biên độ. Với cửa sổ chữ nhật, thùy phụ còn giữ tới $0.22387^2 \approx 5.01\%$ năng lượng; khi giọng nói phát ra một nguyên âm có các tần số cộng hưởng formant bậc cao (ví dụ F_1, F_2 ở dải $800 \ldots 2500$ Hz), năng lượng rò rỉ qua các thùy phụ này sẽ tạo nên các gợn sóng nhấp nhô có chu kỳ ngắn đè lên hàm tự tương quan. Các đỉnh phụ này có thể vượt qua đỉnh chu kỳ thật của dây thanh quản, dẫn đến hiện tượng thuật toán bắt nhầm đỉnh phụ và báo sai cao độ lên gấp đôi hoặc xuống một nửa. Cửa sổ Hanning ép năng lượng rò rỉ xuống chỉ còn $0.02512^2 \approx 0.063\%$ (giảm gần 80 lần năng lượng rò so với chữ nhật), giữ cho đáy hàm tương quan giữa các chu kỳ luôn bằng phẳng và sạch sẽ.

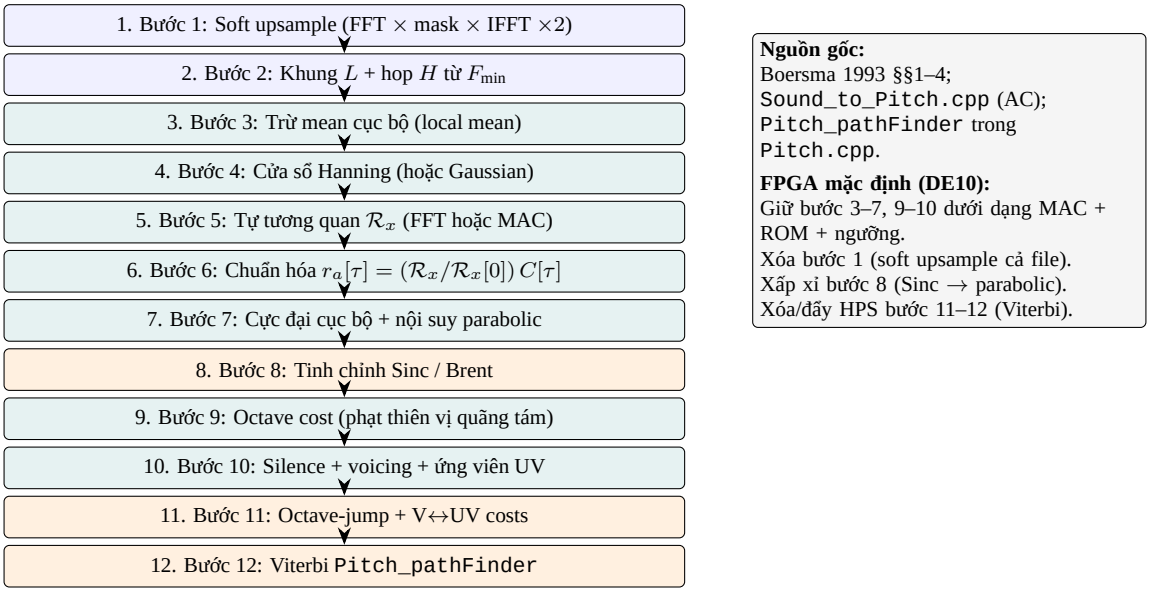
Chương 1. Nguyên lý Praat từ vật lý đến toán (Boersma 1993)

▶ MỤC TIÊU HỌC TẬP

Chương này lần theo đúng mười hai bước của thuật toán đo cao độ F_0 trong phần mềm Praat (Boersma 1993; mã nguồn C++ `Sound_to_Pitch.cpp` và `Pitch.cpp`, nhánh autocorrelation). Với mỗi bước, bạn sẽ: hiểu hiện tượng vật lý mà bước đó bảo vệ; dẫn công thức từ định nghĩa gốc; tính tay một ví dụ số nhỏ đã kiểm bằng tập lệnh `verify/verify_chuong_00_01.py`; thấy ánh xạ silicon trên kit DE10-Standard (giữ, xấp xỉ hay xóa); và nắm mô hình Python khớp bit cùng ngân sách ALM/DSP/M10K/RTF. Các khái niệm đã giải thích ở Chương 0 (ADC, Nyquist, aliasing, framing, mainlobe/sidelobe, Hanning, tự tương quan, Wiener–Khinchin, butterfly, BFP, Q$m.n$, Avalon-ST, RTF,…) chỉ được *tham chiếu*, không định nghĩa lại.

1.0. Bản đồ mười hai bước: giấy Boersma ↔ C++ ↔ silicon

Hình 9 tóm tắt chuỗi đầy đủ Praat AC và ghi chú ngay trên bản đồ những bước mà phần cứng streaming trên Cyclone V sẽ giữ nguyên, xấp xỉ hoặc xóa có chủ đích. Kỹ sư phải hiểu bước đã xóa *trước khi* xóa — đó là nguyên tắc xuyên suốt Chương 1.



Hình 9: Chuỗi mười hai bước Praat autocorrelation. Ô xanh ngọc (`teal!10`): giữ gần bit-exact trên fabric (bước 3–7, 9–10). Ô cam (`orange!12`): xấp xỉ hoặc đẩy sang HPS (bước 8, 11–12). Ô xanh dương nhạt (`blue!6`): soft upsample / framing bị xóa hoặc thay trên streaming (bước 1–2; Bước 1 thay bằng FIR chống alias ở Chương 4).

1.1. Bước 1 — Soft upsampling: dọn mép Nyquist trước khi cắt khung

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: năng lượng sát Nyquist bị cửa sổ "quăng" xuống vùng F_0

Chương 0 đã giải thích lấy mẫu, tần số Nyquist $f_N = f_s/2$, aliasing và rò phổ (mainlobe/sidelobe) của cửa sổ Hanning. Hệ quả thực tế đối với đo cao độ là: nếu một phát ngôn (*utterance*) còn năng lượng gần f_N (tiếng rít quạt, nhiễu cơ học, sóng hài cao), thì khi nhân cửa sổ Hanning dài 40 ms, các thùy phụ khoảng -32 dB sẽ phát tán năng lượng đó xuống đúng dải tìm kiếm $75 \ldots 600$ Hz. Thuật toán sẽ thấy các đỉnh tự tương quan giả — không phải dây thanh tạo ra chúng.

Boersma (1993, Step 1) xử lý cả một utterance trên máy tính như sau: (1) tính FFT toàn chuỗi; (2) nhân phổ với mặt nạ mềm $G[k]$ giảm tuyến tính từ $0{,}95 \cdot f_N$ xuống 0 đúng tại f_N; (3) chèn zero giữa nửa phổ dương và nửa phổ âm rồi IFFT dài $2N$. Kết quả là upsample "mềm" $\times 2$ đồng thời đã dọn sạch mép Nyquist. Đây là thao tác tính toán trên cả câu nói nằm trong RAM, không phải hiện tượng vật lý mới của dây thanh.

Toán từng bước. Đặt $f_N = f_s/2$. Mặt nạ phổ được định nghĩa theo bin tần số k (với N điểm FFT, bin Nyquist là $k = N/2$):

$$G[k] = \begin{cases} 1, & \dfrac{|k|}{N} < 0{,}95 \cdot \dfrac{1}{2}, \\ \dfrac{f_N - |k|\, f_s/N}{f_N - 0{,}95\, f_N}, & 0{,}95\, f_N \le |k|\, f_s/N \le f_N, \\ 0, & |k|\, f_s/N > f_N. \end{cases}$$

Chuỗi sau xử lý là $x_{\text{up}} = \text{IFFT}_{2N}\{\text{pad}(G[k]\, X[k])\}$. Phép zero-pad trong miền tần số rồi IFFT dài gấp đôi chính là nội suy Fourier lý tưởng: lưới thời gian dày gấp hai, phổ hữu ích dưới Nyquist cũ được giữ nguyên (xem lại định nghĩa zero-padding ở Chương 0).

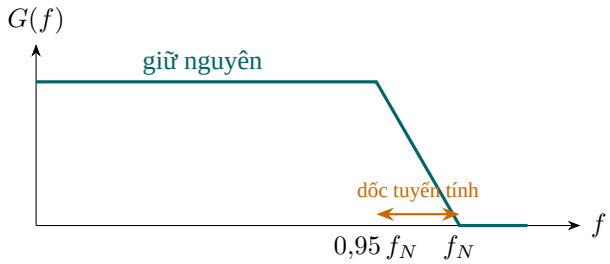
Ví dụ tính tay phổ 8 điểm. Lấy tín hiệu "cứng" đúng tại tần số Nyquist của lưới 8 điểm:

$$x = [1,\ 0,\ -1,\ 0,\ 1,\ 0,\ -1,\ 0].$$

DFT 8 điểm (không chuẩn hóa) cho năng lượng tập trung ở bin gần mép (bin $k = 2$ và đối xứng), còn bin Nyquist $k = 4$ bằng 0 trên tín hiệu này vì chu kỳ khớp lưới:

k	Re $X[k]$	Im $X[k]$	$\lvert X[k]\rvert^2$
0	0	0	0
1	0	0	0
2	4	0	16
3	0	0	0
4 (Nyquist)	0	0	0

Soft filter nhân mỗi bin với $G[k]$: giữ nguyên khi $k/N < 0{,}95/2$, rồi giảm tuyến tính về 0 tại bin Nyquist. Sau đó IFFT dài 16 mẫu — tương đương nội suy dày gấp đôi trên lưới thời gian. (Các số DFT trên đã kiểm bằng `numpy.fft.fft`; xem `verify/verify_chuong_00_01.py`.)



Hình 10: Mặt nạ phổ mềm Boersma Step 1: giữ nguyên dưới $0{,}95\ f_N$, dốc tuyến tính về 0 tại Nyquist, rồi zero-pad và IFFT dài $2N$ để upsample $\times 2$.

Ý nghĩa phần cứng. Trên FPGA streaming, mẫu âm thanh đến từng nhịp đồng hồ: không thể đợi hết cả câu nói rồi FFT toàn utterance. DE10 thay bước 1 bằng (1) lọc chống alias analog trước ADC của codec WM8731, và (2) FIR/decimation sạch từ 48 kHz xuống 16 kHz (Chương 4, FIR Compiler II). Cùng mục tiêu vật lý — đừng để năng lượng gần Nyquist giả thành chu kỳ thấp trong vùng F_0 — khác công cụ.

Twin Python bit-exact. Twin mặc định *bỏ qua* soft upsample và bắt đầu từ PCM đã anti-alias. Lab đối chiếu Praat đầy đủ có thể chạy offline: `np.fft.rfft` → nhân $G[k]$ (cùng dốc $0{,}95\ f_N \to f_N$) → `irfft(n=2N)`, rồi mới đưa vào đường pitch. Sai điểm cắt mask sẽ lệch năng lượng mép phổ so với Praat gốc.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 1 Soft upsample.

- Trên fabric streaming: **0 ALM / 0 DSP / 0 M10K** cho Step 1 (không thực hiện FFT cả utterance).
- Thay thế: anti-alias analog + FIR Compiler II (Chương 4) — vài chục khối DSP và vài M10K hệ số lọc.
- Twin đối chiếu Praat: soft-upsample offline trên Python/HPS trước khi đưa PCM vào đường fixed-point — không tính vào RTF thời gian thực.
- Quên dọn Nyquist: tone $\sim 7 \ldots 8$ kHz giả thành chu kỳ dài trong vùng $75 \ldots 600$ Hz, làm sai F_0 dù khối MAC phía sau hoàn toàn đúng.

Bài tập 1.1. Một utterance tại $f_s = 16$ kHz còn nhiễu 9 kHz. (a) Tần số alias sau ADC là bao nhiêu? (b) Soft upsample của Praat có ngăn được aliasing *đã xảy ra ở* ADC không, hay chỉ dọn năng lượng còn lại dưới Nyquist trước khi cửa sổ hóa? Giải thích.

Lời giải. (a) $f_{\text{alias}} = |16 - 9| = 7$ kHz (đã giải ở Chương 0). (b) Soft upsample *không* đảo ngược aliasing đã xảy ra trong ADC: thành phần 7 kHz đã nằm trong dải gốc. Nó chỉ cắt/làm mềm phần phổ còn lại gần f_N *trước khi* nhân cửa sổ, giảm rò phổ xuống vùng F_0. Trên FPGA, việc chống alias thật sự phải làm bằng lọc analog + FIR trước hạ mẫu (Chương 4).

1.2. Bước 2 — Khung, time step và độ dài cửa sổ từ pitch floor

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: cần thấy ít nhất ba chu kỳ dây thanh trong một khung

Để hàm tự tương quan "nhìn thấy" đỉnh tại $\tau = T_0$ và cả đỉnh phụ tại $\tau = 2T_0$, khung phải dài hơn vài chu kỳ dây thanh. Dây thanh nam có thể chậm tới khoảng 75 Hz, tức $T_0 = 1/75 \approx 13{,}33$ ms. Boersma chọn `periodsPerWindow` $= 3$ với cửa sổ Hanning: ba chu kỳ $= 40$ ms — đủ dài để hai đỉnh nằm trong cửa sổ, nhưng không dài đến mức F_0 đổi nhiều trong một khung (giả định *local-stationary*: cao độ gần như đứng yên trong cửa sổ). Tuỳ chọn Gaussian (`very_accurate`) dùng 6 chu kỳ vì đuôi Gaussian dài hơn trong miền thời gian; mã nguồn nhân đôi `periodsPerWindow` rồi tăng độ sâu nội suy Brent (`Sound_to_Pitch.cpp`, khoảng dòng 405–415).

Toán từng bước. Với $F_{\min}$ (pitch floor, mặc định 75 Hz), f_s (tần số lấy mẫu sau hạ mẫu, 16 kHz trên DE10) và số chu kỳ trong cửa sổ $P = 3$:

$$L_{\text{sec}} = \frac{P}{F_{\min}} = \frac{3}{75} = 0{,}04\,\text{s},$$
$$L = L_{\text{sec}} \cdot f_s = 0{,}04 \times 16000 = 640\,\text{mẫu}.$$

Bước nhảy thời gian mặc định `time_step` $= 0{,}01$ s cho hop

$$H = 0{,}01 \times 16000 = 160\,\text{mẫu}.$$

Tỉ lệ chồng lấn giữa hai khung liên tiếp:

$$\frac{L - H}{L} = \frac{640 - 160}{640} = 0{,}75 = 75\%.$$

Mỗi ước lượng F_0 mới tái sử dụng 480 mẫu đã thấy ở khung trước — đường cong mượt hơn, đổi lại gấp bốn lần số lần tính so với hop $= L$. Nếu `time_step` $= 0$, Praat tự chọn $0{,}75/F_{\min}$ để có khoảng bốn phép đo trên mỗi cửa sổ.

Khoảng trễ tìm kiếm cao độ:

$$\tau_{\min} = \lfloor f_s/F_{\max} \rfloor = \lfloor 16000/600 \rfloor = 26 \quad \text{(lab thường lấy 27 hoặc 32)},$$
$$\tau_{\max} = \lfloor f_s/F_{\min} \rfloor = \lfloor 16000/75 \rfloor = 213.$$

Ngoài đoạn $[\tau_{\min}, \tau_{\max}]$ không có ứng viên F_0 hợp lệ. Trên fabric DE10, vòng MAC quét $\tau = 26 \ldots 214$ (đúng 189 giá trị trễ) để khớp ngân sách đã tính ở Chương 0.

Ví dụ tính tay. $F_{\min} = 75$ Hz, $f_s = 16$ kHz, Hanning 3 chu kỳ: $L = 640$, $H = 160$, chồng lấn 75%, $\tau_{\min} = 26$, $\tau_{\max} = 213$. Đổi $F_{\min} \to 50$ Hz sẽ cho $L = 960$, $\tau_{\max} = 320$ — phải dựng lại toàn bộ ROM cửa sổ và bảng $C[\tau]$, không chỉ sửa một hằng số. (Đã kiểm: `verify_chuong_00_01.py`, hàm `test_hardware_budget`.)

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 2 Frame/hop.

- Giữ cố định $L = 640$, $H = 160$ trên FPGA: khớp đúng 1 khối M10K đệm khung ($640 \times 16 = 10240$ bit) cộng ROM $C[\tau]$ sâu khoảng 189 từ.
- Đổi $F_{\min} \Rightarrow$ đổi L, $\tau_{\max}$, và *bắt buộc rebuild* ROM w lẫn $C[\tau]$.
- RTF: mỗi 10 ms phải xong một khung; tại 50 MHz với 1 MAC/chu kỳ, $640 \times 189 = 120960$ phép nhân mất 2,42 ms (RTF = 0,242), dư sức — xem lại hộp RTF Chương 0.

Bài tập 1.2. Cho $F_{\min} = 100$ Hz, $f_s = 16$ kHz, Hanning 3 chu kỳ, `time_step` $= 0{,}01$ s. Tính L, H, tỉ lệ chồng lấn và $\tau_{\max}$. So sánh số phép MAC mỗi khung với cấu hình $F_{\min} = 75$ Hz (giả sử $\tau_{\min} = 26$ không đổi và τ quét đến $\tau_{\max}$).

Lời giải. $L = 3 \cdot 16000/100 = 480$ mẫu; $H = 160$; chồng lấn $(480 - 160)/480 = 66{,}7\%$; $\tau_{\max} = \lfloor 16000/100 \rfloor = 160$. Số lag $= 160 - 26 + 1 = 135$. MAC $= 480 \times 135 = 64800$ — khoảng một nửa so với 120960 của cấu hình 75 Hz. Khung ngắn hơn và dải τ hẹp hơn đều giảm chi phí, nhưng mất khả năng đo giọng rất trầm dưới 100 Hz.

1.3. Bước 3 — Trừ mean cục bộ (local mean)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: DC và trôi chậm phá đỉnh $\tau = 0$

Áp suất không khí trung bình, offset mạch mic và rung máy đều cộng một hằng số A vào mọi mẫu. Nhắc lại định nghĩa tự tương quan không vòng trên khung N mẫu (đã nêu ở Chương 0):

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{N-1-\tau} x[n]\, x[n+\tau].$$

Tại $\tau = 0$ chỉ còn tổng bình phương: $\mathcal{R}_x[0] = \sum_{n=0}^{N-1} x[n]^2$ — đúng bằng năng lượng khung. Nếu tín hiệu mang offset, đặt $y[n] = x[n] + A$:

$$\mathcal{R}_y[0] = \sum_{n=0}^{N-1} (x[n] + A)^2 = \sum x[n]^2 + 2A \sum x[n] + NA^2 = \mathcal{R}_x[0] + 2A \sum x[n] + NA^2.$$

Đỉnh tại $\tau = 0$ phình tỉ lệ với A^2 và dễ "nuốt" đỉnh chu kỳ khi so với ngưỡng voicing. Praat trừ mean cục bộ trên khoảng $\pm$ một chu kỳ dài nhất quanh tâm khung (`localMean /= 2*nsamp_period` trong mã nguồn).

Ví dụ tính tay 4 mẫu. Cho $x = [3, 5, 3, 5]$, mean $\mu = 4$, tín hiệu không mean $x_{\text{zm}} = [-1, 1, -1, 1]$. Với $N = 4$:

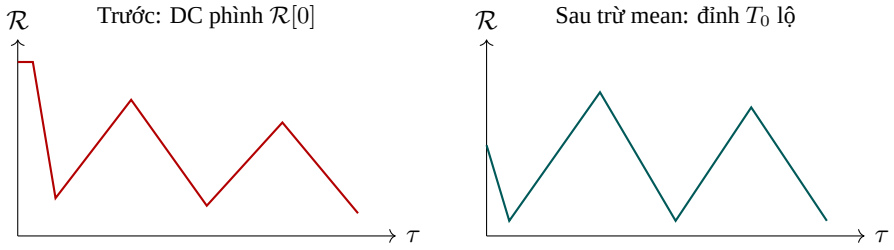
$$\mathcal{R}_{\text{zm}}[0] = (-1)^2 + 1^2 + (-1)^2 + 1^2 = 4,$$
$$\mathcal{R}_{\text{zm}}[2] = (-1) \cdot (-1) + 1 \cdot 1 = 2 \quad \text{(chỉ còn } N - \tau = 2 \text{ cặp)}.$$

Tỉ số thô $\mathcal{R}[2]/\mathcal{R}[0] = 2/4 = 0{,}5$ — *không* bằng 1, dù tín hiệu tuần hoàn chu kỳ 2. Muốn tỉ số về 1 phải chuẩn hóa theo số cặp (unbiased) $\frac{\mathcal{R}[\tau]/(N-\tau)}{\mathcal{R}[0]/N}$, hoặc theo đúng Praat chia thêm tự tương quan cửa sổ $\mathcal{R}_w$ (Bước 6). Với unbiased: $\frac{2/2}{4/4} = 1$.

Nếu quên trừ mean, trên chính $x = [3, 5, 3, 5]$:

$$\mathcal{R}_x[0] = 3^2 + 5^2 + 3^2 + 5^2 = 68,$$
$$\mathcal{R}_x[2] = 3 \cdot 3 + 5 \cdot 5 = 34.$$

Tỉ số thô $34/68 = 0{,}5$; sau unbiased vẫn ra 1 trên tín hiệu tuần hoàn này, nhưng đỉnh $\tau = 0$ đã phình gấp $68/4 = 17$ lần so với bản không mean — đúng bệnh DC làm méo thang đo trước ngưỡng voicing trên tín hiệu thật. (Đã kiểm Python.)



Hình 11: Cùng một tín hiệu có offset DC: trừ mean cục bộ trước cửa sổ là bắt buộc để đỉnh chu kỳ không bị nuốt bởi $\mathcal{R}[0]$.

Ý nghĩa phần cứng / Twin. FPGA tách hai lớp: DC-blocker IIR streaming (Chương 3) nuốt offset chậm; thêm mean khung (cộng dồn L mẫu rồi trừ) trước nhân cửa sổ nếu twin muốn khớp Praat chặt. Twin Python: `x = x - np.mean(x)` trên đúng cửa sổ $\pm$ một chu kỳ dài nhất quanh tâm, hoặc tương đương DC-blocker + mean khung.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 3 Local mean.

- DC-blocker IIR 1-cực 1-zero (Chương 3): vài ALM, 0 DSP, 1 thanh ghi 32-bit.
- Mean khung: 1 adder tích lũy + nhân $1/L$ định dạng Q — dưới 50 ALM.
- Không trừ mean: $\mathcal{R}[0]$ phình, ngưỡng voicing và r_a sai tỉ lệ — hỏng cả chuỗi sau dù MAC đúng.

Bài tập 1.3. Cho $x = [2, 4, 2, 4]$. Tính μ, x_{zm}, $\mathcal{R}_{\text{zm}}[0]$, $\mathcal{R}_{\text{zm}}[2]$, tỉ thô và tỉ unbiased. Tính thêm $\mathcal{R}_x[0]$ khi không trừ mean và hệ số phình $\mathcal{R}_x[0]/\mathcal{R}_{\text{zm}}[0]$.

Lời giải. $\mu = 3$, $x_{\text{zm}} = [-1, 1, -1, 1]$ — trùng ví dụ trên: $\mathcal{R}_{\text{zm}}[0] = 4$, $\mathcal{R}_{\text{zm}}[2] = 2$, thô $0{,}5$, unbiased 1. Không trừ: $\mathcal{R}_x[0] = 4 + 16 + 4 + 16 = 40$, $\mathcal{R}_x[2] = 4 + 16 = 20$, phình $40/4 = 10$ lần.

1.4. Bước 4 — Cửa sổ Hanning (và tuỳ chọn Gaussian)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: cắt khung đột ngột gây rò phổ; Hanning vuốt biên

Chương 0 đã định nghĩa framing, cửa sổ, mainlobe, sidelobe và đơn vị dB. Nhắc ngắn: framing *không* cắt đứt tín hiệu gốc rồi bỏ phần còn lại, mà nhân toàn bộ $x[n]$ với một cửa sổ trượt $w[n - kH]$ chỉ khác không trên đúng L mẫu của khung k. Cửa sổ chữ nhật nhảy xuống 0 ở hai đầu; phổ sinc của nó có thùy phụ đầu khoảng -13 dB (biên độ còn $\approx 1/4{,}5$). Cửa sổ Hanning vuốt hai đầu về gần 0 bằng nửa chu kỳ cosine, kéo thùy phụ xuống khoảng -32 dB (biên độ $\approx 1/40$), bảo vệ dải F_0 khỏi bóng ma tần số cao.

Toán từng bước — công thức Praat dùng $N + 1$. Praat (`Sound_to_Pitch.cpp`) định nghĩa trên N mẫu của khung:

$$w[i] = 0{,}5 - 0{,}5 \cos \frac{2\pi i}{N + 1}, \qquad i = 1, 2, \ldots, N.$$

Các ký hiệu: i là chỉ số mẫu *trong* khung (chạy $1..N$, không phải $0..N-1$); N là số mẫu khung (640 trên DE10); mẫu số $N + 1$ là lựa chọn có chủ đích của Praat để $w[1]$ và $w[N]$ vẫn > 0 nhỏ — mọi ô trong bộ nhớ đều đóng góp năng lượng, không "chết" hai mẫu biên (giáo trình cổ điển thường dùng $N - 1$ và cho $w[0] = w[N-1] = 0$).

Ví dụ tính tay $N = 4$ (đã kiểm Python).

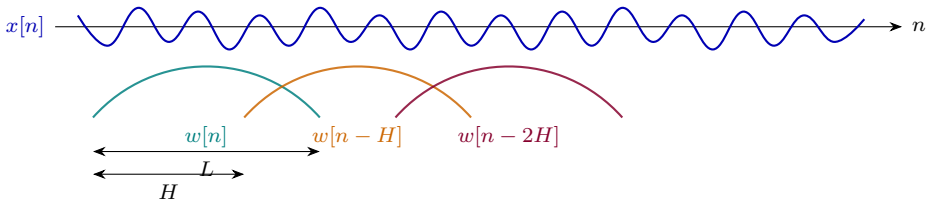
$$w[i] = 0{,}5 - 0{,}5 \cos \frac{2\pi i}{5}, \quad i = 1, 2, 3, 4.$$

$$
\begin{aligned}
i=1:\ & 0{,}5-0{,}5\cos(2\pi/5)=0{,}5-0{,}5(-0{,}309017)=0{,}345491,\\
i=2:\ & 0{,}5-0{,}5\cos(4\pi/5)=0{,}5-0{,}5(-0{,}809017)=0{,}904508,\\
i=3:\ & 0{,}904508, \qquad i=4:\ 0{,}345491.
\end{aligned}
$$

Vậy $w \approx [0{,}3455,\ 0{,}9045,\ 0{,}9045,\ 0{,}3455]$ — đối xứng. Từ đây

$$
\begin{aligned}
\mathcal{R}_w[0] &= \sum_{i=1}^{4} w_i^2 = 1{,}875,\\
\mathcal{R}_w[1] &= \sum_{i=1}^{3} w_i w_{i+1} \approx 1{,}443136,\\
C[1] &= \mathcal{R}_w[0]/\mathcal{R}_w[1] \approx 1{,}299254.
\end{aligned}
$$

Tích $w[n]w[n+\tau]$ suy giảm khi τ tăng — đúng gốc của bảng $C[\tau]$ ở Bước 6 và Chương 8.



Hình 12: Framing = nhân $x[n]$ với cửa sổ trượt $w[n-kH]$. Hop H dịch cửa sổ; chồng lấn 75% khi $L=640$, $H=160$.

Tuỳ chọn Gaussian. Gaussian $w[n] \propto \exp\!\left(-\tfrac{1}{2}(n/\sigma)^2\right)$ hẹp hơn trong tần số (sidelobe thấp hơn) nhưng đuôi dài hơn trong thời gian. Praat khi bật `very_accurate` nhân đôi `periodsPerWindow` ($3 \to 6$) và dùng nội suy SINC700. Trên FPGA điều đó nghĩa là khoảng $2\times$ M10K cửa sổ và $2\times$ MAC. Mặc định silicon: Hanning 3 chu kỳ.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 4 Window.

- ROM Hanning: 640×16-bit (Q1.15) $= 10240$ bit $= 1$ khối M10K; thực tế chỉ cần nửa bảng nhờ đối xứng $w[n] = w[L-1-n]$ (Chương 6).
- Nhân mẫu $\tilde{x}[n] = (x[n]-\mu)\,w[n]$: 1 khối DSP.
- Gaussian 6 chu kỳ: $L \approx 1280$, ROM+MAC roughly $\times 2$ — chỉ bật khi lab đối chiếu “very accurate”.

Bài tập 1.4. Tính $w[1]$ và $w[2]$ theo công thức Praat với $N=8$. Kiểm đối xứng $w[i] = w[N+1-i]$.

Lời giải. $w[i] = 0{,}5 - 0{,}5\cos(2\pi i/9)$. $w[1] = 0{,}5 - 0{,}5\cos(2\pi/9) \approx 0{,}5 - 0{,}5\cdot 0{,}7660 = 0{,}1170$; $w[2] = 0{,}5 - 0{,}5\cos(4\pi/9) \approx 0{,}5 - 0{,}5\cdot 0{,}1736 = 0{,}4132$. Đối xứng: $w[8]=w[1]$, $w[7]=w[2]$,…

1.5. Bước 5 — Tự tương quan: Wiener–Khinchin hoặc MAC trực tiếp

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: đỉnh tại $\tau = T_0$ đo chu kỳ dây thanh

Chương 0 đã định nghĩa hàm tự tương quan $\mathcal{R}_x[\tau]$, độ trễ (lag) và Định lý Wiener–Khinchin: $\mathcal{R}_x = \mathcal{F}^{-1}\{|X|^2\}$. Khi dây thanh rung tuần hoàn với chu kỳ T_0 ($\tau_0 = f_s \cdot T_0$ mẫu), trượt đúng $\tau = \tau_0$ làm các cặp mẫu cùng pha nhân với nhau đạt cực đại. Đo được τ_0 thì $F_0 = f_s/\tau_0$. Định lý Wiener–Khinchin mở hai đường thực thi tương đương toán học: FFT $\to |X|^2 \to$ IFFT, hoặc MAC thời gian trực tiếp.

Toán từng bước. Praat AC path pad zero đến

$$
\texttt{nsampFFT} = 2^{\lceil \log_2(N\cdot(1+\texttt{depth}))\rceil}
$$

với Hanning depth 0,5: $640 \times 1{,}5 = 960 \to 1024$ điểm FFT. Sau IFFT lấy $\mathcal{R}_x[\tau]$ tại các trễ nguyên. Đường FPGA mặc định thay bằng:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} \tilde{x}[n]\,\tilde{x}[n+\tau], \quad \tau \in [\tau_{\min}, \tau_{\max}],$$

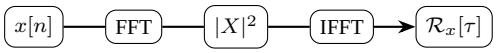
một phép nhân-cộng (MAC) mỗi chu kỳ đồng hồ trên 1 khối DSP.

Ví dụ tính tay DFT 8 điểm + butterfly. Cho $x = [1, 1, -1, -1, 1, 1, -1, -1]$ (chu kỳ 4). Stage 1 radix-2 với twiddle $W = 1$ (cố ý để thấy lõi cộng/trừ; stage sau mới nhân twiddle phức — xem định nghĩa butterfly/twiddle ở Chương 0):

Butterfly	a	b	$a \pm Wb$ ($W = 1$)
pair0	$x[0] = 1$	$x[4] = 1$	2, 0
pair1	1	1	2, 0
pair2	-1	-1	-2, 0
pair3	-1	-1	-2, 0

Sau khi có $X[k]$, bình phương $|X|^2$, rồi IFFT cho $\mathcal{R}[\tau]$ lớn tại $\tau = 0$ và $\tau = 4$ — đúng chu kỳ của x.

Ví dụ Parseval sạch hơn với xung chữ nhật $x = [1, 1, 1, 1, 0, 0, 0, 0]$: $\sum |x|^2 = 4$; DFT cho $X[0] = 4$, $X[1] = 1 - j(1 + \sqrt{2})$, $X[2] = 0$,…và $\frac{1}{N} \sum |X|^2 = 4$ khớp Parseval. (Đã kiểm `test_dft_8_manual`.)



Praat: pad đến $nsampFFT = 1024$ (Hanning depth 0,5).
FPGA mặc định: MAC trực tiếp (cùng $\mathcal{R}$, khác đường sai số).

Hình 13: Đường AC trong `Sound_to_Pitch.cpp`: FFT → power → IFFT. Silicon DE10 thay bằng MAC (Chương 7).

BFP nhắc ngắn. Nếu lab bật FFT IP, nhớ Block Floating Point: khi lấy $|X|^2$ phải nhân lại 2^{2e} trước IFFT (đã giải ở Chương 0). Quên bước này làm sai toàn bộ tỉ lệ $\mathcal{R}[\tau]$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 5 ACF.

- MAC mặc định: 1 DSP, accumulator 48-bit, 120960 chu kỳ/khung @ 50 MHz $\Rightarrow$ 2,42 ms, RTF = 0,242.
- FFT 1024 tùy chọn: vài nghìn butterfly + twiddle ROM + BFP — đắt hơn khi $\tau_{\max} \ll L$; chỉ lab đối chiếu.
- Twin: vòng `for tau` nhân dồn khớp bit với RTL, hoặc `np.fft` khi so Praat float.

Bài tập 1.5. Với $x_{\text{zm}} = [-1, 1, -1, 1]$, tính $\mathcal{R}[0]$, $\mathcal{R}[1]$, $\mathcal{R}[2]$ theo định nghĩa không vòng. Đỉnh chu kỳ nằm ở τ nào?

Lời giải. $\mathcal{R}[0] = 4$; $\mathcal{R}[1] = (-1)(1) + (1)(-1) + (-1)(1) = -3$; $\mathcal{R}[2] = 2$. Đỉnh dương tại $\tau = 2$ (và $\tau = 0$), đúng chu kỳ 2.

1.6. Bước 6 — Chuẩn hóa cho tự tương quan cửa sổ $\mathcal{R}_w[\tau]$

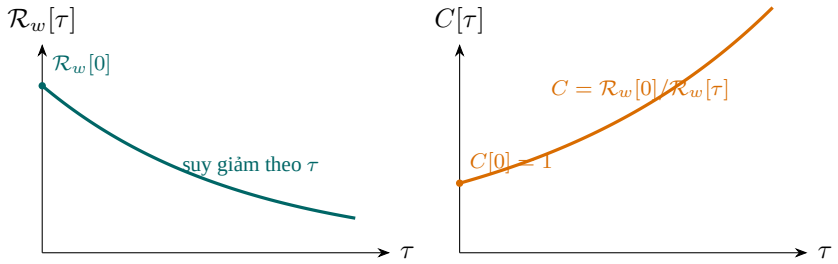
★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng / toán Boersma: hai quả núi Hanning ít chồng khi τ lớn

Sau khi nhân cửa sổ, ngay cả tín hiệu tuần hoàn hoàn hảo cũng có $\mathcal{R}_x[\tau]$ suy giảm khi τ tăng, vì hai bản sao cửa sổ chồng ít hơn. Nếu không bù, thuật toán thiên vị chọn τ nhỏ (cao độ cao giả). Boersma chuẩn hóa:

$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0] \cdot \mathcal{R}_w[\tau]/\mathcal{R}_w[0]} = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau], \quad C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}.$$

Mã: `r[i] = ac[i+1] / (ac[1] * windowR[i+1])` (`Sound_to_Pitch.cpp`). $C[\tau]$ chỉ phụ thuộc cửa sổ — tính offline một lần.

Ví dụ tính tay $N = 4$**,** $\tau = 1$**.** Với $w \approx [0{,}3455, 0{,}9045, 0{,}9045, 0{,}3455]$ đã tính ở Bước 4: $\mathcal{R}_w[0] = 1{,}875$, $\mathcal{R}_w[1] \approx 1{,}443$, $C[1] \approx 1{,}299$. Nhân r thô với $1{,}299$ kéo đỉnh gần biên về công bằng với đỉnh giữa. (Đã kiểm Python.)



(a) Tự tương quan cửa sổ (b) Hệ số bù tăng theo τ

Hình 14: Bước 6: hai quả núi Hanning chồng ít hơn khi τ lớn nên $\mathcal{R}_w[\tau]$ giảm (a); hệ số $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ tăng để kéo các đỉnh gần biên về công bằng với đỉnh giữa (b).

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 6 Norm.

- Bảng $C[\tau]$ tính offline từ đúng w đã dùng, lượng tử Q3.13, nạp ROM (Chương 8): $189 \times 16\,\text{bit} = 3024\,\text{bit}$ — chưa đầy 1 M10K.
- Runtime: 1 nhân DSP mỗi τ (nhân r thô với $C[\tau]$). Divider runtime từng τ là phương án khác, tốn ALM/latency — không mặc định.
- Twin: cùng bảng hex $C[\tau]$ với ROM phần cứng; so khớp từng từ Q3.13.

Bài tập 1.6. Với $w = [0{,}5, 1{,}0, 1{,}0, 0{,}5]$, tính $\mathcal{R}_w[0]$, $\mathcal{R}_w[1]$, $\mathcal{R}_w[2]$ và $C[1]$, $C[2]$.

Lời giải. $\mathcal{R}_w[0] = 0{,}25 + 1 + 1 + 0{,}25 = 2{,}5$; $\mathcal{R}_w[1] = 0{,}5 + 1{,}0 + 0{,}5 = 2{,}0$; $\mathcal{R}_w[2] = 0{,}5 + 0{,}5 = 1{,}0$. $C[1] = 2{,}5/2 = 1{,}25$; $C[2] = 2{,}5/1 = 2{,}5$. C tăng theo τ — đúng thiên vị cần bù.

1.7. Bước 7 — Trích cực đại cục bộ và nội suy parabolic

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: nhiều đỉnh = $T_0, 2T_0, \ldots$ cộng nhiễu formant

Tín hiệu tuần hoàn lý tưởng có đỉnh $\mathcal{R}$ tại mọi kT_0. Formant và nhiễu tạo đỉnh giả. Praat lấy tối đa 14 cực đại cục bộ thỏa $r[i] > r[i-1]$, $r[i] \geq r[i+1]$, và $r[i] > 0{,}5 \cdot$ `voicingThreshold`, cộng một ứng viên Unvoiced $\Rightarrow$ `maxnCandidates` $= 15$.
Đỉnh thật thường nằm *giữa* hai mẫu nguyên trên lưới $\tau \in \mathbb{Z}$. Nội suy parabolic khớp ba điểm quanh đỉnh bằng một parabola rồi lấy đỉnh của parabola — rẻ về số học, đủ chính xác cho FPGA (sai số thường dưới vài cent trên sine sạch). Nội suy sinc/Brent tinh hơn nhưng cần nhiều MAC; silicon mặc định dừng ở parabolic (xem lại so sánh ở Chương 0).

Toán từng bước. Đặt ba mẫu quanh đỉnh nguyên τ^*: $r_- = r[\tau^* - 1]$, $r^* = r[\tau^*]$, $r_+ = r[\tau^* + 1]$. Đặt gốc tại τ^* ($x = 0$ ở đỉnh). Parabola $y = ax^2 + bx + c$ đi qua $(-1, r_-)$, $(0, r^*)$, $(1, r_+)$. Đỉnh parabola tại

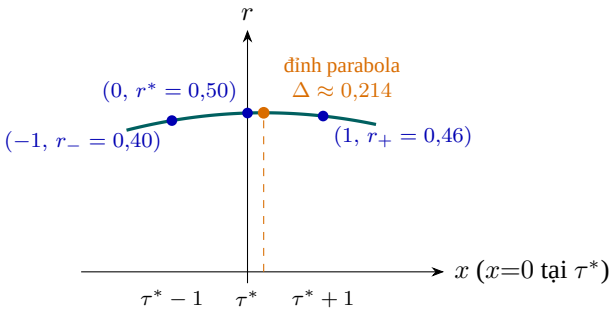
$$\Delta = \frac{1}{2} \cdot \frac{r_- - r_+}{r_- - 2r^* + r_+} = \frac{r_- - r_+}{2(r_- - 2r^* + r_+)}.$$

Trễ tinh: $\tau_{\text{fine}} = \tau^* + \Delta$. Cao độ: $F_0 = f_s / \tau_{\text{fine}}$.

Ví dụ tính tay 3 điểm. $r_- = 0{,}40$, $r^* = 0{,}50$, $r_+ = 0{,}46$ tại $\tau^* = 80$:

$$\Delta = \frac{0{,}40 - 0{,}46}{2\big(0{,}40 - 2 \cdot 0{,}50 + 0{,}46\big)} = \frac{-0{,}06}{2(-0{,}14)} = \frac{-0{,}06}{-0{,}28} = 0{,}214286.$$

$\tau_{\text{fine}} = 80{,}214286$ mẫu; tại $f_s = 16\,\text{kHz}$: $F_0 = 16000/80{,}214286 \approx 199{,}466\,\text{Hz}$. So với ước lượng thô $16000/80 = 200\,\text{Hz}$: lệch khoảng $4{,}63$ cent. (Đã kiểm Python.)



Hình 15: Bước 7: ba điểm (r_-, r^*, r_+) xác định duy nhất một parabola; đỉnh liên tục lệch khỏi mẫu nguyên một lượng $\Delta = (r_- - r_+)/(2(r_- - 2r^* + r_+))$. Ví dụ tay: $\tau^* = 80$, $\Delta \approx 0{,}214$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 7 Peak/parabolic.

- Argmax một đỉnh (K= 1): comparator quét $\tau_{\text{min}}..\tau_{\text{max}}$, vài ALM, 0 DSP.
- Parabolic: 2 phép trừ, 1 chia (LPM_DIVIDE), 1 nhân $\frac{1}{2}$ — vài chu kỳ, 0–1 DSP (Chương 9).
- Top-14 chỉ khi HPS cần đa ứng viên cho Viterbi; mặc định fabric K= 1 + UV.

Bài tập 1.7. Cho $r = \{0{,}20,\ 0{,}60,\ 0{,}50\}$ tại $\tau^* = 100$. Tính Δ, τ_{fine} và F_0 @ 16 kHz.

Lời giải. $\Delta = (0{,}20 - 0{,}50)/\big(2(0{,}20 - 1{,}20 + 0{,}50)\big) = (-0{,}30)/(2(-0{,}50)) = 0{,}30$. $\tau_{\text{fine}} = 100{,}3$; $F_0 = 16000/100{,}3 \approx 159{,}52\,\text{Hz}$.

1.8. Bước 8 — Nội suy Sinc / Brent (đỉnh phụ mẫu tinh)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: lưới mẫu rời ⇒ đỉnh liên tục nằm giữa hai mẫu

$\mathcal{R}(\tau)$ liên tục bị lấy mẫu tại $\tau \in \mathbb{Z}$. Nội suy sinc khôi phục tín hiệu band-limited. Praat: lần 1 parabolic; lần 2 `NUMimproveMaximum` với `NUM_PEAK_INTERPOLATE_SINC70` (Hanning) hoặc `SINC700` (Gaussian / very accurate) — `Sound_to_Pitch.cpp` khoảng dòng 247–254. "Depth" là bề rộng cửa sổ sinc quanh đỉnh: SINC70 lấy ± 35 mẫu; SINC700 lấy ± 350.

Ví dụ tính tay sinc 3 điểm. Giá trị tại $x \in \{-1, 0, 1\}$: $y = 0{,}40,\ 0{,}50,\ 0{,}46$. Xấp xỉ sinc cửa sổ 3 tại $t = 0{,}2$:

$$\hat{y}(t) = \sum_{k=-1}^{1} y[k]\,\text{sinc}(t - k), \quad \text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}\ (0 \mapsto 1).$$

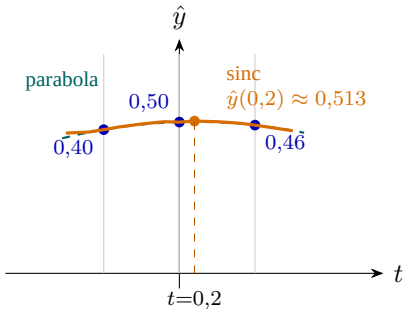
Các mẫu sinc:

$$\begin{aligned}\operatorname{sinc}(1{,}2) &\approx -0{,}155915,\\ \operatorname{sinc}(0{,}2) &\approx 0{,}935489,\\ \operatorname{sinc}(-0{,}8) &\approx 0{,}233872.\end{aligned}$$

Suy ra

$$\begin{aligned}\hat{y} &\approx 0{,}40(-0{,}155915) + 0{,}50(0{,}935489) + 0{,}46(0{,}233872)\\ &\approx 0{,}512960 > y[0].\end{aligned}$$

Đỉnh hơi lệch phải, khớp dấu Δ parabolic dương (đã kiểm `numpy.sinc`).



Hình 16: Bước 8: cùng ba điểm mẫu, nội suy parabolic (nét đứt xanh ngọc) so với xấp xỉ sinc cửa sổ hẹp (nét cam). Sinc nâng đỉnh lên $\approx 0{,}513$ tại $t \approx 0{,}2$, cùng dấu lệch Δ parabolic dương — tinh hơn nhưng tốn MAC.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 8 Sinc/Brent.

- Mặc định fabric: **dừng ở parabolic** (Chương 9) — 0 MAC sinc.
- SINC70: ~ 70 MAC/đỉnh; 14 đỉnh ~ 980 MAC/khung — vẫn nhỏ so với MAC ACF ($\sim 10^5$), nhưng thêm FSM địa chỉ và latency khó đóng timing 10 ms nếu làm naive.
- SINC700: ~ 700 MAC/đỉnh — chỉ optional lab trên HPS, không đưa vào fabric real-time.
- Twin: parabolic bắt buộc khớp bit với RTL; sinc chỉ khi so Praat `very_accurate`.

Bài tập 1.8. Giải thích tại sao FPGA chọn parabolic dù SINC70 chỉ thêm ~ 980 MAC — chưa đến 1% ngân sách MAC ACF.

Lời giải. Chi phí không chỉ là số MAC: cần bộ đệm hàng chục mẫu quanh mỗi đỉnh, FSM địa chỉ, và (nếu đa ứng viên) lặp 14 lần. Trên đường K$=1$ real-time, parabolic cho sai số < 2 cent trên sine sạch — đủ ngưỡng lab 20 cent so Praat (Chương 13) — trong khi thêm Sinc tăng độ phức tạp kiểm chứng mà lợi ích biên nhỏ trên giọng nói thật (nhiễu và formant đã chiếm phần lớn sai số).

1.9. Bước 9 — Octave cost: phạt thiên vị quãng tám

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: $2T_0$ cũng là chu kỳ hoàn hảo — sai octave

Nếu $x(t) = x(t+T_0)$ thì cũng $x(t) = x(t+2T_0)$. Tự tương quan vì thế có đỉnh tại $2T_0$ gần bằng đỉnh tại T_0 sau khi đã bù cửa sổ. Không phạt thì thuật toán thích chọn $F_0/2$ (giọng trầm giả) — đó là **sai octave** (*octave error*): nhầm một quãng tám. Hiện tượng gốc là tính tuần hoàn toán học, không phải dây thanh thật sự rung chậm gấp đôi (đã nêu ở Chương 0).
Boersma trừ một octave cost

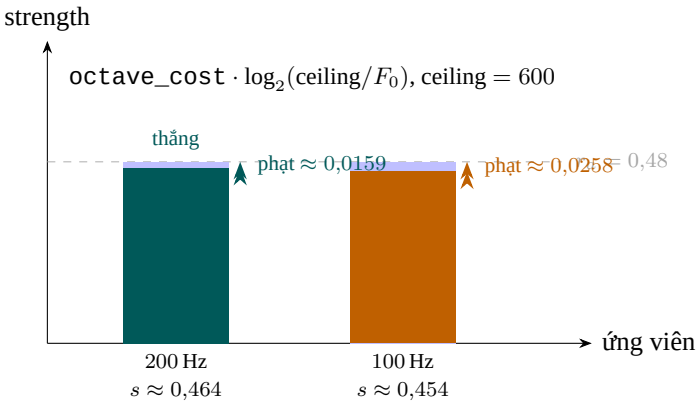
$$\texttt{octave_cost} \cdot \log_2(\text{ceiling}/F_0)$$

khỏi strength của mỗi ứng viên (trong `Pitch_pathFinder`, mảng `delta[]`). Ứng viên tần số thấp bị phạt nặng hơn một chút — đủ để phá thế cân bằng giả giữa T_0 và $2T_0$.

Ví dụ tính tay. `octave_cost` $= 0{,}01$, ceiling $= 600$, hai ứng viên 200 Hz và 100 Hz, cùng $r_a = 0{,}48$:

$$s_{200} = 0{,}48 - 0{,}01 \log_2(600/200) = 0{,}48 - 0{,}01 \cdot 1{,}58496 \approx 0{,}46415,$$
$$s_{100} = 0{,}48 - 0{,}01 \log_2(600/100) = 0{,}48 - 0{,}01 \cdot 2{,}58496 \approx 0{,}45415.$$

200 Hz thắng — đúng F_0, không phải nửa. Tại 500 Hz: phạt chỉ $\approx 0{,}00263$; tại ceiling 600 Hz: phạt $= 0$. (Đã kiểm Python.)



Hình 17: Bước 9: hai ứng viên cùng $r_a = 0{,}48$. Octave cost phạt 100 Hz nặng hơn 200 Hz (thêm một quãng tám so với ceiling), phá thế cân bằng giả $T_0 \,/\, 2T_0$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 9 Octave cost.

- Fabric K$= 1$: không tính delta đầy đủ (không có đa ứng viên để so). Ceiling/floor vẫn cắt dải τ tìm kiếm — *có* ảnh hưởng fabric qua giới hạn vòng MAC.
- HPS pathFinder: soft-float $\log_2$ hoặc LUT nhỏ.
- Twin: cùng công thức khi bật Viterbi soft; đường median-3 bỏ qua octave cost từng ứng viên nhưng vẫn dùng $\tau_{\min}/\tau_{\max}$.

Bài tập 1.9. Cùng $r_a = 0{,}50$, ceiling $= 600$, `octave_cost` $= 0{,}01$. So s tại 150 Hz và 300 Hz. Ai thắng? Đây có phải bẫy octave không?

Lời giải. $s_{150} = 0{,}50 - 0{,}01 \log_2(600/150) = 0{,}50 - 0{,}02 = 0{,}48$; $s_{300} = 0{,}50 - 0{,}01 \log_2(2) = 0{,}50 - 0{,}01 = 0{,}49$. Ứng viên 300 Hz thắng trên strength đơn khung — đúng thiên vị nhẹ về cao. Nếu 300 Hz là bẫy octave của 150 Hz thật, Bước 11–12 (jump cost + Viterbi) sẽ phạt chuyển $150 \to 300$ giữa các khung bằng $0{,}35$ để kéo đường về 150 Hz.

1.10. Bước 10 — Silence threshold, Voicing threshold và Ứng viên vô thanh

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: ba trạng thái âm thanh và bóng ma nhiễu nền

Tín hiệu tiếng nói của con người gồm ba trạng thái vật lý phân biệt rõ rệt:

1. **Âm hữu thanh (Voiced):** Hai mép dây thanh quản khép sát và rung đóng mở tuần hoàn dưới áp lực luồng khí từ phổi (các nguyên âm /a/, /u/, /i/ hay phụ âm mũi /m/, /n/). Dạng sóng âm trong miền thời gian có tính chu kỳ rất mạnh, tạo nên các đỉnh tự tương quan chuẩn hóa cao vút, thường đạt $r_a[\tau^*] \in [0{,}60;\ 0{,}95]$.
2. **Âm vô thanh (Unvoiced):** Dây thanh quản mở rộng hoàn toàn, không rung; luồng không khí đi qua các khe hẹp trong khoang miệng sinh ra hiện tượng xoáy khí hỗn loạn (các âm xát /s/, /x/, /f/ hay âm bật /t/, /p/, /k/). Tín hiệu vô thanh có bản chất ngẫu nhiên tựa như nhiễu trắng, khiến hàm tự tương quan suy giảm nhanh về 0 và các đỉnh cục bộ chỉ dao động ngẫu nhiên ở mức rất thấp ($r_a[\tau] < 0{,}30$).
3. **Khoảng lặng (Silence):** Người nói ngừng phát âm để lấy hơi hoặc ngắt câu. Màng rung micro chỉ thu được tiếng ồn nền của phòng thu, tiếng quạt gió máy tính hoặc tạp âm nhiệt của mạch tiền khuếch đại điện tử (preamplifier hiss).

Nếu không có cơ chế chặn khoảng lặng, thuật toán chuẩn hóa sẽ chia tự tương quan cho năng lượng khung $\mathcal{R}_x[0]$. Khi năng lượng khung tiến gần về 0, những gợn sóng nhiễu ngẫu nhiên chỉ cỡ micro-volt cũng bị khuếch đại giả tạo thành các đỉnh tương quan chuẩn hóa cao tới $0{,}8$ hoặc $0{,}9$. Hậu quả là bộ ước lượng sẽ xuất ra các giá trị cao độ F_0 giả mạo, nhảy nhót loạn xạ ngay trong những đoạn người nói hoàn toàn im lặng.

Toán từng bước từ định nghĩa gốc. Praat giải quyết triệt để vấn đề này thông qua cặp tham số kép. **Ngưỡng im lặng** (`silenceThreshold`) mặc định $\theta_{\text{sil}} = 0{,}03$ theo biên độ. **Ngưỡng hữu thanh** (`voicingThreshold`) mặc định $\theta_{\text{vox}} = 0{,}45$.

Để có hệ quy chiếu công bằng giữa các khung phát âm to nhỏ khác nhau, Praat quét toàn bộ phát ngôn để tìm năng lượng khung lớn nhất làm mốc tham chiếu toàn cục:

$$E_{\text{ref}} = \max_k \mathcal{R}_x[0]_k.$$

Ngưỡng biên độ $0{,}03$ tương đương với ngưỡng năng lượng (bình phương biên độ):

$$\theta_{\text{energy}} = \theta_{\text{sil}}^2 = (0{,}03)^2 = 0{,}0009 \quad (\approx -30{,}46\,\text{dB so với đỉnh tham chiếu}).$$

Sức mạnh (strength) của ứng viên vô thanh (*unvoiced candidate*) trong khung thứ k được Boersma thiết lập theo hàm phi tuyến:

$$s_{\text{uv}} = \theta_{\text{vox}} + \max\left(0,\ 2 - \frac{\mathcal{R}_x[0]_k / E_{\text{ref}}}{\theta_{\text{sil}}/(1+\theta_{\text{vox}})}\right).$$

Phân tích biểu thức giải tích này cho thấy hai trạng thái biên rõ ràng:

- **Trong khoảng lặng ($\mathcal{R}_x[0]_k \ll E_{\text{ref}}$):** Tỷ số năng lượng trong ngoặc tiến về 0. Giá trị s_{uv} tăng vọt lên mức cực đại:
$$s_{\text{uv}} \to \theta_{\text{vox}} + 2 = 0{,}45 + 2 = 2{,}45.$$
Do đỉnh tự tương quan của bất kỳ sóng âm nào sau chuẩn hóa cũng không bao giờ vượt quá $1{,}0$, giá trị $s_{\text{uv}} = 2{,}45$ này đè bẹp tuyệt đối mọi đỉnh tương quan giả do nhiễu sinh ra, ép khung âm thanh phải được phán quyết là Vô thanh ($F_0 = 0$).
- **Trong khung phát âm rõ ràng ($\mathcal{R}_x[0]_k \geq 2\frac{\theta_{\text{sil}}}{1+\theta_{\text{vox}}} E_{\text{ref}} \approx 0{,}0414 E_{\text{ref}}$):** Số hạng thứ hai bị chặn bằng 0. Khi đó sức mạnh vô thanh trở về đúng bằng ngưỡng danh định:
$$s_{\text{uv}} = \theta_{\text{vox}} = 0{,}45.$$
Lúc này, nếu đỉnh tự tương quan cục bộ mạnh nhất $r_a[\tau^*] \geq 0{,}45$, ứng viên hữu thanh sẽ chiến thắng và hệ thống công nhận khung đó có cao độ $F_0 = f_s/\tau_{\text{fine}}$. Ngược lại, nếu $r_a[\tau^*] < 0{,}45$, khung được coi là vô thanh.

Ví dụ tính tay từng bước (kiểm chứng bằng Python). Xét hai khung âm thanh với $\theta_{\text{sil}} = 0{,}03$, $\theta_{\text{vox}} = 0{,}45$. Mẫu số chuẩn hóa:

$$\text{Mẫu số} = \frac{\theta_{\text{sil}}}{1+\theta_{\text{vox}}} = \frac{0{,}03}{1+0{,}45} = \frac{0{,}03}{1{,}45} \approx 0{,}02068966.$$

- **Trường hợp 1 (Khung thì thầm rất nhỏ, $\mathcal{R}_x[0]/E_{\text{ref}} = 0{,}02$):** Tỷ số năng lượng chuẩn hóa:

$$\text{ratio} = \frac{0{,}02}{0{,}02068966} \approx 0{,}966667.$$

  Sức mạnh ứng viên vô thanh:

$$s_{\text{uv}} = 0{,}45 + \max(0,\ 2 - 0{,}966667) = 0{,}45 + 1{,}033333 = 1{,}483333.$$

  Giả sử nhiễu nền tạo ra một đỉnh tương quan cực đại giả có biên độ thô $r_a = 0{,}70$. Vì $r_a = 0{,}70 < s_{\text{uv}} = 1{,}483333$, ứng viên vô thanh hoàn toàn áp đảo; khung được kết luận là vô thanh ($F_0 = 0$), bảo vệ hệ thống khỏi báo động giả.

- **Trường hợp 2 (Khung phát âm nguyên âm mạnh, $\mathcal{R}_x[0]/E_{\text{ref}} = 0{,}80$):** Tỷ số năng lượng:

$$\text{ratio} = \frac{0{,}80}{0{,}02068966} \approx 38{,}667 \gg 2 \implies 2 - \text{ratio} < 0.$$

  Sức mạnh vô thanh duy trì ở mức sàn: $s_{\text{uv}} = 0{,}45$. Nếu đỉnh tự tương quan đạt $r_a[\tau^*] = 0{,}78$, do $0{,}78 > 0{,}45$, khung được gắn nhãn Hữu thanh (Voiced).

Toàn bộ các giá trị trên đã được kiểm chứng khớp chính xác từng chữ số (tập lệnh trong thư mục `verify/`).

Ý nghĩa phần cứng — dẫn hai cổng số nguyên từ quy tắc float. Trên FPGA streaming không đợi hết utterance để lấy E_{ref} toàn cục; thực tế thay bằng ngưỡng tuyệt đối hoặc *moving peak detector*. Quan trọng hơn: hai phép so sánh số nguyên trong twin/RTL *không* lấy từ s_{uv} mà từ hai quy tắc float riêng — và mỗi bên phải biết rõ đại lượng nào đang bị bình phương / chuẩn hóa.

Silence Gate (năng lượng, có bình phương θ_{sil}). Praat khai báo `silenceThreshold` theo *biên độ*: $\theta_{\text{sil}} = 0{,}03$. Năng lượng khung tỉ lệ với bình phương biên độ, nên ngưỡng năng lượng là

$$\theta_{\text{energy}} = \theta_{\text{sil}}^2 = (0{,}03)^2 = 0{,}0009 = \frac{9}{10{,}000}.$$

Quy tắc float (cứng, abort sớm — khác với cạnh tranh mềm s_{uv} ở trên):

$$\frac{\mathcal{R}_x[0]}{E_{\text{ref}}} < \theta_{\text{energy}} \quad \Longleftrightarrow \quad \frac{\mathcal{R}_x[0]}{E_{\text{ref}}} < \frac{9}{10{,}000}.$$

Nhân hai vế với $10{,}000 \cdot E_{\text{ref}}$ (dương):

$$\mathcal{R}_x[0] \cdot 10{,}000 < 9 \cdot E_{\text{ref}}.$$

Đây chính là Silence Gate số nguyên. Ví dụ bài tập:

$$\begin{aligned} \mathcal{R}_x[0] &= 500{,}000, \quad E_{\text{ref}} = 600{,}000{,}000, \\ \frac{\mathcal{R}_x[0]}{E_{\text{ref}}} &= \frac{500{,}000}{600{,}000{,}000} \approx 0{,}000833 < 0{,}0009, \\ \mathcal{R}_x[0] \cdot 10{,}000 &= 5 \times 10^9 < 5{,}4 \times 10^9 = 9 \cdot E_{\text{ref}}. \end{aligned}$$

Cùng một kết luận "im lặng" trên cả hai dạng float và số nguyên.

Lưu ý tách bạch: công thức s_{uv} dùng mẫu số $\theta_{\text{sil}}/(1+\theta_{\text{vox}})$ trên tỉ số $\mathcal{R}_x[0]/E_{\text{ref}}$ để *nâng sức mạnh ứng viên UV*; Silence Gate cứng dùng θ_{sil}^2 để *cắt sớm* khung quá yếu. Hai cơ chế cùng nguồn θ_{sil} nhưng không cùng bất đẳng thức.

Voicing Gate (chuẩn hóa r_a, *không* bình phương). Quy tắc float Praat trên khung không bị silence cắt: khung hữu thanh khi

$$r_a[\tau^*] \geq \theta_{\text{vox}} = 0{,}45,$$

với r_a *đã chuẩn hóa* (không thứ nguyên):

$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau] = \frac{S[\tau]}{\mathcal{R}_x[0]}, \qquad S[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau].$$

Trên twin/RTL, `best_score` là đúng $S[\tau^*]$ sau nhân ROM $C[\tau]$ (Q3.13, dịch $\gg 13$) — cùng đơn vị năng lượng với $\mathcal{R}_x[0]$, *không* phải r_a đã chia. Do đó

$$\frac{\texttt{best_score}}{\mathcal{R}_x[0]} \geq 0{,}45 = \frac{45}{100} \quad \Longleftrightarrow \quad \texttt{best_score} \cdot 100 \geq 45 \cdot \mathcal{R}_x[0].$$

Nếu nhầm đưa r_a không thứ nguyên $(0{,}78)$ vào chỗ `best_score`, phép so sánh $0{,}78 \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$ trộn hai thang đo và luôn sai khi $\mathcal{R}_x[0] \gg 1$. Twin `golden/lab_1_3_5.py` giữ `best_score` = lag energy đã bù $C[\tau]$ để khớp bất đẳng thức trên.

Tóm tắt hai cổng số nguyên (không chia, không DSP):

Silence: $\mathcal{R}_x[0] \cdot 10{,}000 < 9 \cdot E_{\text{ref}}$ (từ $\theta_{\text{sil}}^2 = 9/10{,}000$),

Voicing: $\texttt{best_score} \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$ (từ $S/\mathcal{R}_x[0] \geq 45/100$; S = năng lượng đã $\times C$).

Đoạn mã Verilog tổng hợp dưới đây minh họa khối phán quyết trên FPGA:

Listing 2: cùng đơn vị năng lượng với R[0], không phải r_a chuẩn hóa.]Silence + Voicing Gate số nguyên: best_score là S[tau*] cùng đơn vị năng lượng với R[0], không phải r_a chuẩn hóa.

```
// Silence: R[0]/E_ref < (0.03)^2 = 9/10000
// Voicing: S[tau*]/R[0] >= 0.45 = 45/100  (best_score = S)
wire [63:0] score_scaled = best_score * 64'd100;
wire [63:0] r0_scaled    = r0_energy  * 64'd45;
wire is_silence = (r0_energy * 64'd10000) < (64'd9 * ref_energy);
wire is_voiced  = (!is_silence) && (score_scaled >= r0_scaled);

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        f0_out_reg <= 16'd0;
        vuv_flag   <= 1'b0;
    end else if (frame_done) begin
        vuv_flag   <= is_voiced;
        f0_out_reg <= is_voiced ? f0_interpolated : 16'd0;
    end
end
```

Mô hình song sinh phần mềm (Bit-Exact Twin). Trong mô hình Python tham chiếu (tệp `lab_1_3_5.py` thư mục `golden/`), `best_score` được gán từ `scores[idx] = (r_tau * scoring_rom + 4096) >> 13` — tức $S[\tau]$ — rồi so khớp bit với RTL:

```
# Silence: r0/E_ref < 0.03**2 = 0.0009 = 9/10000
if ref_energy > 0 and r0 * 10000 < 9 * ref_energy:
    return FrameResult(f0=0.0, is_voiced=False, f0_q12_4=0)

# Voicing: best_score/r0 < 0.45  <=>  best_score*100 < 45*r0
# best_score = S[tau*] (C-compensated lag energy), NOT dimensionless r_a
if best_score * 100 < 45 * r0:
    return FrameResult(f0=0.0, is_voiced=False, f0_q12_4=0)
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 10 Voicing Gate.

- **Tài nguyên logic (ALM / LUT):** Chiếm khoảng $35 \ldots 45$ ALMs cho hai bộ so sánh 48-bit và các cổng logic bắt cờ.
- **Khối nhân DSP cứng:** 0 khối (nhân hằng số 45 và 100 được tổng hợp tối ưu qua mạng cộng dịch LUT).
- **Độ trễ xử lý (Latency):** Đúng 1 chu kỳ xung nhịp đồng hồ tại thời điểm kết thúc quét mảng tương quan τ.
- **Ảnh hưởng đến RTF:** Độ trễ 1 chu kỳ (20 ns tại 50 MHz) hoàn toàn không đáng kể so với ngân sách 10 ms của bước nhảy khung (0,0002%).

Bài tập 1.10. Một khung âm thanh có năng lượng đo được $\mathcal{R}_x[0] = 500{,}000$, trong khi năng lượng đỉnh tham chiếu là $E_{\text{ref}} = 600{,}000{,}000$. Đỉnh tự tương quan chuẩn hóa đạt $r_a[\tau^*] = 0{,}82$. Khung này có được công nhận là có cao độ F_0 hay không?

Lời giải. Quy tắc float Silence: $\mathcal{R}_x[0]/E_{\text{ref}} < \theta_{\text{sil}}^2 = 0{,}0009$. Tỉ số thực tế $500{,}000/600{,}000{,}000 = 1/1200 \approx 0{,}000833 < 0{,}0009$ — đã im lặng. Dạng số nguyên cùng nguồn ($0{,}0009 = 9/10{,}000$):

$$\mathcal{R}_x[0] \cdot 10{,}000 = 5 \times 10^9, \qquad 9 \cdot E_{\text{ref}} = 5{,}4 \times 10^9.$$

Vì $5 \times 10^9 < 5{,}4 \times 10^9$, Silence Gate kích hoạt, ép $F_0 = 0$ dù $r_a = 0{,}82$ (nhiễu nền bị chuẩn hóa phóng đại). Voicing Gate ($\texttt{best_score} \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$) không còn được xét: cổng im lặng cắt trước. Nếu giả sử năng lượng đủ lớn và $\texttt{best_score} = S[\tau^*] = 0{,}82 \cdot \mathcal{R}_x[0]$, thì $0{,}82 \cdot \mathcal{R}_x[0] \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$ đúng — nhưng chỉ sau khi đã qua Silence Gate.

1.11. Bước 11 — Chi phí bước nhảy quãng tám và Chi phí chuyển trạng thái Hữu/Vô thanh

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: Quán tính cơ học của dây thanh quản và sự mượt mà sinh lý

Dây thanh quản của con người cấu tạo từ các lớp cơ vân và màng nhầy (cơ giáp-phễu *thyroarytenoid* và cơ nhẫn-giáp *cricothyroid*). Cơ cấu sinh học này có khối lượng và quán tính vật lý thực sự: sức căng cơ không thể nào biến thiên tức thời gấp đôi hoặc tụt giảm một nửa trong khoảng thời gian chỉ 10 ms giữa hai khung liên tiếp. Trong phát âm tự nhiên, quỹ đạo cao độ F_0 luôn biến thiên một cách mượt mà và liên tục; tốc độ biến đổi cao độ thông thường chỉ dưới $5 \ldots 10\%$ mỗi khung (< 1 bán âm).
Nếu chỉ dựa vào cực đại tự tương quan của từng khung đơn lẻ, hiện tượng nhiễu dải tần hoặc cộng hưởng formant có thể đẩy một đỉnh hài phụ lên cao hơn đỉnh chính, gây ra hiện tượng nhảy vọt cao độ gấp đôi ($2F_0$) hoặc tụt một nửa ($F_0/2$) chỉ trong một khung rồi lập tức quay về giá trị cũ. Tương tự, tại các ranh giới chuyển tiếp giữa nguyên âm và phụ âm, trạng thái Hữu thanh và Vô thanh có thể bị chập chờn (flickering: V–UV–V–UV) nếu năng lượng khung mấp mé ngưỡng so sánh. Để ngăn chặn các bước nhảy phi tự nhiên này, giải thuật Praat áp dụng hai hàm phạt chuyển trạng thái (*transition costs*) giữa hai khung thời gian liên tiếp.

Toán từng bước từ định nghĩa gốc. Praat định nghĩa hai thành phần chi phí chuyển trạng thái từ ứng viên i ở khung $t-1$ sang ứng viên j ở khung t:

1. **Chi phí bước nhảy quãng tám (Octave-jump cost):** Áp dụng khi cả hai khung liên tiếp đều là âm Hữu thanh ($F_0 > 0$), tỷ lệ thuận với khoảng cách quãng tám logarit giữa hai tần số:

$$C_{\text{jump}}\big(F_0(t-1),\, F_0(t)\big) = \texttt{octaveJumpCost} \times \left|\log_2 \frac{F_0(t-1)}{F_0(t)}\right|.$$

   Trong phần mềm Praat, tham số mặc định là $\texttt{octaveJumpCost} = 0{,}35$.

2. **Chi phí chuyển đổi trạng thái Hữu/Vô thanh (Voiced/Unvoiced cost):** Phạt mỗi khi đường cao độ chuyển từ trạng thái Hữu thanh sang Vô thanh hoặc ngược lại:

$$C_{\text{vuv}} = \texttt{voicedUnvoicedCost} = 0{,}14.$$

   Đối với quá trình giữ nguyên trạng thái Vô thanh liên tục (UV $\to$ UV), chi phí chuyển đổi bằng 0: $C_{\text{uv}\to\text{uv}} = 0$.

Ví dụ tính tay chi tiết (đã kiểm chứng bằng Python).

- **Biến thiên mượt tự nhiên ($150\,$Hz $\to 160\,$Hz):** Khoảng cách quãng tần:

$$\Delta_{\text{oct}} = \left|\log_2 \frac{150}{160}\right| = \left|\frac{\ln(150/160)}{\ln(2)}\right| \approx 0{,}093109\,\text{octave}.$$

  Chi phí chuyển trạng thái:

$$C_{\text{jump}} = 0{,}35 \times 0{,}093109 \approx 0{,}032588.$$

  Mức phạt chỉ khoảng $0{,}033$, rất nhỏ so với sức mạnh đỉnh tương quan ($0{,}6 \ldots 0{,}9$), hoàn toàn cho phép đường cong cao độ uốn lượn tự nhiên.

- **Bẫy nhảy quãng tám giả mạo ($150\,$Hz $\to 300\,$Hz):** Khoảng cách quãng tần:

$$\Delta_{\text{oct}} = \left|\log_2 \frac{150}{300}\right| = |\log_2(0{,}5)| = 1{,}0000\,\text{octave}.$$

  Chi phí chuyển trạng thái:

$$C_{\text{jump}} = 0{,}35 \times 1{,}0000 = 0{,}3500.$$

  Mức phạt $0{,}35$ là cực kỳ nặng: nó làm giảm độ ưu tiên của ứng viên $300\,$Hz tương đương với việc hạ sức mạnh đỉnh từ $0{,}80$ xuống chỉ còn $0{,}45$, chặn đứng hoàn toàn hiện tượng nhảy quãng tám giả do nhiễu.

Ý nghĩa phần cứng và Bộ lọc Trung vị 3 điểm (Median-3 Filter). Trên fabric FPGA streaming thời gian thực, việc duy trì một ma trận đồ hình chuyển trạng thái đa chiều đòi hỏi phải đệm hàng chục khung vào RAM ngoài, gây trễ lớn. Thay vào đó, kiến trúc phần cứng chuyên dụng trên kit DE10-Standard sử dụng một khối xử lý phi tuyến cực kỳ thanh lịch và hiệu quả: **Bộ lọc trung vị 3 điểm (Median-3 Filter)** trực tiếp trên dòng dữ liệu F_0:

$$F_{0,\text{filt}}[t] = \text{median}\big(F_0[t-1],\, F_0[t],\, F_0[t+1]\big).$$

Bộ lọc trung vị có đặc tính toán học xuất sắc: triệt tiêu tuyệt đối mọi xung gai nhọn đơn lẻ (như khung bị nhảy quãng tám đột ngột lên $2F_0$ hoặc rớt về 0 do mất khung) trong khi bảo toàn hoàn hảo độ dốc và biên dạng của các đoạn chuyển tiếp cao độ thực. Độ trễ phần cứng tạo ra đúng bằng 1 bước nhảy khung ($10\,$ms).

Đoạn mã Verilog tổng hợp được dưới đây mô tả cấu trúc phần cứng của bộ lọc trung vị 3 điểm:

Listing 3: Thiết kế RTL khối lọc trung vị 3 điểm cho dòng cao độ F0 trên FPGA.

```
module median3_f0 (
    input  wire        clk,
    input  wire        rst_n,
    input  wire        valid_in,
    input  wire [15:0] f0_in,       // Dinh dang Q12.4
    output reg         valid_out,
    output reg  [15:0] f0_out
);
    reg [15:0] d0, d1, d2;

    // Mach so sanh 3 cap so de tim trung vi:
    // median(a,b,c) = a > b ? (b > c ? b : (a > c ? c : a))
    //                       : (a > c ? a : (b > c ? c : b))
    function [15:0] get_median;
        input [15:0] a, b, c;
        begin
            if (a > b) begin
                if (b > c)      get_median = b;
                else if (a > c) get_median = c;
                else            get_median = a;
            end else begin
                if (a > c)      get_median = a;
                else if (b > c) get_median = c;
                else            get_median = b;
            end
        end
```

```
    endfunction

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            d0 <= 16'd0; d1 <= 16'd0; d2 <= 16'd0;
            valid_out <= 1'b0; f0_out <= 16'd0;
        end else if (valid_in) begin
            d0 <= f0_in;
            d1 <= d0;
            d2 <= d1;
            valid_out <= 1'b1;
            f0_out    <= get_median(f0_in, d0, d1);
        end else begin
            valid_out <= 1'b0;
        end
    end
endmodule
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 11 Transition Costs / Median-3.

- **Tài nguyên logic (ALM / LUT):** Bộ lọc Median-3 chỉ tốn khoảng $22 \ldots 28$ ALMs gồm 3 thanh ghi 16-bit và mạng so sánh phân nhánh 3 ngõ vào.
- **Khối nhân DSP cứng:** 0 DSP (không dùng phép toán số học nào ngoài phép so sánh độ lớn nguyên).
- **Bộ nhớ M10K:** 0 khối RAM (dữ liệu lưu hoàn toàn trong các flip-flop nội của ALM).
- **Độ trễ thời gian (Latency):** Đúng 1 khung thời gian âm thanh ($10\,\text{ms}$). Với các hệ thống tương tác giọng nói, độ trễ $10\,\text{ms}$ nằm sâu dưới ngưỡng phát hiện của tai người ($\approx 30 \ldots 50\,\text{ms}$).

Bài tập 1.11. Chuỗi giá trị cao độ thô xuất ra từ bộ dò cực đại qua 5 khung liên tiếp (bước nhảy $10\,\text{ms}$) có giá trị là: $148\,\text{Hz}$, $150\,\text{Hz}$, $300\,\text{Hz}$ (bẫy hài bậc hai do nhiễu), $152\,\text{Hz}$, $151\,\text{Hz}$.

a) Xác định chuỗi cao độ ngõ ra sau khi đi qua bộ lọc trung vị 3 điểm Median-3.

b) Tính chi phí bước nhảy quãng tám C_{jump} giữa khung 2 ($150\,\text{Hz}$) và khung 3 ($300\,\text{Hz}$) theo công thức Praat và giải thích tại sao bộ lọc trung vị lại triệt tiêu được lỗi này mà không cần giải thuật quy hoạch động phức tạp.

Lời giải.

a) Áp dụng cửa sổ trung vị trượt 3 điểm trên từng khung (bỏ qua điều kiện biên đầu/cuối):

- Khung 2 (nhìn 3 mẫu $148, 150, 300$): Trung vị của $\{148, 150, 300\}$ là $150\,\text{Hz}$.
- Khung 3 (nhìn 3 mẫu $150, 300, 152$): Sắp xếp $\{150, 152, 300\}$, giá trị ở giữa là $152\,\text{Hz}$.
- Khung 4 (nhìn 3 mẫu $300, 152, 151$): Sắp xếp $\{151, 152, 300\}$, giá trị ở giữa là $152\,\text{Hz}$.

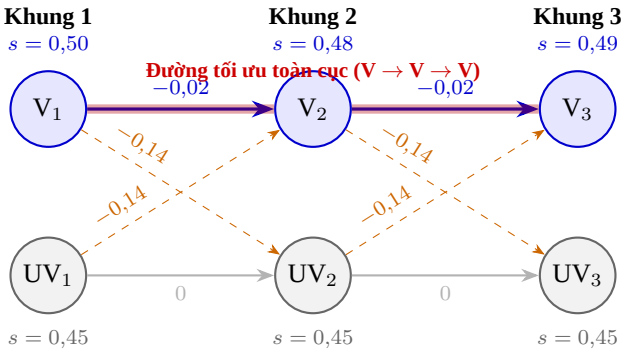
Giá trị gai nhiễu $300\,\text{Hz}$ đã bị gạt phăng hoàn toàn ra khỏi đường cong; chuỗi đầu ra duy trì mượt mà quanh $150 \ldots 152\,\text{Hz}$.

b) Chi phí bước nhảy: $C_{\text{jump}} = 0{,}35 \times |\log_2(150/300)| = 0{,}3500$. Trong khi Viterbi phải tính tổng tích lũy qua nhiều khung và duyệt ngược, bộ lọc trung vị tận dụng tiên đề thống kê rằng một xung đột biến chỉ tồn tại trong 1 khung thì luôn là phần tử ngoại lai (outlier) nằm ở đầu mút của tập 3 điểm, do đó không bao giờ được chọn làm giá trị trung vị. Mạch RTL đạt hiệu quả tương đương Viterbi đối với các xung nhiễu cô lập mà chỉ tiêu tốn chưa đầy 30 ALMs.

1.12. Bước 12 — Thuật toán tìm đường tối ưu toàn cục Viterbi (`Pitch_pathFinder`)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: Sự đánh đổi giữa cực đại cục bộ và sự mượt mà toàn cục

Trong thực tế, quyết định cao độ tối ưu không thể chỉ nhìn vào một thời điểm duy nhất. Xét trường hợp một người đang nói câu có ngữ điệu trầm, tại một khung âm thanh nào đó, đỉnh tự tương quan ở quãng tám cao ($2F_0$) tình cờ cao hơn đỉnh ở tần số cơ bản (F_0) chỉ $0{,}02$ đơn vị do một cộng hưởng vòm họng. Nếu chỉ chọn theo cực đại cục bộ (greedy search), đường cao độ sẽ bị gãy một nốt nhô cao rồi thụt xuống.
Thuật toán Viterbi (`Pitch_pathFinder` trong mã nguồn `Pitch.cpp` của Praat) giải quyết bài toán này dưới dạng **quy hoạch động (dynamic programming)**: tìm đường đi xuyên qua lưới không gian trạng thái (trellis) sao cho tổng điểm tích lũy toàn cục là lớn nhất, hài hòa giữa sức mạnh đỉnh của từng khung và sự liên tục tự nhiên theo thời gian.



Hình 18: Đồ hình lưới Trellis 3 khung thời gian của giải thuật Viterbi trong Praat. Đường màu đỏ thể hiện quỹ đạo có tổng điểm tích lũy lớn nhất sau khi đã trừ các chi phí chuyển trạng thái.

Toán từng bước: Hệ phương trình truy hồi Viterbi. Tại mỗi khung thời gian $t \in \{1, 2, \ldots, T\}$, giả sử ta có K ứng viên cao độ (tối đa 14 ứng viên hữu thanh và 1 ứng viên vô thanh). Đặt:

- $s_t(j)$: Sức mạnh đơn khung của ứng viên thứ j tại khung t.
- $C(i, j)$: Chi phí chuyển đổi từ ứng viên i ở khung $t-1$ sang ứng viên j ở khung t.
- $\delta_t(j)$: Điểm số tích lũy lớn nhất của đường đi kết thúc tại ứng viên j ở khung t.
- $\psi_t(j)$: Con trỏ ngược (backpointer) lưu chỉ số ứng viên ở khung $t-1$ đã mang lại điểm số tích lũy tối đa cho ứng viên j.

Phương trình quy hoạch động Viterbi được xác lập theo hai pha:

1. **Pha lan truyền thuận (Forward recursion):**

$$\begin{aligned} &\text{Khởi tạo } (t=1): && \delta_1(j) = s_1(j), \qquad \forall j \in \{1, \ldots, K\}, \\ &\text{Quy nạp } (t \geq 2): && \delta_t(j) = s_t(j) + \max_{1 \leq i \leq K} \big[\delta_{t-1}(i) - C(i, j)\big], \\ & && \psi_t(j) = \arg\max_{1 \leq i \leq K} \big[\delta_{t-1}(i) - C(i, j)\big]. \end{aligned}$$

2. **Pha truy hồi ngược (Backtracking):** Tại khung kết thúc T, chọn trạng thái có điểm tích lũy cao nhất:

$$j_T^* = \arg\max_{1 \leq j \leq K} \delta_T(j).$$

Truy hồi ngược lần lượt từ $t = T-1$ lùi dần về $t = 1$:

$$j_t^* = \psi_{t+1}(j_{t+1}^*).$$

Ví dụ tính tay lưới 3 khung (đã kiểm chứng trong `verify_chuong_00_01.py`). Xét bài toán 3 khung thời gian với hai trạng thái: Hữu thanh (V) và Vô thanh (UV):

- Sức mạnh ứng viên: Khung 1: $s_1(\text{V}) = 0{,}50$, $s_1(\text{UV}) = 0{,}45$. Khung 2: $s_2(\text{V}) = 0{,}48$, $s_2(\text{UV}) = 0{,}45$. Khung 3: $s_3(\text{V}) = 0{,}49$, $s_3(\text{UV}) = 0{,}45$.
- Chi phí chuyển trạng thái: $C_{\text{v}\to\text{v}} = 0{,}02$ (độ lệch cao độ nhỏ); $C_{\text{v}\to\text{uv}} = C_{\text{uv}\to\text{v}} = 0{,}14$; $C_{\text{uv}\to\text{uv}} = 0{,}00$.

  Tại Khung 1 ($t = 1$):

  $$\delta_1(\text{V}) = 0{,}50, \qquad \delta_1(\text{UV}) = 0{,}45.$$

  Tại Khung 2 ($t = 2$):

  - Điểm đến trạng thái V_2: Từ V_1: $\delta_1(\text{V}) - C_{\text{v}\to\text{v}} = 0{,}50 - 0{,}02 = 0{,}48$. Từ UV_1: $\delta_1(\text{UV}) - C_{\text{uv}\to\text{v}} = 0{,}45 - 0{,}14 = 0{,}31$. Chọn cực đại: $\max(0{,}48,\, 0{,}31) = 0{,}48$ (đến từ V_1). $\Longrightarrow$ $\delta_2(\text{V}) = s_2(\text{V}) + 0{,}48 = 0{,}48 + 0{,}48 =$ **0**,**96**, con trỏ $\psi_2(\text{V}) = \text{V}$.
  - Điểm đến trạng thái UV_2: Từ V_1: $\delta_1(\text{V}) - C_{\text{v}\to\text{uv}} = 0{,}50 - 0{,}14 = 0{,}36$. Từ UV_1: $\delta_1(\text{UV}) - C_{\text{uv}\to\text{uv}} = 0{,}45 - 0{,}00 = 0{,}45$. Chọn cực đại: $\max(0{,}36,\, 0{,}45) = 0{,}45$ (đến từ UV_1). $\Longrightarrow$ $\delta_2(\text{UV}) = s_2(\text{UV}) + 0{,}45 = 0{,}45 + 0{,}45 =$ **0**,**90**, con trỏ $\psi_2(\text{UV}) = \text{UV}$.

  Tại Khung 3 ($t = 3$):

  - Điểm đến trạng thái V_3: Từ V_2: $\delta_2(\text{V}) - C_{\text{v}\to\text{v}} = 0{,}96 - 0{,}02 = 0{,}94$. Từ UV_2: $\delta_2(\text{UV}) - C_{\text{uv}\to\text{v}} = 0{,}90 - 0{,}14 = 0{,}76$. $\Longrightarrow$ $\delta_3(\text{V}) = s_3(\text{V}) + \max(0{,}94,\, 0{,}76) = 0{,}49 + 0{,}94 =$ **1**,**43**, con trỏ $\psi_3(\text{V}) = \text{V}$.
  - Điểm đến trạng thái UV_3: Từ V_2: $\delta_2(\text{V}) - C_{\text{v}\to\text{uv}} = 0{,}96 - 0{,}14 = 0{,}82$. Từ UV_2: $\delta_2(\text{UV}) - C_{\text{uv}\to\text{uv}} = 0{,}90 - 0{,}00 = 0{,}90$. $\Longrightarrow$ $\delta_3(\text{UV}) = s_3(\text{UV}) + \max(0{,}82,\, 0{,}90) = 0{,}45 + 0{,}90 =$ **1**,**35**, con trỏ $\psi_3(\text{UV}) = \text{UV}$.

  Pha truy hồi ngược: Tại khung cuối $t = 3$, so sánh hai điểm tích lũy:

  $$\delta_3(\text{V}) = 1{,}43 > \delta_3(\text{UV}) = 1{,}35 \implies j_3^* = \text{V}.$$

Lần ngược theo con trỏ: $j_2^* = \psi_3(\text{V}) = \text{V}$; $j_1^* = \psi_2(\text{V}) = \text{V}$. Đường tối ưu được chọn là $\{\text{V}_1 \to \text{V}_2 \to \text{V}_3\}$ với tổng điểm $1{,}43$.

Phân bổ kiến trúc hệ thống: Fabric FPGA và HPS ARM Cortex-A9. Trên chip vi mạch Cyclone V SX của kit DE10-Standard, kiến trúc được phân định rạch ròi:

- **Fabric FPGA:** Đảm trách toàn bộ đường xử lý tín hiệu thông lượng cao theo thời gian thực (streaming datapath từ Chương 2 đến 10) gồm bộ lọc IIR, FIR decimation, bộ đệm khung trượt M10K, khối nhân dồn MAC tính tự tương quan, bộ chuẩn hóa ROM và bộ chia nội suy parabol. Chế độ mặc định sử dụng bộ lọc Median-3 thời gian thực với độ trễ chỉ 10 ms, đạt hệ số RTF $= 0{,}242 \ll 1{,}0$.
- **HPS (Hard Processor System):** Nếu ứng dụng cần phân tích âm học ngôn ngữ chuẩn mực phòng thí nghiệm với đầy đủ 15 ứng viên và duyệt Viterbi toàn câu nói, luồng dữ liệu ứng viên từ FPGA sẽ được DMA đẩy lên bộ nhớ DDR3 của bộ xử lý ARM Cortex-A9 kép chạy Linux để thực thi hàm `Pitch_pathFinder` nguyên bản của Boersma.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 12 Viterbi.

- **Quyết định thiết kế cốt lõi: Tắt Viterbi trên fabric FPGA streaming**.
- **Lý do kỹ thuật:** Triển khai Viterbi trên fabric đòi hỏi hàng chục khối M10K để lưu bảng con trỏ và làm trễ tín hiệu hàng giây, vi phạm hoàn toàn tiêu chí thời gian thực biên (Edge Real-Time).
- **Phương án trên fabric:** Dùng khối lọc trung vị 3 điểm Median-3 (chỉ tốn 25 ALMs, 0 DSP, 0 M10K) hoặc xuất trực tiếp cặp giá trị $\{F_0,\, \text{is_voiced}\}$ ra thanh ghi giao tiếp.

Bài tập 1.12. Giả sử tại Khung 2, sức mạnh của ứng viên hữu thanh bị nhiễu làm sụt giảm nghiêm trọng xuống mức $s_2(\mathrm{V}) = 0{,}20$, trong khi các thông số khác giữ nguyên như ví dụ tính tay trên. Tính lại $\delta_2(\mathrm{V})$, $\delta_2(\mathrm{UV})$, $\delta_3(\mathrm{V})$, $\delta_3(\mathrm{UV})$ và cho biết đường tối ưu toàn cục có còn giữ trạng thái Hữu thanh liên tục hay không.

Lời giải.

- Tại Khung 2: $\delta_2(\mathrm{V}) = s_2(\mathrm{V}) + \max(0{,}50 - 0{,}02,\, 0{,}45 - 0{,}14) = 0{,}20 + 0{,}48 = 0{,}68$ (đến từ V_1). $\delta_2(\mathrm{UV}) = 0{,}45 + 0{,}45 = 0{,}90$ (đến từ UV_1).
- Tại Khung 3: $\delta_3(\mathrm{V}) = 0{,}49 + \max\big(0{,}68 - 0{,}02,\, 0{,}90 - 0{,}14\big) = 0{,}49 + \max(0{,}66,\, 0{,}76) = 0{,}49 + 0{,}76 = 1{,}25$ (đến từ UV_2!). $\delta_3(\mathrm{UV}) = 0{,}45 + \max\big(0{,}68 - 0{,}14,\, 0{,}90 - 0{,}00\big) = 0{,}45 + \max(0{,}54,\, 0{,}90) = 0{,}45 + 0{,}90 = 1{,}35$ (đến từ UV_2).

Tại khung 3: $\delta_3(\mathrm{UV}) = 1{,}35 > \delta_3(\mathrm{V}) = 1{,}25 \implies$ trạng thái kết thúc là UV_3. Truy hồi ngược: $\psi_3(\mathrm{UV}) = \mathrm{UV}_2$, $\psi_2(\mathrm{UV}) = \mathrm{UV}_1$. Đường tối ưu toàn cục chuyển hoàn toàn thành $\{\mathrm{UV}_1 \to \mathrm{UV}_2 \to \mathrm{UV}_3\}$. Khi một mắt xích bị sụt giảm quá sâu dưới ngưỡng, Viterbi tự động tái định tuyến toàn bộ để tránh một khung hữu thanh đơn lẻ bị cô lập.

1.13. Bảng giữ / xấp xỉ / xóa trên FPGA

Trước khi mở Quartus, kỹ sư phải biết khối nào *không* có trên fabric — tránh tìm IP không tồn tại và tránh im lặng lệch Praat rồi đổ lỗi twin.

Giữ (gần bit-exact)	**Xấp xỉ**	**Xóa trên fabric**
L=640, H=160, Hanning	Silence: IIR/max-hold thay `globalPeak` cả file	Soft upsample FFT cả utterance (Bước 1)
Mean/DC-blocker, MAC ACF	K= 1 thay 14 ứng viên	Sinc/Brent (Bước 8)
ROM $C[\tau]$, parabolic, f_s/τ	Median-3 thay Viterbi	`Pitch_pathFinder` đầy đủ (Bước 12)
Voicing gate, đóng gói Q12.4	—	CC method (`to_Pitch_cc`)

Mỗi ô "xóa/xấp xỉ" đã có lý do chi phí ở mục silicon tương ứng. Ngân sách sai số lab: $|\Delta F| < 20$ cent trên sine sạch so Praat (Chương 13). Mỗi bước xóa phải ghi trong báo cáo kiểm chứng — không im lặng.

CHI PHÍ (LUT / DSP / chu kỳ)

Vì sao bảng này kết thúc Chương 1? Để bản đồ "giấy Boersma → C++ → silicon" khép lại trước khi Chương 2 mở PLL và I^{2}S. Từ đây trở đi, mỗi khối phần cứng chỉ còn việc hiện thực những ô "Giữ" và "Xấp xỉ".

1.14. Mười tham số `Sound_to_Pitch` — ý nghĩa vật lý từng cái

Tham số	Default	Ý nghĩa vật lý / thiết kế một câu
time step	$0 \to$ auto	Khoảng giữa hai đo; auto $= 0{,}75/F_{\text{min}}$
pitch floor F_{min}	75 Hz	Chu kỳ dài nhất $\Rightarrow L$ và τ_{max}
very accurate	off	Hanning/SINC70 vs Gaussian/SINC700
silence threshold	0,03	Tỉ local/global dưới mức $\to$ nghỉ lặng
voicing threshold	0,45	Sàn strength cho ứng viên UV
octave cost	0,01	Phạt $\log_2$ thiên vị $F_0/2$
octave jump cost	0,35	Phạt $\lvert\log_2 f_1/f_2\rvert$ giữa khung
voiced/unvoiced cost	0,14	Phạt đổi trạng thái V/UV
ceiling	600 Hz	F_{max} + mốc octave cost
maxn candidates	15	14 đỉnh + UV

1.15. Tóm tắt ký hiệu dùng xuyên suốt các chương sau

Ký hiệu	Đơn vị / kiểu	Định nghĩa
f_s	Hz	Tần số lấy mẫu sau hạ mẫu (16 kHz trên DE10)
F_0	Hz	Cao độ $= 1/T_0$ dây thanh
T_0, τ_0	s hoặc mẫu	Chu kỳ cơ bản
τ	mẫu	Trễ (lag) trong tự tương quan
$\mathcal{R}_x[\tau]$	năng lượng	$\sum x[n]x[n+\tau]$ (không vòng)
$r_a[\tau]$	1	Autocorr chuẩn hóa Boersma
$C[\tau]$	1	$\mathcal{R}_w[0]/\mathcal{R}_w[\tau]$
L, H	mẫu	Độ dài cửa sổ và hop khung
Q$m.n$	—	Fixed-point: m bit nguyên (kể dấu), n fraction

1.16. Bài tập tổng hợp và Lời giải chi tiết

Bài tập T1 (Chuỗi số từ mean đến r_a). Cho $x = [3, 5, 3, 5]$ và cửa sổ Praat $N = 4$ đã tính

$$w \approx [0{,}3455,\ 0{,}9045,\ 0{,}9045,\ 0{,}3455], \qquad C[2] \approx \mathcal{R}_w[0]/\mathcal{R}_w[2].$$

a) Trừ mean, nhân cửa sổ, tính $\mathcal{R}_{\tilde{x}}[0]$ và $\mathcal{R}_{\tilde{x}}[2]$.

b) Tính $C[2]$ rồi $r_a[2] = (\mathcal{R}[2]/\mathcal{R}[0]) \cdot C[2]$. Nhận xét so với tỉ thô $0{,}5$.

Lời giải giải thích.

a) $x_{\text{zm}} = [-1, 1, -1, 1]$; $\tilde{x}[i] = x_{\text{zm}}[i-1] \cdot w[i]$ (chỉ số $i = 1..4$):

$$\tilde{x} \approx [-0{,}3455,\ 0{,}9045,\ -0{,}9045,\ 0{,}3455].$$

Vì $|x_{\text{zm}}| = 1$ mọi mẫu nên $\mathcal{R}[0] = \sum \tilde{x}^2 = \mathcal{R}_w[0] = 1{,}875$.

$$\mathcal{R}[2] = (-0{,}3455)(-0{,}9045) + (0{,}9045)(0{,}3455) = 2 \cdot 0{,}3455 \cdot 0{,}9045 \approx 0{,}6249.$$

b) $\mathcal{R}_w[2] = w_1 w_3 + w_2 w_4 \approx 0{,}6249$, nên $C[2] = 1{,}875/0{,}6249 \approx 3{,}000$.

$$r_a[2] = (0{,}6249/1{,}875) \cdot 3{,}000 = 0{,}3333 \cdot 3 = 1{,}000.$$

Chuẩn hóa Boersma kéo tỉ số về đúng 1 trên tín hiệu tuần hoàn sau cửa sổ — đó là lý do bắt buộc có bảng $C[\tau]$ trên ROM.

Bài tập T2 (Ngân sách RTF và quyết định giữ/xóa). Tại 50 MHz, hop 10 ms, MAC 120960 chu kỳ. Giả sử thêm SINC70 cho 1 đỉnh (70 MAC) và Viterbi soft trên HPS (không đụng fabric).

a) RTF fabric sau khi thêm SINC70 còn bao nhiêu?

b) Vì sao tài liệu vẫn khuyến cáo *không* đưa SINC70 vào fabric mặc định?

Lời giải giải thích.

a) Tổng chu kỳ $\approx 120960 + 70 = 121030$; $T_{\text{compute}} = 121030/(50 \cdot 10^6) = 2{,}4206$ ms; RTF $= 0{,}24206$ — gần như không đổi.

b) Lý do không phải RTF mà là độ phức tạp kiểm chứng, FSM địa chỉ, và lợi ích biên nhỏ trên speech thật so với parabolic (đã bàn ở Bài tập 1.8). Nguyên tắc: mỗi khối thêm phải chứng minh cải thiện sai số cent trên bộ kiểm thử Chương 13, không chỉ "còn chỗ trên chip".

Bài tập T3 (Checklist Feynman trước khi cắt bước Praat). Với mỗi bước trong {1, 8, 12}, trả lời năm câu: (1) Bảo vệ khỏi hiện tượng vật lý nào? (2) Sai số khi xóa trên sine? trên speech? (3) Silicon: LUT/DSP/M10K/latency? (4) Twin còn so được Praat không? (5) Có đẩy HPS thay vì xóa hoàn toàn không?

Lời giải giải thích (tóm tắt).

- **Bước 1:** Bảo vệ khỏi rò phổ mép Nyquist; xóa trên fabric thay bằng FIR — sai số kiểm soát được nếu stopband FIR đủ sâu; twin offline vẫn soft-upsample được; không cần HPS real-time.
- **Bước 8:** Bảo vệ khỏi bias lưới mẫu; xóa $\rightarrow$ parabolic: lệch vài cent trên sine, thường nhỏ hơn nhiễu speech; 0 DSP thêm; twin parabolic khớp RTL; HPS optional Sinc.
- **Bước 12:** Bảo vệ khỏi nhảy octave giữa khung; xóa $\rightarrow$ median-3: mất tối ưu đường dài nhưng đủ cho lab; $<$ 200 ALM; twin median-3 khớp fabric, Viterbi soft trên HPS khi cần "Praat đầy đủ".

▶ NHÌN THẤY VẬT LÝ

Sai một công thức ở Chương 1 thì không một khối DSP hay bộ nhớ M10K nào ở các chương sau có thể cứu vãn được hệ thống. Hãy chạy lại toàn bộ `verify/verify_chuong_00_01.py` trước khi sang Chương 2.

Chương 2. PLL, MCLK, I²S và CDC mã Gray

▶ MỤC TIÊU HỌC TẬP

Sau Chương 2, người học tạo được đúng xung nhịp âm thanh từ thạch anh 50 MHz trên DE10-Standard, thu mẫu PCM từ codec WM8731 qua I²S, và đưa mẫu sang miền đồng hồ xử lý 50 MHz bằng DCFIFO mã Gray mà không gây metastability. Các khái niệm PLL, MCLK, I²S, DCFIFO và mã Gray đã được định nghĩa trực giác ở mục 0.4 (ô Feynman 8) của Chương 0 — chương này đi sâu vào cấu hình, tính tay, nội tại IP, RTL từng dòng và twin Python.

2.1. Từ thạch anh 50 MHz đến MCLK 12,288 MHz

▶ NHÌN THẤY VẬT LÝ

Thạch anh trên bo mạch DE10 dao động cơ–điện rất ổn định (sai số thường chỉ vài phần triệu — ppm). Chip codec **Wolfson WM8731** không dùng trực tiếp 50 MHz: nó cần một xung chủ riêng gọi là **MCLK** đúng 12,288 MHz (bằng 256×48 kHz) để chạy bộ lọc oversampling và ADC delta-sigma bên trong. Nếu MCLK lệch vài chục ppm so với giá trị danh định, chu kỳ lấy mẫu sẽ trôi chậm theo thời gian; thuật toán F_0 vẫn "đúng công thức" nhưng đo trên một trục thời gian bị kéo giãn, tạo ra sai số cao độ giả. Ví dụ đời thường: một đồng hồ đeo tay chạy nhanh 50 ppm sẽ lệch khoảng 4,3 giây mỗi ngày — với audio 16 kHz, cùng tỷ lệ đó làm trôi 48 mẫu mỗi phút, đủ để lệch một phần bước nhảy khung $H = 160$.

Khối **PLL** (*Phase-Locked Loop* — vòng khóa pha) trên Cyclone V tạo ra MCLK bằng cách *khóa* một dao động nội chip vào thạch anh ngoài. Trong vòng đó có ba khối vật lý cần gọi đúng tên. **VCO** (*Voltage-Controlled Oscillator* — dao động điều khiển bằng điện áp) là mạch LC/ring trên silicon: điện áp vào cao hơn thì tần số ra cao hơn, giống vặn núm radio nhưng bằng volt. **PFD** (*Phase–Frequency Detector* — bộ so pha–tần số) không phải phép trừ số trên giấy: mỗi lần cạnh xung tham chiếu (f_{in}) và cạnh feedback (sau chia M) lệch nhau, PFD mở một khóa bơm điện tích (*charge pump*) vào **loop filter** (tụ và điện trở lọc). Điện áp trên tụ kéo VCO tới khi hai cạnh khớp — đó mới gọi là *khóa*. Chỉ khi đã khóa, bộ chia C mới cho ra MCLK sạch. Counter mềm bằng flip-flop không đạt jitter mức audio (jitter lớn làm méo bit I²S và làm ADC delta-sigma "run" không đều); vì thế trên Quartus Standard/Lite bắt buộc dùng IP **Altera PLL** (ALTPLL / PLL Intel FPGA IP), không tự viết PLL bằng RTL.

Toán từng bước — integer-N. Trong chế độ integer-N mọi hệ số đều nguyên:

$$f_{\text{VCO}} = f_{\text{in}} \cdot \frac{M}{N}, \qquad f_{\text{out}} = \frac{f_{\text{VCO}}}{C} = f_{\text{in}} \cdot \frac{M}{N \cdot C}.$$

Với $f_{\text{in}} = 50$ MHz và $f_{\text{out}} = 12{,}288$ MHz:

$$\frac{M}{N \cdot C} = \frac{12{,}288}{50} = 0{,}24576.$$

Tỷ số $0{,}24576$ không phân tích thành bộ (M, N, C) nguyên nhỏ nằm trong dải VCO Cyclone V (600–1600 MHz) với sai số đúng 0 ppm. Có thể tìm bộ nguyên lớn cho sai số khoảng 0,2 ppm — ví dụ $M = 1507$, $N = 73$, $C = 84$:

$$f_{\text{VCO}} = 50 \cdot \frac{1507}{73} \approx 1032{,}19178 \text{ MHz} \quad \text{(nằm trong 600–1600)},$$
$$f_{\text{out}} = \frac{f_{\text{VCO}}}{84} = 50 \cdot \frac{1507}{73 \cdot 84} \approx 12{,}28799739 \text{ MHz}.$$

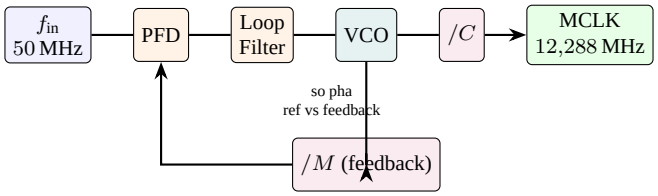
Sai số tuyệt đối so với 12,288 MHz là khoảng 2,61 Hz, tương đương

$$\varepsilon_{\text{ppm}} = \frac{|f_{\text{out}} - 12{,}288|}{12{,}288} \cdot 10^6 \approx 0{,}212 \text{ ppm}.$$

Fractional PLL trên DE10. MegaWizard/IP Catalog cho phép đặt ngõ ra đúng 12,288 MHz; phần mềm chọn hệ số phân số (fractional-M hoặc tương đương tùy family) và ghi M, N, C vào báo cáo Compilation. Một nghiệm phân số minh họa trên giấy: $N = 5$, $M = 61{,}44$, $C = 50$ cho

$$f_{\text{VCO}} = 50 \cdot \frac{61{,}44}{5} = 614{,}4 \text{ MHz}, \qquad f_{\text{out}} = \frac{614{,}4}{50} = 12{,}288 \text{ MHz}$$

đúng tuyệt đối theo số học. Trên silicon, "$61{,}44$" được hiện thực bằng bộ đếm phân số trung bình theo thời gian; jitter dư rất nhỏ so với yêu cầu audio nếu loop filter được cấu hình đúng trong IP.



Hình 19: Luồng tín hiệu trong PLL: PFD so sánh cạnh tham chiếu với feedback sau chia M; khi khóa, chia C cho ra MCLK dùng cho codec.

Ví dụ tính tay — trôi mẫu theo ppm. Giả sử sai số MCLK là $\varepsilon = 50\,\text{ppm} = 50 \cdot 10^{-6}$. Trong $T = 60\,\text{s}$ ở tần số lấy mẫu xử lý $f_s = 16\,\text{kHz}$, số mẫu bị trôi tuyệt đối là

$$\Delta N = \varepsilon \cdot T \cdot f_s = 50 \cdot 10^{-6} \cdot 60 \cdot 16000 = 48 \text{ mẫu.}$$

Bốn mươi tám mẫu ở 16 kHz tương đương 3 ms — đủ để lệch một phần đáng kể của bước nhảy khung $H = 160$ mẫu (10 ms). Với sai số integer-N ở trên ($\approx 0{,}212$ ppm):

$$\Delta N \approx 0{,}212 \cdot 10^{-6} \cdot 60 \cdot 16000 \approx 0{,}204 \text{ mẫu/phút,}$$

không đáng kể so với 48. Mục tiêu lab: giữ sai số audio dưới 5 ppm ($\Delta N \approx 4{,}8$ mẫu/phút), và luôn chờ tín hiệu `locked` của PLL trước khi bật FSM I^{2}S. Nếu bỏ qua `locked`, vài trăm miligiây đầu sau reset MCLK chưa ổn định — codec ghi rác vào FIFO và đỉnh tự tương quan nhảy lung tung.

IP Altera PLL — tham số thật. Trong IP Catalog chọn PLL Intel FPGA IP (hoặc ALTPLL trên tool cũ): Reference Clock 50 MHz; Output $c0 = 12{,}288$ MHz (đưa chân MCLK codec); có thể thêm $c1 = 50$ MHz hệ thống nếu muốn cùng nguồn khóa pha với audio (giảm drift tương đối giữa datapath và MCLK). Bật cổng `locked`; nối `areset` về nút reset toàn cục đã đồng bộ. Báo cáo PLL List ghi rõ M, N, C, f_{VCO}, ppm residual — lab phải chép các số này vào sổ, không đoán.

RTL và twin. RTL chỉ được *chia* sau khi đã có MCLK (ví dụ tạo SCLK $=$ MCLK$/4$ cho frame 64 bit), không thay VCO bằng counter. Mô hình twin Python không mô phỏng jitter analog; nó giả lập trôi thời gian lấy mẫu

$$t_n = \frac{n}{f_s}\left(1 + \varepsilon_{\text{ppm}} \cdot 10^{-6}\right)$$

rồi nội suy sóng hình sin F_0 biết trước lên lưới t_n lệch và quan sát độ lệch cao độ ước lượng khi $\varepsilon \neq 0$. Script `verify/verify_chuong_02.py` kiểm mọi số ppm/drift nêu trong chương.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — PLL (mọi viết tắt).

- **PLL** cứng: 1 / 6 trên Cyclone V 5CSXFC6D6F31C6N ($\approx 16{,}7\%$ ngân sách PLL bảng Chương 0).
- **ALM** (*Adaptive Logic Module* — ô logic cơ bản của Cyclone V, gồm LUT và flip-flop): vài ALM cho logic `locked`/reset. **LUT** (*Look-Up Table*) là bảng chân lý nhỏ trong ALM.
- **DSP** (*Digital Signal Processing* block — khối nhân cứng 18×19): 0.
- **M10K** (block RAM ≈ 10 kbit): 0. **MLAB** (*Memory Logic Array Block* — RAM nhỏ trong LAB): 0.
- **Fmax** (*maximum frequency* — tần số đồng hồ lớn nhất vẫn đạt timing): không phải nút thắt (PLL cứng chạy VCO hàng trăm MHz).
- **Latency**: thời gian từ lúc cấp clock đến khi `locked`=1 (hàng trăm μs đến vài ms tùy IP) — FSM I^{2}S phải chờ.
- **RTF** (*real-time factor* = thời gian xử lý / thời gian audio): khối PLL không đếm RTF theo mẫu; nó chỉ cung cấp nhịp.
- Vì sao bắt buộc IP: soft-counter không đạt ppm/jitter của VCO silicon cho MCLK audio.

2.2. Giao thức I²S với codec WM8731

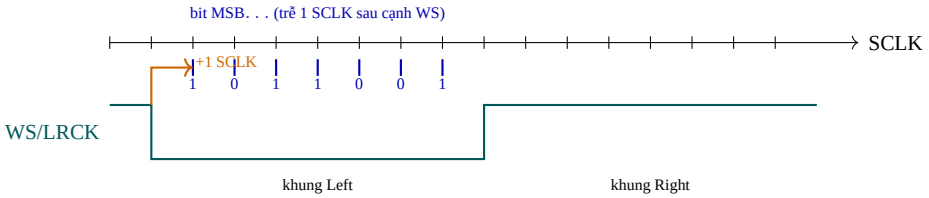
▶ NHÌN THẤY VẬT LÝ

Sau khi ADC trong WM8731 đã lượng tử hóa sóng âm, dữ liệu không nhảy sang FPGA dưới dạng mảng số nguyên song song, mà đi trên ba dây nối tiếp theo chuẩn **I²S** (*Inter-IC Sound* — đã giới thiệu ở Chương 0): dây **SCLK**/BCLK chốt từng bit, dây **WS**/LRCK chọn kênh trái/phải, dây **SDAT** mang dòng bit. Đây là quy ước giao diện điện tử, không phải hiện tượng dây thanh; nhưng đếm sai một bit hoặc trễ sai một cạnh SCLK sẽ làm hỏng mẫu Q1.15 trước khi tín hiệu kịp vào DC-blocker. Cấu hình thanh ghi codec (định dạng I²S, độ dài word, master/slave clock) đi qua bus **I²C** riêng — lab phải khởi tạo I²C trước khi kỳ vọng SDAT có nghĩa.

Quan hệ đồng hồ. Với cấu hình stereo phổ biến trên DE10 (mỗi slot 32 bit, hai kênh = 64 bit/frame) tại $f_s = 48\,\text{kHz}$:

$$\text{MCLK} = 256 \cdot f_s = 256 \cdot 48000 = 12{,}288{,}000\,\text{Hz},$$
$$\text{SCLK} = 64 \cdot f_s = 64 \cdot 48000 = 3{,}072{,}000\,\text{Hz}.$$

Nếu lab chỉ lấy 16 bit/kênh (32 bit/frame tổng cộng L+R) thì SCLK $= 32 \cdot f_s = 1{,}536{,}000$ Hz. Chuẩn Philips I²S: MSB đi trước, dữ liệu hợp lệ bắt đầu sau đúng một cạnh SCLK kể từ cạnh đổi của WS. Trên đường ước lượng F_0 mono, FSM chỉ chốt kênh Left rồi bỏ Right — đủ thông tin cao độ, tiết kiệm băng thông FIFO.



Hình 20: Timing I²S đơn giản hóa: cạnh WS mở khung Left; dữ liệu trên SDAT hợp lệ bắt đầu sau đúng một cạnh SCLK (chuẩn Philips).

Ví dụ tính tay — đóng gói nibble rồi word Q1.15. Bốn cạnh SCLK liên tiếp nhận các bit $1, 0, 1, 1$ (MSB trước). Thanh ghi dịch trái lần lượt:

$$0 \to 1 \to 2 \to 5 \to 11 \quad (0\text{b}1011 = 11_{10}).$$

Mười sáu bit MSB của slot Left mang mẫu có dấu. Ví dụ bit-stream $0100\,0000\,0000\,0000$ (MSB trước) đóng thành

$$0\text{b}0100\,0000\,0000\,0000 = 16384_{10},$$

đúng mức $0{,}5$ trong Q1.15 (ô Feynman 7, Chương 0). Lab lấy 16 bit cao của slot 32 bit (bỏ 16 bit đệm thấp) rồi đẩy vào DCFIFO.

Hai lộ trình phần cứng. (1) **University Program Audio Core**: IP Avalon bọc cấu hình I²C của codec và FIFO L/R — gắn nhanh Platform Designer nhưng che mất đường mẫu, khó đối chiếu bit-exact. (2) **RTL tự viết (default dạy)**: FSM Idle→WaitWS→Skip1→Shift16→Push, đồng bộ cạnh WS sang miền SCLK bằng hai flip-flop, dịch 16 bit hữu ích, nâng `wrreq` mỗi mẫu Left. Twin phần mềm: tệp `.wav` int16 little-endian chính là đầu ra bit-exact sau khi bỏ padding slot.

RTL giải thích từng nhóm lệnh.

```
// Đồng bộ WS (async từ codec) sang miền SCLK bằng 2FF
reg ws_ff1, ws_ff2, ws_prev;
always @(posedge sclk) begin
  ws_ff1 <= ws_async;   // FF1: hấp thụ metastability
  ws_ff2 <= ws_ff1;     // FF2: bit đã ổn định
```

```
  ws_prev <= ws_ff2;    // nhớ cạnh trước để phát hiện rising
end
wire ws_rise = ws_ff2 & ~ws_prev;  // rising = start Left

// FSM rút gọn: sau ws_rise chờ 1 SCLK (Philips), rồi dịch 16 bit
// reg [4:0] bit_cnt; reg [15:0] shreg;
// khi bit_cnt==15: data_left <= {shreg[14:0], sdat}; wrreq_fifo <= 1;
```

Dòng `ws_ff1 <= ws_async` là chỗ metastability được phép xảy ra — chỉ trên *một* bit điều khiển. Dòng `ws_ff2` cho bit thời gian ổn định trước khi dùng. `ws_rise` phát hiện cạnh; sau đúng một SCLK (trễ Philips) mới lấy MSB. Khi đủ 16 bit, ghi DCFIFO phía `wrclk = sclk`. Phía `rdclk = 50 MHz` đọc ra Avalon-ST (`valid`/`ready`) sang DC-blocker.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — I²S RX (mọi viết tắt).

- **ALM/LUT**: khoảng 50–100 ALM (shift 16–32 bit + cạnh WS + FSM). 0 **DSP**, 0 **M10K/MLAB**.
- **Fmax**: miền SCLK 3,072 MHz rất dư so với silicon tốc độ 6.
- **Latency**: từ cạnh WS đến mẫu Left hợp lệ $\approx 1 + 16$ cạnh SCLK (Philips + 16 bit) $\approx 5{,}5\ \mu$s.
- **RTF**: mỗi frame audio 64 cạnh SCLK mang đúng một mẫu Left hữu ích; FSM không bỏ mẫu $\Rightarrow$ RTF thu thập $= 1$ (xử lý kịp thời gian thật).
- Vì sao mono Left: F_0 không cần stereo; stereo nhân đôi băng thông CDC mà không làm sắc đỉnh $\mathcal{R}[\tau]$.

2.3. CDC: DCFIFO bất đồng bộ và mã Gray

▶ NHÌN THẤY VẬT LÝ

Miền đồng hồ âm thanh (MCLK 12,288 MHz, SCLK 3,072 MHz) và miền xử lý lõi (50 MHz) dao động từ các nguồn độc lập — chúng **bất đồng bộ**. Nếu nối một thanh ghi 16 bit từ miền SCLK sang miền 50 MHz bằng dây thô, một số bit có thể chốt đúng lúc tín hiệu đang ở vùng điện áp "không phải 0 cũng không phải 1". Hiện tượng đó gọi là **metastability** (bán ổn định): flip-flop đích đứng lình xình ở mức giữa trong một khoảng thời gian không đoán trước được, rồi mới đổ về 0 hoặc 1 — giống quả bóng cân bằng trên đỉnh tường rồi rơi sang một bên. Hậu quả trên đường F_0 là mẫu rác đột biến, đỉnh tự tương quan nhảy lung tung, đôi khi FSM tưởng FIFO full/empty sai.

Giải pháp chuẩn là bộ đệm **DCFIFO** (*Dual-Clock FIFO* — đã nêu ở Chương 0): cổng ghi chạy theo clock nguồn (SCLK), cổng đọc chạy theo clock đích (50 MHz). Dữ liệu mẫu nằm trong RAM dual-port nội bộ; cái bắt chéo miền clock *không* phải là word dữ liệu 16 bit mà là **con trỏ** ghi/đọc. Con trỏ khi bắt chéo miền phải được mã hóa **mã Gray**: mỗi lần tăng địa chỉ chỉ đúng một bit thay đổi. Nhờ đó, dù bên kia lấy mẫu lúc con trỏ đang chuyển, nó chỉ có thể đọc nhầm tối đa một bước địa chỉ liền kề — không bao giờ đọc phải từ bit-pattern trung gian vô nghĩa của mã nhị phân thường.

Toán mã Gray. Với số nguyên không dấu n, mã Gray phản xạ là

$$G(n) = n \oplus \lfloor n/2 \rfloor = n \oplus (n \gg 1).$$

Bảng tính tay $n = 0 \ldots 7$ (3 bit):

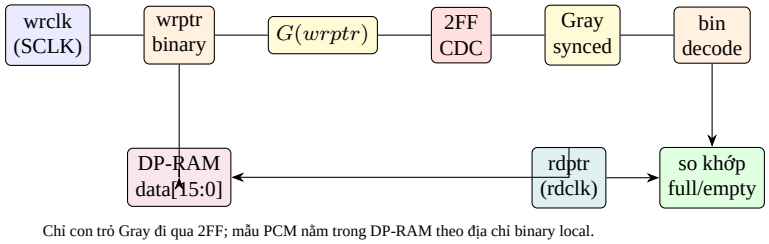
n (thập phân)	Nhị phân	Gray $G(n)$
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Hai mã Gray liên tiếp luôn khác đúng một bit: ví dụ $G(3) = 010$ và $G(4) = 110$ chỉ khác bit cao. Đảo ngược Gray→binary trên giấy: $b_2 = g_2$, $b_1 = g_2 \oplus g_1$, $b_0 = g_2 \oplus g_1 \oplus g_0$.

Nội tại DCFIFO Intel — đồng bộ con trỏ. IP FIFO (dcfifo) thực hiện đúng chuỗi sau mỗi lần ghi/đọc:

1. Tăng con trỏ nhị phân phía local clock ($wrptr$ hoặc $rdptr$).
2. Đổi sang Gray: $wrptr_gray = G(wrptr)$.
3. Đưa $wrptr_gray$ qua chuỗi **hai flip-flop** sang miền clock bên kia (đúng chuẩn 2FF CDC cho từng bit Gray — vì mỗi bit Gray chỉ đổi đơn lẻ, metastability trên một bit không kéo theo bit khác).
4. Ở miền đích, đổi Gray→binary rồi so với con trỏ local để tính `wrfull`/`rdempty`/`rdusedw`.

Word dữ liệu 16 bit *không* đi qua 2FF: nó nằm trong dual-port RAM, được ghi theo địa chỉ binary phía write và đọc theo địa chỉ binary phía read; chỉ có "vé địa chỉ" (con trỏ Gray) là tín hiệu bất đồng bộ cần đồng bộ hóa.



Hình 21: Luồng CDC trong DCFIFO: mã hóa Gray → đồng bộ 2FF từng bit → giải mã binary → so full/empty; dữ liệu mẫu không bắt chéo thô.

Ví dụ tính tay — vì sao nhị phân thường nguy hiểm. Con trỏ nhị phân từ 3 (011) lên 4 (100) đổi *cả ba bit* cùng lúc. Bên đọc bắt chéo miền có thể chốt thành 000, 111, 001,…— địa chỉ giả cách xa hàng đợi thật, đọc nhầm mẫu cũ hoặc đè cờ full/empty. Với Gray, bước $010 \to 110$ chỉ một bit; sai lệch tối đa là ± 1 ô.

Ví dụ tính tay — chọn depth. Tốc độ ghi mono $= 48$ kHz. Giả sử jitter tương đối và burst đọc phía hệ thống tạo độ trễ hấp thụ tối đa $100\,\mu$s. Số mẫu cần đệm:

$$N_{\text{depth}} \geq 48000 \cdot 100 \cdot 10^{-6} = 4{,}8 \quad \Rightarrow \quad \text{chọn depth } 8 \text{ hoặc } 16.$$

Depth 8 từ $\times 16$ bit $= 128$ bit — vừa một MLAB; không cần M10K. Lab mặc định depth 16 để dư margin khi FSM phía đọc thỉnh thoảng `ready`=0.

Twin Python (mô hình logic, không mô phỏng metastability analog).

```
def bin_to_gray(n):
    return n ^ (n >> 1)

# Kiểm bảng 0..7 và tính chất một bit
grays = [bin_to_gray(i) for i in range(8)]
assert [format(g, "03b") for g in grays] == [
```

```
    "000","001","011","010","110","111","101","100"]
for a, b in zip(grays, grays[1:]):
    assert bin(a ^ b).count("1") == 1
```

Twin mức hệ thống: hàng đợi `collections.deque` với hai "clock tick" độc lập; assert không đọc khi empty và không ghi khi full — tương ứng cờ IP.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — CDC / DCFIFO (mọi viết tắt).

- **MLAB/M10K**: depth 8–16 từ $\times 16$ bit $=$ 128–256 bit — thường 1 MLAB (hoặc mảnh M10K nếu tool gộp).
- **ALM/LUT**: vài chục ALM cho Gray + full/empty + chuỗi 2FF. 0 **DSP**.
- **Latency** CDC: khoảng 2–4 chu kỳ clock đích (50 MHz) $\approx$ 40–80 ns — so với hop 10 ms (500 000 chu kỳ) là $< 0{,}001\%$.
- **Fmax**: phía `rdclk`=50 MHz dễ đạt; phía `wrclk`=3,072 MHz cũng dư.
- **RTF**: FIFO không "tính toán" mẫu; nó chỉ chuyển miền. RTF bị phá khi `wrfull`=1 (mất mẫu) — depth 16 chính là bảo hiểm RTF$= 1$.
- Vì sao không 2FF cho cả word 16 bit: 2FF chỉ an toàn từng bit điều khiển; word cần FIFO + Gray để mẫu nhất quán theo thứ tự ghi.

2.4. Chi phí tổng hợp và chỗ đứng trong đường tín hiệu

Khối Chương 2 nằm ngay sau codec WM8731 và ngay trước DC-blocker trên bản đồ đường tín hiệu Chương 0: thạch anh $\rightarrow$ PLL $\rightarrow$ MCLK/SCLK $\rightarrow$ I^{2}S RX $\rightarrow$ DCFIFO $\rightarrow$ Avalon-ST 50 MHz. Nhiệm vụ của cả cụm là *đưa đúng một mẫu PCM mỗi chu kỳ lấy mẫu sang miền xử lý*, không phải tự ước lượng F_0. Vì vậy ngân sách silicon phải cố tình rẻ: dành gần như toàn bộ **DSP** và phần lớn **M10K** cho FIR (Chương 4) và MAC tự tương quan (Chương 7).

► NHÌN THẤY VẬT LÝ

Gom ba ô chi phí mục 2.1–2.3 lại thành một bức tranh. **ALM** (*Adaptive Logic Module*) là "gạch" logic của Cyclone V; mỗi ALM chứa vài **LUT** (*Look-Up Table*) và flip-flop. **DSP** là khối nhân cứng. **M10K** là block RAM khoảng 10 240 bit; **MLAB** là RAM nhỏ hơn nằm trong logic array. **Fmax** là tần số đồng hồ lớn nhất còn thỏa timing. **Latency** là trễ mẫu/chu kỳ từ vào đến ra. **RTF** (*real-time factor*) $=$ thời gian xử lý / thời gian audio: RTF≤ 1 nghĩa là kịp thời gian thật, không bỏ mẫu.

Tổng hợp số liệu lab. Trên 5CSXFC6D6F31C6N có khoảng 41 910 ALM và 6 PLL cứng. Cụm Chương 2 tiêu thụ:

- 1 PLL Fractional (MCLK 12,288 MHz),
- $<$ 150 ALM cho I^{2}S RX + Gray/2FF/cờ FIFO (tức $< 0{,}4\%$ ALM chip),
- 0 DSP,
- tối đa 1 MLAB (hoặc mảnh M10K) cho DCFIFO depth 8–16.

Fmax phía `sysclk`=50 MHz dễ vượt 150 MHz trên silicon tốc độ 6 — điểm nghẽn *không* phải diện tích hay Fmax, mà là kỷ luật CDC và việc AND tín hiệu `locked` vào enable FSM.

Ví dụ tính tay — RTF và phần trăm hop. Giả sử phía đọc DCFIFO luôn `ready`=1. Mỗi mẫu Left vào ở 48 kHz đi ra đúng một mẫu trên Avalon-ST 50 MHz: thời gian audio mỗi mẫu là $1/48000 \approx 20{,}833\,\mu$s, trong khi latency CDC chỉ khoảng 3 chu kỳ $\times 20$ ns $= 60$ ns. Tỷ số

$$\mathrm{RTF}_{\mathrm{CDC}} = \frac{60 \cdot 10^{-9}}{20{,}833 \cdot 10^{-6}} \approx 0{,}0029 \ll 1.$$

Trong một hop xử lý 10 ms ở 50 MHz có $500\,000$ chu kỳ; 3 chu kỳ CDC chiếm

$$\frac{3}{500\,000} = 6 \cdot 10^{-6}$$

(tức $0{,}0006\%$) — không bao giờ là lý do trễ F_0. Ngược lại, nếu FSM quên `locked` hoặc depth FIFO quá nông khiến `wrfull`=1, RTF "vỡ" theo nghĩa mất mẫu: một mẫu mất ở đây không sửa được bằng thêm DSP ở Chương 7.

Vì sao thiết kế này khớp bài toán. Ước lượng F_0 cần MAC hàng trăm nghìn phép nhân-cộng mỗi hop; cụm thu thập chỉ cần *không làm hỏng mẫu*. Giữ < 150 ALM và 0 DSP ở Chương 2 là lựa chọn có chủ đích, không phải "chưa tối ưu". Lab nghiệm thu cụm: (1) PLL List ghi đúng $12{,}288$ MHz và `locked`=1 trước khi thu; (2) Fitter báo 0 DSP cho khối này; (3) không có warning timing CDC xuyên miền thô; (4) script `verify_chuong_02.py` xanh trên mọi số ppm/Gray/SCLK.

2.5. Bài tập và lời giải giải thích

Bài tập 2.1 (Tỷ số PLL và sai số ppm). Từ thạch anh $f_{\text{in}} = 50$ MHz, kỹ sư cần MCLK $= 12{,}288$ MHz.

a) Tính tỷ số $M/(N \cdot C)$ lý thuyết.

b) Nếu dùng integer-N với $M = 1507$, $N = 73$, $C = 84$, tần số thực tế là bao nhiêu và sai số so với $12{,}288$ MHz tính bằng ppm?

c) Trong 60 giây ở 16 kHz, sai số đó làm trôi bao nhiêu mẫu? So với trường hợp 50 ppm thuần.

Lời giải giải thích.

a) $M/(N \cdot C) = 12{,}288/50 = 0{,}24576$.

b)

$$f_{\text{out}} = 50 \cdot \frac{1507}{73 \cdot 84} \approx 12{,}28799739\,\text{MHz}, \quad \varepsilon_{\text{ppm}} \approx 0{,}212.$$

(Script `verify_chuong_02.py`.)

c) $\Delta N \approx 0{,}204$ mẫu/phút — không đáng kể so với 48 mẫu/phút khi sai 50 ppm. Fractional PLL vẫn tiện hơn vì MegaWizard điền đúng $12{,}288$.

Bài tập 2.2 (SCLK và nibble I²S). Codec chạy $f_s = 48$ kHz, frame stereo 64 bit.

a) Tính SCLK.

b) Dịch bốn bit $1, 0, 1, 1$ (MSB trước) — giá trị thập phân?

c) Bit-stream 16 bit $0100\,0000\,0000\,0000$ đóng thành số Q1.15 nào?

d) Vì sao lab mono Left đủ cho F_0?

Lời giải giải thích.

a) SCLK $= 64 \cdot 48000 = 3{,}072{,}000$ Hz.

b) $0\text{b}1011 = 11_{10}$.

c) 16384 — đúng mức $0{,}5$ trong Q1.15.

d) Cao độ là thuộc tính nguồn kích thích dây thanh, gần như đồng nhất trên hai kênh micro gần nhau; lấy thêm Right chỉ nhân đôi băng thông CDC.

Bài tập 2.3 (Gray và metastability). Con trỏ FIFO 3 bit tăng từ 3 lên 4.

a) Viết mã nhị phân và mã Gray của hai giá trị.

b) Giải thích tại sao bắt chéo miền clock trên mã nhị phân có thể đọc ra địa chỉ xa, còn Gray thì không.

Lời giải giải thích.

a) Nhị phân: $3 = 011$, $4 = 100$ (đổi 3 bit). Gray: $G(3) = 010$, $G(4) = 110$ (đổi 1 bit).

b) Nhiều bit đổi cùng lúc cho phép tổ hợp trung gian bất kỳ (địa chỉ xa). Gray bảo đảm mọi thời điểm trung gian chỉ sai tối đa một bước liền kề — lý do DCFIFO Intel gray-code hóa con trỏ xuyên miền.

Bài tập 2.4 (Depth FIFO và 2FF).

a) Với mono 48 kHz và jitter hấp thụ $100\,\mu$s, depth tối thiểu lý thuyết là bao nhiêu? Lab chọn 8 hay 16 thì vì sao?

b) Vì sao chuỗi 2FF áp dụng cho từng bit Gray chứ không cho cả word mẫu 16 bit?

Lời giải giải thích.

a) $48000 \cdot 100 \cdot 10^{-6} = 4{,}8$ — tối thiểu 5, chọn 8 hoặc 16 để dư margin khi `ready`=0 phía đọc. Depth 16 an toàn hơn khi FSM FIR/DC thỉnh thoảng dừng.

b) 2FF chỉ đảm bảo một bit điều khiển thoát metastability; 16 bit dữ liệu có thể mỗi bit ổn định ở giá trị khác nhau tạo word rác. FIFO giữ data trong RAM và chỉ đồng bộ con trỏ Gray.

Bài tập 2.5 (`locked` và trôi 5 ppm).

a) Tính ΔN trong 60 s ở 16 kHz nếu sai số còn 5 ppm.

b) FSM I^{2}S có được phép chạy khi `locked`=0 không? Giải thích hậu quả.

Lời giải giải thích.

a) $\Delta N = 5 \cdot 10^{-6} \cdot 60 \cdot 16000 = 4{,}8$ mẫu/phút — mục tiêu lab chấp nhận được.

b) Không. Khi chưa `locked`, MCLK lệch mạnh hoặc dừng; codec xuất bit vô nghĩa, FIFO đầy mẫu rác, đỉnh $\mathcal{R}[\tau]$ không ổn định. Phải AND `locked` vào tín hiệu enable của FSM thu.

Chương 3. DC-blocker IIR (cực/zero, accumulator 32-bit)

▶ MỤC TIÊU HỌC TẬP

Sau Chương 3, người học giải thích được vì sao offset một chiều phá đỉnh tự tương quan, viết đúng hàm truyền một cực–một zero với $\alpha = 1 - 2^{-8}$, hiện thực RTL accumulator 32 bit theo công thức $y_{\text{acc}} \leftarrow y_{\text{acc}} + x_\Delta - (y_{\text{acc}} \ggg 8)$ rồi $\text{sat}_{16}(y_{\text{acc}} \ggg 15)$, đối chiếu bit-exact với twin Python, và tính tay từng chu kỳ y_{acc}. Khái niệm điểm cực/điểm không và Avalon-ST đã có ở ô Feynman 12 của Chương 0.

3.1. Hiện tượng: offset DC nuốt đỉnh chu kỳ

▶ NHÌN THẤY VẬT LÝ

Micro, codec WM8731 và đường dây analog luôn cộng thêm một thành phần gần đứng yên: điện áp offset một chiều (DC), cộng thêm trôi chậm do nhiệt. Trong miền mẫu, đó là một hằng A cộng vào mọi $x[n]$. Phép tự tương quan (ô Feynman 5, Chương 0) không phân biệt năng lượng hữu ích và năng lượng offset. Nếu A lớn, đỉnh tại độ trễ $\tau = 0$ phình theo A^2 và có thể át hoàn toàn đỉnh chu kỳ T_0 của dây thanh — thuật toán sẽ báo vô thanh hoặc treo ở cao độ sai. Đời thường: đo chiều cao sóng trên mặt biển khi mực nước thủy triều đang dâng — nếu không trừ mực nước trung bình, "đỉnh sóng" chỉ là mực nước chứ không phải sóng.

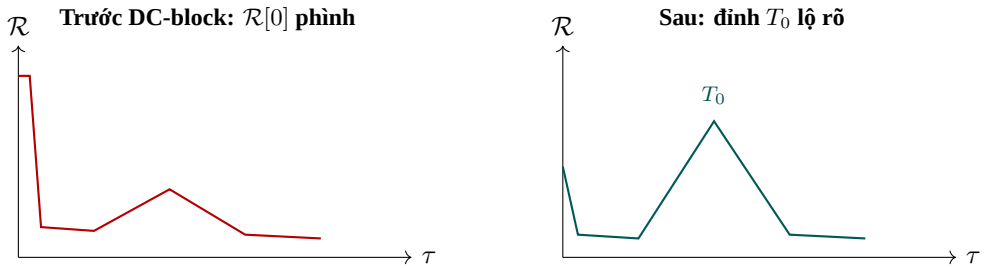
Viết tường minh với tín hiệu $x[n] + A$:

$$\begin{aligned}\mathcal{R}_{x+A}[0] &= \sum_{n=0}^{N-1} (x[n] + A)^2 \\ &= \sum_{n=0}^{N-1} x[n]^2 + 2A \sum_{n=0}^{N-1} x[n] + NA^2 = \mathcal{R}_x[0] + 2A \sum_n x[n] + NA^2.\end{aligned}$$

Số hạng NA^2 luôn dương và tỉ lệ với bình phương offset. Với độ trễ $\tau \neq 0$:

$$\mathcal{R}_{x+A}[\tau] = \mathcal{R}_x[\tau] + A \sum_n x[n] + A \sum_n x[n+\tau] + NA^2,$$

nên toàn bộ hàm tự tương quan bị nâng một "bệ" DC — đỉnh chu kỳ bị che. Praat trừ mean cục bộ từng khung; trên FPGA streaming ta dùng bộ lọc IIR liên tục gọi là **DC-blocker**: một bộ thông cao rất nhẹ, ổn định, rẻ silicon, đặt ngay sau I²S/CDC và trước FIR hạ mẫu (hoặc ngay sau CDC ở 48 kHz, tùy lab — cutoff vẫn dưới $F_{0,\text{min}}$).



Hình 22: Cùng một tín hiệu có offset: bên trái đỉnh $\tau = 0$ nuốt chu kỳ; bên phải sau DC-blocker đỉnh T_0 hiện ra để Chương 9 nội suy.

Ví dụ tính tay — DC thuần và offset cộng vào sóng. Chuỗi $x = [1, 1, 1, 1]$ (toàn DC) cho $\mathcal{R}[0] = \sum x_n^2 = 4$. Sau khi trừ mean (mọi mẫu về 0) thì $\mathcal{R}[0] = 0$: DC thuần phải đi tiếp như tín hiệu vô thanh. Chuỗi $x_0 = [1, -1, 1, -1]$ cộng $A = 2$ thành $[3, 1, 3, 1]$:

$$\sum (x_0 + A)^2 = 9 + 1 + 9 + 1 = 20, \qquad \sum x_0^2 = 4.$$

Offset đã nhân năng lượng $\tau = 0$ lên năm lần — đủ để ngưỡng voicing và so sánh đỉnh T_0 lệch hoàn toàn nếu không chặn DC.

3.2. Toán: cực, zero và chọn $\alpha = 1 - 2^{-8}$

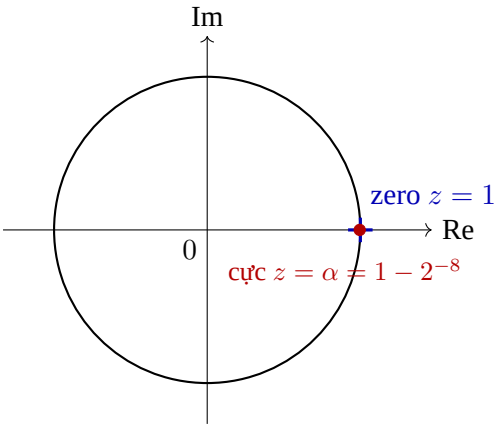
Hàm truyền chuẩn của DC-blocker bậc nhất:

$$H(z) = \frac{1 - z^{-1}}{1 - \alpha z^{-1}}, \qquad 0 < \alpha < 1.$$

Zero (nghiệm tử số) tại $z = 1$ — đúng tần số 0 Hz — nên $H(1) = 0$: mọi tín hiệu hoàn toàn đứng yên bị triệt tiêu. **Cực** (nghiệm mẫu số) tại $z = \alpha$, nằm sát zero nhưng vẫn trong vòng đơn vị nên bộ lọc ổn định. Phương trình sai phân tương đương:

$$y[n] = x[n] - x[n-1] + \alpha\, y[n-1].$$

Đáp ứng tần số gần DC: $|H(e^{j\omega})|$ tăng gần tuyến tính với ω khi ω nhỏ; hằng thời gian xấp xỉ $1/(1-\alpha)$ mẫu.



Zero trên vòng đơn vị tại DC: $H(1) = 0$.
Cực gần zero (trong vòng): HPF nhẹ, ổn định.

Hình 23: DC-blocker trên mặt phẳng z: zero tại $z = 1$, cực tại $\alpha = 1 - 2^{-8} = 0{,}99609375$.

Trên silicon ta chọn $\alpha = 1 - 2^{-8} = 0{,}99609375$ vì phép nhân $\alpha \cdot y$ thu gọn thành một phép dịch và trừ:

$$\alpha y = y - (y \ggg 8) = y \cdot (1 - 2^{-8}),$$

không tốn khối DSP. Hằng thời gian $1/(1-\alpha) = 256$ mẫu. Tại $f_s = 16$ kHz (sau FIR), cutoff xấp xỉ

$$f_c \approx \frac{1-\alpha}{2\pi} f_s = \frac{2^{-8}}{2\pi} \cdot 16000 \approx 9{,}95\,\text{Hz},$$

nằm dưới $F_{0,\min} = 75$ Hz và trên DC — đúng chỗ cần chặn. Ở 48 kHz ngay sau codec, $f_c \approx 29{,}84$ Hz, vẫn an toàn trước khi hạ mẫu.

Ví dụ tính tay — đáp ứng với $\alpha = 0{,}5$ (cực xa để thấy nhanh). Đầu vào hằng $x = [1, 1, 1, 1, 1]$, $y[-1] = 0$:

n	$x[n]$	$y[n] = (x[n] - x[n-1]) + \alpha y[n-1]$	giá trị
0	1	$1 - 0 + 0$	1
1	1	$0 + 0{,}5 \cdot 1$	0,5
2	1	$0 + 0{,}5 \cdot 0{,}5$	0,25
3	1	$0 + 0{,}5 \cdot 0{,}25$	0,125
4	1	$0 + 0{,}5 \cdot 0{,}125$	0,0625

Năng lượng bị nuốt theo cấp số nhân về 0. Với $\alpha = 0{,}75$ trên cùng đầu vào hằng: 1, 0,75, 0,5625, 0,421875, 0,31640625, …— chậm hơn $\alpha = 0{,}5$ nhưng vẫn về 0. Với $\alpha = 0{,}99609375$ suy giảm chậm khoảng 256 mẫu cho một hằng số thời gian.

3.3. Phần cứng: accumulator 32-bit, ví dụ chu kỳ và RTL

FIR Compiler II không thay thế được IIR có phản hồi: kiến trúc FIR không có vòng $y[n-1]$. Viết khoảng mười dòng Verilog với accumulator 32 bit là lựa chọn dạy và lab.

Nếu giữ y chỉ 16 bit Q1.15, mỗi lần dịch $\ggg 8$ sẽ cắt mất phần thập phân khi $|y|$ nhỏ, tạo vùng chết (deadband) hoặc dao động limit-cycle. Giải pháp: giữ thanh ghi phản hồi `y_acc` signed 32 bit; mẫu vào Q1.15 được kéo lên cùng thang bằng dịch trái 15 bit trước khi trừ; ngõ ra lấy lại Q1.15 bằng dịch phải 15 bit rồi bão hòa.

Phương trình bit-exact.

$$\begin{aligned} x_\Delta &= (x[n] \ll 15) - (x[n-1] \ll 15), \\ y_{\text{acc}}[n] &= y_{\text{acc}}[n-1] + x_\Delta - (y_{\text{acc}}[n-1] \ggg 8), \\ y_{\text{out}} &= \text{sat}_{16}(y_{\text{acc}}[n] \ggg 15), \end{aligned}$$

với sat_{16} ghim vào $[-32768, +32767]$. Trên số có dấu, Python/NumPy `>>` và Verilog `>>>` cùng là arithmetic floor shift.

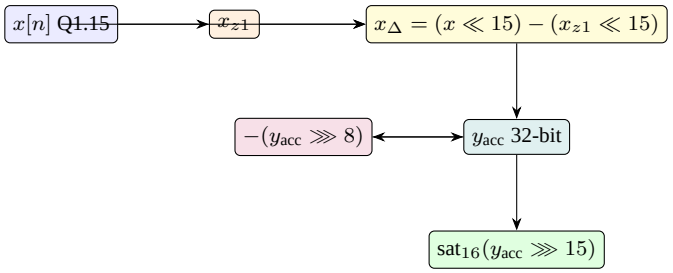
Ví dụ tính tay — từng chu kỳ sau bước Q1.15. Đầu vào $[0, 0, 0]$ rồi bước DC 16384 ($0{,}5$ trong Q1.15). Bảng trung gian (đã chạy script):

n	x	x_Δ	y_{acc} trước	y_{acc} sau	y_{out}
0..2	0	0	0	0	0
3	16384	536870912	0	536870912	16384
4	16384	0	536870912	534773760	16320
5	16384	0	534773760	532684800	16256
6	16384	0	532684800	530604000	16192
7	16384	0	530604000	528531329	16129
8	16384	0	528531329	526466754	16066

Tại $n = 3$: $x_\Delta = 16384 \ll 15 = 536870912$, $y_{\text{acc}} = 536870912$, $y_{\text{out}} = 536870912 \ggg 15 = 16384$. Tại $n = 4$:

$$y_{\text{acc}} \leftarrow 536870912 + 0 - (536870912 \ggg 8) = 534773760, \quad y_{\text{out}} = 16320.$$

Mỗi bước đầu $|y_{\text{out}}|$ giảm khoảng 64 — đúng $(y \ggg 8) \ggg 15 = y_{\text{out}} \ggg 8$ khi chưa bão hòa. Sau khoảng $5 \cdot 256$ mẫu, đầu ra tụt dưới 50 LSB.



Hình 24: Datapath DC-blocker: sai phân kéo thang $\ll 15$, phản hồi 32 bit với $\ggg 8$, ngõ ra bão hòa Q1.15.

RTL giải thích từng nhóm.

```
module dc_blocker (
  input  wire               clk, rst_n,
  input  wire               in_valid,
  output wire               in_ready,
  input  wire signed [15:0] x_in,       // Q1.15
  output reg                out_valid,
  input  wire               out_ready,
  output reg  signed [15:0] y_out
);
  reg signed [15:0] x_z1;               // trễ 1 mẫu
```

```
  reg signed [31:0] y_acc;              // phản hồi 32-bit (KHÔNG cắt 16)
  assign in_ready = out_ready | ~out_valid;  // Avalon-ST backpressure

  wire signed [31:0] x_delta =
      ($signed({x_in, 15'b0})) - ($signed({x_z1, 15'b0}));
  // {x,15'b0} = x<<15 trong cùng bề rộng 31+1 bit có dấu
  wire signed [31:0] y_next =
      y_acc + x_delta - (y_acc >>> 8);  // >>> = arithmetic shift
  wire signed [31:0] y_shift = y_next >>> 15;

  function automatic signed [15:0] sat16;
    input signed [31:0] v;
    begin
      if (v > 32'sd32767)       sat16 = 16'sd32767;
      else if (v < -32'sd32768) sat16 = -16'sd32768;
      else                      sat16 = v[15:0];
    end
  endfunction

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      x_z1 <= 0; y_acc <= 0; y_out <= 0; out_valid <= 0;
    end else if (in_valid && in_ready) begin
      y_acc <= y_next;     // update feedback
      x_z1  <= x_in;
      y_out <= sat16(y_shift);
      out_valid <= 1'b1;
    end else if (out_valid && out_ready) out_valid <= 1'b0;
  end
endmodule
```

`in_ready = out_ready | ~out_valid` cho phép nhận mẫu mới khi đầu ra trống hoặc bên dưới đã lấy. `x_delta` dùng nối bit `{x,15'b0}` thay vì dịch có dấu để tránh mở rộng sai. `y_acc >>> 8` là chỗ hiện thực $\alpha = 1 - 2^{-8}$. `sat16` bắt buộc: nếu bỏ bão hòa, tràn 32→16 tạo tiếng click giả đỉnh F_0.

3.3b. Hiện tượng limit-cycle và chọn α trên silicon

▶ NHÌN THẤY VẬT LÝ

Trên giấy, bộ lọc IIR tuyến tính với $|\alpha| < 1$ luôn đưa trạng thái về 0 khi đầu vào tắt. Trên silicon, mỗi phép nhân/dịch bị *lượng tử hóa*: sai số làm tròn không còn là nhiễu nhỏ thụ động mà bị khuếch đại quanh cực sát vòng đơn vị. Kết quả có thể là một dao động biên độ nhỏ *bền vững* — gọi là **limit cycle** (chu trình giới hạn). Hình dung quả lắc đồng hồ: nếu mỗi bước chỉnh kim chỉ được nhảy nguyên "một vạch" thô, kim có thể mãi mãi nhảy qua lại giữa hai vạch quanh vị trí nghỉ thay vì dừng hẳn. DC-blocker với α sát 1 (cực rất gần $z = 1$) đặc biệt nhạy khi thanh ghi y chỉ rộng 16 bit.

Hai mặt của cùng một lượng tử. (1) **Vùng chết (deadband):** với cập nhật $y \leftarrow y - (y \ggg 8)$ trên số nguyên, khi $|y| < 256$ thì $y \ggg 8 = 0$, nên y *đứng im* dù vẫn còn residual — offset nhỏ đọng lại, nuôi $\mathcal{R}[0]$ giả. (2) **Limit cycle lưỡng cực:** nếu phép làm tròn kèm đổi dấu (mô hình toy của nhánh phản hồi lượng tử), trạng thái có thể nhảy $+\ell \leftrightarrow -\ell$ mãi mãi. Cả hai đều là hệ quả của lưới số nguyên quá thô so với cực gần vòng đơn vị.

Ví dụ tính tay — deadband trên lưới dịch 8 bit. Giả sử sau khi DC lớn đã suy giảm, residual còn $y = 255$ và đầu vào đã về 0. Mỗi bước

$$y \leftarrow y - (y \ggg 8).$$

Vì $255 \ggg 8 = 0$, ta được $y = 255$ *ở mọi* bước tiếp theo. Cùng công thức với $y = 256$:

$$\begin{aligned} y_0 &= 256, \\ y_1 &= 256 - (256 \ggg 8) = 256 - 1 = 255, \\ y_2 &= 255 - (255 \ggg 8) = 255. \end{aligned}$$

Residual kẹt ở 255 LSB — đó chính là vùng chết. Trên Q1.15, $255/32768 \approx 0{,}0078$ vẫn đủ làm phình nhẹ $\mathcal{R}[0]$ nếu đọng xuyên khung.

Ví dụ tính tay — limit cycle lưỡng cực (mô hình làm tròn). Xét ánh xạ minh họa tối giản $y \leftarrow -\text{round}(0{,}875 \cdot y)$ (minh họa phản hồi lượng tử có đổi dấu; không phải RTL lab). Xuất phát $y_0 = 5$:

$$\begin{aligned} y_1 &= -\text{round}(0{,}875 \cdot 5) = -\text{round}(4{,}375) = -4, \\ y_2 &= -\text{round}(0{,}875 \cdot (-4)) = +4, \\ y_3 &= -\text{round}(0{,}875 \cdot 4) = -4. \end{aligned}$$

Từ bước 1 trở đi dãy là $-4, +4, -4, +4, \ldots$ (biên độ không về 0): đó là limit cycle chu kỳ 2.

Script `verify_chuong_03.py` khẳng định cả deadband 255 và dao động ± 4.

Vì sao accumulator 32 bit triệt cả hai. RTL lab giữ y_{acc} signed 32 bit; phép $y_{\text{acc}} \leftarrow y_{\text{acc}} + x_\Delta - (y_{\text{acc}} \ggg 8)$ làm việc trên lưới mịn hơn LSB ngõ ra Q1.15 khoảng 2^{15} lần. Nhiễu lượng tử bị đẩy xuống dưới LSB của mẫu 16 bit lấy ra bằng $y_{\text{acc}} \ggg 15$ rồi bão hòa — người nghe và khối MAC không còn thấy vòng dao động hay residual đứng im ở thang 16 bit. Đó là lý do kỹ thuật, không phải "thói quen 32 bit".

Chọn α trên silicon. Có thể lấy $\alpha = 1 - 2^{-7}$ (cutoff gần gấp đôi) hoặc $1 - 2^{-9}$ (chậm hơn) chỉ bằng đổi hằng dịch trong RTL; không cần DSP. Lab khóa 2^{-8} để mọi twin và mọi hình dùng một số. Đổi α phải đổi cả bảng f_c và bài tập.

Ví dụ tính tay — đáp ứng xung $\alpha = 0{,}5$ (thấy nhanh cực xa). Đặt $x = [1, 0, 0, 0, 0]$, $y[-1] = 0$, $\alpha = 0{,}5$:

$$\begin{aligned} y[0] &= (1 - 0) + 0{,}5 \cdot 0 = 1, \\ y[1] &= (0 - 1) + 0{,}5 \cdot 1 = -0{,}5, \\ y[2] &= (0 - 0) + 0{,}5 \cdot (-0{,}5) = -0{,}25, \\ y[3] &= -0{,}125, \\ y[4] &= -0{,}0625. \end{aligned}$$

Đáp ứng đổi dấu rồi về 0 — đúng HPF: tổng mẫu tiến về 0 ($H(1) = 0$). Trên Q1.15, xung 16384 cho dãy tương tự sau khi nhân thang; với accumulator 32 bit, đuôi không kẹt deadband.

3.4. Twin Python bit-exact

```
import numpy as np

def sat16(v):
    return int(np.clip(v, -32768, 32767))

def dc_blocker_q15(x, alpha_shift=8):
    """Bit-exact với RTL: y_acc int32, x Q1.15 int16."""
    y_acc = np.int32(0)
    x_z1 = np.int32(0)
    out = np.empty(len(x), dtype=np.int16)
    for i, xi in enumerate(np.asarray(x, dtype=np.int32)):
        dx = np.int32((xi << 15) - (x_z1 << 15))
        y_acc = np.int32(y_acc + dx - (y_acc >> alpha_shift))
```

```
        x_z1 = xi
        out[i] = sat16(int(y_acc >> 15))
    return out
```

Vòng lặp trên phải khớp từng LSB với dump simulation RTL. Đổi `np.int32` thành `int` Python không giới hạn bề rộng sẽ *sai* khi gần tràn 32 bit — twin phải giữ đúng modulo hai's complement 32 bit như silicon. Script `verify/verify_chuong_03.py` khẳng định $y[3{:}6] = [16384, 16320, 16256]$, $f_c(16\,\text{kHz}) \approx 9{,}947\,\text{Hz}$, và sau 1600 mẫu bước thì $|y| < 50$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — DC-blocker (mọi viết tắt).

- **ALM** (*Adaptive Logic Module*) / **LUT** (*Look-Up Table*): khoảng 80–120 ALM cho adder/sub 32 bit + dịch số học + bão hòa.
- **DSP** (*Digital Signal Processing* block): 0 — nhờ $\alpha = 1 - 2^{-8}$ hiện thực bằng dịch, không nhân.
- **M10K** / **MLAB**: 0 (không cần đệm mẫu trong khối này).
- **Latency**: 1 mẫu (pipeline Avalon-ST một nhịp) $\approx 20{,}8\,\mu$s ở 48 kHz hoặc $62{,}5\,\mu$s ở 16 kHz.
- **Fmax** (*maximum frequency*): dễ > 200 MHz trên tốc độ 6.
- **RTF** (*real-time factor*): một mẫu vào / một mẫu ra mỗi lần bắt tay `valid`/`ready` $\Rightarrow$ RTF$= 1$ khi không back-pressure.
- Vì sao không cắt 16 bit sớm: tiết kiệm ~ 30 ALM nhưng gây deadband/limit-cycle — đắt hơn mọi LUT trên đường F_0. Vì sao không FIR Compiler II: IIR phản hồi không map vào kiến trúc FIR taps.

Đạo hàm cutoff từ cực. Với $H(z) = (1 - z^{-1})/(1 - \alpha z^{-1})$, trên vòng đơn vị $z = e^{j\omega}$ và $\omega \ll 1$ ta xấp xỉ $1 - e^{-j\omega} \approx j\omega$ và $1 - \alpha e^{-j\omega} \approx (1 - \alpha) + j\alpha\omega$. Biên độ

$$|H| \approx \frac{\omega}{\sqrt{(1-\alpha)^2 + (\alpha\omega)^2}}.$$

Đặt $|H| = 1/\sqrt{2}$ và $\alpha \approx 1$ cho $\omega_c \approx 1 - \alpha$, tức $f_c \approx (1 - \alpha) f_s/(2\pi)$ — đúng công thức đã dùng. Sai số của xấp xỉ này ở $\alpha = 1 - 2^{-8}$ dưới vài phần trăm so với nghiệm đúng phương trình $|H| = 1/\sqrt{2}$; đủ cho chọn tham số lab, không đủ thay đo đạc nếu đổi α lớn.

Vị trí trên datapath và quan hệ với FIR. Có hai chỗ hợp lệ: (1) ngay sau CDC ở 48 kHz rồi mới vào chuỗi hạ mẫu; (2) sau FIR 16 kHz. Chỗ (1) triệt DC sớm, MAC và cửa sổ không bao giờ thấy offset; chỗ (2) rẻ chu kỳ hơn một chút vì chạy chậm hơn nhưng FIR/half-band phải "chở" DC qua. Tài liệu này mặc định **sau CDC, trước FIR** — khớp bản đồ Chương 0. DC-blocker *không* thay high-pass 60 Hz kiểu telephone nếu lab cần cắt rumble mạnh; $\alpha = 1 - 2^{-8}$ chỉ là HPF rất nhẹ.

Tràn 32-bit và chọn bề rộng. Với $|x| \le 32767$, $|x_\Delta| \le 32767 \cdot 2^{15} \cdot 2 < 2^{31}$, và nhánh $-(y \ggg 8)$ cùng Magnitut, tổng $y_{\text{acc}} + x_\Delta - (y \ggg 8)$ trong vận hành bình thường (sau reset, có Avalon-ST) không vượt signed 32 bit nếu đầu vào không phải xung full-scale xen kẽ dấu liên tục ở tần số cực đại. Lab vẫn giữ `sat16` ở ngõ ra; nếu muốn chắc hơn nữa có thể bão hòa y_{acc} vào $\pm(2^{31} - 1)$ trước khi ghi thanh ghi — hiếm khi kích hoạt trên tiếng nói thật.

So sánh với trừ mean theo khung. Trừ mean Praat: $\tilde{x}[n] = x[n] - \frac{1}{L}\sum_{k=0}^{L-1} x[k]$ trên từng khung $L = 640$. Cách đó cần cộng 640 mẫu rồi chia (hoặc nhân $1/L$) mỗi hop — tốn MAC và trễ một khung. IIR DC-blocker cập nhật mỗi mẫu với hai phép cộng/trừ và một dịch, latency một mẫu, phù hợp streaming FPGA. Sai khác: IIR có "đuôi" bộ nhớ y_{acc} xuyên khung; trừ mean không. Trên tiếng nói, đuôi 256 mẫu (16 ms ở 16 kHz) chấp nhận được so với hop 10 ms.

3.4b. Avalon-ST quanh DC-blocker và đo latency

Trên đường tín hiệu, DC-blocker là khối streaming mẫu-đơn: mỗi khi `in_valid & in_ready` nhận đúng một $x[n]$, đúng một nhịp sau (hoặc cùng nhịp pipeline tổ hợp + latch) phát `out_valid` với $y[n]$. Không dùng `sop`/`eop` vì không có biên khung ở đây — biên khung chỉ xuất hiện khi sliding RAM (Chương 5) phát burst L mẫu. Backpressure: nếu FIR phía dưới kéo `out_ready`=0, DC-blocker hạ `in_ready`, DCFIFO phía trên dần đầy; depth 16 đã tính ở Chương 2 hấp thụ vài trăm microgiây. Đo latency lab: đếm chu kỳ từ cạnh lên `in_valid` đến cạnh lên `out_valid` trên ModelSim/Questa với testbench twin — phải ra đúng 1 (theo RTL trên) hoặc đúng số pipeline đã chèn.

Kiểm chứng năng lượng sau khi chặn DC. Lấy chuỗi step Q1.15 dài 1600 mẫu như script: năng lượng $\sum y^2$ tập trung ở đoạn đầu (đáp ứng HPF với bậc thang) rồi đuôi gần 0. Nếu quên `sat16` và tràn, năng lượng đuôi có thể bùng — đó là tín hiệu debug. So sánh với twin float $\alpha = 0{,}99609375$ trên cùng tín hiệu chuẩn hóa: sai số từng mẫu sau đoạn transient phải nằm trong vài LSB khi lượng tử đúng.

Người chấm lab đối chiếu dump RTL với bảng y_{acc} trong mục 3.3: lệch một LSB là chưa đạt. Người học chỉ được chuyển Chương 4 khi script `verify_chuong_03.py` chạy OK và RTL khớp dump.

3.5. Bài tập và lời giải giải thích

Bài tập 3.1 (Phình $\mathcal{R}[0]$.) Cho $x_0 = [1, -1, 1, -1]$ và offset $A = 2$. Tính $\sum (x_0[n] + A)^2$ và giải thích vì sao DC-blocker phải đứng trước MAC.

Lời giải. $\sum (x_0 + A)^2 = 20$, trong khi $\sum x_0^2 = 4$. Offset nhân năng lượng $\tau = 0$ lên năm lần; không chặn DC thì ngưỡng voicing và đỉnh T_0 đều lệch.

Bài tập 3.2 (Đệ quy $\alpha = 0{,}5$ và $\alpha = 0{,}75$).

a) Với $x = [1, 1, 1, 1, 1]$, $\alpha = 0{,}5$, lập bảng $y[0..4]$.

b) Lặp lại với $\alpha = 0{,}75$; nhận xét tốc độ về 0.

Lời giải. (a) $1, 0{,}5, 0{,}25, 0{,}125, 0{,}0625$. (b) $1, 0{,}75, 0{,}5625, 0{,}421875, 0{,}31640625$ — α lớn hơn (cực sát vòng đơn vị hơn) suy giảm chậm hơn; $\alpha = 1 - 2^{-8}$ còn chậm hơn nữa (~ 256 mẫu).

Bài tập 3.3 (Bit-exact step từng chu kỳ). Dùng công thức $y_{\text{acc}} \mathrel{+}= x_\Delta - (y_{\text{acc}} \ggg 8)$, tính y_{out} tại $n = 3, 4, 5$ của bước 16384 (sau ba số 0). Viết cả y_{acc} trung gian ở $n = 4$.

Lời giải. $y_{\text{out}}[3] = 16384$, $y_{\text{acc}}[4] = 534773760$, $y_{\text{out}}[4] = 16320$, $y_{\text{out}}[5] = 16256$ (script). Sai một LSB đồng nghĩa twin và RTL lệch thang.

Bài tập 3.4 (Cutoff và $F_{0,\text{min}}$). Tính f_c tại 16 kHz và 48 kHz với $\alpha = 1 - 2^{-8}$. Vì sao cả hai đều chấp nhận được trước $F_{0,\text{min}} = 75$ Hz?

Lời giải. $f_c \approx 9{,}95$ Hz và $29{,}84$ Hz — cả hai dưới 75 Hz nên không ăn vào dải cao độ; vẫn trên DC nên triệt offset.

Bài tập 3.5 (Deadband 16-bit). Giả sử ai đó giữ phản hồi chỉ 16 bit và làm $y \leftarrow y - (y \ggg 8)$ trên Q1.15. Khi $|y| < 256$, giải thích hiện tượng vùng chết.

Lời giải. $y \ggg 8 = 0$ khi $|y| < 256$, nên y dừng cập nhật dù vẫn còn offset nhỏ — residual DC đọng lại, nuôi $\mathcal{R}[0]$ giả. Accumulator 32 bit giữ phần thập phân dưới ngưỡng 16 bit nên không chết.

Bài tập 3.6 (Trừ mean vs IIR). So sánh chi phí: trừ mean trên khung $L = 640$ mỗi hop $H = 160$ ở 50 MHz cần tối thiểu bao nhiêu phép cộng mỗi hop? DC-blocker cần bao nhiêu phép cộng/trừ mỗi mẫu? Trong 10 ms có bao nhiêu mẫu ở 48 kHz?

Lời giải. Trừ mean: ≥ 640 phép cộng + 1 phép nhân/chia mỗi hop (cộng thêm L phép trừ để tách mean). DC-blocker: 2–3 phép cộng/trừ mỗi mẫu. Trong $10\,\text{ms}$ ở $48\,\text{kHz}$ có 480 mẫu — khoảng $480 \times 3 \approx 1440$ phép, vẫn rẻ hơn nhiều so với MAC tự tương quan ($\sim 189 \times 640$ nhân-cộng mỗi hop, Chương 0), và latency một mẫu thay vì một khung.

Chương 4. FIR Compiler II — hạ mẫu $48 \to 24 \to 16$ kHz (polyphase $2/3$)

▶ MỤC TIÊU HỌC TẬP

Sau Chương 4, người học hạ mẫu đúng chuỗi $48\,\text{kHz} \to 24\,\text{kHz} \to 16\,\text{kHz}$: lọc chống alias trước mỗi lần bỏ mẫu, cấu hình FIR Compiler II fractional rate $I = 2, D = 3$ (polyphase E_0/E_1), giải thích folding/pre-adder và cách IP chia sẻ DSP theo thời gian với số liệu cụ thể, viết twin Python cùng hệ số Q1.15. Thuật ngữ FIR, taps, polyphase, transition band đã có ở ô Feynman 9 của Chương 0; aliasing ở ô 2.

4.1. Hiện tượng: bỏ mẫu không lọc sẽ nuốt F_0

▶ NHÌN THẤY VẬT LÝ

Codec WM8731 xuất PCM ở $48\,\text{kHz}$. Ngân sách Praat/FPGA làm việc ở $16\,\text{kHz}$ (Nyquist $8\,\text{kHz}$ đủ cho F_0 và vài formant thấp). **Hạ mẫu (decimation)** nghĩa là giữ lại một phần mẫu và bỏ phần còn lại. Nếu bỏ mẫu mà chưa lọc thông thấp, mọi thành phần trên nửa tần số mới sẽ **alias** (gương qua Nyquist — Chương 0) đổ xuống vùng $75 \ldots 600\,\text{Hz}$ đúng chỗ ta tìm cao độ. Ví dụ: một formant hoặc nhiễu ở $9\,\text{kHz}$ khi hạ thẳng xuống $16\,\text{kHz}$ sẽ hiện ra như thành phần $7\,\text{kHz}$ hoặc thấp hơn tùy phép gập — đủ để tạo đỉnh giả trên phổ và làm nhiễu tự tương quan.

Lộ trình thực tế trên đường pitch:

1. $48\,\text{kHz} \xrightarrow{\downarrow 2} 24\,\text{kHz}$: hạ nguyên lần 2 (FIR half-band hoặc decimate có lọc), Nyquist mới $12\,\text{kHz}$. Half-band tận dụng gần nửa hệ số bằng 0 xen kẽ — tiết kiệm DSP.

2. $24\,\text{kHz} \xrightarrow{2/3} 16\,\text{kHz}$: tỉ số hữu tỉ — dùng FIR Compiler II Fractional Rate $I = 2, D = 3$ (mỗi 3 mẫu vào tạo 2 mẫu ra).

CIC (*Cascaded Integrator–Comb*) chỉ thắng khi tỷ lệ nguyên $R \gg 1$; với R hiệu dụng $1{,}5$, CIC méo sinc^N và cần bù — loại khỏi lộ trình này.

Ví dụ tính tay — alias số học đơn giản. Tín hiệu $x[n] = \cos(2\pi \cdot 9000 \cdot n/48000)$ tại $48\,\text{kHz}$. Nếu lấy mỗi mẫu thứ ba mà không lọc (minh họa thô cho ý gập phổ), thành phần cao gập vào dải thấp; sau lọc thông thấp cắt trên $\sim 8\,\text{kHz}$ rồi mới bỏ mẫu, biên độ thành phần $9\,\text{kHz}$ bị suy giảm mạnh trước khi gập — đúng vai trò anti-alias.

4.1b. Ngân sách phổ trước và sau hạ mẫu

Trước hạ mẫu, phổ hữu ích của tiếng nói cho bài toán F_0 tập trung dưới khoảng $1\,\text{kHz}$ cho bản thân cao độ, nhưng formant và hài có thể kéo lên vài kHz. Sau bước $\downarrow 2$, Nyquist còn $12\,\text{kHz}$ (tần số lấy mẫu $24\,\text{kHz}$) — vẫn rộng. Sau bước xuống $16\,\text{kHz}$, Nyquist còn $8\,\text{kHz}$: đó là ngưỡng cuối. Mọi năng lượng từ $8\,\text{kHz}$ đến $12\,\text{kHz}$ (và phần còn sót từ miền $48\,\text{kHz}$ trước đó) trên tín hiệu trung gian phải bị dải chặn của FIR $2/3$ đè xuống *trước* khi bỏ mẫu.

Stopband là gì trong thực tế. Bộ lọc số chia phổ thành ba vùng tên gọi chuẩn: **passband** (dải thông — tần số đi qua gần như nguyên vẹn), **transition band** (dải chuyển — suy giảm dần), và **stopband** (dải chặn — nơi biên độ phải bị đè xuống dưới một ngưỡng, ví dụ $-60\,\text{dB}$). Stopband không phải "chỗ không có tín hiệu"; nó là chỗ ta *ép* năng lượng đủ nhỏ để khi gập phổ (alias) xuống dải thấp, phần còn lại không đủ mạnh để tạo đỉnh giả trên tự tương quan. Nếu chỉ "lấy 2 mẫu bỏ 1" mà không lọc, tỷ lệ $2/3$ vẫn đúng về số mẫu nhưng sai về phổ.

Ví dụ tính tay — tone $10\,\text{kHz}$ gập vào đâu. Tại miền $24\,\text{kHz}$, một tone $f = 10\,\text{kHz}$ biên độ $0\,\text{dBFS}$ (biên độ tuyến tính 1). Khi hạ xuống $f_{s,\text{out}} = 16\,\text{kHz}$ mà *không* lọc, tần số alias gần nhất là

$$f_{\text{alias}} = |10\,\text{kHz} - 16\,\text{kHz}| = 6\,\text{kHz},$$

nằm gọn trong Nyquist 8 kHz — đúng vùng formant/cạnh phổ có thể nhiễu đỉnh T_0. Sau FIR có stopband -60 dB, biên độ tuyến tính còn

$$A_{-60} = 10^{-60/20} = 10^{-3} = 0{,}001$$

(tức công suất còn 10^{-6} so với 0 dBFS). Tự tương quan của nhiễu đó nhỏ hơn tín hiệu tiếng nói thông thường hàng chục dB — đủ để đỉnh T_0 không bị át. Nếu stopband chỉ -20 dB:

$$A_{-20} = 10^{-20/20} = 0{,}1,$$

công suất còn $0{,}01$ — nhiễu vẫn lớn, đỉnh phụ có thể thắng ngưỡng voicing. Lab khóa mục tiêu stopband ≤ -60 dB trước 8 kHz.

4.2. Toán FIR, folding và lượng tử hệ số

Bộ lọc **FIR** (*Finite Impulse Response* — đáp ứng xung hữu hạn) là phép tích chập với một dãy hệ số dài hữu hạn N. Định nghĩa gốc, chưa rút gọn:

$$y[n] = \sum_{k=0}^{N-1} h[k]\, x[n-k].$$

Mỗi hệ số $h[k]$ gọi là một **tap** ("vòi" lấy mẫu trễ k). Trên phần cứng, mỗi tap ứng với một phép nhân và một phép cộng vào accumulator — trừ khi ta khai thác đối xứng.

Từ tổng đầy đủ đến folding. Khi thiết kế pha tuyến tính (trễ nhóm không đổi theo tần số — quan trọng để không méo dạng xung dây thanh), hệ số đối xứng:

$$h[k] = h[N-1-k] \qquad (k = 0, 1, \ldots, N-1).$$

Viết lại tổng bằng cách ghép từng cặp $(k,\ N-1-k)$. Với N chẵn:

$$\begin{aligned} y[n] &= \sum_{k=0}^{N/2-1} \Big(h[k]\, x[n-k] + h[N-1-k]\, x[n-(N-1-k)] \Big) \\ &= \sum_{k=0}^{N/2-1} h[k] \Big(x[n-k] + x[n-(N-1-k)] \Big), \end{aligned}$$

vì $h[N-1-k] = h[k]$. Đây là **symmetric folding**: chỉ còn $N/2$ phép nhân thay vì N. (Với N lẻ còn thêm một tap giữa nhân một lần.)

Pre-adder là gì trên silicon. Khối **DSP** 18×19 của Cyclone V có sẵn một bộ cộng cứng ngay trước nhân, gọi là **pre-adder**: nó tính $x_a + x_b$ trong cùng chu kỳ rồi đưa tổng vào nhân với hệ số h. Đúng khớp công thức folding ở trên — vì thế FIR đối xứng map gọn vào silicon mà không tốn ALM để cộng tay trước nhân. Nếu bỏ folding, mỗi cặp tốn hai nhân DSP thay vì một nhân + một pre-adder.

Ví dụ tính tay — folding $N = 4$ từng bước. Cho $h = [h_0, h_1, h_2, h_3] = [1, 2, 2, 1]$ (đối xứng) và cửa sổ mẫu $[x_0, x_1, x_2, x_3] = [a, b, c, d] = [3, 4, 5, 6]$. Tổng thẳng, chưa folding:

$$\begin{aligned} y &= h_0 a + h_1 b + h_2 c + h_3 d \\ &= 1 \cdot 3 + 2 \cdot 4 + 2 \cdot 5 + 1 \cdot 6 \\ &= 3 + 8 + 10 + 6 = 27 \end{aligned}$$

(bốn phép nhân). Cùng y theo folding ($N/2 = 2$ nhân):

$$\begin{aligned} y &= h_0(a + d) + h_1(b + c) \\ &= 1 \cdot (3 + 6) + 2 \cdot (4 + 5) \\ &= 1 \cdot 9 + 2 \cdot 9 \\ &= 9 + 18 = 27. \end{aligned}$$

Hai đường trùng nhau; pre-adder cứng chính là chỗ tính $(3 + 6)$ và $(4 + 5)$ trước khi nhân.

Ví dụ tính tay — convolution 3-tap từng số hạng. $h = [1, 2, 1]$, $x = [3, 4, 5, 0, 0]$, quy ước $x[m] = 0$ khi $m < 0$:

$$y[0] = 1 \cdot x[0] + 2 \cdot x[-1] + 1 \cdot x[-2] = 1 \cdot 3 + 2 \cdot 0 + 1 \cdot 0 = 3,$$
$$y[1] = 1 \cdot x[1] + 2 \cdot x[0] + 1 \cdot x[-1] = 1 \cdot 4 + 2 \cdot 3 + 1 \cdot 0 = 10,$$
$$y[2] = 1 \cdot x[2] + 2 \cdot x[1] + 1 \cdot x[0] = 1 \cdot 5 + 2 \cdot 4 + 1 \cdot 3 = 16.$$

Với $h = [1, 2, 1]/4$ và $x = [4, 0, 4, 0, 4]$: $y = [1, 2, 2, 2, 2]$ — thông thấp làm phẳng thành phần gần Nyquist (script kiểm).

Lượng tử hệ số Q1.15 từng bước. Đưa hệ số thực $h \in [-1, 1)$ sang số nguyên có dấu 16 bit:

$$h_q[k] = \text{sat}_{16}\big(\text{round}(h[k] \cdot 2^{15})\big), \qquad 2^{15} = 32768.$$

Ví dụ $h = [0{,}25, 0{,}5, 0{,}25]$:

$$0{,}25 \cdot 32768 = 8192, \quad 0{,}5 \cdot 32768 = 16384 \quad \Rightarrow \quad h_q = [8192, 16384, 8192].$$

Ví dụ dài hơn $h = [0{,}1, 0{,}2, 0{,}4, 0{,}2, 0{,}1]$:

$$0{,}1 \cdot 32768 = 3276{,}8 \to 3277,$$
$$0{,}2 \cdot 32768 = 6553{,}6 \to 6554,$$
$$0{,}4 \cdot 32768 = 13107{,}2 \to 13107,$$

nên $h_q = [3277, 6554, 13107, 6554, 3277]$ và $\sum h_q = 32769$ (lệch 1 LSB khỏi 32768 do làm tròn — lab ghi nhận và dùng đúng vector này trong IP lẫn twin). Twin, ROM và FIR IP phải dùng *cùng* h_q.

4.3. Polyphase $2/3$: từ $H(z)$ đến commutator E_0, E_1

Viết $k = mM + r$ với $r = 0, \ldots, M - 1$:

$$H(z) = \sum_{r=0}^{M-1} z^{-r} E_r(z^M), \qquad E_r(z) = \sum_m h[mM + r] z^{-m}.$$

Noble identity: sau khi hạ mẫu $\downarrow M$, các nhánh E_r chạy ở tốc độ thấp hơn — tiết kiệm tới hệ số M số MAC so với FIR thẳng rồi bỏ mẫu.

Với fractional $I/D = 2/3$: upsample $\uparrow 2$ (chèn 0) $\to$ lọc $\to$ downsample $\downarrow 3$, được gộp thành polyphase. **Chiều đúng:** mỗi 3 mẫu vào tạo 2 mẫu ra. Ví dụ tách $h = [h_0, \ldots, h_5]$, $M = 3$:

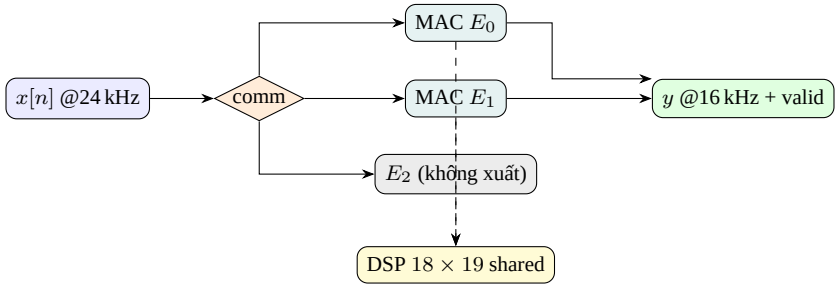
$$E_0 = [h_0, h_3], \quad E_1 = [h_1, h_4], \quad E_2 = [h_2, h_5].$$

Với số nguyên $h = [1, 2, 3, 4, 5, 6]$: $E_0 = [1, 4]$, $E_1 = [2, 5]$, $E_2 = [3, 6]$. Commutator phân phối mẫu vào các nhánh; trên đường $24 \to 16$ kHz chỉ xuất hai pha mỗi ba chu kỳ vào.

Ví dụ lịch mẫu và convolution đầy đủ. Sáu mẫu vào $x = [1, 2, 3, 4, 5, 6]$ qua $h = [1, 2, 3, 4, 5, 6]$ cho

$$y_{\text{full}} = [1, 4, 10, 20, 35, 56, 70, 76, 73, 60, 36]$$

(mode `full`). Lịch $2/3$ giữ 2 trong mỗi 3 chỉ số ra sau khi lọc (chi tiết lịch phụ thuộc phase của commutator trong IP). Tỉ số độ dài hữu ích: sáu vào $\to$ bốn ra cho đoạn ổn định tỷ lệ $4/6 = 2/3$.



Commutator = bộ đếm pha + mux. Chỉ E_0, E_1 xuất theo lịch $2/3$;
một DSP time-mux nhiều tap nhờ folding và clock dư.

Hình 25: Polyphase fractional $I = 2, D = 3$: ba nhánh pha, hai nhánh xuất; DSP chia sẻ theo thời gian.

4.3b. Half-band $48 \to 24$ trước bước $2/3$

Trước fractional $2/3$, bước $\downarrow 2$ từ $48 \to 24$ kHz nên dùng FIR half-band: gần một nửa hệ số đúng bằng 0 (các tap xen kẽ), transition band đối xứng quanh $f_s/4$. Số nhân logic gần còn một nửa so với FIR thông thấp tổng quát cùng độ dài. FIR Compiler II có tùy chọn half-band decimator; lab có thể tách thành hai IP nối tiếp (half-band $\downarrow 2$ rồi fractional $2/3$) để báo cáo DSP từng khối rõ ràng. Twin Python: cùng h_q half-band, `convolve` rồi lấy mẫu chẵn (hoặc lẻ — khớp phase IP).

Ngân sách chu kỳ toàn chuỗi. Một mẫu ra 16 kHz "ăn" trung bình $48/16 = 3$ mẫu vào gốc. Ở 50 MHz, mỗi mẫu ra có

$$\frac{50 \cdot 10^6}{16000} = 3125 \text{ chu kỳ}$$

để toàn bộ half-band + $2/3$ + DC + MAC chạy xong trước mẫu tiếp theo nếu muốn pipeline sâu theo mẫu; thực tế hệ thống làm theo hop 10 ms (500 000 chu kỳ) nên FIR streaming chỉ cần không bỏ `valid` — RTF phần FIR $\ll 1$.

Ví dụ tính tay — đếm MAC polyphase. Cho h dài 6 như trên, mỗi mẫu ra từ nhánh E_0 cần $|E_0| = 2$ nhân, nhánh E_1 cần 2 nhân; nhánh E_2 không xuất nên 0 nhân cho mẫu ra. Trung bình mỗi mẫu vào "gánh" ít MAC hơn FIR 6-tap thẳng (6 nhân) rồi bỏ $1/3$ kết quả. Đúng tinh thần Noble identity: lọc sau khi đã giảm tốc.

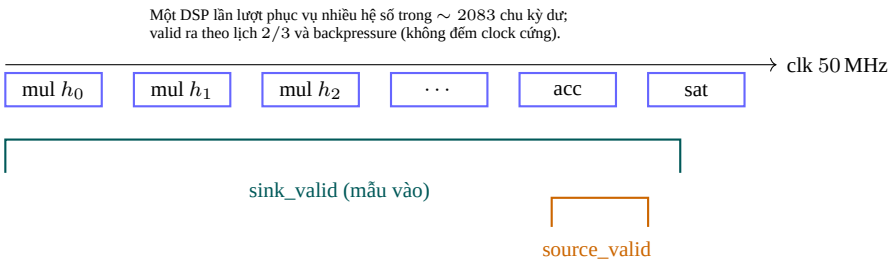
4.4. Intel FIR Compiler II — folding, chia sẻ DSP và số liệu

FIR Compiler II (UG 683208) hỗ trợ Distributed Arithmetic, Mult-add hoặc Systolic. Trên Cyclone V map vào khối DSP Variable Precision: mode 18×19 (đủ Q1.15×Q1.15) hoặc 27×27 khi hệ số/accum rộng hơn. Cổng Avalon-ST: `data`/`valid`/`ready`; `sop`/`eop` là biên *kênh* multi-channel — **không** phải biên khung pitch 40 ms. Lab mono liên tục chỉ cần data/valid/ready.

Cách IP gấp và chia sẻ DSP (số liệu lab). Giả sử thiết kế FIR đối xứng $N = 48$ taps cho nhánh $24 \to 16$ kHz. Folding còn $N/2 = 24$ phép nhân "logic". Polyphase $M = 3$ với $I = 2$ chỉ xuất 2 nhánh — số MAC hữu ích mỗi mẫu ra thấp hơn FIR thẳng. Clock hệ thống 50 MHz và tốc độ mẫu vào 24 kHz cho

$$\frac{50 \cdot 10^6}{24000} \approx 2083 \text{ chu kỳ clock mỗi mẫu vào.}$$

FIR Compiler II khai thác dư địa này: một khối DSP 18×19 được **time-multiplex** qua nhiều cặp (h_k, x) trong cùng một khoảng giữa hai mẫu audio. Báo cáo generate điển hình cho order lab (vài chục tap, fractional $2/3$) ghi khoảng vài đến vài chục DSP — *phải đọc Fitter*, không đoán. Nếu bật "Fully Parallel", số DSP ≈ số nhân sau folding và tăng vọt; lab chọn chế độ cho phép folding/sharing để giữ ngân sách DSP toàn hệ thống khoảng 3 khối (bảng Chương 0) hoặc nới có kiểm soát nếu đo Fmax/RTF vẫn đạt.



Hình 26: Chia sẻ DSP theo thời gian giữa hai mẫu vào 24 kHz: IP gấp nhiều nhân vào một khối 18×19; `source_valid` theo lịch fractional.

Cấu hình dạy. Fractional Rate $I = 2$, $D = 3$; ghi rõ rounding (truncate hoặc convergent) và saturation vào twin. Coefficient bit-width quá hẹp làm stopband nông (alias vào F_0); quá rộng lãng phí DSP. FIR $2/3$ là anti-alias hạ mẫu, *không* thay high-pass 60 Hz — DC-blocker Chương 3 lo offset chậm.

RTL thay thế. Khi không gọi IP: systolic MAC + coeff ROM, folding tay nếu $N \leq 32$. Không tự viết Distributed Arithmetic trừ khi tối ưu diện tích cực đoan — DA khó bit-exact với IP. Luôn giữ cùng h_q giữa ROM, IP và Python.

Twin bit-exact.

```
import numpy as np
# h_q: int16 đã lượng tử giống FIR IP / ROM
acc = np.convolve(x.astype(np.int64), h_q.astype(np.int64), mode="full")
# áp dụng đúng round/sat rồi lấy mẫu theo lịch 2/3 của IP
# so dump Avalon-ST beat-by-beat; thêm test ready=0 vài chu kỳ
```

Phải mô phỏng `ready`=0: IP dừng tiến mẫu vào nhưng không được làm twin "tự đếm clock".

Stopband và nguy cơ alias vào F_0. Thông số thiết kế tối thiểu: passband đến khoảng 5–6 kHz (giữ formant thấp phục vụ phân tích), transition xuống stopband trước 8 kHz (Nyquist 16 kHz). Độ sâu stopband khuyến nghị ≤ -60 dB trong lab để nhiễu alias sau hạ mẫu nằm dưới sàn tự tương quan điển hình. Hệ số Q1.15 với N quá ngắn không đạt -60 dB; đó là lúc tăng N hoặc dùng half-band dài hơn chứ không phải "tăng bit hệ số vô hạn".

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — FIR $2/3$ (mọi viết tắt).

- **DSP** (*Digital Signal Processing* block 18×19, có **pre-adder**): số DSP $\propto$ (taps sau folding)/(hệ số chia sẻ theo thời gian) — *đọc Fitter*, không đoán. Mục tiêu vài DSP để toàn hệ ≈ 3 DSP (bảng Chương 0), hoặc ghi rõ nếu lab nới.
- **M10K** (block RAM ≈ 10 kbit): hệ số + delay line, thường 1–2 khối. **MLAB**: có thể dùng cho delay ngắn; depth lớn nên ưu tiên M10K.
- **ALM/LUT**: vài trăm cho control + Avalon-ST (không kể soft-multiplier nếu hết DSP).
- **Latency**: độ trễ nhóm FIR + pipeline IP (ví dụ ~ 40 chu kỳ) — cộng vào hop nhưng $\ll 500\,000$.
- **Fmax**: thường > 150 MHz khi dùng DSP cứng; sụp nếu Fitter nhét nhân mềm vào ALM.
- **RTF**: với ≈ 2083 chu kỳ dư mỗi mẫu @ 24 kHz/50 MHz, sharing giữ RTF$\ll 1$ miễn không bỏ `valid`.
- Vì sao polyphase: bỏ MAC của pha bị discard, tiết kiệm tới $\sim 66\%$ nhân. Vì sao không CIC: $2/3$ không phải R nguyên lớn.

4.4b. Từ báo cáo Generate đến bảng chi phí điền số

Sau khi Generate FIR Compiler II, mở file báo cáo (hoặc tab Resource Utilization trong IP GUI) và chép tay vào sổ lab các mục: Number of DSP blocks, Number of memory bits / M10K, Latency (clock cycles), Input/output data widths, Coefficient storage (logic/M10K). Ví dụ minh họa (số giả định để luyện đọc — *thay bằng số Fitter thật của bạn*):

Mục báo cáo	Giá trị ví dụ lab $N \approx 48$, $I/D = 2/3$
DSP (18×19)	4
Memory bits	~ 2000 (coeff + delay)
Latency	12–40 chu kỳ (phụ thuộc pipeline)
Avalon-ST	sink/source valid/ready

Latency IP phải cộng vào ngân sách hop: 40 chu kỳ không đáng kể so với $500\,000$, nhưng FSM nối tiếp phải chờ `valid` thật, không trừ latency bằng đếm cứng trừ phi đã đo. Coefficient storage chọn M10K khi N lớn để khỏi ăn ALM; với N nhỏ logic đủ.

So sánh Fully Parallel vs Multicycle. Fully Parallel: mỗi nhân một DSP (sau folding) — Fmax cao, diện tích lớn, dễ vượt ngân sách 3 DSP của bảng Chương 0. Multicycle/sharing: ít DSP, cần nhiều chu kỳ giữa hai mẫu; với 2083 chu kỳ dư ở 24 kHz thì sharing là lựa chọn mặc định dạy. Distributed Arithmetic: ít/không DSP, dùng LUT — khó khớp bit với twin nhân trực tiếp; chỉ dùng khi hết DSP và chấp nhận viết lại twin theo DA.

Twin mở rộng — lịch $2/3$ giả lập.

```
def decimate_2_over_3(y_full):
    """Giữ 2 mẫu, bỏ 1 theo chu kỳ cố định (minh họa).
    Twin khớp IP phải theo đúng phase report của FIR II."""
    out = []
    for i, yi in enumerate(y_full):
        if i % 3 != 2:   # giữ i mod 3 in {0,1}
            out.append(yi)
    return out
assert len(decimate_2_over_3(range(6))) == 4
```

Đoạn trên chỉ minh họa tỷ lệ; phase thật (mod $3 = 0, 1$ hay $1, 2$) phải khớp dump IP. Sai phase làm lệch toàn bộ thời gian đến sliding RAM.

Điều kiện qua chương: script `verify_chuong_04.py` OK và đọc được số DSP từ báo cáo Generate (không bịa).

4.5. Bài tập và lời giải giải thích

Bài tập 4.1 (Folding). Với $h = [1, 2, 2, 1]$, $x = [3, 4, 5, 6]$, chứng minh folding cho cùng kết quả và đếm số nhân.

Lời giải. Thẳng: $3 + 8 + 10 + 6 = 27$ (4 nhân). Folding: $1 \cdot (3 + 6) + 2 \cdot (4 + 5) = 27$ (2 nhân). Pre-adder DSP làm đúng phép cộng trước nhân.

Bài tập 4.2 (Polyphase split). $h = [0, 1, 2, 3, 4, 5]$, $M = 3$. Viết E_0, E_1, E_2. Tỉ số ra/vào của lịch $2/3$ trên 6 mẫu vào là bao nhiêu?

Lời giải. $E_0 = [0, 3]$, $E_1 = [1, 4]$, $E_2 = [2, 5]$. Sáu vào $\rightarrow$ bốn ra: $4/6 = 2/3$.

Bài tập 4.3 (Valid không đều). Vì sao FSM sau FIR II fractional không được đếm clock cố định "cứ 3 ck lấy 2"?

Lời giải. Backpressure Avalon-ST (`ready`=0) làm giãn khoảng cách mẫu vào; lịch valid ra phụ thuộc trạng thái IP. Chỉ bắt mẫu khi `valid`=1.

Bài tập 4.4 (Chu kỳ dư và DSP share).

a) Tính số chu kỳ clock $50\,\text{MHz}$ giữa hai mẫu vào $24\,\text{kHz}$.

b) Với $N = 48$ đối xứng, sau folding còn bao nhiêu nhân logic? Vì sao một DSP vẫn có thể đủ nếu IP time-mux?

Lời giải. (a) $50 \cdot 10^6 / 24000 \approx 2083$ chu kỳ. (b) 24 nhân logic; $2083 \gg 24$ nên một DSP có thể lần lượt thực hiện nhiều nhân trong khoảng giữa hai mẫu — đúng cơ chế sharing của FIR Compiler II ở chế độ không Fully Parallel.

Bài tập 4.5 (Lượng tử hệ số). Cho $h = [0{,}1, 0{,}2, 0{,}4, 0{,}2, 0{,}1]$, tính $h_q = \text{round}(h \cdot 2^{15})$ và $\sum h_q$. Vì sao twin phải dùng đúng vector này?

Lời giải. $h_q = [3277, 6554, 13107, 6554, 3277]$, tổng 32769 (lệch 1 LSB khỏi 32768 do làm tròn). Sai một hệ số làm lệch stopband — alias có thể xuyên vào dải F_0.

Bài tập 4.6 (Half-band và chu kỳ ở $16\,$kHz).

a) Vì sao half-band phù hợp bước $48 \to 24$ hơn CIC?

b) Mỗi mẫu ra $16\,$kHz tương ứng bao nhiêu chu kỳ clock $50\,$MHz? So với $500\,000$ chu kỳ/hop thì FIR streaming chiếm tỷ lệ nào nếu latency IP $= 40$ chu kỳ mỗi mẫu ra và hop có 160 mẫu ra?

Lời giải. (a) Half-band tận dụng hệ số 0 xen kẽ và transition quanh $f_s/4$ đúng cho $\downarrow 2$; CIC cần $R \gg 1$ và méo sinc. (b) 3125 chu kỳ/mẫu ra. $160 \times 40 = 6400$ chu kỳ latency tích lũy theo pipeline mẫu (thực tế pipeline chồng nên wall-clock gần latency một mẫu + thời gian streaming); dù đếm tối đa $6400/500000 \approx 1{,}3\%$ — FIR không phải nút thắt so với MAC.

Ghi chú triển khai Platform Designer. Khi kéo FIR Compiler II vào Platform Designer, nối Avalon-ST từ DC-blocker (hoặc từ half-band) sang `avalon_streaming_sink`, và từ `avalon_streaming_source` sang khối sau. Đừng nối MM (memory-mapped) trừ khi cấu hình hệ số động — lab hệ số tĩnh dùng ROM trong IP. Reset của IP phải đồng bộ miền $50\,$MHz; không reset bằng tín hiệu async thô từ nút bấm.

Lỗi thường gặp khi nối FIR II. (1) `ready` phía sink luôn gắn 1 nhưng quên kiểm `valid` nguồn — mất mẫu khi IP tạm dừng. (2) Dùng `sop`/`eop` như biên khung pitch — sai ngữ nghĩa multi-channel. (3) Hệ số nạp Q2.14 trong khi twin Q1.15 — lệch 6 dB toàn cục. (4) Fully Parallel trên $N = 128$ làm hết DSP kit, Fitter nhét soft-multiplier vào ALM, Fmax sụp. Mỗi lỗi trên đều có triệu chứng riêng trên waveform: học viên nên ghi checklist trước khi hỏi "sao F_0 sai".

Ôn lại tỷ lệ hữu tỉ. $16/24 = 2/3$ là phân số tối giản; mọi lịch valid ra dài hạn phải có đúng 2 nhịp có dữ liệu trên mỗi 3 mẫu vào (khi ready luôn 1). Đo trên simulation: đếm N_{in} và N_{out} trong $0{,}1\,$s lý thuyết — N_{out}/N_{in} phải $\approx 2/3$ với sai số biên pipeline. Sai lệch lớn báo phase commutator hoặc backpressure không mô phỏng đúng trong twin.

Chương 5. Bộ nhớ trượt M10K (True Dual-Port, Old Data)

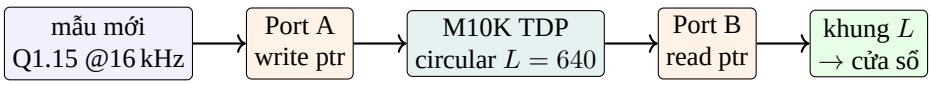
▶ MỤC TIÊU HỌC TẬP

Sau Chương 5, người học dựng bộ đệm vòng $L = 640$ mẫu trên M10K True Dual-Port, chọn đúng mixed-port read-during-write = Old Data, khớp latency đọc 1 hoặc 2 chu kỳ với twin và FSM trên sơ đồ timing chu kỳ-cụ-thể, và tính ngân sách bit/M10K. Thuật ngữ M10K, True Dual-Port, Old Data, ALM, Fmax đã có ở ô Feynman 10 và bài tập 0.3 của Chương 0 — chương này triển khai datapath.

5.1. Hiện tượng: vì sao phải nhớ L mẫu trên chip

▶ NHÌN THẤY VẬT LÝ

Giọng nói là sóng theo thời gian liên tục. Praat cắt khung L mẫu (ví dụ $L = 640$ tại 16 kHz với $F_{0,\min} = 75$ Hz, khoảng 40 ms) rồi trượt theo bước nhảy $H = 160$ (10 ms). Trên FPGA, mỗi mẫu từ FIR chỉ đến một lần rồi biến mất khỏi thanh ghi vào — không thể "tua lại" micro. **Bộ nhớ trượt** (circular buffer) giữ đúng L mẫu gần nhất: cổng A ghi mẫu mới mỗi beat audio hợp lệ, cổng B đọc burst L mẫu khi FSM cần một khung để nhân cửa sổ và MAC. Đời thường: hộp đen máy bay chỉ giữ L giờ ghi gần nhất; mỗi phút mới đè lên phút cũ nhất — đúng phép modulo trên địa chỉ.



Hình 27: Datapath sliding window: ghi một mẫu mỗi beat audio, đọc burst L mẫu khi FSM yêu cầu khung.

Công thức chọn L theo tinh thần Praat: $L \approx 3 \cdot f_s / F_{0,\min}$. Với $f_s = 16$ kHz, $F_{0,\min} = 75$ Hz:

$$L \approx 3 \cdot \frac{16000}{75} = 640 \text{ mẫu.}$$

Nếu lab nâng $F_{0,\min}$ lên 100 Hz thì $L \approx 480$ — tiết kiệm RAM nhưng giảm độ bền đỉnh với giọng trầm. Không chọn L tùy hứng: quá ngắn làm đỉnh $\mathcal{R}[\tau]$ nông; quá dài tốn M10K và tăng MAC.

Vì sao on-chip, không SDRAM. Đọc 640 mẫu liên tiếp mỗi hop 10 ms đòi băng thông và latency Deterministic. SDRAM/HPS có refresh, arbitration, cache — RTF và jitter địa chỉ phá deadline 500 000 chu kỳ. Một khối M10K cho đúng 10240 bit là vừa khít và hoàn toàn Deterministic.

5.1b. So sánh với FIFO tuyến tính và với bộ đệm phần mềm

FIFO tuyến tính (chỉ đẩy mẫu mới, đọc tuần tự từ đầu) không đủ cho khung trượt chồng lấp: sau khi đọc hết L mẫu một lần, dữ liệu cũ mất; trong khi hop chỉ tiến $H < L$ nên cần giữ lại $L - H$ mẫu. Circular buffer giải quyết bằng cách *không* xóa khi đọc — đọc không phá nội dung, chỉ ghi mới đè mẫu cũ nhất. Trong phần mềm Praat, mảng vòng hoặc copy L mẫu mỗi hop làm cùng việc; trên FPGA, copy 640 mẫu mỗi 10 ms vào BRAM khác sẽ nhân đôi M10K và thêm 640 chu kỳ chép — lãng phí so với đọc tại chỗ từ circular.

An toàn đa khung. Nếu FSM chậm và yêu cầu khung mới khi khung trước chưa đọc xong, cần hoặc chặn `frame_req` (backpressure) hoặc double-buffer hai vùng L (tốn 2 M10K). Lab mặc định backpressure: một burst tại một thời điểm, đơn giản và đủ vì $642 \ll 500000$. Double-buffer chỉ cần khi pipeline MAC quá sâu đến mức đọc khung chồng nhau — không phải trường hợp thiết kế DE10 ở Chương 0.

Liên hệ RTF. **RTF** (*real-time factor*) = thời gian xử lý / thời gian audio. Thời gian đọc khung $642/(50 \times 10^6) \approx 12{,}84\ \mu$s trên 10 ms audio cho RTF riêng phần nhớ $\approx 0{,}00128$. Toàn hệ thống RTF bị chi phối bởi MAC (Chương 7), không bởi M10K — đúng chỗ đặt ngân sách silicon.

5.2. Toán con trỏ vòng và ví dụ tính tay

Con trỏ ghi $n_w \in \{0, \ldots, L-1\}$ chạy modulo L: mỗi beat `valid` ghi $\text{mem}[n_w] \leftarrow x_{\text{in}}$ rồi $n_w \leftarrow (n_w + 1) \bmod L$. Khi FSM cần khung (sau khi đã có đủ mẫu mới theo hop H), đọc oldest-first:

$$x_{\text{frame}}[k] = \text{mem}\big[(n_w + k) \bmod L\big], \qquad k = 0, \ldots, L-1,$$

với quy ước n_w đang trỏ ô trống tiếp theo (mẫu cũ nhất nằm đúng tại n_w).

Ví dụ tính tay — circular 8 ô. $L = 8$, ghi lần lượt $1, 2, 3, 4, 5, 6, 7, 8, 9$ (mẫu 9 đè ô 0):

addr	0	1	2	3	4	5	6	7
sau 8 ghi	1	2	3	4	5	6	7	8
sau ghi 9	**9**	2	3	4	5	6	7	8

Khi $n_w = 1$ (vừa ghi 9 vào addr 0 rồi tăng ptr), khung oldest→newest lấy tuần tự addr 1,2,3,4,5,6,7,0, tức dãy mẫu 2,3,4,5,6,7,8,9.

Ví dụ tính tay — quan hệ hop và số mẫu mới. Mỗi hop $H = 160$ mẫu ở 16 kHz tương ứng 10 ms. Bộ đệm $L = 640$ giữ 4 hop lịch sử. Sau mỗi hop, chỉ 160 ô bị ghi đè; 480 ô còn lại vẫn là quá khứ hữu ích cho cửa sổ chồng lấp (overlap) — đúng tinh thần phân tích khung trượt Praat.

5.2b. Overlap, hop và số lần đọc mỗi giây

Với $H = 160$, $f_s = 16$ kHz, số khung mỗi giây là $16000/160 = 100$ khung/s (đúng 10 ms/khung). Mỗi khung đọc $L = 640$ mẫu từ M10K $\Rightarrow 64000$ phép đọc/giây — với clock 50 MHz là tầm thường. Overlap $L - H = 480$ mẫu (75% overlap) làm liền mạch đường F_0 theo thời gian: hai khung liên tiếp chia sẻ 480 mẫu, nên đỉnh cao độ ít nhảy hơn so với hop không chồng. Trên silicon, overlap *không* tốn thêm RAM: circular buffer vốn đã giữ L; chỉ cần phát `frame_req` thường hơn.

Ví dụ tính tay — địa chỉ sau nhiều hop. $L = 8$, $H = 2$ (minh họa thu nhỏ). Bắt đầu $n_w = 0$ sau khi đã đầy 1..8. Sau một hop ghi thêm 2 mẫu (giá trị 9,10): $n_w = 2$, mem$= [9, 10, 3, 4, 5, 6, 7, 8]$, khung $= [3, 4, 5, 6, 7, 8, 9, 10]$. Sau hop nữa (11,12): $n_w = 4$, khung $= [5, 6, 7, 8, 9, 10, 11, 12]$. Quan sát: mỗi hop khung trượt đúng H mẫu trên dòng thời gian — đúng định nghĩa hop Praat.

Đọc đồng thời với ghi — định mức chu kỳ. Trong 10 ms có 160 mẫu cần ghi và một burst 640 đọc. Tổng giao dịch bộ nhớ ≈ 800 mỗi hop, vẫn $\ll 500\,000$. True Dual-Port cho phép ghi và đọc cùng chu kỳ trên hai địa chỉ khác nhau mà không mất mẫu; chỉ khi trùng địa chỉ mới cần quy tắc Old Data đã phân tích.

5.3. Old Data, latency 1/2 và timing chu kỳ-cụ-thể

Trên Cyclone V, khối **M10K** True Dual-Port cho phép Port A ghi và Port B đọc *cùng lúc*. Khi hai cổng chạm *cùng một địa chỉ* trong cùng chu kỳ (mixed-port read-during-write), cần một quy tắc: silicon trả về giá trị nào? **Old Data** nghĩa là cổng đọc nhận đúng nội dung ô *trước* cạnh ghi — như nhìn vào ngăn kéo trước khi ai đó thay đồ trong đó. **New Data** nghĩa là đọc ngay giá trị vừa ghi; trên mixed-port Cyclone V chế độ này *không được bảo đảm* (chỉ Don't Care hoặc Old Data). Twin Python phải trả về giá trị cũ trong cùng "delta-cycle"; nếu không, testbench xanh trên mô hình tưởng tượng nhưng đỏ trên chip thật. Chương 0 đã cảnh báo điều này ở bài tập 0.3 — đây là chỗ hiện thực.

Cấu hình IP. On-Chip Memory (RAM: 2-PORT), True Dual-Port, width 16 (Q1.15), depth $\geq L$ (có thể làm tròn 1024). Mixed-port RDW = **Old Data**. Không dùng MLAB cho $L = 640$ (depth lớn, timing kém). Single-port + time-mux ghi/đọc sẽ buộc FSM không ghi trong lúc đọc khung — phức tạp hóa và dễ mất mẫu audio.

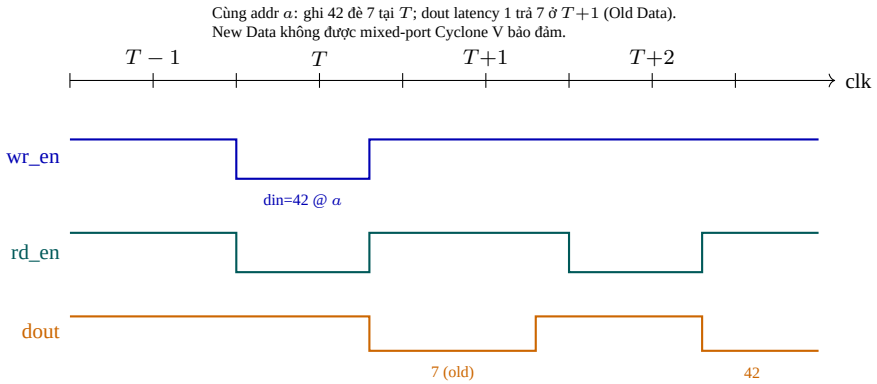
Latency đọc — điểm dễ lệch khung.

- RTL inferred `dout <= mem[rd_ptr]`: latency **1 chu kỳ** sau `rd_en`.
- MegaWizard bật Output Registers: thường **2 chu kỳ** (block RAM register + output register).

FSM Chương 11 và twin **phải** cùng model. Lệch một chu kỳ làm lệch cả 640 mẫu cửa sổ và toàn bộ MAC.

Sơ đồ timing — Old Data cùng địa chỉ. Giả sử tại chu kỳ T Port A ghi giá trị mới 42 vào địa chỉ a đang giữ 7, đồng thời Port B đọc a với `rd_en`=1, cấu hình Old Data, latency 1:

Chu kỳ	Port A (write)	Port B (read addr)	`dout` (lat 1)
$T-1$	—	—	(giá trị cũ hơn)
T	ghi 42 vào a (cũ = 7)	đọc a	— (chưa ra)
$T+1$	—	—	7 (Old Data)
$T+2$	—	đọc a lại	42 (đã là nội dung mới)



Hình 28: Timing chu kỳ-cụ-thể mixed-port read-during-write = Old Data, latency đọc 1:dout ở $T+1$ là giá trị trước khi ghi.

Nếu IP bật output register (latency 2), mọi cạnh `dout` trong hình dịch thêm một chu kỳ — FSM phải chờ đúng 2 nhịp sau `rd_en` đầu trước khi lấy mẫu đầu khung.

RTL inferred (latency 1) giải thích từng dòng.

```
localparam int L = 640;
reg signed [15:0] mem [0:L-1];  // M10K via ramstyle
reg [$clog2(L)-1:0] wr_ptr, rd_ptr;
reg signed [15:0] dout;
always @(posedge clk) begin
  if (valid_i) begin
    mem[wr_ptr] <= din;              // Port A: ghi mẫu audio mới
    wr_ptr <= (wr_ptr == L-1) ? '0 : wr_ptr + 1'b1;
  end
  if (rd_en) begin
    dout <= mem[rd_ptr];         // Port B Old Data
    rd_ptr <= (rd_ptr == L-1) ? '0 : rd_ptr + 1'b1;
  end
end
```

Dòng `dout <= mem[rd_ptr]` tạo thanh ghi ngõ ra — latency 1. Hai tiến trình trong cùng `always` là True Dual-Port khi công cụ suy diễn được; để chắc, lab dùng IP 2-PORT và nối tay hai con trỏ. Không ghi `dout = mem[...]` (blocking tổ hợp) nếu muốn khớp M10K đồng bộ.

Twin địa chỉ khớp Old Data.

```
L = 640
mem = [0] * L
wr = 0
def write(x):
    global wr
    old = mem[wr]       # giá trị trước khi đè --- Old Data
    mem[wr] = int(x)
    wr = (wr + 1) % L
    return old
def read_frame():
    return [mem[(wr + k) % L] for k in range(L)]  # oldest-first
```

Hàm `write` trả về `old` khớp dout silicon khi đọc trùng địa chỉ đang ghi. Script `verify_chuong_05.py` kiểm khung $[2, \ldots, 9]$ ($L = 8$) và ngân sách $640 \times 16 = 10\,240$ bit.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — sliding M10K (mọi viết tắt).

- **M10K** (block RAM ≈ 10 kbit): $L \times 16 = 640 \times 16 = 10\,240$ bit — vừa khít **1 khối**. Depth 1024 từ vẫn khoảng 1–2 M10K. Không dùng **MLAB** cho $L = 640$ (depth lớn, timing kém).
- **ALM/LUT**: khoảng 50–100 cho hai con trỏ + FSM đọc. 0 **DSP**.
- **Latency** đọc: 1 hoặc 2 chu kỳ (cấu hình IP) $\Rightarrow$ đọc hết khung 641 hoặc 642 chu kỳ.
- **Fmax**: dễ > 150 MHz khi ép `ramstyle=M10K`; RAM logic làm Fmax và hành vi Old Data lệch twin.
- **RTF**: $642/500\,000 \approx 0{,}00128$ trên hop 10 ms @ 50 MHz — nhớ không phải nút thắt; MAC Chương 7 mới là nút thắt.
- Vì sao True Dual-Port + Old Data: ghi/đọc song song mỗi chu kỳ; New Data không bảo đảm trên mixed-port Cyclone V.

5.3b. Khởi động ấm máy và đồng bộ hop

Sau reset, bộ đệm chứa 0 (hoặc don't care). Nếu FSM phát khung ngay, MAC sẽ nhân cửa sổ với chuỗi toàn 0 hoặc rác — đỉnh F_0 vô nghĩa. Quy trình ấm máy: chờ ít nhất $L = 640$ mẫu `valid` từ FIR (khoảng 40 ms ở 16 kHz) trước khung đầu tiên; sau đó mỗi $H = 160$ mẫu phát một khung mới. Cờ `warm` trong FSM: đếm mẫu ghi cho tới L, rồi cho phép `frame_req`. Twin Python phải có cùng quy tắc nếu so sánh bit-exact từng khung.

Xung đột ghi/đọc khi burst dài. Trong lúc Port B đọc 640 chu kỳ, Port A vẫn có thể nhận mẫu audio mới mỗi ≈ 3125 chu kỳ hệ thống (một mẫu 16 kHz). Xác suất trùng địa chỉ đúng lúc đang đọc địa chỉ đó là có thật: đó chính là lúc Old Data quyết định đúng/sai. New Data "hy vọng đọc giá trị vừa ghi" sẽ làm một mẫu trong khung mang giá trị tương lai — lệch cửa sổ một cách không lặp lại theo seed, cực khó debug. Old Data bảo đảm khung là ảnh nhất quán của quá khứ tại thời điểm bắt đầu burst (plus các ô chưa bị đè trong lúc đọc — đúng mô hình circular đang trôi).

Hai con trỏ đọc: oldest và tuần tự. Có thể khởi rd_ptr $\leftarrow n_w$ khi bắt burst (oldest), rồi tăng mỗi chu kỳ; hoặc giữ rd_base cố định trong burst và rd_ptr = rd_base + k. Cách hai tránh lệch nếu n_w bị ghi tăng trong lúc đọc — khuyến nghị lab: **chốt** rd_base $= n_w$ **lúc nhận** `frame_req`, rồi đọc rd_base + k mod L, trong khi n_w vẫn tăng trên Port A. Twin:

```
def read_frame_stable(wr_at_req):
    return [mem[(wr_at_req + k) % L] for k in range(L)]
```

Thuộc tính `ramstyle` và suy diễn.

```
(* ramstyle = "M10K" *) reg signed [15:0] mem [0:L-1];
```

Ép map M10K, tránh công cụ phân tán MLAB khi depth lớn. Kiểm Fitter: 1 M10K, không cảnh báo inferred ROM/RAM conflict. Nếu thấy "RAM logic", latency và Old Data có thể không còn đúng như block RAM — phải sửa attribute hoặc dùng IP.

Điều kiện qua chương: script `verify_chuong_05.py` OK, Fitter báo 1 M10K, và waveform RDW khớp Old Data.

5.4. Bài tập và lời giải giải thích

Bài tập 5.1 (Ngân sách bit). $L = 640$, width 16: bao nhiêu bit? Cần tối thiểu bao nhiêu M10K? Nếu $L = 480$ ($F_{0,\min} = 100$ Hz) thì sao?

Lời giải. $640 \times 16 = 10\,240$ bit $= 1$ M10K vừa khít. $480 \times 16 = 7680$ bit vẫn 1 M10K (không chia nhỏ khối) nhưng dư chỗ — có thể dùng dual-purpose lưu thêm hệ số cửa sổ nếu cần.

Bài tập 5.2 (Old Data timing). Cùng địa chỉ: Port A ghi 42 vào ô đang giữ 7, Port B đọc cùng ô trong cùng chu kỳ, latency 1, Old Data. Dout ở chu kỳ sau là gì? Ở hai chu kỳ sau nếu đọc lại?

Lời giải. Chu kỳ $T{+}1$: dout $= 7$. Chu kỳ $T{+}2$ nếu đọc lại addr đó: dout $= 42$. New Data không được mixed-port Cyclone V bảo đảm.

Bài tập 5.3 (Đọc khung). Sau khi ghi mẫu 9 vào $L = 8$ như bảng trên ($n_w = 1$), liệt kê khung oldest-first.

Lời giải. $[2, 3, 4, 5, 6, 7, 8, 9]$ — khớp script `verify/verify_chuong_05.py`.

Bài tập 5.4 (Latency 1 vs 2). Vì sao lab phải ghi rõ "inferred 1 ck" hay "IP output register 2 ck" trước khi viết FSM? Hậu quả lệch một chu kỳ lên MAC là gì?

Lời giải. Mỗi mẫu cửa sổ lệch một chu kỳ làm lệch toàn bộ tích $\mathcal{R}[\tau]$; sai số là dịch cả khung — đỉnh F_0 lệch hệ thống, khódebug vì "có vẻ có đỉnh" nhưng sai cao độ.

Bài tập 5.5 (Ngân sách chu kỳ hop).

a) Trong 10 ms ở 50 MHz có bao nhiêu chu kỳ?

b) Đọc $L = 640$ với latency 2 mất bao nhiêu chu kỳ? Tỷ lệ so với ngân sách hop?

Lời giải. (a) $500\,000$ chu kỳ. (b) 642 chu kỳ $\approx 0{,}128\%$ ngân sách hop — đọc khung gần như "miễn phí" so với MAC 189×640.

Bài tập 5.6 (Warm-up và chốt rd_base**).**

a) Sau reset, cần tối thiểu bao nhiêu mẫu `valid` trước khung đầu? Tương đương bao nhiêu ms ở 16 kHz?

b) Vì sao phải chốt rd_base $= n_w$ lúc `frame_req` thay vì luôn đọc theo n_w đang chạy trong lúc burst?

Lời giải. (a) 640 mẫu $= 40$ ms. (b) Trong burst 640 chu kỳ, n_w có thể tăng khi có mẫu mới; nếu rd_ptr bám n_w động, tập địa chỉ bị trượt trong lúc đọc, khung không còn là một cửa sổ nhất quán tại thời điểm yêu cầu. Chốt rd_base đóng băng ảnh circular tại thời điểm req.

Bài tập 5.7 (`ramstyle` và Fitter). Fitter báo RAM suy diễn thành logic thay vì M10K. Hậu quả lên Old Data/latency? Cách ép?

Lời giải. RAM logic không còn hành vi block mixed-port Old Data chuẩn; timing và latency đổi, twin M10K không còn vàng. Ép `(* ramstyle = "M10K" *)` hoặc dùng IP On-Chip Memory 2-PORT, rồi đọc lại báo cáo Fitter xác nhận 1 M10K.

5.5. Liên kết Chương 0 và chương sau

Bài tập 0.3 của Chương 0 đã cảnh báo New Data trên Cyclone V: Chương 5 là chỗ hiện thực câu trả lời đó bằng IP + timing. Chương 6 (cửa sổ) sẽ nhân từng mẫu đọc từ Port B với hệ số Hanning/Hamming từ ROM; lệch latency đọc sẽ lệch luôn tích cửa sổ. Chương 7 (MAC) giả định khung L đã ổn định trên dây Avalon-ST — mọi sai Old Data ở đây biến thành sai $\mathcal{R}[\tau]$ không sửa được bằng "thêm DSP".

Checklist nghiệm thu Chương 5. (1) Fitter: đúng 1 M10K cho buffer 640×16. (2) Waveform: trùng địa chỉ ghi/đọc cho dout = Old Data. (3) Latency đọc khớp ghi chú lab (1 hoặc 2). (4) Warm-up ≥ 640 mẫu. (5) Khung oldest-first khớp twin trên vector $L = 8$ và trên dump $L = 640$ rút gọn. (6) Trong một hop, số chu kỳ đọc + ghi $\ll 500\,000$. Đủ sáu mục mới được coi là xong khối nhớ trượt.

Định mức overlap 75%. Với $L = 640$, $H = 160$, tỷ lệ chồng $480/640 = 0{,}75$. Đây là lựa chọn cân bằng giữa mượt đường F_0 và chi phí MAC mỗi giây (100 khung/s). Giảm overlap (tăng H) làm thưa điểm cao độ; tăng overlap (giảm H) nhân tải MAC gần tuyến tính.

Chương 6. Cửa sổ Hanning ROM × mẫu (Q1.15)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được vì sao khung âm thanh phải nhân cửa sổ Hanning trước khi tự tương quan; viết đúng công thức Praat ($N+1$) và đối chiếu dạng cổ điển ($N-1$); lượng tử hóa hệ số sang định dạng Q1.15; sinh tệp `hanning_rom.hex` khớp bit với twin Python; hiện thực phép nhân ROM×mẫu bằng một khối DSP trên Cyclone V; đọc latency đăng ký của On-Chip Memory; và đọc được ngân sách M10K / DSP / ALM của khối cửa sổ.

6.1. Hiện tượng thật: cắt khung không được "cắt đứt" sóng

Giọng nói là một dòng sóng liên tục trong không khí và trong ống thanh âm. Để đo chu kỳ rung dây thanh T_0, thuật toán Boersma/Praat không nhìn cả câu nói một lần; nó chỉ lấy một đoạn ngắn khoảng 40 ms — tương đương $L = 640$ mẫu ở tần số lấy mẫu $f_s = 16$ kHz — đúng như bản đồ khối ở Chương 0 đã ghi. Đoạn đó gọi là **khung phân tích (analysis frame)**. Mỗi 10 ms (hop $H = 160$ mẫu) khung trượt đi một bước, tạo ra một ước lượng F_0 mới.

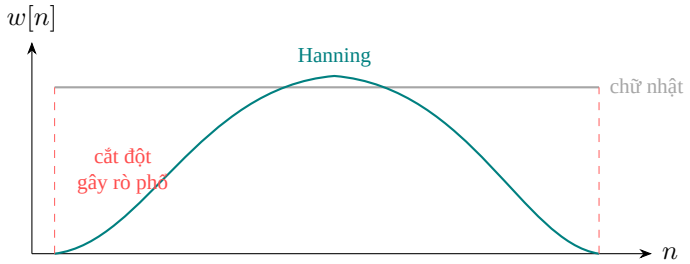
Nếu lấy đoạn đó đúng kiểu cắt bằng kéo — giữ nguyên biên độ đến mẫu cuối rồi nhảy đột ngột về 0 ngoài khung — thì đoạn đó, nhìn từ miền tần số, giống một xung chữ nhật nhân với tín hiệu thật. Phép nhân trong miền thời gian tương đương **tích chập (convolution)** phổ tín hiệu với phổ của xung chữ nhật. Phổ xung chữ nhật là hàm sinc: có một **thùy chính (mainlobe)** rộng và nhiều **thùy phụ (sidelobe)** cao khoảng -13 dB so với đỉnh. Chương 0 đã đổi -13 dB sang tỷ số biên độ $\approx 0{,}224$: nghĩa là mỗi thùy phụ còn khoảng một phần tư biên độ thùy chính — đủ lớn để "rò" năng lượng của hài $2F_0$, $3F_0$ sang đúng vùng tìm cao độ $75 \ldots 600$ Hz. Năng lượng giả đó gọi là **rò phổ (spectral leakage)**.

Hệ quả với cao độ không phải trừu tượng: đỉnh tự tương quan $\mathcal{R}_x[\tau]$ bị méo vì các tần số láng giềng tràn sang. Trực giác đời thường: bạn đang nghe một nốt đàn sạch, nhưng mỗi lần cắt đoạn lại "vỗ" hai đầu đoạn — tiếng vỗ đó tạo thêm tiếng lách cách phổ, không có trong bản gốc. Trên silicon, nếu quên cửa sổ, MAC ở Chương 7 vẫn tính đúng công thức, nhưng đỉnh τ có thể lệch vài mẫu chỉ vì hình dạng cắt khung.

Cửa sổ **Hanning** (còn gọi Hann) làm hai đầu khung về gần 0 một cách mượt, giống vuốt dần biên độ thay vì cắt đứt. Phổ Hanning có thùy phụ khoảng -32 dB (biên độ còn khoảng $1/40$ so với thùy chính — Chương 0). Thùy chính rộng hơn một chút (độ phân giải tần số kém hơn một ít so với chữ nhật cùng độ dài), nhưng với bài đo F_0 bằng tự tương quan thời gian thì đánh đổi này rất đáng: ta cần đỉnh $\mathcal{R}_x$ sạch hơn là phổ sắc như dao. Hamming gần Hanning nhưng nâng hai đầu lên $0{,}08$; gói này theo Praat nên chọn đúng Hanning $N+1$.

▶ NHÌN THẤY VẬT LÝ

Nhân cửa sổ $x[n] \cdot w[n]$ trên silicon chỉ là một phép nhân mỗi mẫu. Toàn bộ "phép màu" giảm rò phổ đã được nướng sẵn vào bảng hệ số $w[n]$. Không cần bộ sinh hàm lượng giác lúc chạy (CORDIC — thuật toán chỉ dùng dịch/cộng để xoay góc), cũng không cần tính cos mỗi mẫu — đó là lý do ROM thắng mọi cách tạo cửa sổ động trên đường pitch streaming.



Hình 29: Cửa sổ chữ nhật giữ biên độ đến sát biên rồi cắt đứt; Hanning vuốt hai đầu về 0, hạ thùy phụ từ khoảng -13 dB xuống khoảng -32 dB.

6.2. Toán từng bước — mọi ký hiệu trước khi dùng

Gọi L (hoặc N) là số mẫu trong một khung phân tích. Với sàn cao độ 75 Hz ở $f_s = 16$ kHz, một chu kỳ tối đa dài $f_s/75 \approx 213{,}3$ mẫu. Praat chọn khung dài khoảng ba chu kỳ của sàn đó: $L = 640$. Sau khi đã trừ mean cục

bộ (Chương 3), tín hiệu cửa sổ là

$$\tilde{x}[n] = (x[n] - \mu)\, w[n], \qquad n = 0, 1, \ldots, L-1.$$

Ký hiệu μ là trung bình số học của L mẫu trong khung; trừ μ trước khi nhân w để đỉnh $\mathcal{R}_x[0]$ không bị DC đẩy lên giả tạo.

Hai định nghĩa Hanning hay gặp — phải chọn một và ghi vào twin, vì khác nhau từng LSB:

1. **Praat ($N{+}1$), dùng trong golden `lab_1_3_5.py`:**

$$w[i] = 0{,}5 - 0{,}5\cos\Big(\frac{2\pi i}{N+1}\Big), \qquad i = 1, 2, \ldots, N.$$

   Hai đầu lớn hơn 0 một chút (không đúng bằng 0). Với $N = 640$, mẫu số là 641. Chỉ số i chạy $1 \ldots N$ rồi ánh xạ sang địa chỉ ROM $0 \ldots N-1$.

2. **Cổ điển / scipy ($N{-}1$):**

$$w[n] = \frac{1}{2}\Big(1 - \cos\frac{2\pi n}{N-1}\Big), \qquad n = 0, 1, \ldots, N-1.$$

   Hai đầu đúng bằng 0; đối xứng $w[n] = w[N-1-n]$ sạch — tiện lưu nửa bảng ROM (chỉ $N/2$ từ).

Gói FPGA này lấy dạng **Praat** $N{+}1$ làm hợp đồng bit-exact với `hanning_rom.hex` (đầy đủ 640 từ). Dạng cổ điển chỉ dùng trong ví dụ tính tay nhỏ và khi muốn minh họa tối ưu nửa bảng.

Lượng tử **Q1.15** (đã định nghĩa ở Chương 0: một bit dấu/phần nguyên, 15 bit phần thập phân; giá trị $1{,}0$ ánh xạ thành $2^{15} = 32768$, nhưng số có dấu 16-bit bão hòa tại $+32767$):

$$w_q[n] = \text{sat}_{16}\big(\text{round}(w[n] \cdot 32768)\big) \in [0, 32767]$$

(với hệ số Hanning không âm). Mẫu vào x_q cũng Q1.15. Tích rồi kéo về Q1.15 bằng làm tròn gần nhất:

$$\begin{aligned} y_{\text{acc}} &= x_q \cdot w_q && \text{(Q2.30, 32-bit có dấu),} \\ y_q &= \text{sat}_{16}\big((y_{\text{acc}} + 2^{14}) \ggg 15\big). \end{aligned}$$

Cộng $2^{14} = 16384$ trước khi dịch phải 15 bit chính là round-to-nearest trên số học hai's complement khi dùng arithmetic shift (`>>>` trong Verilog). Nếu quên cộng 16384 thì mọi tích bị thiên lệch xuống khoảng nửa LSB của Q1.15 — đủ để lệch twin vài chục LSB trên cả khung.

Năng lượng cửa sổ (dùng ở Chương 8 để chuẩn hóa):

$$\mathcal{R}_w[0] = \sum_{n=0}^{N-1} w_q[n]^2$$

tính offline trên int64; twin in giá trị này vào header hex để đối chiếu. Trên $N = 640$ Praat, script xác nhận $\mathcal{R}_w[0] = 258\,100\,276\,868$.

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao nhân rồi dịch 15, không giữ 32-bit suốt đường

Giữ y dạng Q2.30 suốt MAC sẽ phình bộ nhớ khung và DSP. Hợp đồng gói này: sau cửa sổ, mẫu vẫn Q1.15 — cùng định dạng với đầu vào — để MAC Chương 7 tái sử dụng cùng kiểu nhân 18×19. Mọi bit-growth được quản lý ở accum 41-bit của MAC, không ở cửa sổ.

6.3. Ví dụ tính tay — mọi số đã chạy Python

Mọi con số dưới đây được kiểm chứng bởi `verify/verify_chuong_06_09.py`.

A. Dạng cổ điển $N = 8$ (đối xứng nửa bảng). Công thức $w[n] = \frac{1}{2}\big(1 - \cos(2\pi n/7)\big)$, $n = 0..7$. Từng bước cho $n = 2$:

$$\cos\Big(\frac{4\pi}{7}\Big) \approx -0{,}222521, \quad w[2] = \tfrac{1}{2}\big(1 - (-0{,}222521)\big) = 0{,}611260.$$

Lượng tử minh họa bằng round$(w \cdot 32767)$ (hệ số dương tối đa vừa khít int16 dương):

n	$w[n]$ float	w_q
0	0.000000	0
1	0.188255	6169
2	0.611260	20029
3	0.950484	31145

Đối xứng: $w_q[7] = w_q[0] = 0$, $w_q[6] = w_q[1] = 6169$, $w_q[5] = w_q[2] = 20029$, $w_q[4] = w_q[3] = 31145$ — ROM vật lý chỉ cần 4 từ. Nhân tay: $x_q = 10000$, $w_q = 20029$:

$$\begin{aligned} \text{prod} &= 10000 \times 20029 = 200\,290\,000, \\ \text{rounded} &= 200\,290\,000 + 16\,384 = 200\,306\,384, \\ y_q &= 200\,306\,384 \ggg 15 = 6112. \end{aligned}$$

Đối chiếu float $10000 \times 0{,}611260 = 6112{,}6$ — khớp trong 1 LSB. Script assert `q15_mul(10000, 20029) == 6112`.

B. Dạng Praat $N = 8$ (khớp tinh thần golden). $w[i] = 0{,}5 - 0{,}5\cos(2\pi i/9)$, $i = 1..8$, lượng tử round$(w \cdot 32768)$ rồi bão hòa:

i	w float	w_q
1	0.116978	3833
2	0.413176	13539
3	0.750000	24576
4	0.969846	31780

Hai đầu không về 0 tuyệt đối — đúng hành vi Praat. Đối xứng quanh giữa: $w[i] = w[N + 1 - i]$ với cách đánh số Praat.

C. Khung thật $N = 640$ Praat. Các mốc Q1.15 (script assert): $w_q[0] = 1$, $w_q[1] = 3$, $w_q[319] = w_q[320] = 32767$, $w_q[638] = 3$, $w_q[639] = 1$. Tổng bình phương $\mathcal{R}_w[0] = \sum w_q^2 = 258\,100\,276\,868$ (int64). Giữa khung bão hòa tại $+32767$ (không phải 32768) vì giới hạn signed 16-bit — đó là chỗ twin và RTL phải cùng `clip`.

D. Dấu âm — cùng quy tắc làm tròn. $x_q = -10000$, $w_q = 20029$:

$$\begin{aligned} \text{prod} &= -200\,290\,000, \\ \text{rounded} &= -200\,290\,000 + 16\,384 = -200\,273\,616, \\ y_q &= -200\,273\,616 \ggg 15 = -6112. \end{aligned}$$

Script xác nhận `q15_mul(-10000, 20029) == -6112`. Arithmetic shift phải giữ dấu; logical shift sẽ phá số âm.

6.4. Ý nghĩa phần cứng — ROM On-Chip Memory + DSP

Cửa sổ Hanning *không* cần FIR Compiler II: hệ số bất biến với L cố định. Chọn IP **On-Chip Memory** (Quartus: `altsyncram` / ROM 1-PORT) khởi tạo bằng `.hex` hoặc `.mif`:

- **Width** $= 16$ (Q1.15 unsigned-looking nhưng đọc ra signed khi nhân với x_q).
- **Depth** $= 640$ (đầy đủ, khớp Praat) hoặc 320 nếu dùng dạng cổ điển nửa bảng.

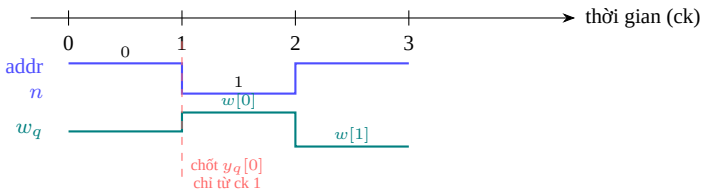
- **Registered output**: bắt buộc bật — trên Cyclone V, đọc M10K có latency đăng ký 1 chu kỳ đồng hồ. FSM phải lấy w_q *một cạnh sau* khi đưa địa chỉ n.
- **Init file**: đường dẫn tới `hanning_rom.hex`; mỗi dòng một từ hex 4 chữ số (`04X`).
- Một khối **DSP** (*Digital Signal Processing* block) trên Cyclone V nhân 18×19 mỗi mẫu: $x_q \times w_q$. Có thể suy diễn thành `*` trong Verilog; Quartus map vào DSP cứng nếu bật "Auto DSP".

Không gọi FIR Compiler chỉ để nhân cửa sổ: lãng phí logic cấu hình systolic và latency hàng chục chu kỳ không cần thiết. Đổi L hoặc công thức w thì **bắt buộc rebuild hex** và nạp lại ROM; quên bước này thì F_0 sai im lặng — testbench vẫn "pass" nếu chỉ kiểm handshake.

Bảng timing chu kỳ — ROM registered + nhân tổ hợp (minh họa). Giả sử nhân suy diễn tổ hợp (0 pipeline DSP), registered ROM:

Chu kỳ k	Địa chỉ ROM	w_q hợp lệ	y_q / handshake
0	đặt n=0, `in_valid`	— (chưa)	chờ
1	n=0 vẫn giữ	$w_q[0]$ ra	$y_q[0]$ chốt nếu `in_ready`
2	đặt n=1	$w_q[0]$ (cũ) rồi $w_q[1]$	$y_q[1]$
⋮	⋮	⋮	⋮
640	n=639 đã đặt từ k=639	$w_q[639]$	mẫu cuối khung

Nếu DSP nhân cũng đăng ký 1 chu kỳ (IP LPM_MULT pipeline $= 1$), thêm một hàng trễ: tổng latency địa chỉ$\rightarrow y_q$ thành 2 chu kỳ. Lab phải ghi rõ "inferred 1 ck" hay "IP output register 2 ck" trước khi viết FSM — lệch một mẫu làm lệch toàn bộ tích MAC $\mathcal{R}[\tau]$ (bài tập Chương 5 đã cảnh báo).



Hình 30: ROM On-Chip Memory bật registered output: địa chỉ đặt ở cạnh 0, w_q chỉ ổn định từ cạnh 1. Nhân/chốt y_q trước cạnh 1 là đọc nhầm hệ số.

Ví dụ tính tay — latency 1 ck trên 3 mẫu. Đặt $x_q = [10000, 20000, 30000]$, $w_q = [20029, 31145, 0]$ (cổ điển $N = 8$). Nếu FSM đúng latency: y lần lượt `q15_mul` cho $(10000, 20029) \rightarrow 6112$, $(20000, 31145) \rightarrow 19009$, $(30000, 0) \rightarrow 0$. Nếu bỏ chờ 1 ck và nhân với w trễ một mẫu, cặp thành $(10000, X)$, $(20000, 20029)$, $(30000, 31145)$ — sai hoàn toàn. Script assert hai tích đầu; tích với 0 là hệ quả trực tiếp của định nghĩa.

RTL datapath — giải thích từng dòng.

```
module window_hanning #(
  parameter N = 640
)(
  input  wire               clk, rst_n,
  input  wire               in_valid,
  output wire               in_ready,
  input  wire signed [15:0] x_q,        // Q1.15
  output reg                out_valid,
  input  wire               out_ready,
  output reg  signed [15:0] y_q         // Q1.15 windowed
);
  (* ram_init_file = "hanning_rom.hex" *)
  reg [15:0] rom [0:N-1];
```

```
  reg [9:0]  n;                          // 0..N-1
  // Lưu ý: rom[n] đọc tổ hợp ở đây; trên silicon thật
  // nên tách reg w_q_r <= rom[n] (latency 1) rồi nhân.
  wire signed [15:0] w_q = rom[n];
  wire signed [31:0] prod = x_q * w_q;  // Q2.30
  wire signed [31:0] rounded = prod + 32'sd16384;
  wire signed [31:0] shifted = rounded >>> 15;
  function automatic signed [15:0] sat16;
    input signed [31:0] v;
    begin
      if (v > 32'sd32767) sat16 = 16'sd32767;
      else if (v < -32'sd32768) sat16 = -16'sd32768;
      else sat16 = v[15:0];
    end
  endfunction
  assign in_ready = out_ready | ~out_valid;
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin n <= 0; out_valid <= 0; y_q <= 0; end
    else if (in_valid && in_ready) begin
      y_q <= sat16(shifted);
      out_valid <= 1'b1;
      n <= (n == N-1) ? 10'd0 : n + 10'd1;
    end else if (out_valid && out_ready) out_valid <= 1'b0;
  end
endmodule
```

Handshake Avalon-ST dùng `in_ready`/`out_ready` (Chương 0): downstream chưa sẵn sàng thì upstream giữ `valid`. Hàm `sat16` bão hòa (không wrap). `ram_init_file` bảo Quartus nạp hex lúc cấu hình.

Nếu muốn tiết kiệm một nửa BRAM với dạng cổ điển: địa chỉ `(n < 320) ? n : (639-n)` và ROM depth 320. Twin phải cùng công thức — không trộn Praat hex với địa chỉ nửa bảng cổ điển.

6.5. Twin Python bit-exact

Twin không phải “code tham khảo đẹp”; nó là hợp đồng số học. Mọi thay đổi round/sat phải sửa **cùng chỗ** RTL và twin.

```
import numpy as np
Q15 = 1 << 15

def hanning_praat_q15(N=640):
    i = np.arange(1, N + 1, dtype=np.float64)
    w = 0.5 - 0.5 * np.cos(2.0 * np.pi * i / (N + 1))
    return np.clip(np.round(w * Q15), 0, Q15 - 1).astype(np.int64)

def apply_window(xq, wq):
    # Khớp RTL: (x*w + 2^14) >>> 15, rồi sat ngoài nếu cần
    prod = xq.astype(np.int64) * wq.astype(np.int64)
    return ((prod + (1 << 14)) >> 15).astype(np.int64)

wq = hanning_praat_q15(640)
with open("hex/hanning_rom.hex", "w") as f:
    for v in wq:
        f.write(f"{int(v) & 0xFFFF:04X}\n")
print("Rw0 =", int(np.sum(wq * wq)))
```

Giải thích từng khối: `arange(1,N+1)` khớp chỉ số Praat; `round` rồi `clip` khớp sat dương; `apply_window`

không dùng float nhân. Script kiểm số tay `verify/verify_chuong_06_09.py` — chạy trước mọi lần commit hex.

6.6. Chi phí silicon — viết tắt mở rộng lần đầu trong chương

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách khối cửa sổ.

- **M10K** (khối BRAM nhúng $10\,240$ bit trên Cyclone V, Chương 0): ROM đầy đủ $640 \times 16 = 10\,240$ bit = đúng 1 M10K. Nửa bảng cổ điển $320 \times 16 = 5\,120$ bit vẫn chiếm 1 M10K (không chia nhỏ hơn một block) — tiết kiệm độ sâu địa chỉ, không tiết kiệm số block nếu chỉ một ROM.
- **DSP**: 1 nhân 18×19 mỗi mẫu cửa sổ (có thể dùng chung với MAC ở Pha 2 nếu FSM tuần tự chia sẻ một DSP theo thời gian).
- **ALM** (*Adaptive Logic Module* — đơn vị logic cơ bản Cyclone V, gồm LUT look-up table + flip-flop): con trỏ n + sat + handshake $\lesssim 80$ ALM.
- **Fmax** (tần số đồng hồ tối đa sau place-and-route): $\gg 50$ MHz — đường nhân một chu kỳ không phải nút thắt.
- **Latency**: 1 chu kỳ đọc ROM (+0/1 chu kỳ nhân pipeline). Trên hop 10 ms $= 500\,000$ chu kỳ @ 50 MHz, cửa sổ 640 chu kỳ chiếm $0{,}128\%$ ngân sách.
- Vì sao ROM: $w[n]$ bất biến — nướng offline một lần rẻ hơn CORDIC/sinh cos lúc chạy; FIR Compiler (Chương 4) dành cho lọc đa tốc độ, không phải bảng 640 từ cố định.

6.7. Bài tập và lời giải giải thích

Bài tập 6.1. Sinh hex Praat $N = 8$; đối chiếu bốn từ đầu với bảng mục 6.3.B.

Bài tập 6.2. RTL sim: khung toàn mẫu 32767 nhân Hanning — mô tả biên độ y ở hai đầu và giữa khung. Giải thích vì sao.

Bài tập 6.3. Cố ý dùng hex Praat nhưng địa chỉ nửa bảng cổ điển: $\mathcal{R}_x[\tau]$ lệch twin thế nào?

Bài tập 6.4. Tính tay $x_q = -10000$, $w_q = 20029$ với cùng quy tắc $+16384$ rồi $\ggg 15$. Kết quả?

Bài tập 6.5. On-Chip Memory bật registered output. Vẽ timeline 3 chu kỳ đầu: khi nào `address`, khi nào `q` ổn định, khi nào được phép chốt y_q. Nếu FSM bỏ qua 1 chu kỳ chờ, mẫu cửa sổ nào bị lệch?
Lời giải giải thích.

1. Kỳ vọng bốn từ đầu 3833, 13539, 24576, 31780 (script đã assert).
2. Giữa khung $w \approx 1$ nên $y \approx 32767$; hai đầu w nhỏ nên y nhỏ — đúng hình quả núi Hanning. Đây là hệ quả trực tiếp của công thức Praat, không phải lỗi bão hòa.
3. Hệ số sai đối xứng làm méo $\mathcal{R}_x$, đỉnh τ lệch vài mẫu, dẫn đến sai vài chục cent khi đối chiếu Praat ở Chương 13.
4. Tích $-200\,290\,000$; cộng 16384 rồi arithmetic $\ggg 15$ cho -6112 (đối xứng với ví dụ dương ở mục 6.3.A). Script xác nhận `q15_mul(-10000, 20029) == -6112`.
5. Chu kỳ 0: đặt address$= 0$. Chu kỳ 1: $q = w_q[0]$ ổn định — chỉ lúc này mới nhân với $x[0]$. Chu kỳ 2: address$= 1$, q chuyển sang $w_q[1]$. Bỏ chờ 1 ck khiến $x[0]$ nhân nhầm w cũ (hoặc X) — cả khung lệch một mẫu, MAC $\mathcal{R}[\tau]$ dịch hệ thống.

Chương 7. Tự tương quan — MAC mặc định / FFT tùy chọn + BFP

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn viết được định nghĩa tự tương quan không vòng từ nguyên lý đầu tiên; tính tay từng giá trị $\mathcal{R}[\tau]$ trên chuỗi ngắn và trên khung cosin chu kỳ 8; thiết kế datapath MAC tuần tự bằng một khối DSP và bộ tích lũy rộng kèm bảng timing chu kỳ; giải thích FFT IP radix-4 / BFP từng tầng cánh bướm và barrel shift 2^{2e} trước IFFT; viết twin Python bit-exact; và tính được ngân sách chu kỳ / RTF so với bước nhảy 10 ms.

7.1. Hiện tượng thật: đo mức "giống chính mình"

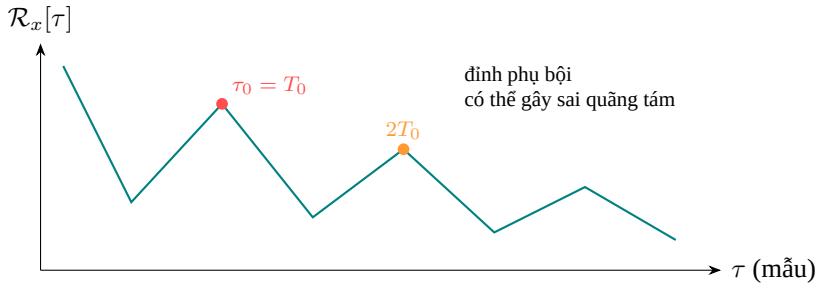
Khi dây thanh quản rung gần tuần hoàn với chu kỳ cơ bản T_0 giây, sóng áp suất tại thời điểm t gần trùng với sóng tại $t + T_0$. Trên lưới mẫu $f_s = 16$ kHz, chu kỳ đó là $\tau_0 = f_s \cdot T_0$ mẫu. Ví dụ giọng nam khoảng 100 Hz cho $\tau_0 = 160$; giọng nữ khoảng 200 Hz cho $\tau_0 = 80$. Phép **tự tương quan (autocorrelation)** đo mức giống nhau đó bằng cách trượt một bản sao đi τ mẫu (**độ trễ — lag**), nhân từng cặp mẫu chồng nhau và cộng dồn. Tại $\tau = \tau_0$, các cặp cùng pha cho tích dương lớn, tạo đỉnh nổi trên đồ thị $\mathcal{R}_x[\tau]$. Đo được τ_0 thì suy ra ngay $F_0 = f_s/\tau_0$.

Đây là hiện tượng vật lý thật của sóng tuần hoàn, không chỉ là quy ước toán học. Định lý **Wiener–Khinchin** (Chương 0) chứng minh rằng $\mathcal{R}_x$ cũng bằng biến đổi Fourier ngược của mật độ phổ công suất $|X[k]|^2$, nên có hai đường silicon tương đương về toán: nhân–cộng trực tiếp (MAC) hoặc FFT $\to |X|^2 \to$ IFFT. Với dải trễ hẹp $\tau = 26 \ldots 214$ (189 giá trị, khớp `lab_1_3_5.py` và bản đồ Chương 0), đường mặc định trên Cyclone V là **MAC** (*Multiply-ACcumulate*): chỉ tính đúng những τ cần thiết, không tính toàn bộ phổ rồi mới cắt. Mỗi hop 10 ms chỉ có 500 000 chu kỳ ở 50 MHz; 98 280 phép MAC chiếm khoảng một phần năm ngân sách đó và vẫn để lại biên cho cửa sổ, chuẩn hóa và chia F_0.

Tại sao không lấy đỉnh phổ trực tiếp? Vì phổ biên độ của nguyên âm có đỉnh formant (cộng hưởng ống thanh) có thể cao hơn đỉnh F_0; tự tương quan thời gian "nhìn" chu kỳ lặp của toàn sóng, ít bị formant đánh lừa hơn — đó là lý do Praat chọn ACF (sau chuẩn hóa) làm lõi đo cao độ.

▶ NHÌN THẤY VẬT LÝ

Tự tương quan không "tạo ra" cao độ. Nó chỉ đo độ trùng pha giữa tín hiệu và bản sao trễ của chính nó. Nếu khung đã bị rò phổ hoặc còn DC (Chương 3, 6), đỉnh giả sẽ cạnh tranh với đỉnh thật — MAC tính đúng công thức vẫn báo sai F_0.



Hình 31: Hàm tự tương quan của khung hữu thanh: đỉnh toàn cục tại $\tau = 0$ (năng lượng), đỉnh cục bộ đầu tiên tại $\tau_0 = T_0$ là ứng viên cao độ; đỉnh $2T_0$ là ứng viên sai quãng tám.

7.2. Toán từng bước — định nghĩa gốc trước mọi phím tắt

Hàm tự tương quan **không vòng (linear / non-circular)** của khung $x[n]$ độ dài L được định nghĩa từ tích vô hướng của hai đoạn chồng:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x[n] \cdot x[n+\tau], \qquad \tau \geq 0.$$

Từng bước: (1) chọn τ; (2) lấy cặp $(x[0], x[\tau])$, $(x[1], x[\tau+1])$, …, $(x[L-1-\tau], x[L-1])$; (3) nhân từng cặp; (4) cộng tất cả tích. Không được viết tắt thành "chỉ một hạng $x[0]x[\tau]$".

Mỗi ký hiệu:

- $x[n]$: mẫu đã trừ mean và đã nhân cửa sổ Hanning (Chương 3, 6), định dạng Q1.15 trên silicon.
- τ: độ trễ tính bằng số mẫu. Dải quét phần cứng: $\tau \in [26, 214]$ (189 giá trị) để đỉnh tìm trong $[27, 213]$ luôn có láng giềng cho nội suy parabolic (Chương 9). Sàn $75\,\text{Hz} \leftrightarrow \tau \approx 213$; trần $600\,\text{Hz} \leftrightarrow \tau \approx 27$.
- Số hạng trong tổng đúng bằng $L - \tau$: trễ càng lớn, càng ít cặp chồng — đây là thiên vị hình học sẽ được bù ở Chương 8.
- Tại $\tau = 0$: $\mathcal{R}_x[0] = \sum_n x[n]^2$ là năng lượng khung (cực đại toàn cục).

  Định lý Wiener–Khinchin cho phép đường tương đương:

$$\mathcal{R}_x[\tau] = \mathcal{F}^{-1}\{|X[k]|^2\}.$$

FFT mặc định tính tự tương quan **vòng (circular)**: mẫu cuối "quấn" sang mẫu đầu. Muốn khớp định nghĩa không vòng ở trên phải zero-pad ít nhất thêm L số 0 trước FFT (thường pad lên $N_{\text{FFT}} = 1024$ hoặc 2048). Đó là một trong những lý do đường mặc định pitch chọn MAC.

Trên silicon, mỗi tích $x[n] \cdot x[n+\tau]$ là Q1.15×Q1.15 = Q2.30 (32-bit). Cộng dồn đến 640 hạng tử đòi hỏi bộ tích lũy rộng: thiết kế dùng **41-bit có dấu** (khớp stub `rtl/mac_autocorr.v` và comment trong `lab_1_3_5.py`). Ước lượng thô: $|x| \le 2^{15}$, $|x \cdot x| \le 2^{30}$, $640 \cdot 2^{30} < 2^{40}$ — 41 bit có dấu vừa đủ biên an toàn một bit.

7.3. Ví dụ tính tay — mọi số đã chạy Python

A. Chuỗi cực ngắn $x = [1, 2, 1, 2]$. Từng cặp:

$$\begin{aligned}
\mathcal{R}[0] &= 1\cdot 1 + 2\cdot 2 + 1\cdot 1 + 2\cdot 2 = 1 + 4 + 1 + 4 = 10,\\
\mathcal{R}[1] &= 1\cdot 2 + 2\cdot 1 + 1\cdot 2 = 2 + 2 + 2 = 6,\\
\mathcal{R}[2] &= 1\cdot 1 + 2\cdot 2 = 1 + 4 = 5,\\
\mathcal{R}[3] &= 1\cdot 2 = 2.
\end{aligned}$$

Tỉ số thô $\mathcal{R}[2]/\mathcal{R}[0] = 0{,}5$. Sau khi nhân $C[\tau]$ (Chương 8) và chia theo số hạng, đỉnh chu kỳ được kéo về gần 1.

B. Khung cosin chu kỳ 8 mẫu, $N = 16$ (Q1.15). $x[n] = \text{sat}_{16}(\text{round}(\cos(2\pi n/8) \cdot 32768))$, $n = 0..15$. Từng bước cho $n = 0$: $\cos 0 = 1$, $\text{round}(32768) = 32768$, sat $\to 32767$. Cho $n = 4$: $\cos \pi = -1$, $\text{round}(-32768) = -32768$ (vừa khít min int16).

n	0	1	2	3	4	5	6	7
x	32767	23170	0	−23170	−32768	−23170	0	23170
n	8	9	10	11	12	13	14	15
x	32767	23170	0	−23170	−32768	−23170	0	23170

Tự tương quan không vòng (script):

$$\begin{aligned}
\mathcal{R}[0] &= 8\,589\,627\,426,\\
\mathcal{R}[8] &= 4\,294\,813\,713 \approx \tfrac{1}{2}\mathcal{R}[0],\\
\mathcal{R}[4] &= -6\,442\,220\,568 \quad \text{(chống pha, không phải đỉnh dương).}
\end{aligned}$$

Tỉ số thô $\mathcal{R}[8]/\mathcal{R}[0] = 0{,}5$ vì chỉ còn 8 cặp; sau chuẩn hóa unbiased $(\mathcal{R}[8]/8)/(\mathcal{R}[0]/16) = 1$. Đây là lý do Chương 8 tồn tại.

C. Ngân sách chu kỳ trên đường MAC thật. Với $\tau = 26 \ldots 214$, tổng số phép nhân–cộng đúng bằng

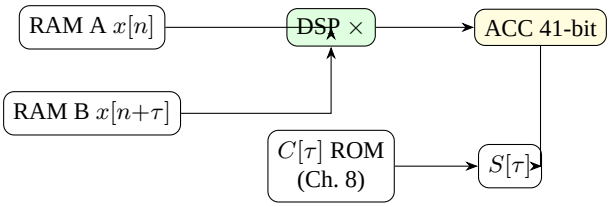
$$N_{\text{MAC}} = \sum_{\tau=26}^{214} (640 - \tau).$$

Tính tổng cấp số: số hạng 189; số hạng đầu $640 - 26 = 614$; số hạng cuối $640 - 214 = 426$; tổng $= 189 \cdot (614 + 426)/2 = 98\,280$. Không phải $189 \times 640 = 120\,960$ — công thức naïve đếm thừa. Tại $f_{\text{clk}} = 50\,\text{MHz}$:

$$T_{\text{compute}} = \frac{98\,280}{50 \times 10^6} = 1{,}9656\,\text{ms}, \qquad \text{RTF} = \frac{1{,}9656}{10} = 0{,}19656 \ll 1.$$

7.4. Ý nghĩa phần cứng — MAC mặc định

Hai con trỏ đọc khung trong BRAM True Dual-Port (Chương 5): một trỏ tới $x[n]$, một tới $x[n+\tau]$. Mỗi chu kỳ một khối DSP nhân 18×18 (hoặc 18×19), cộng vào bộ tích lũy 41-bit có dấu. Quét τ tuần tự. Sau $(L - \tau)$ bước, thanh ghi accum chính là $\mathcal{R}_x[\tau]$.



Hình 32: Datapath MAC: hai toán hạng từ TDP RAM, một nhân DSP, một tích lũy — khớp vòng `for tau` trong `lab_1_3_5.py`.

Bảng timing chu kỳ — một τ cố định (ví dụ $\tau = 2$, $L = 4$).

ck	Trạng thái	n	xa=$x[n]$, xb=$x[n+\tau]$	`acc` sau cạnh
0	IDLE→LOAD	—	—	clear 0
1	RUN	0	$x[0], x[2]$	$x[0]x[2]$
2	RUN	1	$x[1], x[3]$	$x[0]x[2] + x[1]x[3]$
3	DONE	—	—	$= \mathcal{R}[2]$; `done_tau`=1

Với $\tau = 26$, $L = 640$: pha RUN kéo dài đúng 614 chu kỳ. FSM ngoài tăng τ và lặp. Không được clear `acc` giữa các n của cùng một τ.

RTL lõi (ý tưởng, khớp stub `mac_autocorr.v`).

```
module mac_autocorr (
  input  wire                clk, rst_n, start,
  input  wire signed [15:0] xa, xb,   // x[n], x[n+tau] Q1.15
  input  wire [9:0]          n_pairs,  // = L - tau
  output reg  signed [40:0] acc,      // 41-bit signed
  output reg                 done_tau
);
  // FSM: IDLE -> RUN (n_pairs cycles: acc <= acc + xa*xb) -> DONE
  // xa, xb phải ổn định đúng cạnh khi acc cập nhật (đồng bộ với
  // latency đọc TDP RAM --- thường 1 ck registered).
endmodule
```

Không dùng `numpy.correlate` float làm chuẩn silicon: twin phải nhân–cộng int cùng độ rộng accum.

7.5. Đường tùy chọn: FFT IP + BFP + barrel shift trước IFFT

Chỉ bật sau khi MAC đã xanh bit-exact. Dùng **FFT Intel FPGA IP** (User Guide 683374):

- Kiến trúc **radix-4** (hoặc radix-2): mỗi tầng cánh bướm (**butterfly**) kết hợp 4 (hoặc 2) điểm với twiddle $W_N^k = e^{-j2\pi k/N}$ lấy từ ROM nội bộ. Với $N = 1024$ radix-4 cần $\log_4 1024 = 5$ tầng; radix-2 cần 10 tầng.
- Streaming hoặc buffered; tín hiệu SOP/EOP đánh dấu một block N điểm trên Avalon-ST.
- **Block Floating Point (BFP — số chấm động khối)**: mọi bin chia sẻ một số mũ nguyên e; mantissa dịch sao cho max $|m|$ vừa khít bit. Sau mỗi tầng butterfly, nếu có nguy cơ tràn thì toàn vector dịch phải 1 bit và e tăng 1. IP xuất `source_exp`.
- BFP **không** có cổng nhập exponent vào IFFT — kỹ sư phải scale bên ngoài trước khi đưa mantissa vào IFFT.

BFP từng tầng — ví dụ mantissa 8-bit, 2 tầng. Bắt đầu stage 0: $m = [40, -30, 10, -5]$, $e = 0$. Ngưỡng tràn minh họa $= 32$ (giả sử còn 1 bit đầu). Vì max $|m| = 40 > 32$:

Sự kiện	e	Mantissa	Ghi chú
Vào stage 0	0	$[40, -30, 10, -5]$	max $= 40 > 32$
Sau scale stage 0	1	$[20, -15, 5, -2]$	dịch phải 1
Sau stage 1 (không tràn)	1	$[20, -15, 5, -2]$	giữ e
Power bin 0 thô $\|m\|^2$	—	400	quên 2^{2e}
Power đúng $\|m\|^2 \cdot 2^{2e}$	—	1600	$= 40^2$

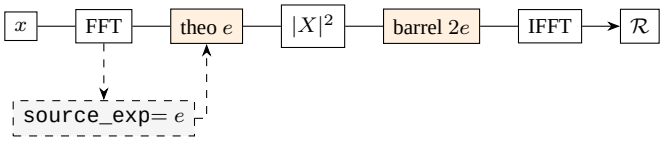
★ NGUYÊN LÝ ĐẦU TIÊN: Sau $|X|^2$, số mũ nhân đôi thành $2e$

Mantissa $y = 3$, exponent $e = 2$ có giá trị thật $3 \cdot 2^{-2}$. Bình phương:

$$(3 \cdot 2^{-2})^2 = 9 \cdot 2^{-4}$$

— số mũ thành $2e$. Đẩy mantissa $|y|^2$ thẳng vào IFFT (IP kỳ vọng fixed-point với exponent $= 0$ ở lối vào) mà quên barrel-shift theo $2e$ sẽ làm cả dải động $\mathcal{R}[\tau]$ sai tỷ lệ, phá ngưỡng voicing.

Barrel shift trước IFFT. RTL bắt buộc: nếu `source_exp`$= e$ sau FFT, sau khi tính $|X[k]|^2$ trên mantissa, dịch trái (hoặc nhân) kết quả theo $2e$ trước khi đưa vào IFFT — hoặc dịch mantissa IFFT input tương đương. Twin phải dùng model `_model` của variation FFT IP, không đoán e.



Hình 33: Đường FFT/Wiener–Khinchin: hai khối cam (scale theo e và barrel 2^{2e}) là RTL bắt buộc. Quên chúng gần như chắc fail bit-exact — đó là lý do mặc định vẫn là MAC.

Vì sao MAC thắng khi $\tau_{\max} \ll L$. MAC: 98 280 nhân thực/khung. FFT 1024 radix-4: 5 tầng, mỗi tầng 256 butterfly phức (mỗi butterfly vài nhân thực) — cộng IFFT tương tự cộng $|X|^2$, rồi vẫn phải cắt τ. Thêm rủi ro bit-reverse và BFP. Chỉ mở FFT khi lab yêu cầu so khớp đường Praat FFT hoặc khi RTF MAC không đủ.

Bit-reverse, zero-pad — hai bẫy của đường FFT. **Bit-reverse** là thứ tự địa chỉ khi đảo từng bit của chỉ số bin (ví dụ bin $6 = 110_2$ thành $011_2 = 3$). FFT IP có thể xuất natural order hoặc bit-reverse tùy cấu hình; twin và RTL sau $|X|^2$ phải cùng một thứ tự trước IFFT, nếu không đỉnh τ_0 sẽ "nhảy chỗ". **Zero-pad** nghĩa là thêm số 0 vào cuối khung trước FFT để tự tương quan vòng không "quấn" đuôi vào đầu: khung $L = 640$ cần pad ít nhất thêm 640 số 0 (thường lên $N_{\text{FFT}} = 2048$). MAC không bao giờ gặp hai bẫy này vì nó chỉ cộng đúng $L - \tau$ hạng thật.

7.6. Twin Python bit-exact

```
import numpy as np

def acf_linear_int(x, tau_min, tau_max):
    x = np.asarray(x, dtype=np.int64)
    L = len(x)
    out = {}
    for tau in range(tau_min, tau_max + 1):
        # Khớp MAC: cộng dồn int, không float
        out[tau] = int(np.dot(x[: L - tau], x[tau:]))
    return out

# Khớp lab: tau 26..214, accum int64 (>= 41-bit signed trên RTL)
R = acf_linear_int(xw_q15, 26, 214)
```

```
# So dump R[tau] với ModelSim: sai lệch 0 bit trên đường MAC
```

Giải thích: `np.dot` trên `int64` tương đương vòng RUN của FSM; không lấy `numpy.correlate` float. Trên nhánh FFT, twin phải gồm cả barrel 2^{2e} giống RTL và so với `_model` của IP.

Ví dụ tính tay — ngân sách RTF an toàn. Với $N_{\text{MAC}} = 98\,280$ đã tính ở mục 7.3.C, thêm giả sử 640 chu kỳ cửa sổ + 189 chu kỳ nhân C + 300 chu kỳ peak/chia $\approx 99\,409$ chu kỳ $\approx 1{,}988$ ms @ 50 MHz. Lab dành biên 3 ms (RTF $0{,}3$) để chứa thêm latency đọc RAM đăng ký và handshake Avalon-ST. Script khóa các số lõi; biên 3 ms là quy ước nghiệm thu, không phải assert tuyệt đối.

7.7. Chi phí silicon

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách MAC autocorr (default).

- **DSP**: 1 khối MAC/chu kỳ; 98 280 phép/khung $\approx 1{,}97$ ms @ 50 MHz.
- **ALM**: ~ 200 cho FSM con trỏ τ, n, handshake Avalon-ST.
- **M10K**: đệm khung dùng chung Chương 5; MAC không thêm BRAM.
- **RTF** $= 0{,}197 \ll 1$ (còn $> 80\%$ thời gian rảnh mỗi hop 10 ms $= 500\,000$ chu kỳ). Vì sao vừa khít: ngay cả khi thêm cửa sổ + $C[\tau]$ + parabolic + chia, tổng vẫn dưới khoảng 3 ms, RTF $< 0{,}3$.
- **FFT optional**: hàng nghìn ALM + nhiều M10K twiddle/buffer + DSP theo variation; barrel shifter vài chục ALM. Chỉ bật khi twin đã khóa và lab yêu cầu.
- Vì sao MAC mặc định: 0 rủi ro BFP/bit-reverse, 0-bit với golden `lab_1_3_5.py`.

7.8. Bài tập và lời giải giải thích

Bài tập 7.1. Với $x = [1, 2, 1, 2]$, tính $\mathcal{R}[1]$ và $\mathcal{R}[2]$ từ định nghĩa gốc. Có được phép viết $\mathcal{R}[2] = x[0]x[2]$ một hạng duy nhất không?

Bài tập 7.2. Trên khung cosin mục 7.3.B, vì sao $\mathcal{R}[8]/\mathcal{R}[0] = 0{,}5$ dù tín hiệu lặp hoàn hảo? Làm thế nào để tỉ số về gần 1?

Bài tập 7.3. Mantissa sau FFT $m = 20$, $e = 1$. Tính power $|m|^2 \cdot 2^{2e}$. Nếu quên nhân 2^{2e}, sai bao nhiêu lần?

Bài tập 7.4. Tính N_{MAC} đúng bằng $\sum_{\tau=26}^{214}(640 - \tau)$ và RTF @ 50 MHz. So với ước lượng naïve 189×640.

Bài tập 7.5. Vẽ bảng timing 4 chu kỳ cho $\tau = 1$, $L = 4$, $x = [1, 2, 1, 2]$: mỗi cạnh `acc` mang giá trị gì? `done_tau` lên khi nào?
Lời giải giải thích.

1. $\mathcal{R}[1] = 6$, $\mathcal{R}[2] = 5$. Không được chỉ lấy một hạng: định nghĩa cộng mọi cặp chồng, với $\tau = 2$ còn đúng hai hạng $1 \cdot 1 + 2 \cdot 2$.
2. Vì chỉ còn $N - \tau = 8$ cặp trong tổng không vòng, trong khi $\mathcal{R}[0]$ có 16 hạng. Chuẩn hóa unbiased chia cho số hạng, hoặc nhân $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (Chương 8), kéo tỉ số về gần 1.
3. Power $= 400 \cdot 4 = 1600$. Quên 2^{2e} còn 400 — sai đúng $4 = 2^{2e}$ lần (khoảng $+6$ dB nếu đo năng lượng sai chiều ngược), đủ phá ngưỡng voicing.
4. $N_{\text{MAC}} = 98\,280$, $T = 1{,}9656$ ms, RTF $\approx 0{,}197$. Naïve 120 960 lớn hơn khoảng 23% vì đếm thừa các hạng không tồn tại khi $\tau > 0$.
5. ck1: `acc`$= 1 \cdot 2 = 2$; ck2: $2 + 2 \cdot 1 = 4$; ck3: $4 + 1 \cdot 2 = 6 = \mathcal{R}[1]$; `done_tau` sau đúng $L - \tau = 3$ bước RUN.

Chương 8. Chuẩn hóa $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (ROM nhân thay chia)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được vì sao phải chia cho tự tương quan của cửa sổ; dẫn ra $C[\tau]$ từ nguyên lý đầu tiên từng bước; tính tay và kiểm Python một bảng nhỏ cùng các mốc trên khung $L = 640$; cấu hình ROM M10K mang $C[\tau]$ dạng Q3.13 kèm latency đăng ký; đối chiếu với LPM_DIVIDE / bộ chia non-restoring khi $\mathcal{R}_w$ đổi lúc chạy; viết twin sinh `.hex` bit-exact; và đọc được hộp chi phí — tại sao nhân thay chia runtime.

8.1. Hiện tượng thật: đỉnh τ lớn bị "đạp" vì hai quả núi ít chồng

Sau khi nhân cửa sổ Hanning (Chương 6), mỗi khung trở thành một "quả núi": biên độ lớn ở giữa, nhỏ ở hai đầu. Nếu nhìn dạng sóng sau cửa sổ trên máy hiện sóng, hai đầu gần im lặng còn giữa khung vẫn giữ chu kỳ dây thanh — đúng ý đồ: giảm discontinuity mà không xóa thông tin T_0 ở phần năng lượng chính. Khi tính $\mathcal{R}_x[\tau]$, ta đang đo độ chồng giữa quả núi đó và bản sao trễ τ mẫu. Trễ càng lớn, hai quả núi chồng nhau càng ít — năng lượng chồng đó chính là tự tương quan của *chính cửa sổ*:

$$\mathcal{R}_w[\tau] = \sum_{n=0}^{L-1-\tau} w[n]\, w[n+\tau].$$

Nếu lấy $\mathcal{R}_x[\tau]$ thô làm điểm số, đỉnh ở τ lớn (giọng trầm, T_0 dài) bị đạp xuống chỉ vì hình học cửa sổ, không phải vì tín hiệu kém tuần hoàn. Ví dụ khung cosin ở Chương 7 cho $\mathcal{R}[8]/\mathcal{R}[0] = 0{,}5$ dù sóng lặp hoàn hảo — đó đúng là hiệu ứng này: tám cặp chồng so với mười sáu hạng ở $\tau = 0$.

Trực giác đời thường: bạn so hai tờ giấy có hình quả núi in mực; kéo lệch chúng theo chiều ngang thì vùng mực chồng nhau nhỏ dần. Không phải vì hình vẽ xấu đi — chỉ vì hình học chồng. $C[\tau]$ chính là hệ số "kéo độ đậm" lại cho công bằng giữa các vị trí lệch.

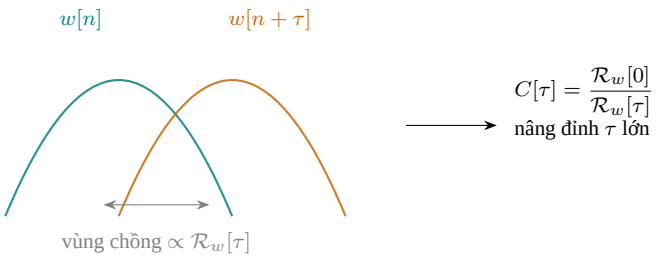
Praat/Boersma nhân hệ số bù

$$C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}$$

để kéo mọi đỉnh về cùng thước gần $[0, 1]$. $C[\tau]$ **không phải hiện tượng dây thanh**: nó là hiệu chỉnh hình học của cửa sổ. Trên FPGA, $\mathcal{R}_w$ chỉ phụ thuộc w và L (bất biến lúc chạy), nên tính offline, nạp ROM, rồi *nhân* thay vì chia mỗi τ. Chia runtime 189 lần mỗi khung vừa tốn latency vừa khó khớp bit với round cố định.

▶ NHÌN THẤY VẬT LÝ

Quên nhân $C[\tau]$ thiên lệch hệ thống về τ nhỏ (giọng cao giả): đỉnh trầm bị đạp, thuật toán thích đỉnh ngắn hơn. Đây là lỗi kiến trúc, không phải nhiễu ngẫu nhiên — cent-error sẽ lệch một phía trên mọi câu nói trầm.



Hình 34: Hai quả núi cửa sổ chồng ít dần khi τ tăng. Hệ số $C[\tau]$ bù đúng độ mất chồng đó.

8.2. Toán từng bước — mọi ký hiệu

Bắt đầu từ định nghĩa Boersma sau khi đã có $\mathcal{R}_x[0]$:

$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau], \qquad C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}.$$

Từng bước dẫn C: (1) tính $\mathcal{R}_w[0] = \sum_n w[n]^2$; (2) với mỗi τ, tính $\mathcal{R}_w[\tau] = \sum_{n=0}^{L-1-\tau} w[n]w[n+\tau]$; (3) chia $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (float offline); (4) lượng tử Q3.13.

- $\mathcal{R}_x[\tau]$: tự tương quan tín hiệu đã cửa sổ (Chương 7), int rộng trên silicon.
- $\mathcal{R}_w[\tau]$: tự tương quan cửa sổ — mẫu số "độ chồng", tính offline từ w_q.
- $C[\tau] \geq 1$ và tăng theo τ (vì $\mathcal{R}_w$ giảm).
- Lượng tử silicon: $C_q[\tau] = \text{sat}_{u16}\big(\text{round}(C[\tau] \cdot 2^{13})\big)$ — định dạng **Q3.13** (3 bit phần nguyên gồm đầu, 13 bit thập phân; $1{,}0 \mapsto 8192$; dải đến gần $4{,}0$). Chọn Q3.13 vì trên $L = 640$, $C[214] \approx 2{,}13$ vượt quá 1 (Q1.15 sẽ bão hòa sai tại $1{,}0$).
- Bit-growth: R_x (vd Q2.30 / accum) $\times C$ (Q3.13) rồi làm tròn về thang điểm số. Twin và RTL dùng cùng quy tắc:
$$S[\tau] = (R_x[\tau] \cdot C_q[\tau] + 2^{12}) \gg 13$$
(làm tròn gần nhất bằng cộng $2^{12} = 4096$ trước khi dịch 13, khớp `lab_1_3_5.py`).

Dải ROM phần cứng: 189 từ cho $\tau = 26 \ldots 214$. Tệp `hex/c_tau_rom.hex` trong gói này còn nhân thêm hệ số octave-cost của Praat (xem `scoring_rom_q13` trong golden) — đó là tiện ích gộp ROM, không đổi bản chất $C[\tau]$ bù cửa sổ. Twin kiểm tra cửa sổ thuần dùng `compensation_rom_q13`; twin kiểm tra điểm số đầy đủ dùng scoring hex.

★ NGUYÊN LÝ ĐẦU TIÊN: Nhân nghịch đảo offline thay chia runtime

Chia số nguyên trên FPGA (LPM_DIVIDE) có latency hàng chục chu kỳ và thương/dư phụ thuộc làm tròn IP. Vì mẫu số $\mathcal{R}_w[\tau]$ biết trước, nghịch đảo một lần trên Python rồi nhân là đủ — 189 nhân cố định latency, bit-exact với twin.

8.3. Ví dụ tính tay — mọi số đã chạy Python

A. $L = 4$, $w = [1, 2, 2, 1]$ **— từng hạng.**

$$\begin{aligned}
\mathcal{R}_w[0] &= 1 \cdot 1 + 2 \cdot 2 + 2 \cdot 2 + 1 \cdot 1 = 1 + 4 + 4 + 1 = 10, \\
\mathcal{R}_w[1] &= 1 \cdot 2 + 2 \cdot 2 + 2 \cdot 1 = 2 + 4 + 2 = 8, \\
\mathcal{R}_w[2] &= 1 \cdot 2 + 2 \cdot 1 = 2 + 2 = 4, \\
\mathcal{R}_w[3] &= 1 \cdot 1 = 1.
\end{aligned}$$

Vậy $C = [10/10,\ 10/8,\ 10/4,\ 10/1] = [1,\ 1{,}25,\ 2{,}5,\ 10]$. Q3.13 từng bước cho $C = 1{,}25$:

$$1{,}25 \cdot 8192 = 10\,240 \quad (\texttt{0x2800}).$$

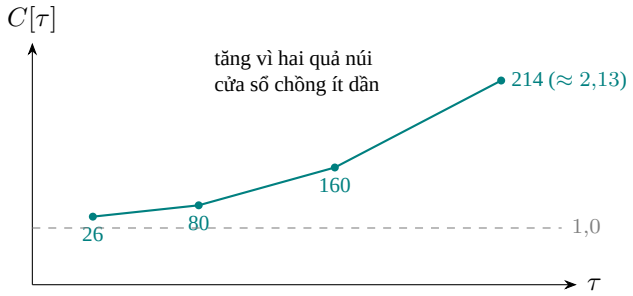
Giả sử $\mathcal{R}_x = [20, 12, 6, 1]$ thì sau nhân C (float): $[20,\ 15,\ 15,\ 10]$ — đỉnh phụ τ được nâng tương đối. Trên int: $S[1] = (12 \cdot 10240 + 4096) \gg 13 = 15$ (khớp).

B. $w = [1, 2, 1]$**,** $\tau = 2$**.** $\mathcal{R}_w[0] = 1 + 4 + 1 = 6$, $\mathcal{R}_w[2] = 1 \cdot 1 = 1$, $C[2] = 6$.

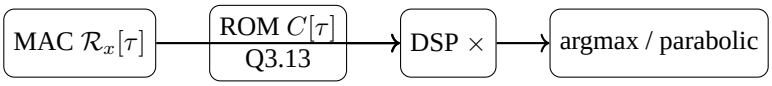
C. Khung thật $N = 640$ **Praat (bù cửa sổ thuần).**

τ	C float	C_q Q3.13	hex
26	1.010885	8281	`2059`
80	1.108152	9078	`2376`
160	1.515087	12412	`307C`
214	2.127577	17429	`4415`

Hex triển khai (`scoring_rom`, có octave-cost): $C_q[26] = 8283$, $C_q[214] = 17070$ — lệch nhẹ so với bảng thuần, đúng thiết kế gộp phạt quãng tám vào cùng một lần nhân. Script `verify_chuong_06_09.py` assert cả hai bảng.



Hình 35: $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ tăng theo τ: càng trễ, vùng chồng cửa sổ càng nhỏ, hệ số bù càng lớn. Các mốc khớp bảng 8.3.C.



Hình 36: Sau nhân C: đưa $S[\tau]$ (hoặc $r_a[\tau]$) vào khối dò đỉnh (Chương 9).

8.4. Ý nghĩa phần cứng — ROM mặc định, chia chỉ khi bắt buộc

Đường mặc định: IP **On-Chip Memory** kiểu ROM (hoặc RAM khởi tạo `.hex`/`.mif`).

- Depth $= 189$ (hoặc $\tau_{\max} + 1$ nếu lưu từ 0); width $= 16$ bit mang Q3.13.
- Registered output: 1 chu kỳ — FSM Chương 11 chờ đúng latency trước khi nhân.
- Một khối DSP nhân $R_x \times C_q$ mỗi τ (có thể dùng chung nhịp với MAC).
- Init file: `c_tau_rom.hex` (scoring) hoặc `compensation` thuần — ghi rõ trong README lab.

Bảng timing — đọc ROM C rồi nhân.

ck	Địa chỉ	C_q	S
0	$\tau - 26$ đặt	—	—
1	giữ	C_q ổn định	bắt đầu nhân (hoặc chốt nếu DSP 0-pipe)
2	τ kế	C mới	S của τ trước hợp lệ nếu DSP 1-pipe

Khi nào mới chia runtime? Chỉ khi $\mathcal{R}_w$ *đổi lúc chạy* (cửa sổ adaptive). Dùng **LPM_DIVIDE** hoặc bộ chia **non-restoring** tự viết bằng ALM (gặp lại ở Chương 9 cho $F_0 = f_s/\tau$). Đường pitch mặc định **không** chia mỗi τ: ROM đã nướng nghịch đảo. IP "ALTDIV_TENT không tồn tại.

Đổi L hoặc công thức w thì **rebuild file hex bắt buộc**; nạp ROM cũ sẽ sai im lặng và đỉnh F_0 thiên lệch.

RTL (ý tưởng) — từng dòng.

```
(* ram_init_file = "c_tau_rom.hex" *)
reg [15:0] C_rom [0:188];          // tau 26..214
reg [15:0] C_q;
reg signed [42:0] S;               // score after *C
always @(posedge clk) begin
  C_q <= C_rom[tau - 26];          // 1-cycle registered read
  // Nhân signed: zero-extend C_q thành số không âm
  S   <= (R_x * $signed({1'b0,C_q}) + 43'sd4096) >>> 13;
end
```

Dòng `C_q <= C_rom[...]` là latency 1; dòng `S <= ...` dùng C_q của chu kỳ trước nếu cùng `always`. Lab phải pipeline đúng: tách hai `always`, hoặc chấp nhận S trễ thêm 1 ck so với R_x mới.

8.4b. LPM_DIVIDE nội bộ khi bắt buộc chia $\mathcal{R}_w$

Khi lab cố tình đổi cửa sổ theo khung (ví dụ adaptive window), ROM cố định không còn đúng. Khi đó cấu hình **LPM_DIVIDE**:

- `LPM_WIDTHN`: độ rộng numerator — đủ chứa $\mathcal{R}_w[0]$ (thường 32–48 bit int).
- `LPM_WIDTHD`: độ rộng denominator $\mathcal{R}_w[\tau]$.
- `LPM_PIPELINE`: số tầng pipeline L (mỗi tầng một hàng thanh ghi). Latency đúng L chu kỳ sau khi `numer`/`denom` ổn định.
- `LPM_NREPRESENTATION` / `LPM_DREPRESENTATION`: unsigned hoặc signed; khớp twin.

Timing LPM_DIVIDE (pipeline $L = 4$ minh họa).

ck	Sự kiện	Ghi chú
0	nạp $\mathcal{R}_w[0]$, $\mathcal{R}_w[\tau]$	start ngầm theo data
1–3	pipeline nội bộ	không đọc quotient
4	`quotient`$\approx C$ thô	mới được nhân với $\mathcal{R}_x$

So với ROM 1 ck: chia chậm hơn và khó bit-exact vì IP có thể cắt thương khác `round` Python. Vì vậy tài liệu giữ ROM làm mặc định.

Non-restoring tự viết — giải thích thuật ngữ. **Non-restoring division** là thuật toán chia số nguyên từng bit: mỗi chu kỳ đồng hồ dịch thanh ghi dư một bit, rồi *trừ* hoặc *cộng* số chia tùy dấu dư hiện tại, và ghi một bit thương. Khác restoring (trừ thử rồi cộng lại nếu âm), non-restoring không hoàn tác — ít multiplex hơn, dễ viết bằng ALM thuần. Với thương 16-bit cần khoảng 16 chu kỳ. Vẫn chậm hơn đọc ROM một chu kỳ, nhưng khi lab bắt buộc chia $\mathcal{R}_w$ lúc chạy và muốn so từng bước với twin, non-restoring dễ quan sát hơn LPM_DIVIDE (IP "đen"). Chương 9 sẽ dùng lại đúng thuật toán này cho $F_0 = f_s/\tau$.

8.4c. Octave-cost gộp vào cùng ROM

Praat phạt ứng viên gần bội của đỉnh mạnh (sai quãng tám) bằng hệ số < 1 nhân thêm vào điểm số. Gói này nhân octave-cost vào cùng từ hex với $C[\tau]$ (scoring_rom) để FSM chỉ một lần nhân DSP. Hệ quả: bảng "thuần C" và bảng "scoring" lệch vài LSB như mục 8.3.C — không phải lỗi làm tròn. Lab phải chọn đúng tệp: so khớp Praat đầy đủ dùng scoring; chứng minh công thức bù cửa sổ dùng compensation thuần.

Ví dụ số: tại $\tau = 214$, thuần $C_q = 17429$, scoring 17070. Tỉ số $17070/17429 \approx 0{,}979$ — octave-cost khoảng vài phần trăm, đủ kéo đỉnh bội xuống dưới đỉnh chính trên nhiều khung hữu thanh.

8.5. Twin Python bit-exact

Twin ở đây là mô hình Python nhân–cộng/lượng tử giống hệt RTL: cùng công thức, cùng làm tròn, cùng bão hòa. **Hợp đồng bit-exact** nghĩa là dump thanh ghi trên FPGA phải trùng từng bit với số twin in ra trên cùng vector vào — không "gần đúng". Mọi thay đổi round/sat phải sửa cùng một chỗ ở twin và RTL, rồi sinh lại hex từ twin (không sửa hex tay).

```
import numpy as np
Q13 = 1 << 13

def build_C(w, tau_min, tau_max):
    w = np.asarray(w, float)
    L = len(w)
    Rw0 = float(np.dot(w, w))
    out = []
    for t in range(tau_min, tau_max + 1):
        Rwt = float(np.dot(w[: L - t], w[t:]))
```

```
        out.append(int(round(Rw0 / max(Rwt, 1e-30) * Q13)))
    return np.clip(out, 0, 65535).astype(np.uint16)

def apply_C(R_x, C_q):
    return (R_x.astype(np.int64) * C_q.astype(np.int64) + (1 << 12)) >> 13

# assert: build_C([1,2,2,1], 0, 3)[1] == 10240  # C=1.25
```

Giải thích: `max(Rwt, 1e-30)` tránh chia 0 khi cửa sổ suy biến; `round` rồi `clip` khớp sat u16; `apply_C` cộng 2^{12} trước $\gg 13$ khớp RTL. Mọi thay đổi công thức octave-cost phải sửa cùng chỗ golden và hex.

8.5b. Quy trình sinh hex và nghiệm thu

1. Chạy twin `build_C` trên đúng w_q Praat $N = 640$ (cùng hex cửa sổ Chương 6).
2. Ghi `compensation_rom_q13.hex` (thuần) và/hoặc nhân octave-cost $\rightarrow$ `c_tau_rom.hex`.
3. So 4 mốc $\tau \in \{26, 80, 160, 214\}$ với bảng mục 8.3.C — sai một mốc là dừng, không nạp bitstream.
4. Trên ModelSim: kích τ tuần tự, so $S[\tau]$ với `apply_C` trên cùng R_x dump từ MAC.
5. Chỉ khi dump xanh mới nối argmax Chương 9.

Nếu đổi Hanning sang Hamming, toàn bộ $\mathcal{R}_w$ đổi — hex cửa sổ và hex C phải rebuild cùng commit. Sai một LSB công thức (quên octave-cost hoặc dùng nhầm Q1.15) lệch hàng trăm đơn vị trên C_q, đủ đổi argmax; vì vậy nghiệm thu theo bốn mốc hex mục 8.3.C, không chỉ theo "trông có vẻ tăng theo τ".

8.6. Chi phí silicon

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách chuẩn hóa $C[\tau]$.

- **M10K**: $189 \times 16 = 3\,024$ bit < 1 M10K ($10\,240$ bit) — dư chỗ ghép thêm bảng nhỏ nếu cần.
- **DSP**: 1 nhân $R_x \times C$ mỗi τ (dùng chung hoặc ngay sau MAC). 189 nhân/khung $\ll 98\,280$ MAC.
- **ALM**: địa chỉ + đăng ký — vài chục.
- So với LPM_DIVIDE mỗi τ: tiết kiệm 189 phép chia/khung và latency không đều — lý do chọn ROM. Một LPM_DIVIDE 32/16 pipeline 16 ck sẽ thêm 189×16 chu kỳ nếu tuần tự — phá RTF.
- Vì sao Q3.13: $C[214] \approx 2{,}13$ vừa trong 3 bit nguyên; Q1.15 sẽ bão hòa tại $1{,}0$ và phá bù. Vì sao không Q4.12: đủ nhưng mất 1 bit thập phân không cần thiết khi $C < 4$.

Trên hop 10 ms, khối C chỉ thêm khoảng 189 chu kỳ nhân (+ latency ROM). Tổng Pha 2 vẫn dễ nằm dưới 3 ms nếu MAC là nút thắt chính. Lab nghiệm thu: dump $S[\tau]$ sau nhân C phải khớp twin trên cùng hex.

Ví dụ tính tay — ba lớp sai số (cùng một khung). Giả sử $\tau^* = 80$ đúng trên twin. (1) Hex $C_q[80]$ lệch $+200$ LSB so twin: $S[80]$ bị nâng giả, có thể thắng τ láng giềng — đỉnh lệch có hệ thống theo τ. (2) FSM đọc ROM sớm 1 chu kỳ: $S[\tau]$ mang $C_q[\tau - 1]$; đỉnh lệch đúng 1 mẫu. (3) Dùng compensation thuần trong khi twin so scoring: lệch vài phần trăm như $17070/17429 \approx 0{,}979$ ở $\tau = 214$, dễ thành sai quãng tám trên nguyên âm ổn định. Ba lớp này cho triệu chứng khác nhau; đừng gộp thành "F_0 sai".

Trên đường mặc định không có khối chia $\mathcal{R}_w$: thanh ghi accum của MAC mang $\mathcal{R}_x[\tau]$, ROM mang $C_q[\tau]$, một DSP nhân rồi dịch 13 bit có làm tròn. IP tên "ALTDIV_TENT" không tồn tại. Phép chia thật chỉ xuất hiện khi $\mathcal{R}_w$ đổi lúc chạy (LPM_DIVIDE / non-restoring ở trên) hoặc khi Chương 9 tính $F_0 = f_s/\tau_{\text{fine}}$ — một lần mỗi khung, không phải 189 lần. So sánh voicing với ngưỡng cũng làm bằng nhân nguyên (best $\cdot\, 100 < 45 \cdot R_x[0]$), không chia float — twin phải cùng quy tắc.

8.7. Bài tập và lời giải giải thích

Bài tập 8.1. $w = [1, 2, 1]$: tính $C[2]$.

Bài tập 8.2. Hex Q3.13 cho $C = 1{,}5$: giá trị nguyên và mã hex?

Bài tập 8.3. Twin dùng floor thay round khi lượng tử C: lệch tối đa bao nhiêu LSB trước khi cắt?

Bài tập 8.4. Quên nhân C: đỉnh τ lớn bị thiên lệch thế nào?

Bài tập 8.5. Liệt kê ba tham số IP ROM bắt buộc: width, depth, init file. Khi nào cân nhắc LPM_DIVIDE thay ROM? Vẽ timeline 2 chu kỳ đọc registered.

Lời giải giải thích.

1. $\mathcal{R}_w[0] = 1 + 4 + 1 = 6$, $\mathcal{R}_w[2] = 1 \cdot 1 = 1$, $C[2] = 6$.
2. $1{,}5 \cdot 8192 = 12288 =$`0x3000`.
3. Floor thay round lệch tối đa $0{,}5$ LSB của thang Q3.13 trước khi ép kiểu nguyên.
4. Quên C: đỉnh τ lớn bị đạp bởi hình học cửa sổ, thuật toán thiên về τ nhỏ (giọng cao giả).
5. Width 16, depth 189 (hoặc $\tau_{\max} - \tau_{\min} + 1$), init `c_tau_rom.hex`. LPM_DIVIDE chỉ khi $\mathcal{R}_w$ đổi lúc chạy; đường mặc định pitch không chia mỗi τ. Timeline: ck0 đặt địa chỉ; ck1 C_q hợp lệ — mới được nhân với R_x.

Chương 9. Peak, parabolic và $F_0 = f_s/\tau$ (LPM_DIVIDE)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được vì sao đỉnh τ trên lưới mẫu nguyên chưa đủ cho độ phân giải cao độ; dẫn ra và tính tay nội suy parabolic ba điểm với mọi trung gian; cấu hình LPM_DIVIDE (pipeline từng tầng) hoặc divider non-restoring tự viết để lấy $F_0 = f_s/\tau_{\text{fine}}$; đóng gói kết quả dạng Q12.4; viết twin chờ đúng latency chia; và đọc được từng dòng hộp chi phí silicon kèm lý do chọn parabolic thay sinc.

9.1. Hiện tượng thật: đỉnh tự tương quan = chu kỳ đóng/mở thanh môn

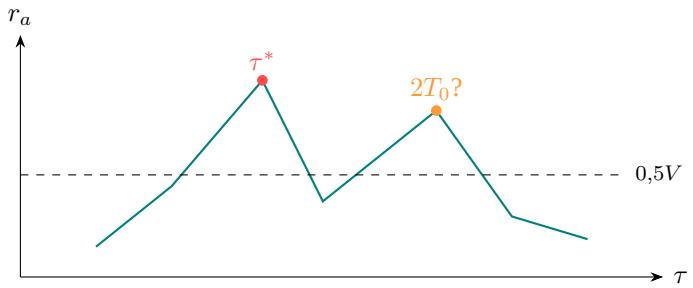
Sau chuẩn hóa Boersma (Chương 8), điểm số $S[\tau]$ (hay $r_a[\tau]$) gần thang $[0, 1]$ đo mức "khung giống chính mình" tại trễ τ. Với dây thanh gần tuần hoàn, đỉnh lớn nhất trong $[\tau_{\min}, \tau_{\max}]$ ước lượng T_0 tính bằng số mẫu. **Argmax** là phép tìm chỉ số τ^* cho $S[\tau^*]$ lớn nhất — thuật ngữ tính toán, nhưng nó bám sát hiện tượng vật lý: khoảng thời gian giữa hai lần đóng thanh môn.

Trên thiết kế này, mảng điểm số có 189 ô cho $\tau = 26 \dots 214$, nhưng argmax bị hạn chặt vào $[27, 213]$ (187 ứng viên) để τ^* luôn còn hai láng giềng $\tau^* \pm 1$ phục vụ nội suy. Ngưỡng voicing Praat mặc định $V = 0{,}45$: nếu đỉnh tốt nhất $< 0{,}45 \cdot \mathcal{R}_x[0]$ (so sánh nguyên: best $\cdot\, 100 < 45 \cdot R_x[0]$) thì khung vô thanh, $F_0 = 0$. So sánh nguyên tránh chia float trên silicon và khớp twin bit-exact.

Vì sao không dừng ở τ^* nguyên? Vì $F_0 = f_s/\tau$: một mẫu lệch ở $\tau = 80$ đổi F_0 khoảng $16000/80 - 16000/81 \approx 2{,}5$ Hz — với giọng nữ đó đã là vài chục cent, đủ fail nghiệm thu Chương 13. Nội suy dưới mẫu kéo sai số xuống dưới 1 Hz điển hình với chi phí vài phép cộng/trừ. Parabolic giả định đỉnh đơn cục bộ sắc; nếu sóng bão hòa cắt ngọn, đỉnh $\mathcal{R}_x$ phẳng và Δ nhảy loạn — khi đó kiểm tra cửa sổ (Chương 6) trước khi đổ lỗi cho divider.

▶ NHÌN THẤY VẬT LÝ

Đỉnh tại $\tau = 2T_0$ luôn tồn tại vì sóng chu kỳ T_0 cũng chu kỳ $2T_0$. Nếu biên độ đỉnh bội bị nhiễu đẩy cao hơn đỉnh chính, thuật toán báo $F_0/2$ — **sai quãng tám (octave error)**. Octave-cost trong ROM điểm số (Chương 8) và cổng voicing (Chương 10) giảm rủi ro này; argmax một mình không đủ. Trên tiếng Việt, sai quãng tám biến thanh sắc thành "trầm giả" và phá tương quan Pearson với Praat.



Hình 37: Argmax chọn đỉnh cao nhất trên $[\tau_{\min}, \tau_{\max}]$; đỉnh bội có thể còn nhưng thường thấp hơn sau chuẩn hóa và octave-cost.

9.2. Nội suy parabolic — từng ký hiệu, không nhảy bước

$\mathcal{R}(\tau)$ (hay $r_a(\tau)$) là hàm của biến liên tục τ, nhưng phần cứng chỉ có mẫu tại số nguyên. Gần một đỉnh đơn, đồ thị trông giống một **parabola** $y = ax^2 + bx + c$. Khớp ba điểm r_-, r^*, r_+ tại $x = -1, 0, +1$:

$$
\begin{aligned}
r_- &= a - b + c, \\
r^* &= c, \\
r_+ &= a + b + c.
\end{aligned}
$$

Trừ hai phương trình biên: $r_- - r_+ = -2b \Rightarrow b = (r_+ - r_-)/2$. Cộng rồi trừ $2r^*$: $r_- + r_+ - 2r^* = 2a \Rightarrow a = (r_- - 2r^* + r_+)/2$. Đỉnh parabola tại $x = -b/(2a)$:

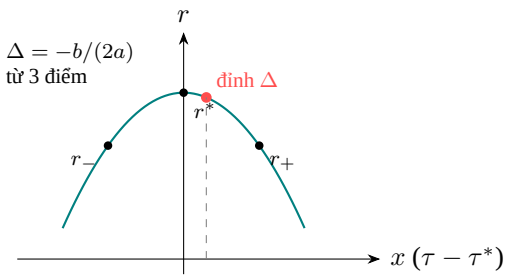
$$\Delta = \frac{1}{2} \cdot \frac{r_- - r_+}{r_- - 2r^* + r_+}, \qquad \tau_{\text{fine}} = \tau^* + \Delta, \qquad F_0 = \frac{f_s}{\tau_{\text{fine}}}.$$

- $r_- = r_a[\tau^* - 1]$, $r^* = r_a[\tau^*]$, $r_+ = r_a[\tau^* + 1]$.
- Mẫu số $r_- - 2r^* + r_+$ xấp xỉ đạo hàm hai (độ cong); phải < 0 tại đỉnh thật (cong xuống). Nếu > 0 đó là đáy — RTL phải bắt.
- $\Delta > 0$ nghĩa là đỉnh hơi lệch phải; $\Delta < 0$ lệch trái.
- Trên silicon fixed-point (Mode B trong golden): $N = S_- - S_+$, $D = 2(S_- - 2S^* + S_+)$,

  $$\tau_{\text{fine,Q16}} = (\tau^* \ll 16) + ((N \ll 16)//D),$$

  rồi chia F_0 bằng số nguyên có làm tròn.

Sai số điển hình $\ll 0{,}5$ mẫu trên đỉnh sắc — đủ cho lab pitch trước khi nghĩ đến sinc (Chương 0 đã so sánh Sinc70/Sinc700 đắt hàng trăm nhân).



Hình 38: Nội suy parabolic: khớp ba điểm r_-, r^*, r_+ bằng $y = ax^2 + bx + c$, lấy đỉnh tại $x = \Delta$. $\Delta > 0$ nghĩa là chu kỳ thật hơi dài hơn τ^* nguyên.

9.3. Ví dụ tính tay — mọi số đã chạy Python

A. Argmax có ngưỡng — từng quyết định. Các cực đại cục bộ $(80, 0{,}50)$, $(160, 0{,}47)$, $(40, 0{,}30)$; $V = 0{,}45$. So $0{,}30$ với $0{,}45$: loại 40. Còn 80 và 160: $0{,}50 > 0{,}47$ nên $\tau^* = 80$. Nếu hai đỉnh bằng nhau, quy ước lab lấy τ nhỏ hơn (cao độ cao hơn) — ghi rõ trong twin.

B. Parabolic và Q12.4 — mọi trung gian. Với $r_- = 0{,}40$, $r^* = 0{,}50$, $r_+ = 0{,}46$, $\tau^* = 80$, $f_s = 16\,000$:

$$\begin{aligned} r_- - r_+ &= 0{,}40 - 0{,}46 = -0{,}06, \\ r_- - 2r^* + r_+ &= 0{,}40 - 1{,}00 + 0{,}46 = -0{,}14, \\ \Delta &= \tfrac{1}{2} \cdot \frac{-0{,}06}{-0{,}14} = \tfrac{1}{2} \cdot \frac{3}{7} = \frac{3}{14} \approx 0{,}214285714. \end{aligned}$$

$\tau_{\text{fine}} = 80 + 3/14 \approx 80{,}2142857$.

$$F_0 = \frac{16000}{80{,}2142857} \approx 199{,}4657\,\text{Hz}.$$

Đóng gói **Q12.4** (12 bit phần nguyên, 4 bit thập phân; $1\,\text{Hz} \mapsto 16$):

$$F_{0,\text{Q12.4}} = \text{round}(199{,}4657 \cdot 16) = \text{round}(3191{,}451) = 3191.$$

Bảng quét thêm (script `verify/verify_chuong_06_09.py`):

y_-	y_*	y_+	Δ	τ_{fine}	F_0 @16 kHz / Q12.4
0.40	0.50	0.46	0.214286	80.214286	199.4657 / 3191
0.30	0.55	0.50	0.333333	100.333333	159.4684 / 2551
0.48	0.50	0.40	−0.333333	119.666667	133.7047 / 2139
0.20	0.60	0.20	0.000000	64.000000	250.0000 / 4000

C. Phép chia $13 \div 3$ (minh họa non-restoring 4-bit) — từng bước. Dividend $D = 13 = 1101_2$, divisor $V = 3 = 0011_2$. Non-restoring 4 bước (ý tưởng):

Bước k	Thao tác	Dư gần đúng	Bit thương
1	dịch, thử trừ V	dương $\rightarrow$ giữ trừ	1
2	dịch, trừ/cộng theo dấu	…	…
3	…	…	…
4	kết thúc	dư $= 1$	thương $= 4$

Vì $3 \cdot 4 + 1 = 13$. Testbench bắt buộc: assert $13/3 = 4$, $16000/80 = 200$.

9.4. Ý nghĩa phần cứng — LPM_DIVIDE pipeline và non-restoring

Trên laptop, $16000/80$ là một lệnh. Trên Cyclone V, chia nguyên thường làm bằng IP **LPM_DIVIDE** (pipeline cố định L chu kỳ) hoặc FSM **non-restoring** tự viết (khoảng W chu kỳ cho thương W bit). Latency là hợp đồng: twin và testbench phải chờ đúng số cạnh clock trước khi đọc quotient. Lệch một chu kỳ gắn F_0 khung trước vào khung sau — lỗi thời gian, không phải lỗi toán.

LPM_DIVIDE — tham số và pipeline từng tầng.

- `LPM_WIDTHN`: numerator (vd 32-bit mang f_s đã scale, hoặc $16000 \cdot 16 \cdot 65536$ theo Mode B).
- `LPM_WIDTHD`: denominator τ_{fine} (vd 16–32 bit Q-format).
- `LPM_PIPELINE`$= L$: IP chèn L hàng thanh ghi; mỗi chu kỳ dữ liệu tiến một tầng.
- `LPM_NREPRESENTATION`/`LPM_DREPRESENTATION`: unsigned khuyến nghị cho F_0 dương.

Bảng timing LPM_DIVIDE ($L = 8$ minh họa).

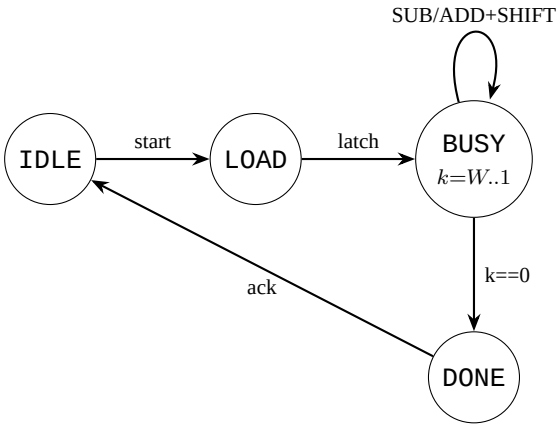
ck	Sự kiện	Ghi chú
0	nạp `numer`=fs_scaled, `denom`=τ_q	sau parabolic xong
1–7	dữ liệu trong pipeline	**không** đọc quotient
8	`quotient` $= F_{0,\text{Q12.4}}$ thô	chốt vào thanh ghi xuất
9	`valid` lên Avalon-ST	Chương 11/12 đóng gói

Golden Mode B:

$$f0_{\text{Q12.4}} = (16000 \cdot 16 \cdot 65536 + (\tau_{\text{fine,Q16}} \gg 1))//\tau_{\text{fine,Q16}}$$

(chia có làm tròn bằng cộng nửa mẫu số).

Non-restoring — trực giác và FSM. **Restoring division** trừ số chia khỏi dư; nếu kết quả âm thì cộng lại ("restore") rồi ghi bit thương 0. **Non-restoring** bỏ bước hoàn tác: nếu dư âm thì ở chu kỳ sau *cộng* số chia và ghi bit 0; nếu dư dương thì trừ và ghi bit 1. Ít multiplex hơn, phù hợp RTL thuần ALM trên Cyclone V. FSM bốn trạng thái: IDLE (chờ start) $\rightarrow$ LOAD (chốt dividend/divisor) $\rightarrow$ BUSY (W vòng dịch + trừ/cộng, mỗi vòng một bit thương) $\rightarrow$ DONE (giữ quotient đến khi ack).



Hình 39: FSM divider non-restoring: IDLE→LOAD→BUSY (W vòng)→DONE. LPM_DIVIDE ẩn FSM này trong IP, nhưng latency vẫn phải khớp twin.

RTL bọc LPM_DIVIDE — từng dòng ý tưởng.

```
// Sau tau_fine_q16 và fs=16000:
// start_div pulse -> chờ L chu kỳ -> f0_quot hợp lệ (Q12.4)
lpm_divide #(.LPM_WIDTHN(32), .LPM_WIDTHD(16), .LPM_PIPELINE(L))
  u_div (.clock(clk), .numer(fs_scaled), .denom(tau_q),
         .quotient(f0_quot));
// FSM ngoài: đếm L cạnh sau start; chỉ khi cnt==L mới assert out_valid
```

Không tồn tại IP `ALTDIV_TENT`. Chỉ LPM_DIVIDE hoặc RTL non-restoring.

Bảng timing argmax tuần tự (187 ứng viên).

ck	Việc làm	Thanh ghi
0	$\tau = 27$, `best`=S[27], $\tau^* = 27$	khởi tạo
1	so S[28] với best; cập nhật nếu lớn hơn	…
⋮	quét đến 213	…
186	xong argmax	τ^*, best
187+	đọc S[$\tau^* \pm 1$], parabolic, start chia	

9.4b. Mode B fixed-point — từng phép trên silicon

Golden `lab_1_3_5.py` không chia float trên đường chính. Sau khi có S_-, S^*, S_+ (int sau nhân C):

1. $N \leftarrow S_- - S_+$ (có dấu).
2. $D \leftarrow 2(S_- - 2S^* + S_+)$ — mẫu số cong; bắt buộc $D < 0$ tại đỉnh.
3. $\Delta_{\text{Q16}} \leftarrow (N \ll 16)//D$ (chia nguyên có dấu, làm tròn tùy golden).
4. $\tau_{\text{fine,Q16}} \leftarrow (\tau^* \ll 16) + \Delta_{\text{Q16}}$.
5. $f0_{\text{Q12.4}} \leftarrow (16000 \cdot 16 \cdot 65536 + (\tau_{\text{fine,Q16}} \gg 1))//\tau_{\text{fine,Q16}}$.

Mỗi phép $//$ trên RTL là một lần LPM_DIVIDE hoặc một vòng non-restoring. Có thể tái sử dụng cùng IP chia hai lần tuần tự (parabolic rồi F_0) nếu FSM cho phép — tiết kiệm một instance IP với giá latency cộng dồn.

Ví dụ số Mode B minh họa với thang đơn giản. Giả sử sau scale $S_- = 400$, $S^* = 500$, $S_+ = 460$, $\tau^* = 80$ (cùng tỉ lệ với ví dụ float 0.40/0.50/0.46):

$$N = 400 - 460 = -60,$$
$$D = 2(400 - 1000 + 460) = 2(-140) = -280,$$
$$\Delta_{\text{float}} = 0{,}5 \cdot (-60)/(-140) = 3/14.$$

Twin float và Mode B phải hội tụ trong sai số làm tròn cố định — script khóa Q12.4 $= 3191$.

9.4c. So sánh parabolic với sinc (nhắc Chương 0)

Sinc nội suy lý tưởng cần nhiều hệ số (Sinc70, Sinc700 trong Praat đầy đủ). Trên streaming FPGA với Viterbi tắt, chi phí hàng trăm nhân/đỉnh không đáng khi parabolic đã giữ sai số dưới khoảng 2 cent trên đỉnh sắc. Lab nâng cao có thể thêm FIR nội suy sau này; đường mặc định gói này dừng ở parabolic ba điểm + một chia.

Khi đỉnh phẳng (ba điểm gần bằng nhau), mẫu số $D \to 0$ — parabolic không ổn định. RTL phải: (a) nếu $|D|$ dưới ngưỡng thì lấy $\Delta = 0$; (b) hoặc dựa voicing để đánh vô thanh. Twin phải cùng ngưỡng — đây là chỗ hay lệch bit nếu quên.

9.5. Twin Python bit-exact

```
import numpy as np

def parabolic_delta(ym1, y0, yp1):
    denom = ym1 - 2.0 * y0 + yp1
    return 0.5 * (ym1 - yp1) / denom

def f0_q12_4(fs, tau_fine):
    return int(round(fs / float(tau_fine) * 16))

d = parabolic_delta(0.40, 0.50, 0.46)
assert abs(d - 3/14) < 1e-12
assert f0_q12_4(16000, 80 + d) == 3191
# TB: sau đúng L posedge, so quotient với twin
```

Giải thích: mẫu số `denom` phải khác 0 và âm tại đỉnh; twin float chỉ để khóa công thức — Mode B int trên RTL mới là hợp đồng silicon. Định dạng xuất (Chương 12): F_0 Q12.4 đóng gói `uint16` little-endian; 0 = vô thanh; trần biểu diễn $4095{,}9375$ Hz.

9.5b. Hợp đồng testbench với latency chia

Một testbench tối thiểu phải:

1. Nạp ba điểm S đã biết $\rightarrow$ kỳ vọng τ_{fine} và $F_{0,\text{Q12.4}}$ từ twin.
2. Pulse `start_div`; đếm đúng L cạnh `posedge clk`.
3. Chỉ khi `cnt==L` mới so `quotient` với twin — so sớm hơn là fail giả.
4. Thêm case vô thanh: best dưới ngưỡng $\rightarrow$ `f0==0`, không cần chờ chia (hoặc chia nhưng bỏ quotient).
5. Case mẫu số $|D|$ nhỏ: $\Delta = 0$, $F_0 = \text{round}(f_s/\tau^* \cdot 16)$.

Trên board DE10-Standard, đo latency thực bằng SignalTap: trigger `start_div`, probe `out_valid`. Nếu khoảng cách cạnh khác L đã khai báo, sửa RTL và twin cùng lúc — không vá delay tùy hứng trong FSM top.

Khi đóng gói Q12.4 ra HPS (Chương 12), nhớ **little-endian** uint16: byte thấp ghi trước (ví dụ $3191 = 0x0C77$ thành các byte `77 0C`). Đọc nhầm big-endian đảo thành $0x770C = 30476$ — divider đúng nhưng file sai. Twin xuất `pitch_f0.bin` cùng endian để so dump.

9.6. Chi phí silicon

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách peak + parabolic + divider.

- **Argmax** quét ≈ 187 giá trị: vài ALM + comparator, 0 DSP, 0 M10K — ≈ 187 chu kỳ.
- **Parabolic**: 2 trừ + 1 chia ngắn + nhân $1/2$ — 0–1 DSP hoặc soft logic, vài chu kỳ.
- **LPM_DIVIDE** 32/16 pipeline: IP dùng ALM (và tùy chọn DSP); *nút thắt là latency L*, không phải diện tích (chỉ 1 phép/khung). Với $L = 16$, thêm 16 chu kỳ — không đáng kể so với 98 280 MAC.
- **Non-restoring** tự viết W-bit: $\approx W$ chu kỳ, thuần ALM, dễ twin từng bước.
- **M10K**: 0 cho khối này (trừ khi đệm kết quả nhiều khung).
- Vì sao parabolic + 1 chia/khung: sai số $<$ 2 cent điển hình (Chương 0); Sinc70/Sinc700 đắt hàng trăm phép nhân/đỉnh trong khi Viterbi đã tắt trên đường streaming. Vì sao không chia float mềm: một LPM_DIVIDE cố định latency dễ FSM hơn FPU.

Tổng Pha peak+div thường < 300 chu kỳ/khung — dưới $0{,}1\%$ hop 10 ms @ 50 MHz. Nút thắt vẫn là MAC Chương 7. Lab nghiệm thu: (1) parabolic float khớp bảng 9.3; (2) Mode B int khớp golden; (3) testbench chờ đúng L ck trước khi đọc quotient.

Ví dụ tính tay — debug theo lớp, không theo tai. Giả sử dump Q12.4 $= 3191$ ($199{,}4375$ Hz) nhưng Praat báo ≈ 100 Hz. Đừng nghe tai trước: kiểm tra lần lượt (a) τ^* có phải ≈ 160 thay vì 80 không (sai quãng tám / quên C); (b) τ_{fine} có khớp parabolic twin không; (c) quotient có được chốt đúng sau L cạnh không; (d) file `pitch_f0.bin` có little-endian uint16 không — đảo byte biến $3191 = 0x0C77$ thành $0x770C = 30476$ "trông hợp lệ" trong hex viewer nhưng vô nghĩa. Script khóa các số (a)–(b); (c)–(d) là hợp đồng testbench và đóng gói Chương 12.

9.7. Bài tập và lời giải giải thích

Bài tập 9.1. $r = [0{,}2,\ 0{,}6,\ 0{,}5]$ tại $\tau = 100, 101, 102$: tính Δ, τ_{fine}, F_0 @ 16 kHz, mã Q12.4.

Bài tập 9.2. Testbench: $16000/80 = 200$. Cố ý đọc quotient sớm 1 chu kỳ — quan sát triệu chứng gắn nhầm khung.

Bài tập 9.3. Với bộ $(0{,}40, 0{,}50, 0{,}46)$, so Δ parabolic với ý nghĩa "đỉnh lệch phải". Nếu mẫu số $r_- - 2r^* + r_+ > 0$, đó có còn là đỉnh không?

Bài tập 9.4. LPM_DIVIDE pipeline $L = 8$. Vẽ timeline từ khi parabolic xong đến `out_valid`. Nếu Fmax tụt buộc tăng L lên 16, RTF đổi bao nhiêu trên hop 10 ms?

Bài tập 9.5. Giải thích vì sao argmax chỉ chạy $[27, 213]$ dù ROM có $\tau = 26$ và 214.
Lời giải giải thích.

1. Đỉnh nguyên tại $\tau^* = 101$ (0,6 lớn nhất). $\Delta = 0{,}5 \cdot (0{,}2 - 0{,}5)/(0{,}2 - 1{,}2 + 0{,}5) = 0{,}5 \cdot (-0{,}3)/(-0{,}5) = 0{,}3$. $\tau_{\text{fine}} = 101{,}3$, $F_0 \approx 157{,}9467$ Hz, Q12.4 $= 2527$ (script assert).
2. Đọc sớm 1 ck lấy thương của lần chia trước (hoặc giá trị chưa ổn định). Triệu chứng điển hình: đường F_0 "trễ một hop" so với twin, cent error nhảy có hệ thống theo thời gian chứ không theo biên độ.
3. $\Delta = +3/14 > 0$: đỉnh thật hơi bên phải τ^*. Nếu mẫu số > 0 thì độ cong hướng lên — đó là đáy (cực tiểu), không được lấy làm chu kỳ dây thanh; RTL phải kiểm tra mẫu số < 0 hoặc dựa vào argmax đã chọn cực đại.
4. Thêm 8 chu kỳ pipeline: $+8/500000$ trên hop — RTF tăng không đáng kể ($\ll 0{,}001$). Tăng L không phải lý do fail thời gian thực; fail thường do quên chờ L.
5. Cần $\tau^* \pm 1$ luôn nằm trong mảng điểm số. Nếu cho phép $\tau^* = 26$, thiếu r_-; $\tau^* = 214$ thiếu r_+. Vì vậy biên argmax thụt vào một mẫu mỗi phía.

Chương 10. Quyết định voicing, chi phí rút gọn và median-3 trên FPGA

▶ MỤC TIÊU HỌC TẬP

Sau khi học xong Chương 10, người học sẽ:

1. Giải thích được hiện tượng vật lý khi dây thanh quản ngừng rung (im lặng, phụ âm vô thanh, hơi thở) và vì sao vẫn báo F_0 trong các khung đó sẽ phá đường cong cao độ.
2. Viết và tính tay được cổng voicing/silence từ $r_a[\tau^*]$ và tỷ số năng lượng $P_{\text{local}}/P_{\text{global}}$, cùng ứng viên Unvoiced strength s_{UV} đúng công thức Praat trong `Pitch.cpp`.
3. Triển khai bộ lọc trung vị ba khung (median-3) có thanh ghi xuất trễ đúng một bước nhảy $H = 160$ mẫu (≈ 10 ms), và chứng minh nó đập được một xung nhiễu đơn mà không cần quy hoạch động Viterbi.
4. Tính tay chi phí chuyển trạng thái octave-jump / V↔UV và một bảng Viterbi 2×2 (hoặc 3 khung) để hiểu `Pitch_pathFinder`, rồi nêu rõ vì sao mặc định trên fabric Cyclone V **tắt** Viterbi và chỉ giữ soft-float trên HPS khi cần đường cong "Praat đầy đủ".
5. Đọc hộp chi phí silicon (ALM, DSP, M10K, latency, RTF) của khối voicing + median-3 và đối chiếu với ngân sách Chương 0.

10.1. Hiện tượng thật: dây thanh không rung liên tục

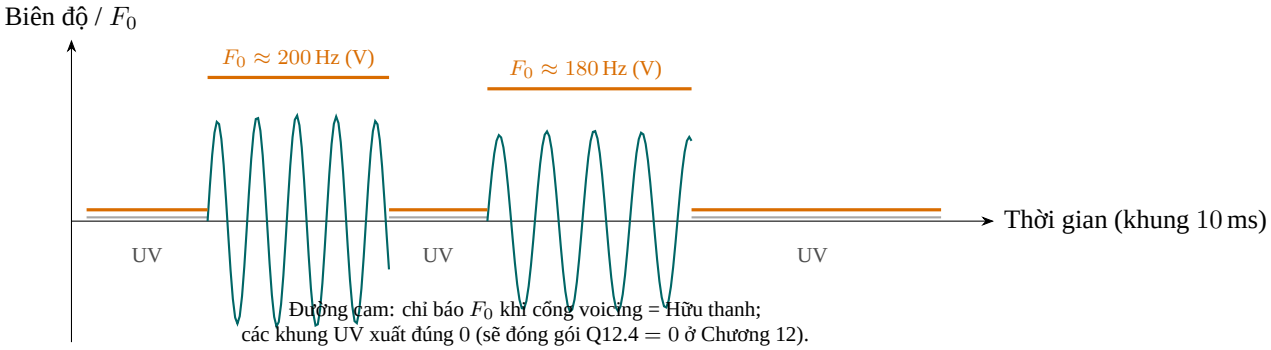
★ NGUYÊN LÝ ĐẦU TIÊN: Khi nào cao độ phải bằng không?

Trong Chương 0 và Chương 1 ta đã thấy $F_0 = 1/T_0$ là tần số đóng/mở thanh môn khi dây thanh rung tuần hoàn. Nhưng trong một câu nói thật, dây thanh chỉ rung trong các đoạn **hữu thanh** (nguyên âm, phụ âm hữu thanh). Ba tình huống sau *không* có chu kỳ dây thanh ổn định:

- **Im lặng / hơi thở:** năng lượng khung rất nhỏ; hàm tự tương quan gần như nhiễu trắng, không có đỉnh r_a nổi.
- **Phụ âm vô thanh** (ví dụ $/s/$, $/f/$): có năng lượng cao tần nhưng không có chu kỳ T_0 của thanh môn.
- **Khoảng chuyển tiếp ngắn:** đỉnh r_a yếu, dễ bị nhiễu đẩy sang đỉnh giả.

Nếu phần cứng vẫn xuất một số Hz "bịa" trong các khung đó, đường cong cao độ sẽ nhảy lung tung và mọi chỉ số đối sánh với Praat (GPE, VDE ở Chương 13) đều xấu. Praat giải quyết bằng hai tầng: (1) cổng silence/voicing + ứng viên Unvoiced ngay trên từng khung; (2) tuỳ chọn thuật toán quy hoạch động `Pitch_pathFinder` (Viterbi) với các chi phí octave và V↔UV. Trên FPGA DE10-Standard, thiết kế mặc định giữ tầng (1) cộng bộ lọc median-3 trên fabric, và **tắt** tầng (2) khỏi silicon logic — chuyển sang HPS khi thật sự cần.

Hình 40 minh họa một đoạn sóng có nguyên âm xen khoảng lặng: chỉ các khung hữu thanh mới được phép mang $F_0 > 0$.



Hình 40: Trực giác voicing: sóng có đoạn rung và đoạn im. Cổng voicing buộc $F_0 = 0$ trên khung vô thanh/im lặng trước khi median-3 và đóng gói.

10.2. Toán từng bước: cổng voicing và ứng viên Unvoiced

Sau Chương 8–9 ta đã có đỉnh chuẩn hóa $r_a[\tau^*] \in [0, 1]$ và (tuỳ chọn) F_0 nội suy. Praat không chỉ nhìn r_a: nó còn so sánh **đỉnh biên độ cục bộ** của khung với một **đỉnh toàn cục chậm**.

Định nghĩa ký hiệu.

- $r_a[\tau^*]$: độ mạnh tự tương quan tại đỉnh tốt nhất sau khi nhân ROM $C[\tau]$ (Chương 8).
- $V = \texttt{voicingThreshold} \approx 0{,}45$: ngưỡng sàn để một đỉnh còn được coi là ứng viên hữu thanh.
- $S = \texttt{silenceThreshold} \approx 0{,}03$: ngưỡng tỷ số năng lượng để nghi ngờ im lặng.
- $P_{\text{local}} = \max_n |x[n]|$ trên khung hiện tại (hoặc $\sqrt{\mathcal{R}_x[0]/L}$ — lab dùng max-abs cho rẻ silicon).
- P_{global}: ước lượng đỉnh biên độ của cả phát ngôn. Offline Praat lấy max cả file; streaming FPGA dùng **max-hold** hoặc IIR chậm vì không nhìn trước cả câu (Chương 0 đã nhấn mạnh ràng buộc streaming).

Cổng hữu thanh rút gọn trên fabric.

$$\text{voiced} = \big(r_a[\tau^*] \geq V\big) \ \wedge \ \big(P_{\text{local}} \geq S \cdot P_{\text{global}}\big). \tag{36}$$

Hai điều kiện đọc bằng lời: (1) đỉnh tương quan đủ cao; (2) khung không quá nhỏ so với đỉnh toàn cục. Nếu một trong hai sai, xuất $F_0 = 0$.

Ứng viên Unvoiced strength (đầy đủ Praat). Trong `Pitch.cpp`, Praat còn tạo một "ứng viên vô thanh" cạnh tranh với mọi đỉnh hữu thanh:

$$s_{\text{UV}} = V + \max\!\Big(0,\ 2 - \frac{P_{\text{local}}/P_{\text{global}}}{S/(1+V)}\Big). \tag{37}$$

Khi khung gần im lặng, phân số năng lượng nhỏ, hạng $\max(0, \ldots)$ dương lớn, nên s_{UV} thắng mọi đỉnh r_a yếu và pathFinder (nếu bật) chọn Unvoiced. Trên fabric mặc định, công thức (37) được dùng để *hiểu* và để twin Python đối chiếu Praat; RTL tối thiểu chỉ cần (36).

▶ NHÌN THẤY VẬT LÝ

V và S là **ngưỡng quy ước tính toán**, không phải hằng số vật lý của dây thanh. Đổi V từ $0{,}45$ xuống $0{,}30$ sẽ khiến nhiều khung nhiễu bị gắn nhầm hữu thanh (VDE tăng ở Chương 13). Đổi S quá cao sẽ nuốt cả nguyên âm nhỏ lúc đầu câu khi P_{global} chưa kịp warmup.

10.3. Ví dụ tính tay — voicing và s_{UV} (đã kiểm Python)

Ngưỡng Q1.14. Lab lưu r_a dạng Q1.14 (15 bit có dấu dùng 14 bit frac). Đổi $V = 0{,}45$ sang số nguyên:

$$V_{\text{int}} = \lfloor 0{,}45 \cdot 2^{14} \rfloor = \lfloor 0{,}45 \cdot 16384 \rfloor = 7372.$$

(Tập lệnh `verify/verify_chuong_10_14.py` khẳng định đúng 7372.)

Khung sine hữu thanh. Cho $r_a = 12000$ (tức $12000/16384 \approx 0{,}732$), $P_{\text{local}} = 3277$, $P_{\text{global}} = 30000$, $S = 0{,}03$:

$$S \cdot P_{\text{global}} = 0{,}03 \cdot 30000 = 900.$$

Vì $12000 \geq 7372$ và $3277 \geq 900$, cổng (36) cho **voiced** $= 1$.

Khung gần im lặng. Cùng $r_a = 12000$ (nhiễu có thể tạo đỉnh giả cao!) nhưng $P_{\text{local}} = 200$:

$$200 < 900 \quad \Rightarrow \quad \text{voiced} = 0.$$

Đây là lý do bắt buộc có nhánh silence: chỉ nhìn r_a sẽ bị đánh lừa.

Tính s_{UV}. Đặt $P_{\text{local}} = 0{,}02$, $P_{\text{global}} = 1$, $S = 0{,}03$, $V = 0{,}45$:

$$\frac{S}{1+V} = \frac{0{,}03}{1{,}45} \approx 0{,}0206897, \qquad \frac{P_{\text{local}}/P_{\text{global}}}{S/(1+V)} = \frac{0{,}02}{0{,}0206897} \approx 0{,}96667.$$

$$s_{\text{UV}} = 0{,}45 + \max(0,\ 2 - 0{,}96667) = 0{,}45 + 1{,}03333 = 1{,}48333.$$

Một đỉnh $r_a = 0{,}30$ thua hoàn toàn ứng viên Unvoiced $\Rightarrow F_0 = 0$. Nếu $P_{\text{local}} = P_{\text{global}}$ thì hạng max tắt và $s_{\text{UV}} = V = 0{,}45$: silence không kích.

10.4. Median-3: làm sạch xung đơn với trễ đúng một hop

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý / trực giác: lấy giá trị "ở giữa" ba khung

Giọng người không đổi cao độ một quãng tám rồi quay lại trong đúng 10 ms. Một xung F_0 đơn (do nhiễu thắng argmax một khung) gần như chắc là lỗi. Bộ lọc **trung vị ba điểm (median-3)** lấy giá trị nằm giữa của ba mẫu liên tiếp: xung lệch bị loại, còn đường cong đơn điệu được giữ. Đây là thao tác thống kê trên chuỗi F_0, không phải hiện tượng khí động học mới — nhưng nó bám sát ràng buộc vật lý "F_0 người biến thiên chậm".

Với ba số a, b, c, một công thức chỉ dùng min / max (thuận RTL, không cần sắp xếp đầy đủ) là:

$$\text{med}(a,b,c) = \max\big(\min(a,b),\ \min\big(\max(a,b),c\big)\big). \tag{38}$$

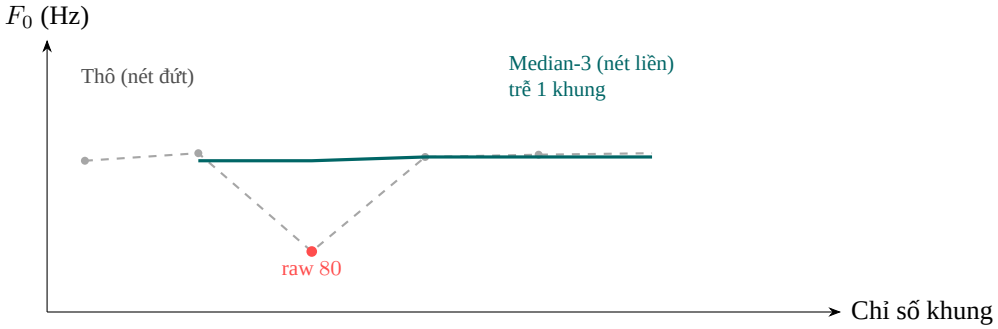
Trên silicon, ba thanh ghi trượt $f_1 \leftarrow f_2 \leftarrow f_3 \leftarrow F_0^{(\text{mới})}$; mỗi khi `in_valid`, tính med rồi **chốt vào** `m_reg`. Ngõ ra `f_out` chỉ cập nhật từ `m_reg` ở nhịp `in_valid` *kế tiếp* — tức trễ nhóm đúng một bước nhảy $H = 160$ mẫu ≈ 10 ms (đúng như Feynman box 11 ở Chương 0).

Ví dụ tính tay. Chuỗi cao độ thô (Hz): $200,\ 210,\ 80,\ 205,\ 208$. Các cửa sổ ba điểm:

Cửa sổ (a,b,c)	Sắp xếp	med	Ý nghĩa
$(200, 210, 80)$	$80, 200, 210$	200	Đập spike 80
$(210, 80, 205)$	$80, 205, 210$	205	
$(80, 205, 208)$	$80, 205, 208$	205	

Spike 80 Hz (thường là bắt nhầm $2T_0$ hoặc nhiễu) biến mất khỏi chuỗi xuất. Kiểm chứng bằng script `verify/verify_chuong_10_14.py`.

Bài tập nhỏ trong bài. Với một cửa $(100, 400, 110)$, med $= 110$ (sắp $100, 110, 400$). Xung 400 Hz (octave nhảy giả) bị loại; kết quả *không* phải 100 — đó là min, không phải median.



Hình 41: Median-3 đập một xung thấp đơn; đường xuất trễ đúng một khung so với quyết định voicing tức thời.

10.5. Chi phí rút gọn và Viterbi — hiểu để biết vì sao tắt trên fabric

Praat gắn mỗi ứng viên một độ mạnh $\delta(t,j)$ (đã trừ `octaveCost`), rồi tìm đường cực đại:

$$V(t,j) = \delta(t,j) + \max_i \big[V(t-1,i) - c_{ij}\big], \tag{39}$$

trong đó chi phí chuyển c_{ij} gồm:

- `octaveJumpCost`$\cdot|\log_2(f_j/f_i)|$ khi cả hai hữu thanh (mặc định $0{,}35$);
- `voicedUnvoicedCost` khi đổi V$\leftrightarrow$UV (mặc định $0{,}14$);
- nhân `timeStepCorrection`$= 0{,}01/\Delta t$ nếu đổi hop.

Ví dụ chi phí chuyển trạng thái.

Từ $\rightarrow$ Sang	f_1	f_2	c_{ij}
V$\rightarrow$V	150	160	$0{,}35 \cdot \lvert\log_2(150/160)\rvert \approx 0{,}03259$
V$\rightarrow$V	150	300	$0{,}35 \cdot 1 = 0{,}35$ (đúng một quãng tám — đắt)
V$\rightarrow$UV	150	0	$0{,}14$

Ví dụ Viterbi 2 khung $\times$ 2 ứng viên. Cho $\delta(t{=}1) = [V{:}0{,}50,\ \text{UV}{:}0{,}40]$, $\delta(t{=}2) = [V{:}0{,}45,\ \text{UV}{:}0{,}48]$, chi phí V–V $= 0{,}05$, V–UV $= 0{,}14$:

$$\text{cum}(V_2) = 0{,}45 + \max(0{,}50 - 0{,}05,\ 0{,}40 - 0{,}14) = 0{,}45 + 0{,}45 = 0{,}90,$$
$$\text{cum}(\text{UV}_2) = 0{,}48 + \max(0{,}50 - 0{,}14,\ 0{,}40 - 0) = 0{,}48 + 0{,}40 = 0{,}88.$$

Đường thắng là V$\rightarrow$V dù $\delta(\text{UV}_2)$ cao hơn $\delta(V_2)$ một chút — đúng tinh thần pathFinder: phạt chuyển trạng thái. (Phiên bản 3 khung với jump $0{,}02$ cho cum$(V_2) = 0{,}96 >$ cum$(\text{UV}_2) = 0{,}90$; xem script verify.)

Vì sao fabric tắt Viterbi? Độ phức tạp $O(TK^2)$ với K ứng viên/khung (Praat tới 15), cần lưu path metric và backpointer — tốn M10K và logic so sánh rộng. Lab chỉ giữ $K{=}1$ (argmax) + UV qua cổng (36) + median-3. Hai lý do không port `pathFinder` lên fabric ở lab này: (1) median-3 đã đập đủ spike đơn cho đo thanh điệu/Pearson trên giọng sạch; (2) HPS ARM Cortex-A9 có thể chạy soft-float pathFinder offline trên chuỗi F_0 thô nếu bài yêu cầu "Praat đầy đủ", không chiếm ALM thời gian thực.

10.6. Phần cứng: thuần ALM, không IP voicing

Không có Intel IP cho voicing hay median. Toàn bộ là so sánh và min / max trên Adaptive Logic Module (ALM).

```
// Voicing gate + registered median-3 (tre dung 1 hop)
// Gia su f0_in da la Hz nguyen hoac Q12.4 thô truoc pack.
reg [15:0] f1, f2, f3, m_reg, f_out;
reg        voiced_r, out_valid;
wire       voiced = (ra_q14 >= V_Q14) && (p_local >= silence_floor);
wire [15:0] med = max(min(f1,f2), min(max(f1,f2),f3));

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    f1<=0; f2<=0; f3<=0; m_reg<=0; f_out<=0; out_valid<=0;
  end else if (in_valid) begin
    f1<=f2; f2<=f3;
    f3<= voiced ? f0_in : 16'd0;   // UV ep 0 truoc median
    m_reg <= med;                  // tinh tren cua so cu
    f_out <= m_reg;                // xuat ban chot truoc (= tre 1 khung)
    out_valid <= 1'b1;
  end else out_valid <= 1'b0;
end
```

Giải thích từng ý: (1) ép UV về 0 *trước* median để một khung vô thanh không bị "kéo" bởi hai hàng xóm hữu thanh thành một giá trị giả; (2) `m_reg` rồi `f_out <= m_reg` tạo pipeline trễ đúng một `in_valid`; (3) không dùng DSP, không dùng M10K.

10.7. Twin Python khớp hành vi

```
import numpy as np

def med3(a, b, c):
    return max(min(a, b), min(max(a, b), c))

def voicing_gate(ra, V, P_local, P_global, S=0.03):
    return (ra >= V) and (P_local >= S * P_global)

def s_uv(P_local, P_global, S=0.03, V=0.45):
    return V + max(0.0, 2.0 - (P_local / P_global) / (S / (1.0 + V)))

def median3_stream(f0, voiced, delay=True):
    """Ep UV=0, median-3, tre 1 khung neu delay=True."""
    x = [float(f if v else 0.0) for f, v in zip(f0, voiced)]
    y = []
    for i in range(len(x)):
        if i < 2:
            m = x[i]  # warmup: xuat thô / 0 tuy chinh lab
        else:
            m = med3(x[i-2], x[i-1], x[i])
        y.append(m)
    if delay and y:
        y = [0.0] + y[:-1]  # thanh ghi xuat tre 1
    return np.asarray(y)
```

Mọi số trong mục 10.3–10.5 được khẳng định bởi `verify/verify_chuong_10_14.py`.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — khối voicing + median-3 (giải thích từng mục).

- **ALM** (*Adaptive Logic Module*): đơn vị logic Cyclone V. Cổng voicing (hai so sánh + AND) $\approx$ 20–40 ALM; median-3 (min / max trên 16 bit + thanh ghi) $\approx$ 60–100 ALM; max-hold P_{global} vài ALM. Tổng khối này $\ll 150$ ALM — chỉ một phần nhỏ ngân sách ≈ 850 ALM của cả luồng F_0 ở Chương 0.
- **DSP / M10K:** 0. Không nhân, không bảng lớn.
- **Latency:** đúng 1 hop $= 10$ ms do thanh ghi xuất median. Không tăng RTF tính MAC (RTF vẫn do Chương 7 quyết định, $\approx 0{,}24$).
- **Viterbi trên fabric:** 0 ALM mặc định. Nếu bật đủ $K{=}15$, ước tính hàng nghìn ALM + vài M10K path metric — phá ngân sách 2% và không cần cho lab median-3.
- **Vì sao chọn median thay Viterbi trên fabric:** đập spike đơn đủ cho đo thanh điệu tiếng Việt và Pearson với Praat trên giọng sạch; giữ RTF và diện tích; vẫn để cửa HPS chạy pathFinder khi bài yêu cầu.

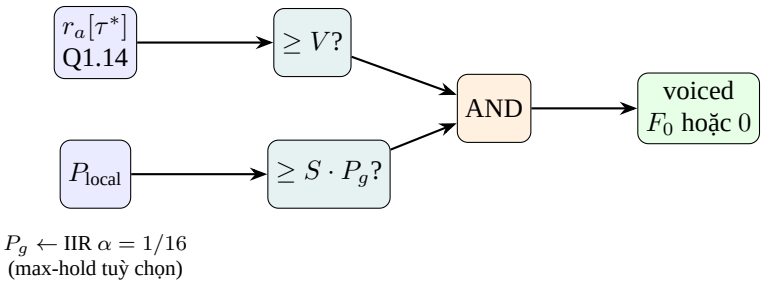
Ví dụ tính tay — warmup P_{global} bằng IIR chậm. Trên streaming FPGA không nhìn được cả câu nói trước, nên P_{global} được ước bằng bộ lọc IIR (*Infinite Impulse Response* — phản hồi vô hạn: lối ra phụ thuộc cả lối vào hiện tại và trạng thái nội đã nhớ). Ở đây dùng IIR bậc một với hệ số $\alpha = 1/16$ (dịch phải bốn bit, rất rẻ trên ALM):

$$P_g[n] = (1-\alpha)\,P_g[n-1] + \alpha\,P_{\text{local}}[n]. \tag{40}$$

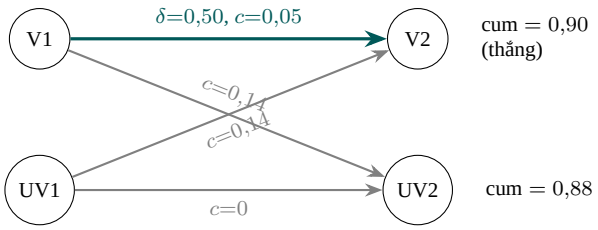
Cho chuỗi đỉnh biên độ cục bộ $P_{\text{local}} = \{100, 200, 500\}$ và $P_g[0] = 0$:

$$\begin{aligned}
P_g[1] &= \tfrac{15}{16}\cdot 0 + \tfrac{1}{16}\cdot 100 = 6{,}25,\\
P_g[2] &= \tfrac{15}{16}\cdot 6{,}25 + \tfrac{1}{16}\cdot 200 = 18{,}359375,\\
P_g[3] &= \tfrac{15}{16}\cdot 18{,}359375 + \tfrac{1}{16}\cdot 500 = 48{,}4619140625.
\end{aligned}$$

Sàn silence $S \cdot P_g$ lúc đó lần lượt $0{,}1875$, $0{,}5508$, $1{,}4539$ — vẫn nhỏ hơn P_{local}, nên cổng voicing *không* nuốt nguyên âm đang lên dần. Nếu α quá lớn (ví dụ $1/2$), P_g nhảy theo từng khung nhiễu và sàn silence dao động, dễ tạo VDE giả. Mọi số trên khớp `verify/verify_chuong_10_14.py`.

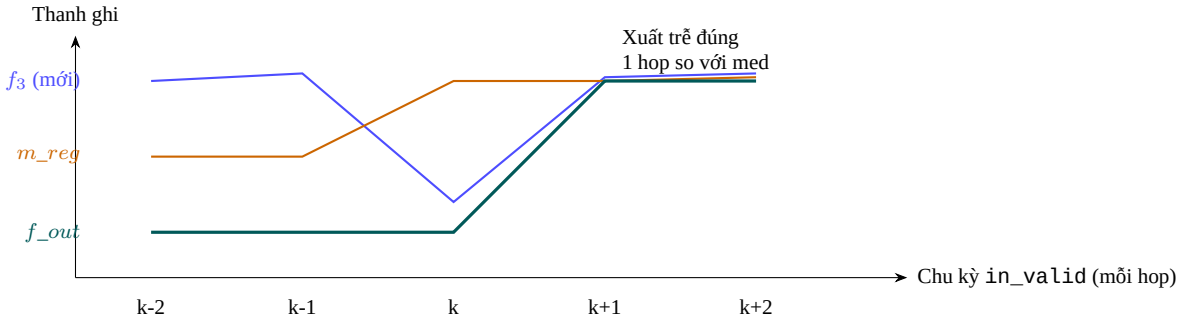


Hình 42: Khối cổng voicing trên fabric: hai so sánh + AND; P_g cập nhật chậm; UV ép $F_0 = 0$ trước median-3.

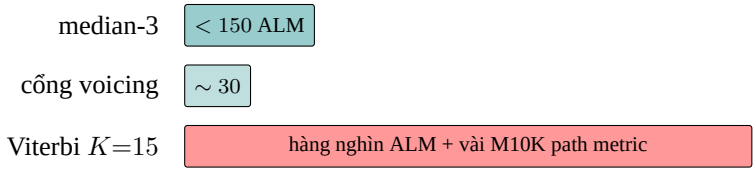


Trellis Viterbi 2×2: đường V→V thắng dù $\delta(\text{UV}_2) > \delta(V_2)$.

Hình 43: Lưới chuyển trạng thái rút gọn của `Pitch_pathFinder`: chi phí c_{ij} phạt nhảy V↔UV và octave.



Hình 44: Pipeline median-3 đăng ký: `f_out` lấy giá trị `m_reg` của nhịp trước, khớp Feynman box 11 (trễ nhóm 10 ms).



So sánh chi phí silicon: giữ median trên fabric; chuyển pathFinder sang HPS nếu cần.

Hình 45: Trực giác ngân sách: median-3 + cổng voicing nằm gọn trong phần nhỏ của ≈ 850 ALM; Viterbi đa ứng viên phá ngân sách lab.

Giải thích RTL từng dòng (mở rộng). Dây `voiced` là tổ hợp thuần (chỉ so sánh và AND, không có xung nhịp bên trong): so sánh r_a với ngưỡng V đã lượng tử Q1.14 thành $V_{\text{Q14}} = 7372$ (tức $0{,}45 \cdot 2^{14}$) và so P_{local} với sàn silence $S \cdot P_g$; không cần khối DSP. Ba thanh ghi f_1, f_2, f_3 tạo cửa sổ trượt ba khung. Trong Verilog, phép gán không chặn (*non-blocking*, toán tử `<=`) trong cùng khối `always` có nghĩa: mọi vế phải đọc giá trị *cũ* của thanh ghi trước khi cập nhật — vì vậy `med` nhìn đúng cửa sổ vừa hoàn tất, rồi mới dịch cửa. Ép khung vô thanh (UV) thành 0 ngay khi vào f_3 để một khung im không bị hai hàng xóm hữu thanh "kéo" thành cao độ giả qua median. Cặp `m_reg` ← med, rồi `f_out` ← `m_reg` thêm đúng một cấp thanh ghi xuất (trễ một hop): đo bằng

tay trên chuỗi $\{200, 210, 80, 205, 208\}$ cho `f_out` $= \{0, 0, 0, 200, 200\}$ (ba nhịp đầu là warmup), khớp script verify.

Vì sao octave jump $150 \to 300$ đắt đúng $0{,}35$. Quãng tám (*octave*) là khoảng tần số nhân đôi: dây thanh dao động gấp đôi số chu kỳ trong cùng thời gian, nên F_0 nhảy từ 150 lên 300 Hz (hoặc ngược lại xuống nửa). Praat phạt bước nhảy đó bằng hệ số $0{,}35$ nhân số octave trên thang $\log_2$:

$$|\log_2(150/300)| = |\log_2(1/2)| = 1, \qquad 0{,}35 \cdot 1 = 0{,}35.$$

Cùng khoảng đó theo cent (thang log âm nhạc, 1200 cent $=$ một octave): $1200 \log_2(400/200) = +1200$ và $1200 \log_2(100/200) = -1200$. Median-3 đập được xung octave *đơn* (một khung lệch); chuỗi octave bền nhiều khung liên tiếp cần pathFinder trên HPS hoặc chỉnh ROM $C[\tau]$ ở Chương 8.

10.8. Bài tập và lời giải giải thích

Bài tập 10.1 (Median-3 và octave spike). Cho chuỗi thô $F_0 = \{100,\ 400,\ 110,\ 105\}$ Hz (giả sử tất cả đã voiced). Tính ngõ ra median-3 *không* trễ thanh ghi cho các cửa ba điểm bắt đầu từ chỉ số 0, rồi nêu cửa nào đập được spike 400.

Lời giải. Cửa $(100, 400, 110)$: sắp $100, 110, 400 \Rightarrow$ med $= 110$. Cửa $(400, 110, 105)$: sắp $105, 110, 400 \Rightarrow$ med $= 110$. Spike 400 không bao giờ là trung vị khi hai hàng xóm nằm gần 100—110. Nếu nhầm dùng min sẽ ra 100 ở cửa đầu — đó là lỗi khái niệm (min $\neq$ median).

Bài tập 10.2 (s_{UV} khi không im lặng). Chứng minh khi $P_{\text{local}} = P_{\text{global}}$ và $S = 0{,}03$, $V = 0{,}45$ thì $s_{\text{UV}} = V$, tức nhánh silence trong (37) không kích.

Lời giải.

$$\frac{P_{\text{local}}/P_{\text{global}}}{S/(1+V)} = \frac{1}{0{,}03/1{,}45} = \frac{1{,}45}{0{,}03} \approx 48{,}333,$$

nên $2 - 48{,}333 < 0$ và $\max(0, \cdot) = 0 \Rightarrow s_{\text{UV}} = V = 0{,}45$. Ý nghĩa: khung đủ lớn so với đỉnh toàn cục thì Unvoiced chỉ còn sàn V, phải cạnh tranh sòng phẳng với r_a.

Bài tập 10.3 (Hai lý do không port pathFinder). Liệt kê hai lý do kỹ thuật/lab để mặc định tắt Viterbi trên fabric Cyclone V của bài này.

Lời giải. (1) **Tài nguyên và độ phức tạp:** $O(TK^2)$ với K tới 15 cần nhiều so sánh song song hoặc FSM dài + BRAM/M10K lưu metric — đẩy chiếm dụng ALM/M10K vượt ngân sách $\approx 2\%$ trong khi median-3 < 150 ALM đã đập spike đơn. (2) **Phân tầng hệ thống:** đo thời gian thực trên FPGA chỉ cần cổng voicing + median; đường cong "Praat đầy đủ" có thể tính soft-float trên HPS từ chuỗi ứng viên thô, không chặn luồng 16 kHz. Thêm nữa, tắt Viterbi buộc lab giữ $K{=}1$, đơn giản hóa argmax Chương 9.

Chương 11. FSM đỉnh và giao thức Avalon-ST (valid/ready/SOP/EOP)

▶ MỤC TIÊU HỌC TẬP

Sau Chương 11, người học sẽ:

1. Giải thích được hợp đồng silicon Avalon Streaming (Avalon-ST): cặp `valid`/`ready`, khái niệm **beat**, tín hiệu gói `SOP`/`EOP`, và hiện tượng áp lực ngược (*backpressure*).
2. Vẽ và đếm tay được một dạng sóng có khoảng nghỉ `ready`$= 0$ và khoảng nghỉ `valid`$= 0$, bảo đảm số beat vào bằng số beat ra.
3. Nối được FSM đỉnh đường F_0 (`IDLE`, `FILL`, `WINDOW`, `MAC`, `PEAK`, `VOICE`, `PACK`, `OUT`); mỗi chuyển trạng thái tôn trọng handshake với khối sau.
4. Viết RTL nguồn không bao giờ tiến con trỏ khi chưa có beat, và twin Python đếm beat khớp testbench.
5. Đọc hộp chi phí adapter Avalon-ST và lý do một mẫu mất im lặng phá cả F_0.

11.1. Hiện tượng thật: ống nước có van hai đầu

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao cần `valid`/`ready`? — không phải hiện tượng âm thanh

Trong Chương 0 (Feynman box 12) đã định nghĩa Avalon-ST là chuẩn điểm-nối-điểm một chiều của Intel cho DSP băng thông cao. Hãy gắn thêm trực giác đời thường: hai khối phần cứng như hai đoạn ống nước. Bên gửi (*source*) có van "tôi đang bơm" (`valid`); bên nhận (*sink*) có van "tôi còn chỗ chứa" (`ready`). Nước (một từ dữ liệu) chỉ chảy qua khớp nối khi *cả hai* van mở trong cùng một chu kỳ xung nhịp. Đó không phải quy luật vật lý của sóng âm — đó là **hợp đồng silicon** để hai khối khác tốc độ không làm mất mẫu, không nhân đôi mẫu, và không ghi đè bộ đệm.

Nếu mất đúng một mẫu trong khung $L = 640$, cửa sổ Hanning sẽ lệch một mẫu, hàm $\mathcal{R}_x[\tau]$ lệch, đỉnh τ^* lệch, và F_0 sai mà Quartus vẫn báo timing sạch. Lỗi handshake là lỗi *im lặng* — testbench phải đếm beat, không chỉ nhìn Fmax.

11.2. Toán / quy ước từng tín hiệu

Beat. Một **beat** là một lần chuyển dữ liệu thành công:

$$\text{beat}[n] = \big(\texttt{valid}[n] = 1\big) \ \wedge \ \big(\texttt{ready}[n] = 1\big). \tag{41}$$

Đếm số cạnh xung có beat $= 1$ trong cả phép mô phỏng phải bằng số mẫu (hoặc số từ) mà nguồn định gửi — trừ các chỗ cố ý hạ mẫu (FIR $2/3$ ở Chương 4).

`valid`. Source khẳng định: dữ liệu trên bus `data` ổn định và có nghĩa trong chu kỳ này. Khi source chưa có gì, hạ `valid`$= 0$ (khoảng nghỉ chủ động — *valid gap*).

`ready`. Sink khẳng định: nếu `valid`$= 1$ thì mình sẽ chốt dữ liệu vào thanh ghi/FIFO. Khi bộ đệm đầy, hạ `ready`$= 0$ — đây là **backpressure**.

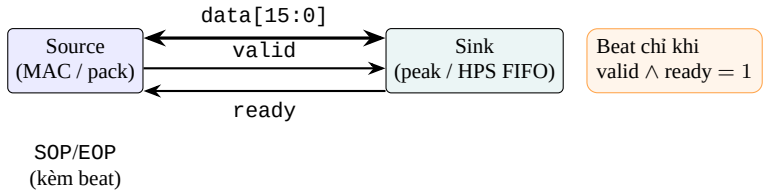
Luật giữ dữ liệu. Khi `valid` $= 1$ và `ready` $= 0$, source **bắt buộc giữ nguyên** cả `data` lẫn `valid`. Chỉ được đổi sang mẫu kế khi một beat đã xảy ra (hoặc khi tự hạ `valid`).

`readyLatency`. Tài liệu Avalon Interface Specifications (doc ID 683091) cho phép sink báo `ready` trước 0 hoặc 1 chu kỳ. Lab này dùng `readyLatency`$= 0$: quyết định nhận nằm cùng chu kỳ với `valid`. Nếu nối IP FIR/FFT cấu hình latency 1 mà FSM của bạn giả định 0, beat đầu/cuối burst sẽ lệch — đọc UG trước khi nối.

SOP / EOP (Start of Packet / End of Packet). Avalon-ST có thể đóng gói một **packet**: `SOP`$=1$ trên beat đầu, `EOP`$=1$ trên beat cuối. Với khung âm thanh $L = 640$ mẫu Q1.15 đưa vào khối cửa sổ/MAC, quy ước lab:

- Beat chỉ số 0: `SOP`$=1$, `EOP`$=0$;
- Beat $1 \ldots 638$: cả hai bằng 0;
- Beat 639: `SOP`$=0$, `EOP`$=1$.

Khối nhận dùng `SOP` để reset con trỏ cửa sổ, dùng `EOP` để kích hoạt MAC/peak. Thiếu `EOP` $\Rightarrow$ FSM treo ở `FILL`; thừa `EOP` $\Rightarrow$ cắt khung sớm.



Hình 46: Avalon-ST điểm-nối-điểm: data đi một chiều; valid từ source; ready từ sink; SOP/EOP đánh dấu biên gói khung.

11.3. Ví dụ tính tay — sáu/bảy chu kỳ có gap (đã kiểm logic)

Nguồn muốn gửi bốn mẫu $\{A, B, C, D\}$. Giữa B và C, sink kéo `ready`$=0$ đúng hai chu kỳ; sau beat C, source tự kéo `valid`$=0$ đúng một chu kỳ (valid gap), rồi gửi D.

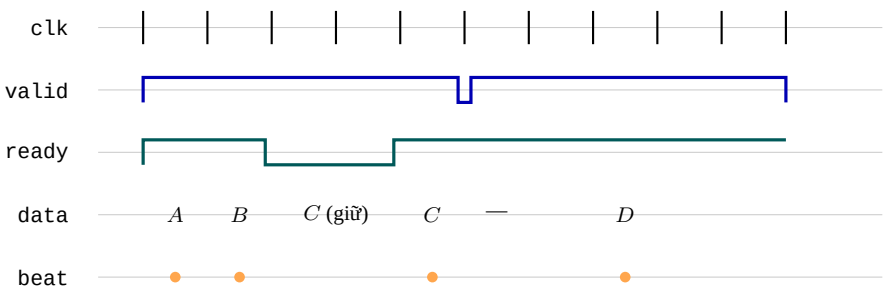
Chu kỳ	`valid`	`ready`	`data` trên bus	Beat?
1	1	1	A	có — chuyển A
2	1	1	B	có — chuyển B
3	1	0	C (giữ)	không
4	1	0	C (giữ)	không
5	1	1	C	có — chuyển C
6	0	1	—	không (valid gap)
7	1	1	D	có — chuyển D

Tổng beat $= 4$, đúng bằng số mẫu gửi. Script `verify/verify_chuong_10_14.py` đếm lại trace này. Nếu RTL tiến con trỏ ở chu kỳ 3 (khi `ready`$=0$), sink sẽ bỏ lỡ C hoặc nhận trùng B — đếm beat thành 3 hoặc 5, testbench phải fail.

Packet 640 **mẫu.** `SOP` trên beat 0, `EOP` trên beat 639. Mọi beat giữa không mang cờ gói. Twin đếm: số lần `SOP` $= 1$ phải bằng số khung; mỗi `SOP` đi kèm đúng một `EOP` sau đúng 639 beat (hoặc $L - 1$ khoảng cách beat).

11.3b. Dạng sóng timing Avalon-ST (vẽ từng cạnh xung)

Hình 47 là cùng ví dụ mục 11.3 vẽ theo trục thời gian chu kỳ: khi `ready`$=0$, `data` phải đứng yên ở C; khi source tự nghỉ, `valid`$=0$ dù sink vẫn sẵn sàng.

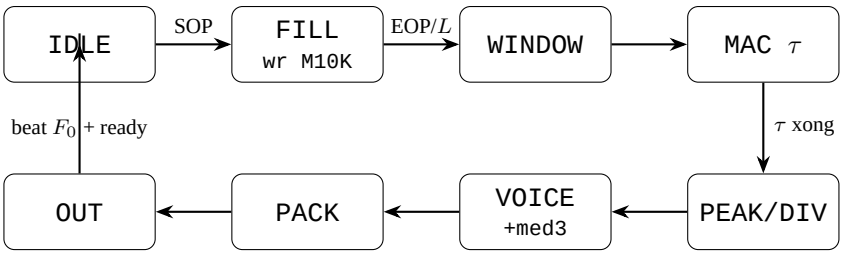


Bốn chấm cam = bốn beat $\{A, B, C, D\}$. Khoảng `ready`$=0$ không được tiến con trỏ.

Hình 47: Timing Avalon-ST `readyLatency`$=0$: beat chỉ tại giao của valid và ready đồng thời cao.

11.4. FSM đỉnh của đường F_0

Hình 48 là máy trạng thái tích hợp (rút gọn nhưng đủ lab). Mỗi trạng thái điều khiển một khối đã học ở các chương trước; mũi tên chỉ chuyển khi khối sau sẵn sàng (`ready`) hoặc khi điều kiện nội hoàn tất.



Hình 48: FSM pitch đỉnh: FILL đủ L mẫu (bám SOP/EOP)→ WINDOW→ MAC mọi τ → PEAK/DIV→ VOICE/median-3→ PACK Q12.4→ OUT Avalon-ST. Quay lại IDLE khi sink đã nhận beat cao độ.

Vai trò từng trạng thái (neo chương trước).

- `IDLE`: chờ `SOP` từ FIR/16 kHz (Chương 4–5).
- `FILL`: ghi True Dual-Port M10K, Old Data (Chương 5); đếm beat đến L hoặc chờ `EOP`.
- `WINDOW`: nhân ROM Hanning Q1.15 (Chương 6).
- `MAC`: quét $\tau = 26 \dots 214$ bằng 1 DSP (Chương 7); đây là trạng thái chiếm nhiều chu kỳ nhất ($\approx 120{,}960$ MAC).
- `PEAK/DIV`: argmax + parabolic + LPM_DIVIDE (Chương 9).
- `VOICE`: cổng voicing + median-3 đăng ký (Chương 10).
- `PACK`/`OUT`: đóng Q12.4 và handshake Avalon-ST ra HPS/Memory (Chương 12).

Nếu `OUT` gặp backpressure (`ready`$= 0$), FSM **không** được nhảy về `FILL` và ghi đè bộ đệm đang chờ — phải giữ `valid` và data Q12.4 như mục 11.2.

11.4b. Ngân sách chu kỳ từng trạng thái FSM

Tại xung nhịp hệ thống $f_{\text{clk}} = 50\,\text{MHz}$, mỗi hop $H/f_s = 160/16000 = 0{,}01\,\text{s}$ có đúng

$$N_{\text{cyc}} = f_{\text{clk}} \cdot 0{,}01 = 50 \cdot 10^6 \cdot 0{,}01 = 500\,000$$

chu kỳ. Phân bổ điển hình (lab MAC tuần tự một DSP):

Trạng thái	Chu kỳ (ước)	Ghi chú
FILL / WINDOW	$\sim$ 640–1280	ghi M10K + nhân Hanning
MAC $\tau \in [26, 214]$	120 960	189×640 (biên trên)
PEAK / DIV	$\sim$ 20–80	argmax + latency LPM_DIVIDE
VOICE / PACK / OUT	$\sim$ 5–20	median + Q12.4 + handshake
Tổng hữu ích	$\lesssim 123\,000$	RTF $\approx 0{,}24$
Biên còn lại	$\approx 377\,000$	chờ hop kế / backpressure

Số $189 = 214 - 26 + 1$ và tích $189 \times 640 = 120\,960$ đã kiểm script. RTF (*Real-Time Factor*) = thời gian xử lý / thời gian âm thanh $= 120\,960/500\,000 \approx 0{,}24192 \ll 1$: fabric kịp thật.

Ví dụ tính tay — bài tập 11.1 chi tiết. Gửi $\{X, Y, Z\}$, sau beat X sink kéo `ready`$= 0$ đúng ba chu kỳ:

ck	valid	ready	data	beat?
1	1	1	X	có
2	1	0	Y giữ	không
3	1	0	Y giữ	không
4	1	0	Y giữ	không
5	1	1	Y	có
6	1	1	Z	có

Tổng beat $= 3$. Script verify đếm lại trace này.

11.5. RTL — giữ data khi backpressure

```
// Source side: KHONG duoc advance rd_ptr khi chua co beat
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    out_valid <= 1'b0;
    out_data  <= 16'd0;
    out_sop   <= 1'b0;
    out_eop   <= 1'b0;
  end else if (!out_valid || out_ready) begin
    // Duoc phep nap beat moi (idle hoa vua chuyen xong beat cu)
    if (have_sample) begin
      out_data  <= next_sample;
      out_sop   <= next_is_first;
      out_eop   <= next_is_last;
      out_valid <= 1'b1;
      // chi o day moi rd_ptr <= rd_ptr + 1
    end else begin
      out_valid <= 1'b0;
      out_sop   <= 1'b0;
      out_eop   <= 1'b0;
    end
  end
  // else: giu nguyen bus (backpressure)
end
```

Dòng then chốt là điều kiện `!out_valid || out_ready`: chỉ khi chưa cầm beat, hoặc sink vừa nhận beat, source mới được đổi bus. Mọi chỗ khác phải giữ nguyên — kể cả `SOP`/`EOP`.

11.5b. Avalon-ST bên trong IP Intel và cầu HPS–FPGA

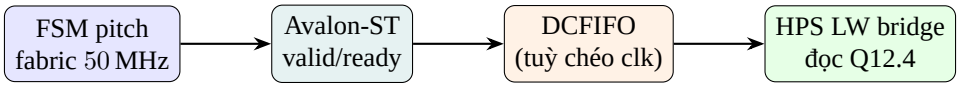
Tài liệu *Avalon Interface Specifications* (doc ID 683091) định nghĩa thêm:

- `empty`/`error`: dùng khi bề rộng biểu tượng > 1 byte; lab pitch dùng biểu tượng 16 bit đầy đủ nên thường bỏ `empty`.
- `channel`: phân kênh ảo trên cùng dây — lab một đường mono không cần.
- FIR Compiler II và FFT IP đều nói Avalon-ST; khi nối FIR→sliding RAM phải khớp `readyLatency`. Lab chọn 0 xuyên suốt để FSM đơn giản: quyết định nhận nằm cùng chu kỳ `valid`.

Cầu HPS–FPGA trên DE10-Standard. Cyclone V SX trên DE10-Standard có Hard Processor System (HPS) — cụm CPU cứng dual Cortex-A9 nằm trên cùng chip với FPGA fabric. Ba cầu chính nối HPS với logic do bạn viết:

- **Lightweight HPS-to-FPGA (LW bridge)** $\approx$ 2 MB không gian địa chỉ: CPU đọc thanh ghi trạng thái / từ Q12.4 từng khung — đủ cho 97 từ/giây.
- **HPS-to-FPGA bridge** rộng hơn: DMA burst nếu sau này ghi WAV+bin xuống SDRAM.
- **FPGA-to-HPS**: fabric làm master (ít dùng ở lab pitch).

Thực tế lab: khối `OUT` đẩy Avalon-ST vào DCFIFO (*Dual-Clock FIFO* — hàng đợi hai miền xung nhịp, IP Intel) nếu HPS và FPGA khác PLL, rồi HPS đọc định kỳ (poll) hoặc nhận ngắt khi FIFO nửa đầy. Một từ mất trên cầu LW không làm Quartus báo lỗi — chỉ làm Pearson thanh điệu sụp; đó là lý do tầng 3 phải đếm beat đến tận biên HPS.



Hình 49: Đường ra cao độ: FSM→Avalon-ST→(FIFO)→HPS. Backpressure từ FIFO đầy phải đứt ngược về `OUT`.

M10K trong FILL. Trạng thái `FILL` ghi True Dual-Port On-Chip Memory (khối M10K 10 kbit). Một khung $640 \times 16 = 10\,240$ bit vừa khít ≈ 1 M10K. Chế độ mixed-port read-during-write trên Cyclone V chỉ **Old Data** hoặc **Don't Care** — không có New Data (Chương 5). FSM không được giả định đọc thấy mẫu vừa ghi trong cùng chu kỳ trên cổng kia.

11.5c. RTL giải thích từng dòng

```
// Dieu kien nap beat moi:
//   !out_valid  : bus dang trong (chua cam beat)
//   out_ready   : sink vua nhan beat (duoc doi mau ke)
// Ngoai hai truong hop do: GIU NGUYEN data/valid/SOP/EOP.
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    out_valid <= 1'b0;
    out_data  <= 16'd0;
    out_sop   <= 1'b0;
    out_eop   <= 1'b0;
  end else if (!out_valid || out_ready) begin
    if (have_sample) begin
      out_data  <= next_sample;   // chi doi khi duoc phep
      out_sop   <= next_is_first;
      out_eop   <= next_is_last;
      out_valid <= 1'b1;
      // rd_ptr <= rd_ptr + 1;  // DUY NHAT noi advance
    end else begin
      out_valid <= 1'b0;          // valid gap chu dong
      out_sop   <= 1'b0;
      out_eop   <= 1'b0;
    end
  end
  // else: backpressure — latched bus khong doi
end
```

Từng ý: (1) `rst_n` bất đồng bộ hạ đưa mọi cờ về 0 an toàn; (2) điều kiện `!out_valid || out_ready` là trái tim hợp đồng 683091 với `readyLatency`$= 0$; (3) con trỏ đọc chỉ tăng trong nhánh `have_sample` — tăng ngoài nhánh này là bug cố ý của bài tập 11.2; (4) nhánh `else` ẩn (không ghi gì) giữ latch — đó chính là "giữ data khi backpressure".

► NHÌN THẤY VẬT LÝ

Handshake không "sửa" thuật toán Boersma. Nó chỉ bảo đảm đúng mẫu đúng lúc đến đúng khối. Một beat mất ở biên MAC↔peak làm parabolic gắn nhầm τ, tạo GPE octave giả trong khi mọi công thức trong twin float vẫn xanh — đó là lớp lỗi tầng 3 ở Chương 13.

11.6. Twin đếm beat và kiểm SOP/EOP

```
def count_beats(trace):
    return sum(1 for c in trace if c["valid"] and c["ready"])

def check_packets(trace, L=640):
    beats = [c for c in trace if c["valid"] and c["ready"]]
    assert count_beats(trace) == len(beats)
    # Moi goi: SOP tren beat dau, EOP tren beat cuoi, dung L beat
    i = 0
    while i < len(beats):
        assert beats[i]["sop"] and not beats[i]["eop"]
        assert i + L - 1 < len(beats)
        for k in range(1, L - 1):
            assert not beats[i+k]["sop"] and not beats[i+k]["eop"]
        assert beats[i+L-1]["eop"] and not beats[i+L-1]["sop"]
        i += L
```

Testbench ModelSim/Questa: LFSR hạ `ready` ngẫu nhiên; LFSR tạo valid-gap phía source; scoreboard so số beat và nội dung Q1.15 từng mẫu với twin.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — FSM + adapter Avalon-ST.

- **Adapter valid/ready/SOP/EOP:** khoảng 0–30 ALM (mux giữ data, vài flip-flop); 0 DSP; 0 M10K.
- **FSM đỉnh:** vài trăm ALM tùy số trạng thái và độ rộng bộ đếm địa chỉ (10 bit cho 640), vẫn nằm trong ngân sách ≈ 850 ALM toàn luồng.
- **Latency điều khiển:** bản thân handshake thêm 0–1 chu kỳ mỗi biên; không đổi RTF MAC $\approx 0{,}24$.
- **Vì sao giữ data khi `ready`$= 0$:** mất một mẫu = lệch khung = sai $\mathcal{R}_x$ = sai F_0 — không có cảnh báo synthesis.
- **Vì sao cần SOP/EOP:** tách biên gói 640 mẫu trong luồng liên tục 16 kHz; thiếu chúng FSM không biết lúc nào kết thúc `FILL`.

Ba luật Avalon-ST cần thuộc trước Signal Tap. (1) Beat chỉ xảy ra khi `valid` và `ready` cùng cao trong một chu kỳ (`readyLatency`$= 0$). (2) Khi `valid`$= 1$ mà `ready`$= 0$, source giữ nguyên `data` lẫn SOP/EOP. (3) Mỗi gói $L = 640$ có đúng một SOP ở beat đầu và một EOP ở beat cuối. Vi phạm luật (2) lệch $\mathcal{R}_x$ và F_0 dù Fmax vẫn xanh — twin đếm beat phải khớp testbench trên mọi seed LFSR.

11.7. Bài tập và lời giải giải thích

Bài tập 11.1 (Waveform 8 chu kỳ). Gửi 3 mẫu $\{X, Y, Z\}$. Sau beat X, sink kéo `ready`$= 0$ trong đúng 3 chu kỳ, rồi nhận Y và Z liên tiếp. Vẽ bảng `valid`/`ready`/`data`/beat và đếm tổng beat.

Lời giải. Một lịch hợp lệ: ck1 beat X; ck2–4 `valid`$= 1$, `ready`$= 0$, giữ Y; ck5 beat Y; ck6 beat Z. Tổng beat $= 3$. Mọi lịch tiến con trỏ sớm sẽ làm sink thấy sai mẫu trên ck5.

Bài tập 11.2 (Bug cố ý). RTL advance `rd_ptr` mỗi khi `valid`$= 1$ dù `ready`$= 0$. Testbench kỳ vọng gì?

Lời giải. Source đã "quên" mẫu đang giữ: khi `ready` lên lại, bus đã sang mẫu kế. Số beat đếm được có thể vẫn bằng N nếu sink chốt nhầm nội dung, nhưng scoreboard nội dung fail; hoặc nếu sink không chốt khi `ready`$= 0$ mà source đã bỏ mẫu, tổng beat $< N$. Cả hai đều phải đỏ.

Bài tập 11.3 (MAC chậm hơn peak). Khối MAC phát $\mathcal{R}_x[\tau]$ từng τ sang peak. Peak cần vài chu kỳ xử lý một τ nên hạ `ready`. Chứng minh source MAC phải giữ đúng $\mathcal{R}_x[\tau]$ hiện tại.

Lời giải. Nếu MAC chuyển sang $\tau + 1$ khi peak chưa nhận, peak sẽ bỏ τ hoặc gắn kết quả parabolic vào trễ sai. Luật Avalon-ST: không beat thì không đổi `data`. Twin đếm beat giữa MAC và peak phải bằng số τ trong $[\tau_{\min}, \tau_{\max}]$ (189 giá trị).

Chương 12. Đóng gói F_0 Q12.4 và tệp `pitch_f0.bin`

▶ MỤC TIÊU HỌC TẬP

Sau Chương 12, người học sẽ:

1. Giải thích được hợp đồng số giữa FPGA và phần mềm phía trên: mỗi khung xuất một từ **uint16** theo thang **Q12.4** (đơn vị $1/16$ Hz), ghi **little-endian**, khung vô thanh gửi đúng 0.
2. Tính tay và đóng gói được bảng $F_0 \to$ Q12.4 $\to$ hai byte LE cho các mốc 0, 75, $150{,}0625$, $199{,}466$, 200, 220, 440, 600, 700 (bão hòa), đã kiểm Python.
3. Đọc được bố trí bitfield (Hình 50–52), tách phần nguyên/frac, và giải mã đúng trên HPS.
4. Ghép được cặp tệp kiểm chứng `audio_16k.wav` (PCM 16 kHz) với `pitch_f0.bin` (chuỗi uint16 LE đồng bộ theo hop 10 ms).
5. Viết RTL pack có bão hòa trần 600 Hz ($q = 9600$) và twin `struct.pack("<H", q)` khớp từng bit; phân biệt lỗi thuật toán với lỗi giao thức (đảo endian, quên chia 16).

12.1. Hiện tượng thật: server đọc gì từ FPGA?

★ NGUYÊN LÝ ĐẦU TIÊN: Hợp đồng số, không phải số thực dấu phẩy động

Khối ước lượng trên Cyclone V không nói chuyện với HPS/server bằng `float64`. Sau median-3 (Chương 10) và FSM OUT (Chương 11), hợp đồng là một từ 16 bit không dấu theo thang **Q12.4**: 12 bit phần nguyên + 4 bit phần thập phân cố định. Giá trị nguyên lưu trong thanh ghi thỏa

$$F_q = \mathrm{sat}_{0..9600}\big(\mathrm{round}(F_0^{\mathrm{Hz}} \cdot 16)\big), \qquad \hat{F}_0 = F_q/16 \text{ (Hz)}. \tag{42}$$

Trọng số bit thấp nhất (LSB) bằng $2^{-4} = 0{,}0625$ Hz. Trên tệp `pitch_f0.bin` (và trên bus Avalon-ST 16 bit), hai byte được ghi **little-endian**: byte thấp trước, byte cao sau — khớp hầu hết CPU little-endian của HPS ARM. Khung vô thanh (**unvoiced**, UV) gửi đúng $F_q = 0$ (`00 00`), không dùng NaN hay mã đặc biệt. Đảo endian hoặc quên chia 16 phía server làm $\hat{F}_0$ lệch $\times 256$ hoặc $\times 16$ trong khi silicon vẫn đúng: đó là lỗi *giao thức*, không phải lỗi pitch.

Vì sao chọn đúng 4 bit frac? Tại $F_0 = 200$ Hz, một LSB $0{,}0625$ Hz tương đương

$$1200 \log_2\big((200 + 0{,}0625)/200\big) \approx 0{,}541 \text{ cent}$$

(đã kiểm script). Ngân sách sai số lab so Praat thường 20–30 cent (Chương 13), nên lượng tử Q12.4 nhỏ hơn nhiều so với sai số thuật toán — không phải nút thắt. Đồng thời $600 \cdot 16 = 9600 < 2^{16} - 1$, cả dải $75 \ldots 600$ Hz nằm gọn trong 16 bit không dấu kèm trần bão hòa rõ ràng.

Trực giác đời thường. Hãy nghĩ Q12.4 như thước kẻ có vạch mỗi $1/16$ Hz. Bạn không gửi "hai trăm phẩy không" bằng chữ; bạn gửi số đếm vạch (3200 vạch = 200 Hz). Phần mềm phía trên chỉ việc chia lại 16 để ra Hz. Nếu cầm thước đọc ngược (big-endian), số đếm vạch thành một số khác hoàn toàn — thước không hỏng, người đọc hỏng.

12.2. Toán từng bước: từ Hz đến hai byte

1. Nhân F_0 (Hz, số thực sau LPM_DIVIDE/parabolic) với 16.
2. Làm tròn tới số nguyên gần nhất: $q_0 = \mathrm{round}(F_0 \cdot 16)$.
3. Nếu không voiced hoặc $F_0 \le 0$: ép $q = 0$.
4. Bão hòa: $q = \min(q_0,\ \texttt{CEIL})$ với $\texttt{CEIL} = 600 \cdot 16 = 9600$.
5. Tách byte: $b_0 = q \bmod 256$, $b_1 = \lfloor q/256 \rfloor$ (little-endian: ghi b_0 rồi b_1).

Định dạng Q$m.n$ đã được Chương 0 (Feynman box 7) định nghĩa: đây là Q12.4 *không dấu* trên dây ra, khác với Q1.15 có dấu của mẫu âm thanh. Quan hệ bit:

$$F_q = (F_q[15:4]) \cdot 16 + F_q[3:0], \qquad \hat{F}_0 = F_q/16. \tag{43}$$

Không cần "nối" thêm dấu phẩy động IEEE 754 trên cầu LW: một lệnh load half-word trên ARM rồi dịch/chia nguyên là đủ.

12.3. Ví dụ tính tay (đã kiểm Python)

F_0 (Hz)	F_q (uint16)	Bytes LE	Ghi chú
0 (UV)	0	`00 00`	Hợp đồng UV
75,0	1200	`B0 04`	Sàn tìm kiếm
150,0625	2401	`61 09`	Đúng một LSB trên 150
150,25	2404	`64 09`	$150 \cdot 16 + 4$
199,466	3191	`77 0C`	Parabolic Chương 9
200,0	3200	`80 0C`	Mốc lab
220,0	3520	`C0 0D`	
440,0	7040	`80 1B`	
600,0	9600	`80 25`	Trần
700,0	9600	`80 25`	Bão hòa về trần

Chi tiết một hàng — 199,466 Hz.

$$\begin{aligned} 199{,}466 \cdot 16 &= 3191{,}456, \\ \text{round}(3191{,}456) &= 3191, \\ 3191 &= \text{0x0C77} = 12 \cdot 256 + 119, \\ b_0 &= \text{0x77}, \quad b_1 = \text{0x0C}. \end{aligned}$$

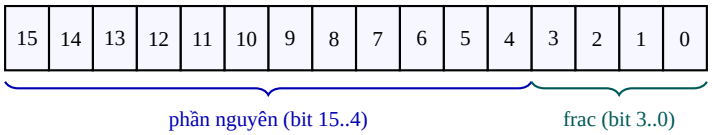
Giải mã ngược: $3191/16 = 199{,}4375$ Hz (phần nguyên 199, frac 7). Sai số lượng tử so với 199,466 chỉ $\approx 0{,}0285$ Hz.

Bẫy endian. Bytes LE của 200 Hz là `80 0C`. Nếu server đọc nhầm big-endian:

$$\texttt{0x0C80} = 3200_{\text{LE}} \text{ đọc BE } \Rightarrow \text{ 0x0C80} = 32780, \qquad 32780/16 = 2048{,}75 \text{ Hz}.$$

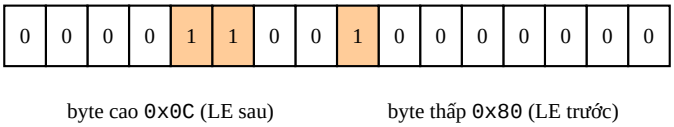
Một lỗi một dòng `struct.unpack` biến 200 Hz thành > 2 kHz — GPE/FFE ở Chương 13 sẽ "nổ", trong khi RTL và twin pack vẫn xanh.

12.4. Bố trí bitfield Q12.4 (Hình 50–52)



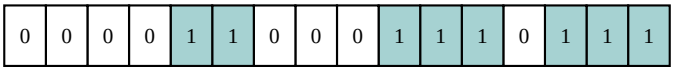
Hình 50: Bitfield Q12.4: bit 0 (LSB = 0,0625 Hz) bên phải; bit 15 bên trái.

$$200 \text{ Hz} \to F_q = 3200 = \text{0x0C80}$$



Hình 51: Bit pattern 200 Hz: trên tệp/bin là `80 0C`.

$$199{,}466\,\text{Hz} \to 3191 = \text{0x0C77}$$



$3191 \gg 4 = 199$, $3191 \bmod 16 = 7 \Rightarrow 199{,}4375\,\text{Hz}$ sau giải mã.

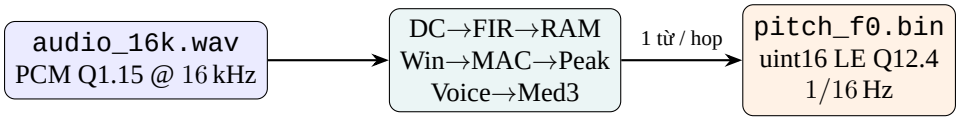
Hình 52: Bit pattern sau parabolic; LE `77 0C`.

Ví dụ tính tay — tách bit đã kiểm. Script `verify/verify_chuong_10_14.py` khẳng định $(3200 \gg 4, 3200\&15) = (200, 0)$ và $(3191 \gg 4, 3191\&15) = (199, 7)$. Đây là cách HPS có thể tránh chia nếu muốn: dịch phải 4 lấy Hz nguyên, lấy bốn bit thấp nhân $0{,}0625$ nếu cần frac.

12.5. Cặp tệp `audio_16k.wav` và `pitch_f0.bin`

Luồng kiểm chứng chuẩn lab.

1. `audio_16k.wav`: PCM mono 16-bit, $f_s = 16\,\text{kHz}$, đã qua (hoặc giả lập) DC-block + FIR hạ mẫu. Đây là đầu vào "sạch" của sliding RAM.
2. Chạy đường float/fixed/RTL theo hop $H = 160$ mẫu (10 ms): số khung
$$N_{\text{frames}} = \left\lfloor \frac{N_{\text{samples}} - L}{H} \right\rfloor + 1$$
với $L = 640$.
3. `pitch_f0.bin`: đúng N_{frames} từ uint16 LE. Khung k ứng với tâm cửa sổ quanh mẫu $kH + L/2$ (cùng quy ước thời gian Praat time step $0{,}01$ s).



Đồng bộ: khung $k \leftrightarrow$ byte $2k, 2k+1$ trong bin

Hình 53: Hợp đồng kiểm chứng: mỗi hop 10 ms trên wav sinh đúng hai byte trong `pitch_f0.bin`.

Ví dụ tính tay — một giây. Một giây âm thanh 16 kHz có 16000 mẫu. Với $L = 640$, $H = 160$:

$$N_{\text{frames}} = \lfloor (16000 - 640)/160 \rfloor + 1 = 97, \qquad |\texttt{bin}| = 97 \cdot 2 = 194 \text{ byte}.$$

Script verify assert các số này.

12.6. RTL pack — giải thích từng dòng

```
module pack_q124 #(
  parameter CEIL_Q = 16'd9600  // 600 Hz * 16
)(
  input  wire        clk, rst_n,
  input  wire        in_valid,
  input  wire        voiced,
  input  wire [15:0] f0_hz_u,   // phan nguyen Hz (tu LPM_DIVIDE)
  input  wire [3:0]  f0_frac,   // 4 bit frac neu mang san Q12.4
  output reg         out_valid,
  output reg  [15:0] q124,
  output reg  [7:0]  b0, b1     // LE: b0 thap, b1 cao
);
```

```
  // raw = f0_hz_u * 16 + f0_frac  (khong dau, 17 bit de so CEIL)
  wire [16:0] raw = {1'b0, f0_hz_u, 4'b0} + f0_frac;
  wire [15:0] sat = (raw > CEIL_Q) ? CEIL_Q : raw[15:0];
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      q124<=0; b0<=0; b1<=0; out_valid<=0;
    end else if (in_valid) begin
      q124 <= voiced ? sat : 16'd0;
      b0   <= voiced ? sat[7:0]  : 8'd0;
      b1   <= voiced ? sat[15:8] : 8'd0;
      out_valid <= 1'b1;
    end else out_valid <= 1'b0;
  end
endmodule
```

Từng dòng then chốt: (1) `{f0_hz_u, 4'b0}` là dịch trái 4 bit $=$ nhân 16 không cần DSP; (2) cộng `f0_frac` nếu datapath đã mang sẵn phần thập phân; (3) so sánh với 9600 trên 17 bit tránh wrap; (4) mux UV ép 0 trên cả từ và cả hai byte; (5) một chu kỳ đăng ký — latency pack $= 1$, không đổi RTF MAC.

Không cần DSP: chỉ dịch, cộng, so sánh bão hòa, mux. Ngõ `b0`/`b1` tiện ghi Memory-Mapped hoặc stream byte vào thẻ SD; ngõ `q124` đi Avalon-ST 16 bit sang HPS qua LW bridge (Chương 11).

12.7. Twin bit-exact và đường giải mã HPS

```
import numpy as np, struct
CEIL = 9600

def pack_q124(f0_hz, voiced=True):
    if (not voiced) or f0_hz <= 0:
        q = 0
    else:
        q = max(0, min(CEIL, int(np.round(f0_hz * 16.0))))
    return q, struct.pack("<H", q)

assert pack_q124(200.0)[1] == b"\x80\x0c"
assert pack_q124(199.466)[0] == 3191
assert pack_q124(0.0, False)[0] == 0
assert pack_q124(700.0)[0] == CEIL
assert pack_q124(220.0)[1] == b"\xc0\x0d"
```

Giải mã phía HPS / Python.

```
import numpy as np, wave
raw = open("pitch_f0.bin", "rb").read()
assert len(raw) % 2 == 0
f0 = np.frombuffer(raw, dtype="<u2").astype(np.float64) / 16.0
with wave.open("audio_16k.wav", "rb") as w:
    assert w.getframerate() == 16000 and w.getnchannels() == 1
    n = w.getnframes()
n_frames = (n - 640) // 160 + 1
assert len(f0) == n_frames
# f0[k] == 0  =>  UV tai khung k
```

Nếu `dtype=">u2"` (big-endian) trên máy LE, mọi kiểm chứng cent/GPE sẽ sai dù RTL đúng.

Bão hòa trần 600 Hz. Mọi $F_0 > 600$ (do nhiễu đỉnh τ nhỏ) bị ghim $q = 9600$. Trần khớp `pitch_ceiling` Praat lab; không ghim sẽ tràn ý nghĩa trên thang đo hoặc tạo giá trị > 600 Hz ngoài hợp đồng tìm kiếm.

Cầu Lightweight HPS-to-FPGA. Thông lượng 97 từ/giây (≈ 194 B/s cho một giây audio) nhỏ hơn hàng nghìn lần băng thông LW bridge trên Cyclone V SX — lab chỉ cần poll thanh ghi hoặc ngắt khi FIFO có dữ liệu, không bắt buộc DMA. Vẫn phải tôn trọng Avalon-ST/Avalon-MM ở biên: mất một ghi = thiếu một khung trên bin.

▶ NHÌN THẤY VẬT LÝ

Q12.4 không phải định dạng "nghe được". Nó là hợp đồng đo lường giữa fabric và phần mềm. Sai hợp đồng làm mọi metric Chương 13 đỏ trong khi MAC/peak vẫn đúng — hãy `hexdump -C pitch_f0.bin | head` trước khi đổ tội Boersma.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — đóng gói Q12.4 (giải thích từng mục).

- **ALM** (*Adaptive Logic Module*): vài chục (dịch, cộng frac, so CEIL, mux UV, thanh ghi). Đơn vị logic cơ bản của Cyclone V.
- **DSP** (*Digital Signal Processing block*): 0 — không nhân rộng.
- **M10K** (block RAM ≈ 10 kbit): 0 cho bản thân pack; FIFO HPS tuỳ chọn 1 M10K nếu gom burst nhiều khung.
- **Latency:** 1 chu kỳ đăng ký — không đổi RTF (*Real-Time Factor* = thời gian xử lý / thời gian âm thanh).
- **Fmax:** khối tổ hợp nông (dịch+so sánh) không phải nút thắt so MAC/DIV.
- **Vì sao Q12.4:** đủ 75–600 Hz trong 16 bit; LSB ≈ 0,54 cent tại 200 Hz, nhỏ hơn ngân sách 20 cent; LE khớp HPS ARM.
- **Vì sao UV= 0:** một giá trị đặc biệt duy nhất — server không cần enum riêng; median-3 và Pearson đều xử lý 0 như "không cao độ".

12.7b. Từ thanh ghi fabric đến tiến trình user-space trên HPS

Sau khi `pack_q124` chốt F_q, có ít nhất ba cách đưa số sang phần mềm — lab chọn cách (1) vì đơn giản và đủ thông lượng:

1. **Thanh ghi Avalon-MM trên LW bridge:** Avalon-MM (*Memory-Mapped*) là bus địa chỉ/đọc-ghi; FSM OUT viết F_q vào một thanh ghi, CPU đọc định kỳ mỗi 10 ms (hoặc khi bit "new" bật). Phù hợp 97 từ/giây.
2. **DCFIFO + ngắt:** fabric đẩy Avalon-ST vào FIFO; khi mực nước (watermark) đạt ngưỡng, HPS chạy ISR (*Interrupt Service Routine* — hàm đáp ứng ngắt) để rút hết. Tốt khi lịch CPU bị jitter lớn.
3. **DMA qua bridge rộng:** DMA (*Direct Memory Access*) ghi liên tục vào SDRAM rồi tiến trình user-space `mmap`. Chỉ cần khi sau này ghi cả PCM + bin tốc độ cao.

Ví dụ tính tay — poll period. Giả sử CPU poll mỗi 5 ms trong khi hop là 10 ms. Mỗi khung được đọc khoảng hai lần; phần mềm phải dùng bit "new" hoặc đếm khung để không nhân đôi mẫu trên `pitch_f0.bin`. Nếu poll 20 ms, có nguy cơ bỏ khung khi ISR/trễ lịch — FIFO depth tối thiểu nên ≥ 2 từ (an toàn hơn ≥ 8).

Endian trên dây Avalon-ST 16 bit. Trên bus 16 bit, một beat mang cả từ F_q; "little-endian" lộ rõ khi *phá* thành byte để ghi tệp hoặc gửi UART. Nhầm chỗ phá byte (coi bit 15..8 là byte thấp) chính là bug `0C 80` thay vì `80 0C`. Test tối thiểu: golden 200 Hz phải dump đúng hai byte `80 0C` trên logic analyzer hoặc trong file.

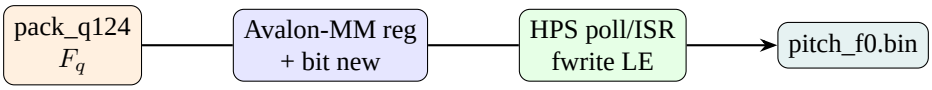
Vì sao không xuất float32. Một số nhóm gửi `float32` little-endian (4 byte/khung). Lab giữ Q12.4 vì không cần FPU trên đường ra, dễ khớp twin nguyên từng bit, và LSB đã nhỏ hơn ngân sách cent. Nếu bài bắt buộc float, convert một lần duy nhất trên HPS sau khi đã xác nhận hai byte Q12.4 đúng.

Ví dụ tính tay — bão hòa và UV cạnh nhau. Chuỗi thô sau peak: $\{700,\ 0,\ 220\}$ Hz với cờ voiced $\{1, 0, 1\}$. Pack:

$$\begin{aligned} 700 &\to \text{sat}(11200) = 9600 \to \texttt{80 25}, \\ 0\ (\text{UV}) &\to 0 \to \texttt{00 00}, \\ 220 &\to 3520 \to \texttt{C0 0D}. \end{aligned}$$

Tệp bin sáu byte: `80 25 00 00 C0 0D`. Median-3 (Chương 10) nếu chạy *trước* pack sẽ nhìn chuỗi Hz đã ép UV$=0$ và bão hòa tùy thiết kế; lab chuẩn: voicing→median→pack, và bão hòa nằm trong pack để trần hợp đồng thống nhất.

Đồng bộ thời gian với Praat. Mỗi khung lab k ứng $t = 0{,}01 \cdot k$ giây (hoặc neo tâm cửa $t = 0{,}01 \cdot k + L/(2f_s)$). Lệch một hop làm hệ số Pearson thanh điệu giảm dù từng Hz đúng — lỗi giao thức thời gian, cùng họ với lỗi endian.



Hình 54: Đường lab mặc định: pack → reg LW → HPS → bin. Mỗi mũi tên là chỗ có thể đảo endian hoặc nhân đôi/bỏ khung.

12.7c. Kiểm thử giao thức tách khỏi thuật toán

Trước khi chạy GPE/VDE, hãy có bộ test *chỉ* pack/unpack:

- Vector F_0 biết trước → so byte bin với bảng mục 12.3.
- Cố ý unpack BE → assert fail rõ ràng (bắt regression).
- Đếm `len(bin)==2*n_frames` với n_frames từ wav.
- Khung UV xen kẽ: mọi vị trí UV là `00 00`, không "rò" 1 LSB.

Chỉ khi bộ này xanh mới mở tầng so Praat. Quy tắc này tiết kiệm ngày debug "vì sao GPE $= 1$ trên sine hoàn hảo".

12.7d. Sai số lượng tử Q12.4 trong ngân sách cent

Tại F_0 gần sàn 75 Hz, một LSB $0{,}0625$ Hz tương đương

$$1200 \log_2\big((75 + 0{,}0625)/75\big) \approx 1{,}443 \text{ cent},$$

vẫn nhỏ hơn ngân sách 20 cent. Tại trần 600 Hz:

$$1200 \log_2\big((600 + 0{,}0625)/600\big) \approx 0{,}180 \text{ cent}.$$

Vậy Q12.4 "thô" nhất ở tần số thấp nhưng vẫn không phải thủ phạm GPE (script verify khẳng định hai số cent trên). Thủ phạm giao thức (endian) tạo sai số hàng nghìn Hz — lớn hơn mọi lượng tử.

Ví dụ tính tay — làm tròn nửa LSB. $F_0 = 200{,}03125$ Hz (đúng nửa LSB trên 200): $200{,}03125 \cdot 16 = 3200{,}5$. Quy ước `round` half-away / banker's tùy ngôn ngữ; Python `round` dùng banker's (về số chẵn) nên `round(3200.5)`=3200. Trên RTL, thường cắt hoặc cộng $0{,}5$ rồi trunc — phải khóa một quy ước trong twin và TB. Lab khuyến nghị: `int(floor(x*16+0.5))` (half-up) cho khớp nhiều LPM. Với $199{,}466$, cả hai quy ước đều cho 3191 vì $3191{,}456$ không nằm nửa.

12.8. Bài tập và lời giải giải thích

Bài tập 12.1. Pack 220 Hz: kỳ vọng F_q và hai byte LE?

Lời giải. $220 \cdot 16 = 3520 = 0\text{x}0\text{DC}0$. LE: `C0 0D`. Khớp bảng mục 12.3 và script verify.

Bài tập 12.2. Đổi big-endian cố ý trên 200 Hz: evaluator đọc ra bao nhiêu Hz?

Lời giải. Bytes thật `80 0C`; đọc BE cho $0\text{x}0\text{C}80 = 32780$, $32780/16 = 2048{,}75$ Hz. Sai số hàng nghìn Hz — rõ ràng là lỗi giao thức, không phải MAC.

Bài tập 12.3. Vì sao UV phải là `00 00` chứ không phải $0{,}0625$ Hz ($q = 1$)?

Lời giải. $q = 1$ vẫn là "có cao độ" rất thấp; các chỉ số VDE/FFE và logic thanh điệu sẽ coi khung đó hữu thanh. Hợp đồng lab: đúng một mã 0 duy nhất cho Unvoiced/Silence sau cổng Chương 10.

Bài tập 12.4. Tách bit 3191: phần nguyên, frac, $\hat{F}_0$?

Lời giải. $3191 \gg 4 = 199$, $3191\&15 = 7$, $\hat{F}_0 = 199 + 7/16 = 199{,}4375$ Hz. Đã kiểm script.

Chương 13. Kiểm chứng ba tầng: float / fixed / RTL

▶ MỤC TIÊU HỌC TẬP

Sau Chương 13, người học sẽ:

1. Vận hành được quy trình kiểm chứng ba tầng: (1) mô hình float đối chiếu Praat; (2) mô hình dấu phẩy tĩnh twin đối chiếu float; (3) RTL/sim đối chiếu twin bit-exact.
2. Định nghĩa và tính tay được ba chỉ số cổ điển **GPE**, **VDE**, **FFE** (Gross Pitch Error, Voicing Decision Error, Fine/F0 Frame Error) kèm ví dụ số đã kiểm Python.
3. Viết testbench Avalon-ST có `valid`-gap và backpressure, đếm beat, so Q12.4 với golden.
4. Điền bảng tài nguyên từ báo cáo Fitter thật (không bịa số), và giải thích vì sao mỗi tầng bắt một lớp lỗi khác nhau.

13.1. Hiện tượng thật: ba tầng bắt ba lớp lỗi

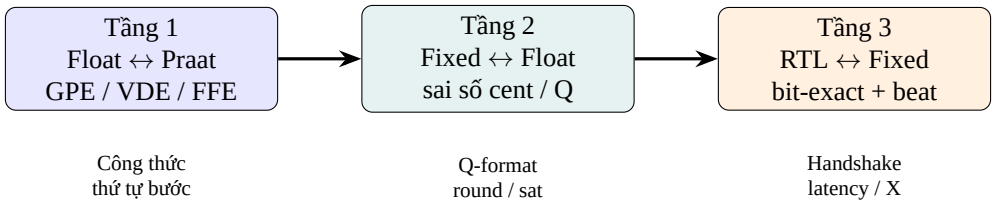
★ NGUYÊN LÝ ĐẦU TIÊN: Silicon không chạy Python — hãy khoanh đúng lớp

Một đường F_0 sai có thể đến từ rất nhiều chỗ: nhầm công thức Boersma, sai Q-format, sai handshake, quên bão hòa, đảo endian. Nếu nhảy thẳng vào Signal Tap trên DE10, bạn sẽ mất ngày để đoán. Ba tầng kiểm chứng tách bạch ba lớp:

- **Tầng 1 — Float vs Praat:** cùng wav, cùng time step 10 ms, cùng floor/ceiling 75–600 Hz. Sai ở đây là sai *thuật toán / thứ tự bước* (quên chuẩn hóa $C[\tau]$, sai parabolic, tắt nhầm silence gate).
- **Tầng 2 — Fixed twin vs Float:** cùng công thức nhưng mọi phép toán dùng Q$m.n$, round/sat/shift giống RTL. Sai ở đây là sai *lượng tử hóa và làm tròn*.
- **Tầng 3 — RTL vs Fixed:** ModelSim/Questa hoặc Signal Tap dump từng mẫu/từng khung. Mục tiêu lab: 0 **bit mismatch** trên vector golden. Sai ở đây là sai *handshake, latency, reset, X-propagation*.

Nhảy thẳng tầng 3 khi tầng 1 còn đỏ chính là lãng phí ngày debug FSM.

Ba vector bắt buộc mọi khối tiền xử lý (đã nêu Chương 3): `zero` (toàn 0), `DC` (hằng khác 0), `clip` (đụng ± 32767). Thêm vector sine $75/100/200/440/590$ Hz, nhiễu trắng, âm xát $/s/$, và hai đoạn giọng thật — cùng tinh thần `verify_tiers.py`.



Hình 55: Ba tầng kiểm chứng: mỗi tầng khoanh một lớp lỗi; chỉ xuống tầng dưới khi tầng trên đã xanh trên cùng bộ vector.

13.2. Toán: GPE, VDE, FFE — định nghĩa từng ký hiệu

Trong đánh giá cao độ chuẩn (Rabiner, Chu & Alwan, v.v.), mỗi khung được gắn nhãn hữu thanh/vô thanh bởi tham chiếu (ở đây: Praat) và bởi hệ thống (float / fixed / RTL). Ký hiệu:

- N: tổng số khung có mặt cả hai hệ thống.
- N_{VV}: số khung cả tham chiếu lẫn hệ thống đều hữu thanh.
- $F_{\text{ref}}[k]$, $F_{\text{hw}}[k]$: cao độ (Hz) trên các khung cùng hữu thanh.

GPE — Gross Pitch Error (Sai cao độ thô).

$$\text{GPE} = \frac{1}{N_{VV}} \left| \left\{ k \in VV : \left| F_{\text{hw}}[k]/F_{\text{ref}}[k] - 1 \right| > 0{,}2 \right\} \right|. \tag{44}$$

Ngưỡng 20% roughly một quãng gần (khoảng lớn hơn sai số tinh). GPE cao thường báo octave error ($F_0/2$ hoặc $2F_0$) — đúng bệnh Chương 8–10 khi quên $C[\tau]$ hoặc voicing nuốt đỉnh đúng.

VDE — Voicing Decision Error (Sai quyết định hữu/vô thanh).

$$\text{VDE} = \frac{N_{\text{FV}} + N_{\text{FU}}}{N}, \tag{45}$$

với N_{FV} = false voiced (hệ thống báo V khi tham chiếu UV), N_{FU} = false unvoiced (ngược lại). VDE cao $\Rightarrow$ xem lại ngưỡng V, S, warmup P_{global} (Chương 10).

FFE — F0 Frame Error (gộp). Nhiều báo cáo lab định nghĩa FFE là tỷ lệ khung "hỏng" theo nghĩa *hoặc* sai voicing *hoặc* GPE trên khung cùng hữu thanh:

$$\text{FFE} = \frac{N_{\text{VDE frames}} + N_{\text{GPE frames}}}{N}. \tag{46}$$

(Một số tài liệu gọi "Fine Error" là sai số cent trung bình trên khung *không* GPE; lab này dùng FFE dạng frame-error tổng hợp như trên và ghi rõ trong báo cáo.)

Sai số cent trên khung "tinh".

$$\Delta_{\text{cent}}[k] = 1200 \log_2\big(F_{\text{hw}}[k]/F_{\text{ref}}[k]\big). \tag{47}$$

Ngân sách lab trên sine/giọng sạch: $\max|\Delta_{\text{cent}}| < 20$—$30$ cent sau khi đã loại khung GPE (nếu có). Mỗi stage (DC, FIR lượng tử, window Q1.15, MAC cắt bit, $C[\tau]$ Q3.13, parabolic, LPM_DIVIDE, Q12.4) góp vài cent; vượt ngân sách thì lần ngược từ peak về FIR, không đoán.

13.3. Ví dụ tính tay — chỉ số và cent (đã kiểm Python)

Cent. $F_{\text{hw}} = 199{,}466\,\text{Hz}$ (parabolic Chương 9), $F_{\text{ref}} = 200\,\text{Hz}$:

$$\Delta_{\text{cent}} = 1200 \log_2(199{,}466/200) \approx -4{,}629\,\text{cent}.$$

Nằm trong ngân sách 20 cent.

Đồ thị nhầm lẫn giả định $N = 20$ khung. Giả sử $N_{VV} = 14$, trong đó 2 khung lệch $> 20\%$ (octave); thêm 3 khung sai voicing (không trùng hai khung GPE):

$$\text{GPE} = \frac{2}{14} \approx 0{,}1429, \qquad \text{VDE} = \frac{3}{20} = 0{,}15, \qquad \text{FFE} = \frac{5}{20} = 0{,}25.$$

Script `verify/verify_chuong_10_14.py` khẳng định các tỷ số này. Trên sine sạch lab, mục tiêu thực tế: GPE ≈ 0, VDE thấp, $|\Delta_{\text{cent}}| < 20$.

LSB Q12.4. Tại 200 Hz, $0{,}0625\,\text{Hz} \approx 0{,}541$ cent — nhỏ hơn nhiều so với $-4{,}6$ cent của parabolic ở ví dụ trên, nên tầng pack hiếm khi là thủ phạm GPE.

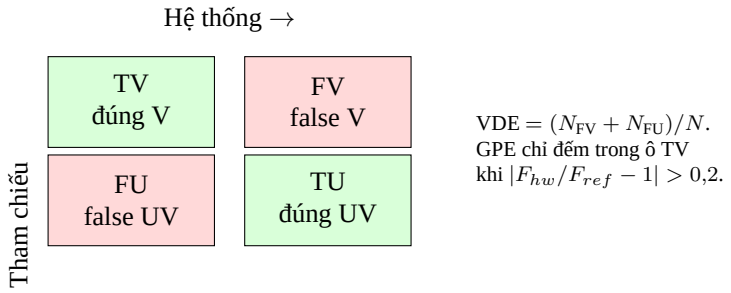
Ví dụ tính tay — ma trận nhầm lẫn năm khung. Cho tham chiếu Praat và hệ thống (Hz; 0 = UV):

k	0	1	2	3	4
F_{ref}	200	200	0	180	0
F_{hw}	199,5	100	0	0	50
Nhãn lỗi	tinh	GPE octave	OK UV	false UV	false V

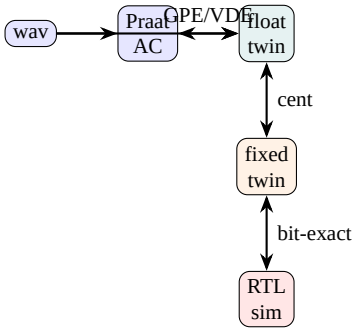
Khung cùng hữu thanh $VV = \{0, 1\}$ ($N_{VV} = 2$). Khung 1 có $|100/200 - 1| = 0{,}5 > 0{,}2 \Rightarrow N_{\text{GPE}} = 1$. Sai voicing ở $k = 3$ (FU) và $k = 4$ (FV) $\Rightarrow N_{\text{VDE}} = 2$.

$$\text{GPE} = \frac{1}{2} = 0{,}5, \qquad \text{VDE} = \frac{2}{5} = 0{,}4, \qquad \text{FFE} = \frac{1+2}{5} = 0{,}6.$$

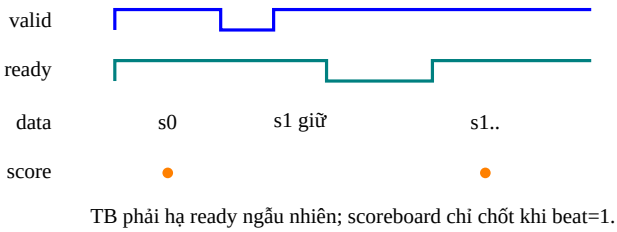
Script verify khẳng định $N_{\text{GPE}} = 1$, $N_{\text{VDE}} = 2$ trên đúng vector này.



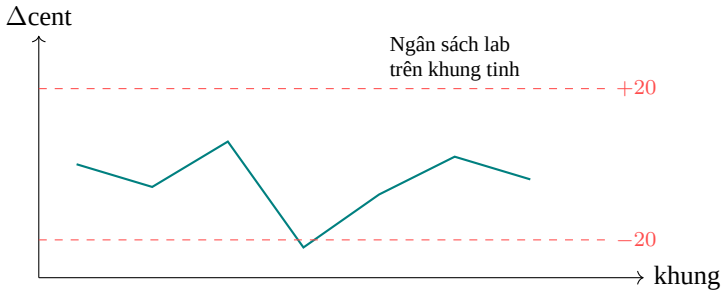
Hình 56: Ma trận hữu/vô thanh: VDE lấy hai ô đỏ; GPE lấy tập con của ô TV.



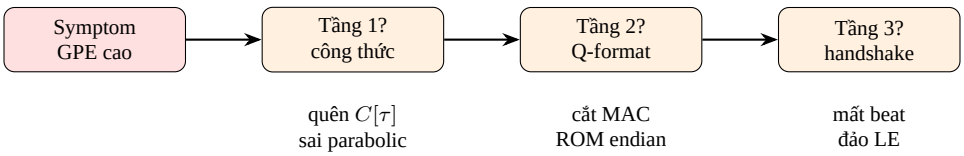
Hình 57: Luồng đối sánh tầng 1→2→3 trên cùng wav.



Hình 58: Testbench Avalon-ST: valid-gap + backpressure; chỉ chấm cam được so golden.



Hình 59: Sai số cent khung-tinh phải nằm trong ±20—30 cent sau khi loại GPE.



Hình 60: Cây chẩn đoán: luôn hỏi tầng trên trước khi đụng Signal Tap.

13.4. Tầng 1: Float vs Praat

Thực hành khuyến nghị (Parselmouth hoặc Praat batch):

```
snd = parselmouth.Sound(wav, sampling_frequency=16000)
```

```
pm = snd.to_pitch_ac(time_step=0.01, pitch_floor=75, pitch_ceiling=600)
f_ref = pm.selected_array["frequency"]  # 0 = UV
# f_float: twin float_pitch_frame tung hop
# Do GPE/VDE/FFE chi tren khung overlap thoi gian
```

Tham số bắt buộc khớp lab: time step $0{,}01$ s, floor 75, ceiling 600, và (nếu đối chiếu sâu) cùng voicingThreshold $0{,}45$. Khác tham số $\Rightarrow$ "sai số" giả tạo.

13.4b. Tầng 1 sâu: khớp tham số Praat

Ngoài time step / floor / ceiling, các tham số hay làm lệch giả tạo:

- `voicingThreshold` (mặc định $0{,}45$): khác số $\Rightarrow$ VDE giả.
- `octaveCost` / `octaveJumpCost`: twin float lab tắt pathFinder thì không so được đường cong "có Viterbi" của Praat GUI mặc định — phải gọi API/`Pitch.cpp` với cùng cờ tắt, hoặc chấp nhận so trên khung đã voiced ổn định.
- Đồng bộ thời gian: tâm cửa sổ lab tại $kH + L/2$; Praat time step $0{,}01$ s phải neo cùng gốc.

Ví dụ tính tay — RTF mô phỏng 1 s. Số khung 97, mỗi khung $\approx 120\,960$ chu kỳ MAC:

$$T_{\text{proc}} = \frac{120\,960 \cdot 97}{50 \cdot 10^6} \approx 0{,}2347\,\text{s}, \qquad \text{RTF} \approx 0{,}2347$$

(RTF = *Real-Time Factor* = thời gian xử lý / thời gian âm thanh; đã kiểm script). Một TB chu kỳ-chính xác cho 1 s audio vì thế chạy khoảng $0{,}23$ s thời gian mô phỏng xung nhịp (wall-clock phụ thuộc tốc độ sim).

13.5. Tầng 2: Fixed twin vs Float

Twin cố định bit dùng cùng ROM Hanning Q1.15, ROM $C[\tau]$ Q3.13, cùng sat/shift như RTL. Đo:

- Sai lệch cent trung bình / max giữa F_{fixed} và F_{float} trên khung cùng voiced;
- Số khung đổi voicing so với float (thường 0 trên sine).

Nếu tầng 1 xanh mà tầng 2 đỏ: nghi ngờ cắt bit MAC, quên dịch BFP (*Block Floating Point* — mỗi khối FFT chia sẻ một exponent; nếu dùng FFT IP), hoặc ROM $C[\tau]$ sai endian trong hex.

13.5b. Tầng 2 sâu: nguồn cent cố định bit

Mỗi khối góp sai số lượng tử xấp xỉ (thứ tự độ lớn, không phải định lý):

- Window Q1.15: hệ số mép khung lệch 1 LSB $\rightarrow$ vài cent.
- MAC cắt về thanh ghi trung gian: nếu cắt quá sớm trước khi nhân $C[\tau]$, đỉnh τ lớn bị thiên vị.
- ROM $C[\tau]$ Q3.13: sai endian hex $\rightarrow$ GPE hàng loạt (lớp này đôi khi lộ sớm ở tầng 1 nếu float cũng đọc nhầm hex — tách test ROM).
- LPM_DIVIDE làm tròn quotient: thường < 1 cent tại 200 Hz.
- Pack Q12.4: LSB $\approx 0{,}54$ cent tại 200 Hz.

Tổng ngân sách 20–30 cent là "hộp" chứa cho cả chuỗi; vượt hộp thì lần từ pack ngược về FIR, đo cent từng điểm neo (sau window, sau MAC, sau $C[\tau]$, sau DIV).

13.6. Tầng 3: RTL vs Fixed — bit-exact và testbench

Hợp đồng bit-exact. Trên cùng `golden_pcm.hex`, mọi khung Q12.4 dump từ sim phải bằng twin fixed: 0 bit lệch. Không "gần đúng" ở tầng này.

Testbench Avalon-ST.

- Đọc PCM hex → stream sample với `valid` có gap ngẫu nhiên (LFSR).
- Sink hạ `ready` ngẫu nhiên ở mọi stage (đặc biệt MAC↔peak, pack↔HPS).
- Đếm beat vào = đếm beat ra (trừ decimate cố ý).
- Scoreboard so F_q với `golden_f0.hex`.
- Vector cạnh: zero / DC / clip phải cho $F_q = 0$ hoặc hành vi DC-blocker đã định nghĩa (Chương 3).

```
delta_cent = 1200 * np.log2(f_hw / f_praat)
assert np.nanmax(np.abs(delta_cent[voiced_both])) < 20.0
assert (q_rtl == q_fixed).all()  # tang 3
```

Tài liệu Intel cần mở đúng số. Avalon 683091, FIR 683208, FFT 683374 (nếu bật), LPM_DIVIDE handbook. Fmax/ALM/DSP/M10K lấy từ *Fitter report* sau Place & Route — không bịa.

13.6b. Tầng 3 sâu: X-propagation và reset

RTL sim phải:

1. Khởi động `rst_n`$= 0$ đủ vài chu kỳ; mọi thanh ghi median/FSM về 0.
2. Không để `X` (trạng thái chưa xác định trong mô phỏng RTL — bit chưa được gán $0/1$ sau reset) trên `valid`/`ready`: một `X` làm scoreboard "xanh giả" hoặc đỏ ngẫu nhiên.
3. Dump F_q chỉ khi beat OUT; so với `golden_f0.hex` từ twin fixed cùng seed LFSR cho ready.
4. Vector cạnh zero/DC/clip: kỳ vọng $F_q = 0$ (hoặc hành vi DC-blocker đã khóa Chương 3).

13.6c. Golden hex và tái lập bit-exact

Mỗi revision RTL phải kèm:

- `golden_pcm.hex` — vector vào (hoặc WAV gốc + script gen).
- `golden_f0.hex` — chuỗi F_q từ twin fixed cùng tham số.
- Seed LFSR cho valid-gap/ready trong TB (ghi vào header test).

Không tái lập được seed thì "bit-exact hôm qua" không chứng minh được hôm nay. Khi đổi V, S, hoặc ROM $C[\tau]$, phải regenerate golden và ghi chú DIFF.

Ví dụ tính tay — ngưỡng GPE 20%**.** Tại $F_{\text{ref}} = 200\,\text{Hz}$, biên GPE là 160 và $240\,\text{Hz}$. Mọi ước lượng trong $(160, 240)$ trên khung VV *không* tính GPE (chỉ đóng góp cent tinh). Octave đúng 100 hoặc 400 nằm ngoài biên — GPE chắc chắn. Đó là lý do GPE cao thường chỉ thẳng ROM $C[\tau]$ hoặc voicing nuốt đỉnh đúng rồi bám hài.

FFE và báo cáo. Khi viết báo cáo lab, luôn định nghĩa FFE một lần (công thức (46)) và không trộn với "fine error trung bình cent". Người chấm cần biết bạn đang đếm *tỷ lệ khung hỏng* chứ không phải trung bình cent.

▶ NHÌN THẤY VẬT LÝ

Ba tầng không phải nghi thức nộp bài. Chúng là cách khoanh vùng nhanh: công thức, lượng tử, hay dây nối. Một TB "end-to-end đỏ" không nói bạn sửa chỗ nào; ba đèn xanh/đỏ nói được.

13.7. Bảng tài nguyên (điền từ Fitter)

Khối	ALM	DSP	M10K	Latency (ck)
PLL + I^2S				
DC-blocker				
FIR $2/3$				
Sliding RAM				
Window				
MAC ACF				
ROM $C[\tau]$				
Peak + DIV				
Voicing + median-3				
FSM + pack Q12.4				
Tổng (mục tiêu $\lesssim 850$ ALM, 3 DSP, 4 M10K)				

Khi nộp bài (Chương 14), không được để trống các ô tổng nếu đã chạy Fitter — điền số thật kèm ảnh chụp report.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách xác minh — giải thích từng mục.

- **Thời gian sim:** vài giây đến vài phút cho 1 s audio @ 50 MHz TB (tuỳ độ chi tiết dump).
- **Disk:** golden hex vài MB — commit cùng revision RTL để tái lập.
- **Người:** không bỏ tầng float; mỗi fail gắn đúng tầng (GPE → thuật toán; cent fixed → Q-format; bit mismatch → RTL).
- **Vì sao ba tầng:** một TB "tất cả" không chỉ ra *loại* lỗi; tách tầng rút ngắn chu kỳ sửa.

13.7b. Cách điền Fitter thật

Sau Place & Route trên Quartus Standard/Lite cho part `5CSXFC6D6F31C6N`:

1. Mở *Fitter* → *Resource Usage Summary*: copy ALM logic, DSP blocks, M10K.
2. Mở *Timing Analyzer*: Fmax của clock 50 MHz (hoặc clock datapath thật).
3. Breakdown theo hierarchy nếu tool cho phép — tách MAC, FIR, FSM.
4. Không lấy số từ Synthesis sớm (trước Fitter) làm số nộp: synthesis ước tính, Fitter mới là silicon mapping.

Mục tiêu Chương 0: $\lesssim 850$ ALM, 3 DSP, 4 M10K, RTF $\approx 0{,}24$. Vượt M10K thường do FIFO/RAM thừa; vượt DSP do nhân cửa sổ + MAC + FIR không chia sẻ — xét time-multiplex trước khi xin thêm.

13.7c. Liên hệ RTF, sim wall-time và Signal Tap

RTF $\approx 0{,}23$ trên 1 s audio nghĩa là mô phỏng chu kỳ-chính xác "bắt kịp" nhanh hơn realtime trên giấy; nhưng ModelSim/Questa có thể chậm hơn realtime tường vì diễn dịch. Đừng nhầm wall-time sim với RTF silicon. Signal Tap trên board đo realtime thật: nếu FIFO đầy liên tục, backpressure đang ăn biên $377\,000$ chu kỳ — cần giảm dump hoặc tăng FIFO, không thêm DSP.

Ví dụ tính tay — đếm khung overlap thời gian. Praat và twin phải so trên cùng tập thời điểm. Nếu Praat trả 100 khung và twin 97 khung cho 1 s (do neo cửa khác), chỉ lấy giao khoảng thời gian chung, ví dụ 90 khung giữa. Tính GPE/VDE trên $N = 90$ đó; không pad 0 giả vào đuôi. Sai bước này tạo VDE giả ở biên câu.

Chuẩn bị báo cáo. Mỗi tầng một mục: lệnh chạy, tham số, bảng chỉ số, kết luận xanh/đỏ. Đính kèm hash git của RTL + twin + golden. Người chấm tái lập được thì điểm phương pháp mới vững.

Trước khi sang Chương 14. Tầng 1 cần GPE≈ 0 trên sine sạch; tầng 2 max cent < 20; tầng 3 không lệch bit và đếm beat khớp. Ngoài zero/DC/clip/sine, giữ một đoạn xát vô thanh (ví dụ $/s/$) và một đoạn giọng có thanh điệu — tránh VDE "xanh giả" vì toàn hữu thanh, và để Pearson có nghĩa.

13.8. Bài tập và lời giải giải thích

Bài tập 13.1. Chạy twin DC-blocker trên zero / DC / clip. Kỳ vọng đầu ra định tính?

Lời giải. Zero $\to$ zero. DC hằng $\to$ sau vài hằng số thời gian α, đầu ra $\to 0$ (zero tại $z = 1$). Clip ± 32767 xen kẽ $\to$ không được wrap thành âm lớn; sat_16 giữ biên. Nếu clip tạo dao động limit-cycle, nghi `y_acc` quá hẹp (Chương 3).

Bài tập 13.2. Cố ý đảo endian Q12.4 trên golden $200\,\text{Hz}$. GPE/cent thay đổi thế nào?

Lời giải. Server đọc $2048{,}75\,\text{Hz}$ (Chương 12). $|2048{,}75/200 - 1| \gg 0{,}2 \Rightarrow$ mọi khung đó thành GPE; Δ_{cent} khổng lồ. Tầng 1 "đỏ" dù thuật toán đúng — phải bắt bằng test pack/unpack trước khi đổ tội MAC.

Bài tập 13.3. Cho $N = 20$, $N_{VV} = 14$, 2 GPE, 3 VDE (rời nhau). Tính GPE, VDE, FFE.

Lời giải. GPE $= 2/14 \approx 0{,}1429$, VDE $= 3/20 = 0{,}15$, FFE $= 5/20 = 0{,}25$ — khớp script verify.

Chương 14. Checklist nghiệm thu cuối cùng (kèm lý do)

▶ MỤC TIÊU HỌC TẬP

Trước khi nộp bài hoặc đóng gói bitfile cho DE10-Standard, người học sẽ:

1. Đọc từng mục checklist như một câu khẳng định có thể ký tên, kèm *lý do kỹ thuật* neo về chương đã học — không chỉ "tick cho có".
2. Chẩn đoán nhanh các triệu chứng đỏ điển hình (Q12.4 = 0, $F_0/2$, đảo endian, FSM treo) và chỉ đúng tầng kiểm chứng cần mở lại.
3. Điền bảng tự ký (RTL lint, twin bit-exact, smoke phần cứng, tài liệu đủ tám yếu tố) với ngày theo múi giờ làm việc.
4. Hiểu vì sao checklist chỉ xuất hiện **một lần** ở Chương 14: tránh lan checklist trùng, mọi khẳng định đã được giải thích ở Chương 2–13.

14.1. Hiện tượng thật: checklist bắt lỗi hệ thống, không phải nghi thức

★ NGUYÊN LÝ ĐẦU TIÊN: Một mắt xích sai — synth vẫn xanh

Đường F_0 trên FPGA là chuỗi quyết định đã khóa từ Chương 0: MCLK 12,288 MHz, M10K Old Data, Hanning $N-1$, ROM $C[\tau]$, parabolic + LPM_DIVIDE, voicing + median-3, Avalon-ST giữ data, Q12.4 LE, Viterbi tắt trên fabric. Sai một mắt xích thường *không* làm Quartus báo lỗi — nó làm F_0 lệch im lặng, GPE/VDE ở Chương 13 đỏ, và Pearson thanh điệu sụp. Checklist buộc bạn xác nhận lại từng mắt xích bằng **bằng chứng** (twin xanh, Fitter report, waveform Avalon, Signal Tap), không chỉ "nhớ là đã làm".

14.2. Chuỗi khẳng định có lý do (ký từng dòng)

1. **Chuỗi đồng hồ và lấy mẫu.** PLL khóa thạch anh 50 MHz tạo $MCLK = 12{,}288$ MHz; ADC codec 48 kHz; hạ mẫu về 24 kHz rồi FIR polyphase tỷ số $2/3$ về 16 kHz.
   - *Lý do:* Sai MCLK vài chục ppm làm trôi T_s, đo lệch F_0 theo thời gian (Chương 0 Feynman box 8; Chương 2). Không dùng MCLK 4,096 MHz cho gói WM8731 trên DE10 này.
2. **DC-blocker.** Thanh ghi phản hồi `y_acc` rộng 32 bit; cập nhật $y \leftarrow y + x_\Delta - (y \ggg 8)$; xuất $\mathrm{sat}_{16}(y \ggg 15)$. Twin `dc_blocker_q15` khớp từng mẫu.
   - *Lý do:* DC phình $\mathcal{R}[0]$ và nuốt đỉnh T_0 (Chương 3). `y` chỉ 16 bit tạo deadband/limit-cycle.
3. **Bộ nhớ trượt.** On-Chip Memory True Dual-Port; mixed-port read-during-write = **Old Data**; twin mô phỏng đúng giá trị cũ khi hai cổng đụng cùng địa chỉ.
   - *Lý do:* Cyclone V M10K **không** hỗ trợ New Data mixed-port (Chương 0, Chương 5). Ép New Data ⇒ bypass ALM, giảm Fmax.
4. **Cửa sổ.** ROM Hanning Q1.15 dạng cổ điển $N-1$, nửa bảng nhờ đối xứng; nhân trước MAC.
   - *Lý do:* Hanning hạ sidelobe ≈ -32 dB, bảo vệ dải F_0 khỏi rò phổ (Chương 0, Chương 6). Sai N vs $N-1$ lệch hệ số mép khung.
5. **Tự tương quan.** Đường mặc định MAC trực tiếp trên DSP; nếu bật FFT thì nhớ BFP và barrel shift 2^{2e} *trước* IFFT.
   - *Lý do:* Quên $2e$ phá tỷ lệ r_a và sụp ngưỡng voicing (Chương 0 Feynman box 6; Chương 7).
6. **Chuẩn hóa $C[\tau]$.** ROM $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (nhân thay chia runtime), trừ khi bắt buộc LPM_DIVIDE động.
   - *Lý do:* Không bù dốc cửa sổ ⇒ thiên vị τ lớn ⇒ octave xuống (Chương 8).

7. **Đỉnh và chia.** Argmax + parabolic; $F_0 = f_s/\tau$ qua LPM_DIVIDE; TB chờ đúng latency L.

   - *Lý do:* Đọc quotient sớm một chu kỳ gắn nhầm khung (Chương 9). Không dùng IP bịa ALTDIV_TENT.

8. **Voicing / median-3.** Cổng silence/voicing trên fabric; median-3 đăng ký (trễ một hop); Viterbi chỉ trên HPS nếu cần.

   - *Lý do:* Báo F_0 trên UV phá đường cong; median đập spike đơn với < 150 ALM; Viterbi $O(TK^2)$ vượt ngân sách lab (Chương 10).

9. **Avalon-ST.** Handshake `valid`/`ready`/`SOP`/`EOP`: source giữ data khi backpressure; đếm beat khớp; gói 640 mẫu có SOP/EOP đúng chỗ.

   - *Lý do:* Mất một mẫu = lệch khung = sai F_0 im lặng (Chương 11; doc 683091).

10. **Đóng gói.** Q12.4 little-endian; UV $= 0$; cặp `audio_16k.wav` $\leftrightarrow$ `pitch_f0.bin` đồng bộ hop 10 ms; golden so Praat trong dung sai đã ghi.

    - *Lý do:* Đảo endian biến 200 Hz thành $2048{,}75$ Hz — lỗi giao thức (Chương 12).

11. **Twin ba tầng.** Vector zero / DC / sine / clip / giọng thật xanh từ float$\rightarrow$fixed$\rightarrow$RTL; GPE/VDE/FFE và cent nằm trong ngân sách báo cáo (Chương 13).

    - *Lý do:* Mỗi tầng khoanh một lớp lỗi; skip tầng float = debug FSM khi công thức còn sai.

12. **Báo cáo tài nguyên.** Ghi ALM (hoặc LE), DSP, M10K, Fmax, latency một khung — lấy từ Fitter, đối chiếu costbox từng chương và ngân sách Chương 0 (≈ 850 ALM, 3 DSP, 4 M10K, RTF $\approx 0{,}24$).

    - *Lý do:* Số bịa không kiểm chứng được trên máy chấm; Fmax thấp có thể do ép New Data hoặc chia tổ hợp dài.

13. **Không dùng tên IP / cấu hình bịa.** Không ALTDIV_TENT; không CIC cho tỷ số $2/3$; không MCLK $4{,}096$ MHz cho gói DE10 WM8731 này; không "New Data" mixed-port M10K trên Cyclone V.

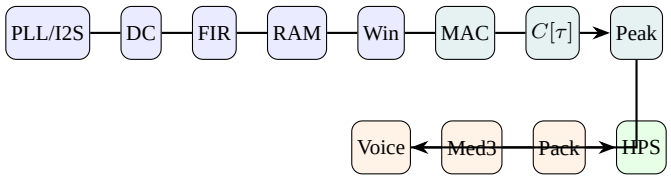
    - *Lý do:* Các lựa chọn đó hoặc không tồn tại trên tool Standard/Lite, hoặc sai silicon thật — điểm trừ cứng.

14. **Giữ nguyên v10.** Phụ lục / các trang v10 (pp.1–38 trong bản đóng gói) không bị sửa nội dung giải thích gốc.

    - *Lý do:* Chính sách DIFF: phần mới là Chương 0–14; không đụng luồng chữ đã khóa.

14.2b. Đọc checklist như chuỗi nhân quả, không như danh sách tick

Mỗi dòng mục 14.2 neo một quyết định silicon đã dạy ở chương trước. Hãy đọc chúng theo dòng tín hiệu: thạch anh nuôi PLL tạo $MCLK$ (Chương 2) — nếu nhầm tần số PLL, mọi τ sau đó lệch theo ppm tích lũy. DC-blocker giữ thanh ghi phản hồi đủ rộng (thường 32 bit) để tránh deadband (vùng lượng tử hóa thành 0 cập nhật, tạo "rung" trên im lặng). Sliding RAM trên M10K chỉ được cấu hình mixed-port **Old Data** hoặc Don't Care trên Cyclone V — không có New Data. Cửa sổ Hanning đúng mẫu số $N-1$ hạ thùy phụ (sidelobe, dải phổ phụ quanh đỉnh chính) khoảng -32 dB. MAC tuần tự tránh BFP của FFT; ROM $C[\tau]$ chống thiên vị octave xuống; LPM_DIVIDE phải chờ đủ latency L trước khi đọc thương. Cổng voicing + median-3 thay Viterbi trên fabric; Avalon-ST và Q12.4 là hai mắt xích giao thức — mất beat lệch thuật toán im lặng, đảo endian làm metric nổ dù silicon đúng.



Hình 61: Chuỗi mắt xích nghiệm thu: mỗi hộp ứng một dòng checklist có lý do neo chương.

14.3. Bảng tự ký

Hạng mục	Người ký	Ngày (Asia/Saigon)
RTL lint sạch (không latch suy đoán, reset bất đồng bộ đúng)		
Twin bit-exact tầng 2–3 xanh trên golden		
Hardware smoke trên DE10 (SOP→Q12.4 ra HPS)		
Tài liệu đủ 8 yếu tố / chương (Chương 0 mục 0.2)		
Bảng Fitter đã điền số thật		

14.3b. Quy trình smoke trên DE10 (30–60 phút)

1. Nạp bitfile; xác nhận PLL lock (LED/status reg).
2. Phát sine 200 Hz từ PC hoặc ROM nội; HPS đọc N từ Q12.4.
3. Kỳ vọng: đa số khung $F_q \in [3180, 3220]$ (tức $\approx 198{,}75$—$201{,}25$ Hz), không toàn 0, không toàn 9600.
4. Tắt nguồn âm: kỳ vọng $F_q = 0$ sau khi P_g ổn.
5. Chạy `verify_chuong_10_14.py` và (nếu có) `verify_tiers.py`; đính log.
6. Chụp Fitter: ALM/DSP/M10K/Fmax; điền bảng Chương 13.

Ví dụ tính tay — ngân sách M10K nhắc lại.

$$640 \times 16 = 10\,240 \text{ bit} = 1 \times \text{M10K (đúng dung lượng)}.$$

Nếu Fitter báo 12 M10K: nghi ping-pong thừa, FIFO dư, hoặc vô tình suy ra RAM cho Viterbi/FFT khi lab không cần.

Ví dụ tính tay — RTF board. Cùng công thức Chương 0/11: $120\,960/500\,000 \approx 0{,}2419$. Đo wall-time trên HPS (thời gian giữa hai SOP khung) phải < 10 ms trung bình; nếu > 10 ms ổn định, nghi FSM chờ chia/hand-shake kẹt — không nghi "thiếu DSP" vì MAC chỉ dùng 1 DSP.

▶ NHÌN THẤY VẬT LÝ

Checklist xanh không có nghĩa lý thuyết Praat đã "chứng minh". Nó có nghĩa: trên bộ vector đã khóa, ba tầng kiểm chứng và các ràng buộc silicon DE10 đã được xác nhận bằng chứng. Đổi L, H, hoặc ceiling thì phải chạy lại suite — không ăn theo chữ ký cũ.

14.4. Chẩn đoán đỏ → xanh trước khi nộp

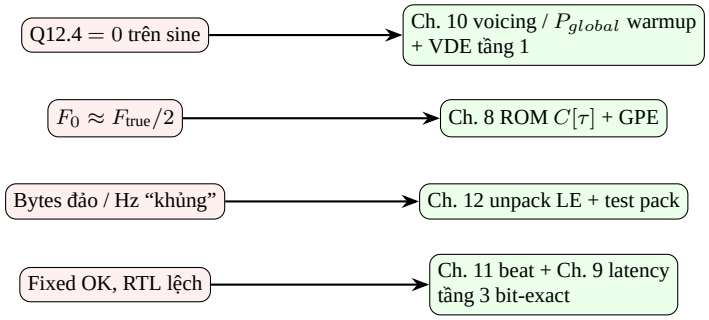
★ NGUYÊN LÝ ĐẦU TIÊN: Một triệu chứng, một nguyên nhân điển hình

- Golden sine 200 Hz ra Q12.4 $= 0$: thường voicing gate nuốt — `silenceThreshold` quá cao hoặc P_{global} chưa warmup. Mở lại Chương 10 và tầng 1 VDE.
- Ra ≈ 100 Hz thay vì 200: thường quên nhân $C[\tau]$ (đỉnh $2T_0$ thắng) hoặc octave bias. Mở Chương 8 và GPE.
- Ra 200 nhưng bytes `0C 80`: đảo endian phía đọc. Mở Chương 12 — silicon có thể đúng.
- Đúng trên twin fixed nhưng sai trên RTL: nghi latency LPM_DIVIDE, backpressure làm mất beat, hoặc reset không xóa thanh ghi median. Mở Chương 9–11 và tầng 3.
- FSM treo ở `FILL`: thiếu `EOP` hoặc đếm beat $< L$. Mở Chương 11.

Twin phải bắt từng lớp trước khi đụng Signal Tap.

Ví dụ tính tay đóng gói cuối. $F_0 = 199{,}466$ Hz (parabolic Chương 9) → round$(199{,}466 \cdot 16) = 3191$ → LE `77 0C`. UV → `00 00`. Khớp `verify/verify_chuong_10_14.py`.

Ngân sách thời gian thực nhắc lại. Tại 50 MHz, một hop 10 ms có 500,000 chu kỳ; $189 \times 640 = 120{,}960$ MAC $\Rightarrow$ RTF $\approx 0{,}2419 \ll 1$ (Chương 0 bài tập 0.2). Nếu đo RTF > 1 trên board, nghi chờ chia/handshake bị kẹt, không nghi "thiếu DSP".



Hình 62: Sơ đồ chẩn đoán nhanh: triệu chứng đỏ $\rightarrow$ chương và tầng cần mở lại trước khi nộp.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách "ngày nộp" — giải thích từng mục.

- **Thời gian người:** 30–60 phút đọc checklist + chạy lại suite twin/verify_chuong_10_14.py + (nếu có board) smoke HPS đọc `pitch_f0.bin`. Đây là chi phí quy trình, không phải ALM.
- **Disk:** giữ nguyên hex ROM w, $C[\tau]$, golden — không regenerate phút chót trừ khi đổi L hoặc H.
- **Vì sao checklist một lần duy nhất ở Chương 14:** tránh lan checklist trùng ở nhiều chương; mọi khẳng định đã neo Chương 2–13 với lý do đầy đủ. Chương 14 chỉ là cổng nghiệm thu.
- **RTF / silicon:** không đổi ở bước này; chỉ xác nhận số Fitter và đo RTF thực tế ≤ 1.

14.4b. Bảng triệu chứng $\leftrightarrow$ bằng chứng cần nộp

Triệu chứng	Mở chương / tầng	Bằng chứng xanh
Q12.4 $= 0$ trên sine 200 Hz	Ch. 10, VDE tầng 1	Log voiced$= 1$, P_g warmup
$F_0 \approx 100$ thay 200	Ch. 8, GPE	Dump $r_a[\tau]$: đỉnh $\tau \approx 80$ thắng sau $C[\tau]$
Bytes `0C 80`	Ch. 12 unpack	`hexdump` hiện `80 0C`
Fixed OK, RTL lệch	Ch. 9–11, tầng 3	Diff 0 bit và đếm beat khớp
FSM treo FILL	Ch. 11 SOP/EOP	Waveform có EOP tại beat 639
Fitter 12 M10K	Ch. 5, 14	Hierarchy: 1 sliding RAM + ROM
RTF đo > 1	Ch. 7, 11	Không kẹt ready; đúng latency DIV

Ví dụ tính tay — ký checklist có số. Trước khi ký mục "Twin ba tầng", hãy điền vào biên bản: GPE$= 0$, VDE$= 0{,}02$, max cent $= 7{,}3$ trên sine 200 Hz (số ví dụ — thay bằng số chạy thật). Số cụ thể biến chữ ký thành bằng chứng; ô trống biến checklist thành nghi thức.

Giữ DIFF và v10. Chính sách gói tài liệu: không sửa nội dung giải thích gốc trước lệnh `\input{chuong/...}` (các file `chuong_00`... đã khóa) và không đụng chương ngoài phạm vi giao. Mục "Giữ nguyên v10" trên checklist nhắc người nộp: khi compilation lệch, sửa chương mới hoặc preamble được phép — không "tiện tay" đụng khối v10.

14.4c. Thứ tự ưu tiên ngày nộp

1. Chạy `python3 verify/verify_chuong_10_14.py` — phải in `ALL ... PASSED` (thêm tiers nếu có).
2. Xác nhận `pitch_f0.bin` trên vector smoke: độ dài chẵn, mẫu 200 Hz đúng `80 0C`.
3. Đọc lại bốn triệu chứng đỏ ở Hình 62; nếu dính một, đừng nộp.
4. Điền Fitter + bảng tự ký với ngày Asia/Saigon.

5. Chỉ sau đó mới đóng bitfile / zip bài.

Thứ tự này cố ý để phần mềm/vector chặn lỗi giao thức trước khi bạn đổ công vào Signal Tap phút chót.

Nếu một chữ viết tắt trên checklist chưa rõ (ALM, DSP, M10K, MLAB, Fmax, RTF, GPE/VDE/FFE, HPS), mở lại costbox của chương neo — đừng ký khi còn đọc như mã bí mật.

14.4d. Cam kết ngắn trước chữ ký

Người ký bảng 14.3 xác nhận đã: (1) chạy verify script và giữ log; (2) nhìn tận mắt `hexdump` vài khung đầu của bin; (3) điền số Fitter không đoán; (4) không bật Viterbi trên fabric ở bản nộp mặc định; (5) không dùng tên IP bịa. Nếu một mục chưa làm, để trống ngày và làm nốt — chữ ký sớm hơn bằng chứng là nợ kỹ thuật sẽ lộ ở chấm phúc khảo.

14.5. Bài tập tự vấn trước khi ký

Bài tập 14.1. Liệt kê ba bằng chứng bạn sẽ đính kèm để chứng minh mục "Twin ba tầng xanh" không phải tự khai.

Lời giải gợi ý. (1) Log chạy `verify_tiers.py` / `verify_chuong_10_14.py` có timestamp. (2) Bảng GPE/VDE/FFE và max cent trên sine 200 Hz và một đoạn giọng. (3) Diff 0 bit giữa dump RTL và twin fixed trên `golden_f0.hex`.

Bài tập 14.2. Nếu Fitter báo 12 M10K trong khi ngân sách Chương 0 là 4, bạn kiểm tra khối nào trước?

Lời giải. Sliding RAM 640×16 chỉ cần ≈ 1 M10K; ROM Hanning nửa bảng và ROM $C[\tau]$ thêm 1–2; FIFO HPS tuỳ chọn 1. Nếu thấy 12, nghi nhân đôi buffer ping-pong không cần, suy luận suy ra block RAM cho Viterbi (đã tắt), hoặc FIR IP sinh RAM dư — mở report theo hierarchy trước khi chấp nhận vượt ngân sách.

Bài tập 14.3. Nêu một lý do *không* port `Pitch_pathFinder` lên fabric ở lần nộp mặc định.

Lời giải. Median-3 + cổng voicing đã đủ đập spike đơn trong ngân sách $< 2\%$ ALM; Viterbi đa ứng viên cần M10K path metric và tăng latency vượt nhu cầu thời gian thực lab; HPS vẫn chạy soft-float khi bài yêu cầu đường cong Praat đầy đủ (Chương 10).

Phụ lục A: Áp dụng lý thuyết F_0/Praat vào mã nguồn Python (`audio_processing.py` và `evaluator.py`)

▶ MỤC TIÊU HỌC TẬP

Phạm vi và Mục tiêu: Phụ lục này đối chiếu hai tệp mã nguồn xử lý âm thanh trong thư mục `Pronunciation_Final_Master` của kho lưu trữ `DucDocDao/Pitch-Project` (nhánh `main`) với hệ thống lý thuyết đo cao độ F_0 bằng thuật toán tự tương quan trong tài liệu này (chuẩn Praat, Boersma 1993). Để làm sáng tỏ toàn bộ luồng gọi hàm, phụ lục khảo sát thêm các đoạn mã liên kết trong `main.py`, `visualizer.py`, `generate_audio.py` và `sentence_data.py`. Mọi hành vi mô tả dưới đây đều được truy xuất trực tiếp từ mã nguồn thực tế và đối chiếu tường minh với mã nguồn C++ gốc của Praat (`fon/Sound_to_Pitch.cpp` và `fon/Pitch.cpp`).

1. Bức Tranh Tổng Thể: Hai Tệp Làm Gì Trong Hệ Thống

1.1. Ranh giới triển khai: Mã nguồn Python nguyên bản vs. Hệ thống FPGA Edge Node

Trước khi đi sâu vào mã nguồn của từng hàm, cần định vị dự án `Pitch-Project` trong bức tranh kiến trúc tổng thể **FPGA Edge Node** $\leftrightarrow$ **Shared Folder** $\leftrightarrow$ **Server** đã thiết lập tại Hình 1 (Mục 0.1):